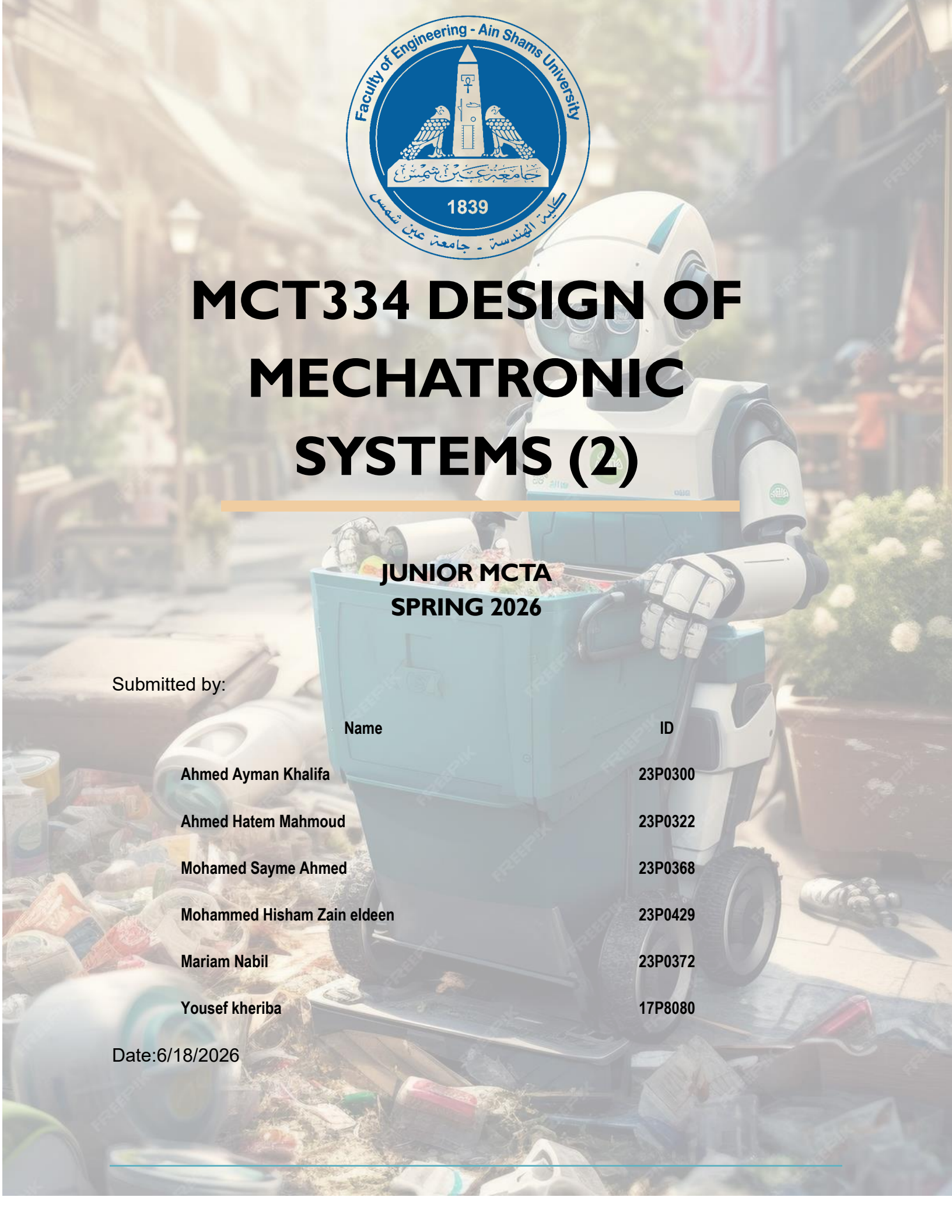


MCT334 DESIGN OF MECHATRONIC SYSTEMS (2)

JUNIOR MCTA
SPRING 2026

Submitted by:

Name	ID
Ahmed Ayman Khalifa	23P0300
Ahmed Hatem Mahmoud	23P0322
Mohamed Sayme Ahmed	23P0368
Mohammed Hisham Zain eldeen	23P0429
Mariam Nabil	23P0372
Yousef kheriba	17P8080

Date:6/18/2026

ACKNOWLEDGEMENT

We are extremely privileged and indebted to all the support, supervision, assistance and guidelines that we are receiving throughout the project.

Firstly, we owe our deep gratitude to the **Department of Mechatronics Engineering, Ain Shams University** for providing this chance to do the project. Then with all due respect, special thanks to our supervisor, Dr. Shady Ahmed Maged Ahmed Mohamed for providing the necessary guidance. And we would like to thank our project co-supervisors Eng. Mohamed Ahmed Mohanna, Eng Miriam Hani Helmy Bichai, Eng. Mina Yousry Halim, Eng. Dina Zakaria Mahmaud, Eng. Amr Sameh Mohamed Mokhtar Sharshar, Eng Youssef Nagaty who is continuously assisting, guiding and encouraging us. Our special thanks go to countless readers of this project report who are the most important critics and commentor. We welcome constructive comments and suggestions which will help us to do better in future. It is our radiant sentiment to place on record our kindest regards whoever were directly and indirectly involved in the successful completion of this project.

ABSTRACT

This report presents the successful design, implementation, and full system integration of an autonomous trash-collection robot developed for Major Task Submission 2. The project has successfully progressed from a conceptual design to a fully operational mechatronic system featuring a dual-link robotic arm mounted on a stabilized mobile chassis.

The mechanical, electrical, control, communication, and vision subsystems were successfully integrated into a unified platform, ensuring seamless interaction between hardware and software components. An RTOS-based motor control system utilizing PID regulation with IMU and encoder feedback was implemented to achieve accurate navigation, autonomous maneuvering, and reliable motion control. Static and dynamic obstacle avoidance algorithms were incorporated to enable safe operation in changing environments.

A robust electrical architecture was designed with organized power distribution, clean wiring, secure mounting, and efficient cable management. The system also includes a functional charging port, emergency-stop recovery without requiring a full reboot, and multiple state indicators to provide clear operational feedback and improve safety.

For intelligent perception, a custom-trained computer vision model was developed and deployed to detect mission-specific trash objects of different types, colors, and sizes with confidence estimation. The vision module is fully integrated with ROS2, enabling real-time communication and autonomous decision-making. High-level control is achieved through modular ROS2 nodes, launch files, and effective micro-ROS communication between the ESP32 and Raspberry Pi, with meaningful GPIO utilization for hardware interaction.

In addition, a wireless Graphical User Interface (GUI) was developed to provide an intuitive user experience. The GUI supports live visualization of robot status and sensor information, displays vision detection results in real time, and offers a manual control mode through Wi-Fi communication for testing and operation.

The complete robot was assembled, powered, and tested as a fully integrated system. Repeated mission execution demonstrated reliable autonomous navigation, successful trash detection and collection, consistent subsystem interaction, and robust overall performance. The final implementation closely matches the original CAD design while satisfying the project's functional, safety, communication, and integration requirements.

INTRODUCTION

The objective of this stage was to bridge the gap between individually developed subsystems and a fully integrated autonomous mechatronic system. While the mechanical components, JGB37 DC motors, high-torque servos, sensors, and embedded controllers were previously selected and validated, this phase focused on combining the Mechanical, Electrical, Control, Vision, and Software domains into a single intelligent robotic platform.

A major milestone of this work was the successful integration of all hardware and software modules, allowing the robot to operate autonomously while maintaining reliable communication between the ESP32 microcontroller and the Raspberry Pi through a micro-ROS framework. Real-time PID control with IMU and encoder feedback enables precise motion, while computer vision and ROS2 processing provide intelligent object detection and decision-making capabilities.

The robot was fully assembled with clean wiring, organized power distribution, secure component mounting, a functional charging interface, emergency-stop recovery functionality, and clear operational state indicators to ensure safety and maintainability. Autonomous navigation was validated through path control, turning, and both static and dynamic obstacle avoidance scenarios.

To enhance usability and monitoring, a custom Graphical User Interface (GUI) was developed with Wi-Fi communication, allowing live visualization of robot status, sensor data, and object detection results while also providing a manual control mode for operation and testing.

The entire system was subjected to multiple test runs to verify subsystem synchronization, mission repeatability, and overall reliability. These tests confirmed successful autonomous trash detection, navigation, collection, and system integration, demonstrating the effectiveness of the final mechatronic design.

This report is structured to demonstrate compliance with the project's objectives and performance standards:

- **Vision:** Detecting various trash objects with different types, colours, and sizes together with confidence estimation and ROS2 integration.
- **Mechanical Realization:** Validating the manufactured chassis and robotic arm against the original CAD models.
- **Electrical Integrity:** Ensuring a safe electrical architecture, organized power distribution, functional charging capability, emergency-stop recovery, and clean hardware integration.
- **Control Theory:** Implementing RTOS-based real-time PID loops using IMU and encoder feedback for accurate navigation and motion control.
- **Communication:** Establishing reliable micro-ROS communication between the ESP32 and Raspberry Pi through modular ROS2 architecture and launch files.
- **GUI and System Integration:** Developing a wireless graphical interface for live monitoring and manual control while validating the complete integration of all project subsystems through successful mission execution.

TABLE OF CONTENTS

Abstract 2

Introduction 4

Brief Description 7

Product-Design-Specification 8

Project Management 15

1. Work Breakdown Structure 15
2. Gantt Chart 16

CAD Design 18

1. Whole Robot 18
   a) Extra mechanism design 19
   b) Wheel Assembly 19
2. Sensors & Electronics 20
   a) Electronics 20
   b) Sensors & indicators 21
3. CAD rendered images 22
4. Working drawings 28

Mechanical Design 67

1. Actuator Sizing Using Solidworks 67
2. Robot Dynamics and Stability Calculations 68
   a) Stability in the Longitudinal Direction: 69
   b) Dynamic Force Balance (Moment Equilibrium)70
   c) Stability Limits and Tip-over Conditions 70
   d) Stability in the Lateral Direction and Roll-over Analysis: 71

Extra Mechanism Analysis 74

a) Joint Torque Analysis 74
b) Tipping Analysis 75
c) Critical Tipping Angle 75

Position Analysis 76

Velocity Analysis 76
Acceleration Analysis 76
3. Stress Analysis 77
a) Stress analysis on all critical robot components77
b) Minimum required diameter of the wheel shafts90
Results 93
4. Bearing Selection and Placement Justification96
Electrical Design..... 103
1. Block Diagram 103
2. Wiring Diagram..... 105
a) Power Supply Section..... 105
b) Microcontroller and Motion Control 105
c) Sensor and Vision Processing Unit..... 105
d) Control and System Integration..... 106
e) Safety Circuit..... 106
3. Battery Sizing Calculations..... 108
a) Current analysis..... 108
b) Calculate battery capacity..... 109
c) power consumption..... 110
4. Safety Circuit 113
a) Simulation 113
b) PCB Design 115
c) Validation 117
Software & Control 118
1. Software Flowchart..... 118
a) Low level Flowchart 119
b) High level Flowchart..... 120
2. Motor Control and PID..... 121
Simulation..... 124

1. Environment Setup 124
2. Static Obstacle Avoidance 125
3. Dynamic Obstacle 127
4. IMU and Encoders Integration 129
5. Goal Communication 131
6. Path Completion 132
7. Extra Mechanism Simulation 134
8. Start and Stop Signals 136

Bill Of Materials 137

Vision Module Evaluation 138

Integrated Manufactured Mechanical System 139

Fully Integrated Electrical System 140

micro-ROS Communication Setup 141

Hardware Requirements 142

1. Main Power Control 142
2. Emergency Stop System 142
3. State Indicators 142
4. Charging Interface 143
5. Wiring and Hardware Integration 143

Navigation 144

1. Autonomous Maneuvering 144
2. Static Obstacle Avoidance 144
3. Dynamic Obstacle Avoidance 144

Graphical User Interface (GUI) 146

1. Connection and Communication 146
2. Robot Operating Modes 146
3. Mission Control 147
4. GUI Performance 147

Mission Execution 148

1. Mission Workflow 148
2. Detection and Collection Process 148
3. Extra Mechanism Functionality 148
4. System Integration and Repeatability 149
High-Level Control 150
1. Modular ROS2 Nodes 150
2. Launch File Implementation 150
3. micro-ROS Communication 150
4. GPIO Utilization 151
System Integration 152
Conclusion 153

LIST OF TABLES

Table 1 Functional Requirements 8
Table 2 components power consumption 13
Table 3 total average power 14
Table 4 radius to safe velocity 73
Table 5 shaft forces calculations 91
Table 6 current and voltage consumption 109
Table 7 bill of material 137

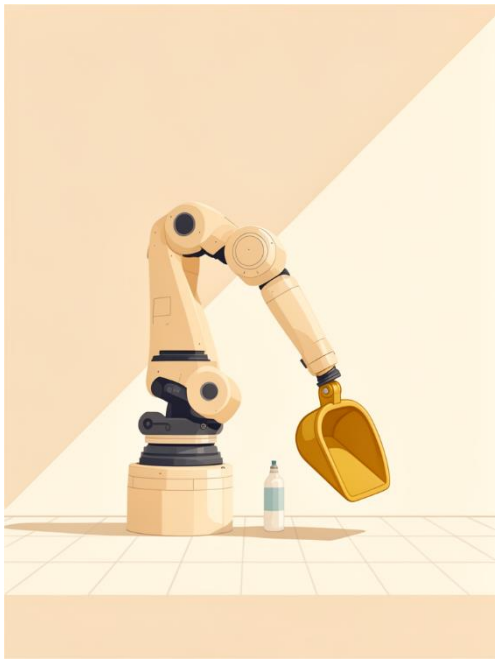
LIST OF FIGURES

Figure 1 Whole robot CAD design 18
Figure 2 extra mechanism design 19
Figure 3 Wheel Assembly 19
Figure 4 Shaft fixation 20
Figure 5 electronics fixation 20
Figure 6 sensors fixation 21
Figure 7 CAD rendered 22
Figure 8 Working drawings 28
Figure 9 torque-time graph 67
Figure 10 Stability in the Longitudinal Direction 69
Figure 11 stability in the Lateral Direction 71
Figure 12 the corresponding velocity curves 73
Figure 13 extra mechanism analysis 74
Figure 14 stress analysis 77
Figure 15 create the shaft and place it 90
Figure 16 shaft datasheet 91
Figure 17 add forces, and material data 92
Figure 18 minimum diameter 93
Figure 19 deflection angle 94
Figure 20 deflection 94
Figure 21 Block Diagram 104
Figure 22 wiring diagram 107
Figure 23 Safety circuit during Short circuit 113
Figure 24 Safety circuit during normal 114
Figure 25 Safety circuit schematic 115
Figure 26 Safety circuit PCB 115

Figure 27 PCB Layout View 116
Figure 28 Normal case 117
Figure 29 Short circuit case 117
Figure 30 Gazebo Environment 124
Figure 31 mechanical configuration ..... **Error! Bookmark not defined.**
Figure 32 electrical implementation 140
Figure 33 emergency switch and indicators 143
Figure 34 GUI interface 147
Figure 35 final prototype 152

BRIEF DESCRIPTION

This section was written to provide a brief yet thorough description of our project idea. This will serve as a comprehensive guide for whom this project proposal may be interested in gaining a complete understanding of the vision our team has set out to accomplish.



Mobile, affordable, and eco-focused: ECOBee is a lightweight autonomous robot with a 2DOF arm and scoop that collects bottles, cans, cups, and paper debris. Designed for restaurants, cafés, wedding halls and event venues to keep spaces clean, safe, and on-brand — continuously and reliably.

Autonomous indoor navigation, vision-based waste detection, an internal storage bin, and continuous operation. ECOBee reduces manual labor, improves guest experience, and supports sustainability initiatives at a fraction of competitor costs.

- Specialized for hospitality waste (bottles, cans, cups, tissues)
- Compact (~5 kg), safe, and always-ready
- Local price point to accelerate adoption

PRODUCT-DESIGN-SPECIFICATION

This project involves the design and development of an autonomous mobile robot capable of identifying, navigating toward, and collecting trash in a structured outdoor or semi-structured indoor environment. The system integrates mechanical design, embedded control systems, and autonomous navigation using ROS 2 and simulation in Gazebo.

The robot must detect trash objects, plan a collision-free path, traverse toward the target, collect the object using a mechanical collection mechanism, and continue operation safely and efficiently. The design emphasizes stability, reliability, modularity, and safe human interaction.

Table 1 Functional Requirements

Construction

Key Considerations	Requirements	Issues/Challenges
Materials and process	**Chassis & Structural Panels** • **Laser-cut acrylic (PMMA)** • Thickness: 3-6 • Load-bearing panels: 6–8 mm • Enclosures / covers: 4–6 mm • Density: ~1.18 g/cm³ • Tensile strength: ~60–70 MPa **Arm Structure** • Material: 3D printed PLA **Wheels** • Rubber tires • Shafts: AISI 1045 Steel • Bearings: pillow block bearing • Load rating ≥ 1.5× total system weight.	• Acrylic is brittle → cracks under shock or over-tightening. • Stress concentration at bolt holes. • Laser-cut edges may initiate cracks. • Low impact resistance (garbage collision risk). • Long-term creep under constant load.

	Electronics Housing • ABS or polycarbonate enclosure. • Flame-retardant grade	
Dimensions	Height (mm): 522.5 Width (mm): 361-522.5 Length (m): 410.7 mass (kg) : 10.669 kg	Make it smaller and portable
Quantity	Initial production of 1 unit, with 10 more in the next 12 months	To achieve a mass product in 12 months
Lifespan	DC motors: 1,000 to 3,000 hours (2 – 4 months non-stop) Servo motors: 20,000 and 30,000 hours (2 – 3 years non-stop) So, the product is expected to function at 100% for 6 months - 1 year before need for maintenance • Design life: Minimum 5 years operational. • Duty cycle: 4–8 hours/day. • Mechanical joints rated for ≥ 500,000 cycles. • Battery life: ≥ 500 charge cycles before 80% capacity. • Electronics MTBF target: ≥ 10,000 hours.	• Arm gear backlash over time. • Bearing contamination from dust/liquid waste. • Battery degradation in hot environments (Egypt heat is a real factor). • UV degradation if used outdoors.
Maintenance	• Modular arm design for quick removal. • Access panel for battery replacement • Standard fasteners only (no proprietary screws). • Maintenance interval: ○ Weekly: Visual inspection	• Dust ingress into actuators. • Cable fatigue at arm joints. • Water exposure in bin area.

	○ Monthly: Lubrication ○ Every 6 months: Bearing check	• Maintenance complexity if wiring isn't routed cleanly.
Experience		
Key Considerations	**Requirements**	**Issues/Challenges**
Aesthetics	• Industrial but friendly appearance. • Consistent color scheme (current black/yellow is good: safety-coded). • Visible status LEDs. • Smooth external surfaces to avoid waste buildup. • Hidden wiring.	• Industrial looks may appear intimidating. • Scratches and dirt reduce visual quality. • Exposed fasteners affect perceived quality.
Ergonomics	• Bin height optimized: 114 mm from ground. • Emergency stop button easily reachable. • Weight ≤ 0.001– 1kg for manageable manual handling.	• Center of gravity too high → tipping risk. • Handle placement interfering with bin opening.
Target audience	• Wedding Halls &Event Organizers • Cafés • Restaurants	• Cluttered environment • Low lighting conditions • Liquid spills • Navigation reliability
Safety		
Key Considerations	Requirements	Issues/Challenges

Safety considerations	**Mechanical Safety** • Pinch-point protection at joints. • Rounded edges (R ≥ 2 mm). **Electrical Safety** • Insulated wiring. • Overcurrent protection. • Battery BMS mandatory. **Operational Safety** • Obstacle detection. • On- Off switch • Emergency stop button. • Audible warning during arm motion.	• Arm collision with humans. • Battery overheating. • Sharp waste objects damaging structure. • Stability when arm fully extended.
Safety standards	Depending on region, applicable standards may include: • ISO 12100 – Safety of machinery risk assessment • ISO 13849 – Safety-related control systems • IEC 60204-1 – Electrical equipment of machines • ISO 10218 (if robotic classification applies) • CE marking requirements (if commercialized in EU) • RoHS compliance for electronics	• Full compliance can be expensive. • Safety certification increases development time. • Requires documented risk assessment and FMEA.
Testing	Safety testing is recommended to validate these features, ensuring the product meets quality control standards and is safe under various usage scenarios.	

Transport		
Key Considerations	**Requirements**	**Issues/Challenges**
Packaging	• Double wall corrugated box or wooden crate. • Foam protection for arm and electronics. • Anti-static protection for PCB. • Shock absorption rating for 0.5 m drop. • Moisture barrier (desiccant included).	• Arm protrusion complicates packaging shape. • Increased packaging cost due to weight. • Shipping vibration damage.
Shipping	Ensure packaging complies with regional shipping standards, considering climate factors (moisture-resistant packaging for humid areas).	• Lithium battery shipping restrictions. • Customs clearance if exported. • Shock damage to arm joints.
Environmental		

<table>
<tr><th>Key Considerations</th><th>Requirements</th><th>Issues/Challenges</th></tr>
<tr><td>Recycling</td><td>• Design for disassembly.
• Separate metal, plastic, electronics.
• Avoid bonded mixed materials.
• Mark plastic types.
• Battery removable for recycling.</td><td>• Mixed-material components (e.g., rubber bonded to metal).
• E-waste disposal regulations.
• Cost of sustainable material choices.</td></tr>
<tr><td colspan="3">Power</td></tr>
<tr><th>Key Considerations</th><th>Requirements</th><th>Issues/Challenges</th></tr>
<tr><td>Power Consumption</td><td>Table 2 components power consumption
<table>
<tr><th>Component</th><th>Qty</th><th>Avg Power (W)</th><th>Peak Power (W)</th><th>Notes</th></tr>
<tr><td>Raspberry Pi 5</td><td>1</td><td>12</td><td>15</td><td>High processing load</td></tr>
<tr><td>Camera</td><td>1</td><td>2</td><td>3</td><td>Active streaming</td></tr>
<tr><td>ESP32</td><td>1</td><td>1</td><td>2</td><td>Wi-Fi active</td></tr>
<tr><td>DC Motors</td><td>2</td><td>60-70</td><td>120+</td><td>During acceleration</td></tr>
<tr><td>Servo Motors</td><td>3</td><td>15–20</td><td>30</td><td>Arm lifting peak</td></tr>
<tr><td>Sensors + IMU + Ultrasonic</td><td>—</td><td>3</td><td>5</td><td>Low draw</td></tr>
</table></td><td>• High motor current spikes.
• Servo brown-out risk on shared supply.
• Buck converter overheating.
• Li-Po safety management required.
• Limited runtime with small battery capacity.</td></tr>
</table>

H-bridge + losses	1	3	5	Heat losses	

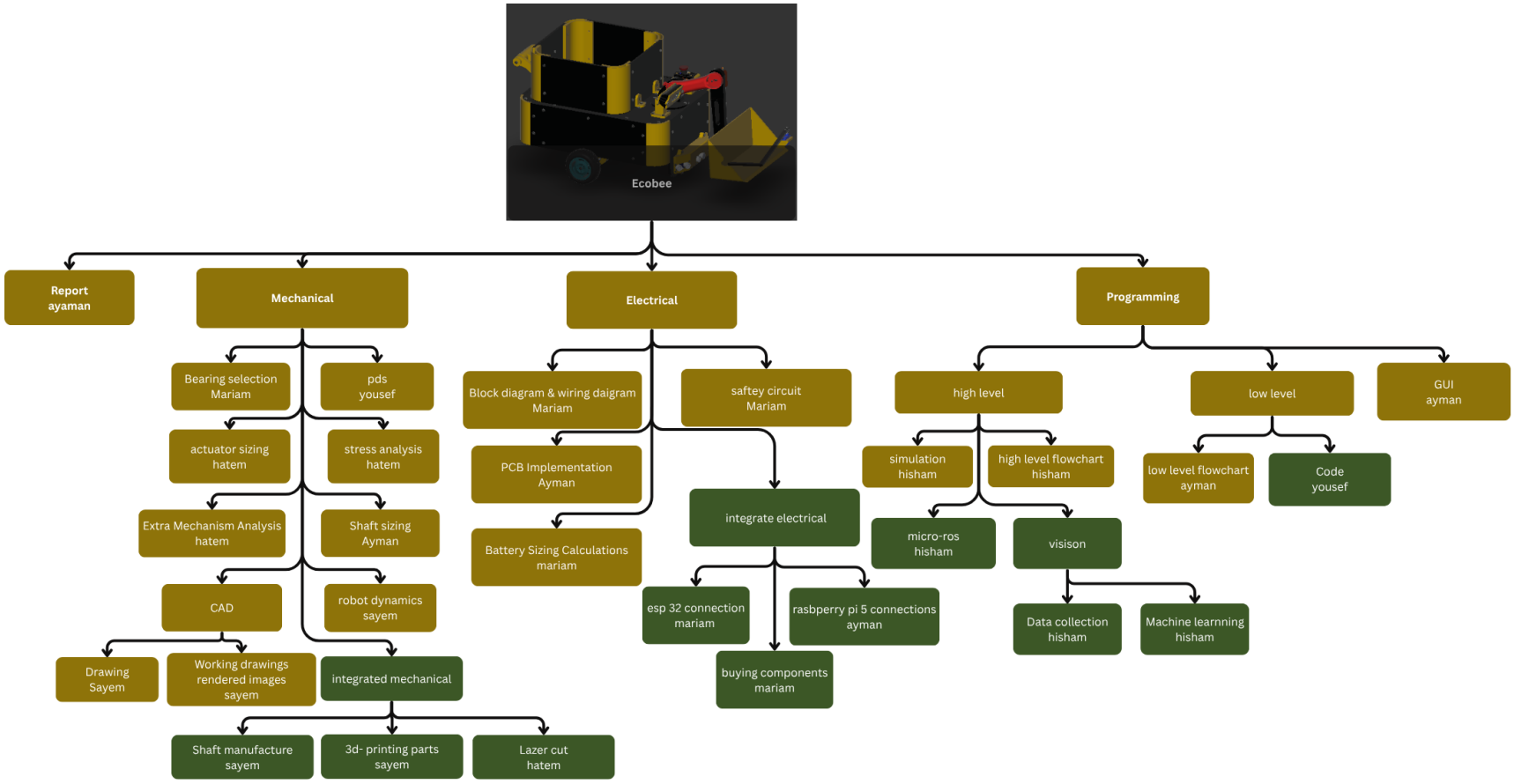
Total Average Power: 72–110W

Table 3 total average power

Parameter	Requirement
Continuous output	≥120W
Peak capability	≥200W
Discharge rating	≥20A recommended
Estimated runtime (138Wh & 125Wh battery)	~1.5 hour

PROJECT MANAGEMENT

1. Work Breakdown Structure



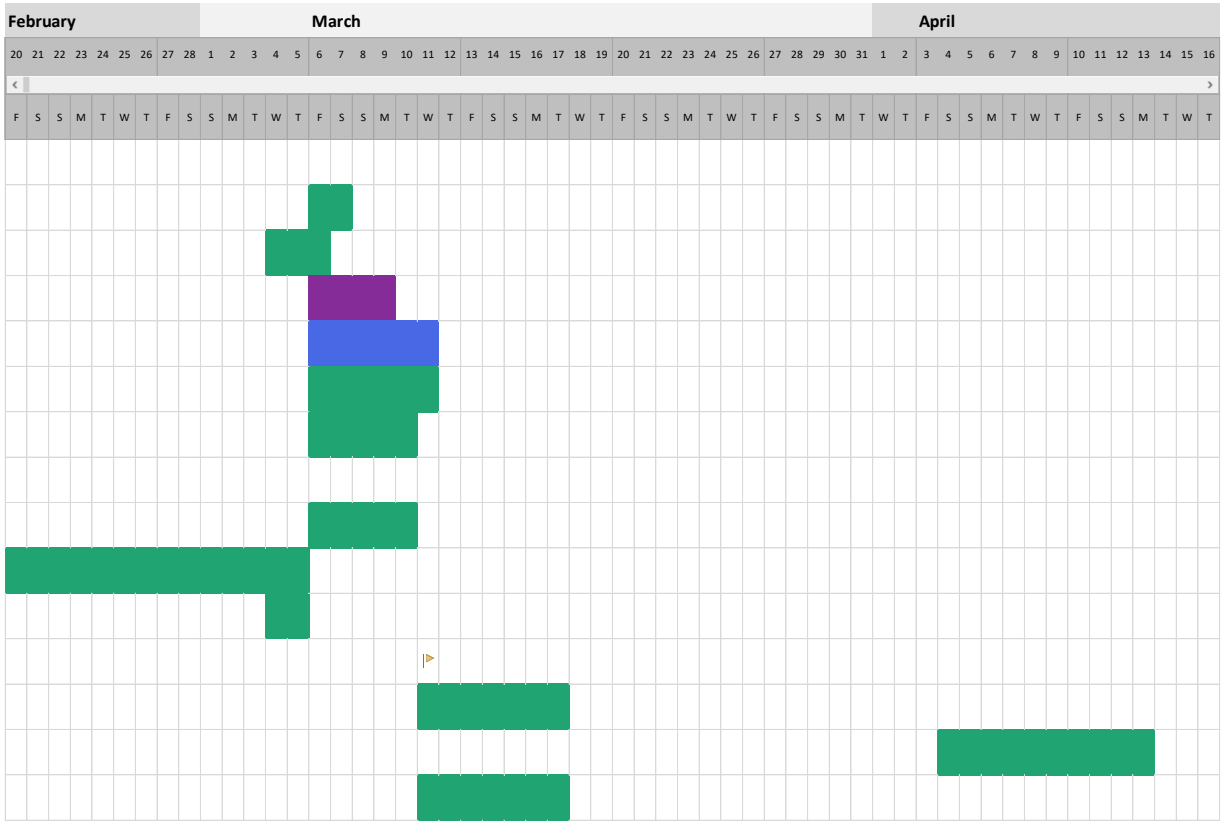
2. Gantt Chart

Legend: On track | Low risk | Med risk | High risk | Unassigned

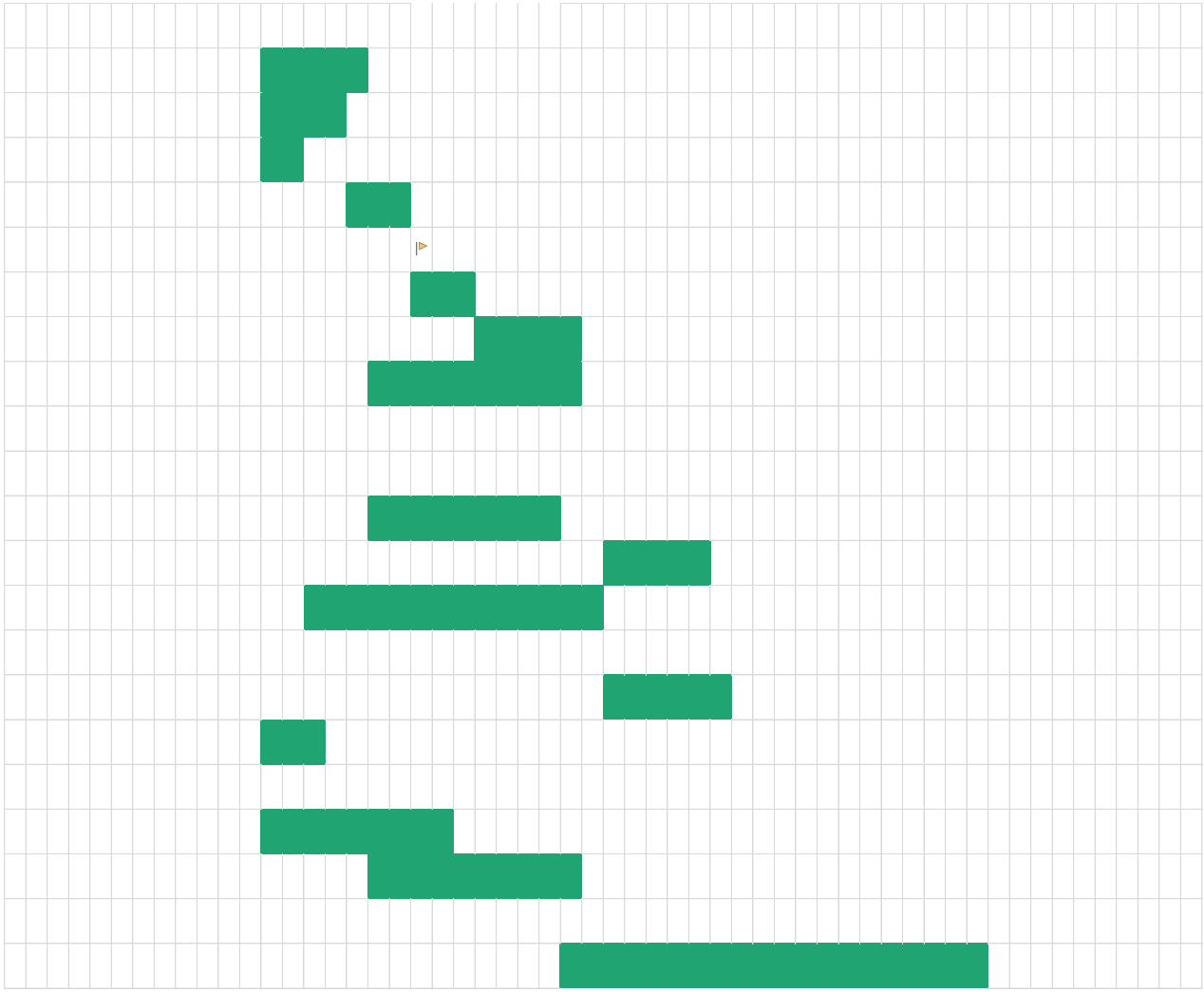
Project start date: 2/20/2026

Scrolling increment: 0

Milestone description	Category	Assigned to	Progress	Start	Days
Phase 1: Mechanical					
1.1 Bearing selection	On Track	Mariam	100%	3/6/2026	2
1.2 Pds	On Track	Yousef	100%	3/4/2026	3
1.3 Actuator sizing	High Risk	Hatem	100%	3/6/2026	4
1.4 stress analysis	Med Risk	Hatem	100%	3/6/2026	6
1.5 Extra Mechanism Analysis	On Track	Hatem	100%	3/6/2026	6
1.6 Shaft sizing	On Track	Ayman	100%	3/6/2026	5
1.7 CAD					
1.7.1 Working drawings rendered images	On Track	Sayem	100%	3/6/2026	5
1.7.2 Drawing	On Track	Sayem	100%	2/20/2026	14
1.8 robot dynamics	On Track	Sayem	100%	3/4/2026	2
1.9 Integrared mechanical	Milestone			3/11/2026	1
1.9.1 Shaft manufacture	On Track	Sayem	2%	3/11/2026	7
1.9.2 3D-printing parts	On Track	Sayem	1%	4/4/2026	10
1.9.3 Laser cut	On Track	Hatem	1%	3/11/2026	7



Phase 2: Electrical					
2.1 Block diagram and wiring daigram	On Track	Mariam	100%	3/4/2026	5
2.2 saftey circuit design	On Track	Mariam	100%	3/4/2026	4
2.3 Battery Sizing Calculations	On Track	Mariam	100%	3/4/2026	2
2.4 PCB Implementation	On Track	Ayman	100%	3/8/2026	3
2.5 integrate electrical	Milestone			3/11/2026	1
2.5.1 esp 22 connection	On Track	Mariam	0%	3/11/2026	3
2.5.2 rasbperry pi 5 connections	On Track	Ayman	0%	3/14/2026	5
2.5.3 buying components	On Track	Mariam	5%	3/9/2026	10
Phase 3: Programming					
3.1 High level					
3.1.1 High level flowchart	On Track	Hisham	100%	3/9/2026	9
3.1.2 Micro-ROS	On Track	Hisham	0%	3/20/2026	5
3.1.3 simulation	On Track	Hisham	100%	3/6/2026	14
3.1.4 vision					
3.1.4.1 Data collection	On Track	Hisham	0%	3/20/2026	6
3.1.4.2 Machine learnning	On Track	Hisham	0%	3/4/2026	3
3.2 low level					
3.2.1 Code	On Track	Ayman	2%	3/4/2026	9
3.2.2 low level flowchart	On Track	Ayman	100%	3/9/2026	10
3.3 GUI	On Track	Ayman	0%	4/18/2026	7
4 Final report writing	On Track	Ayman	30%	3/18/2026	20



CAD DESIGN

1. Whole Robot



Figure 1 Whole robot CAD design

a) Extra mechanism design

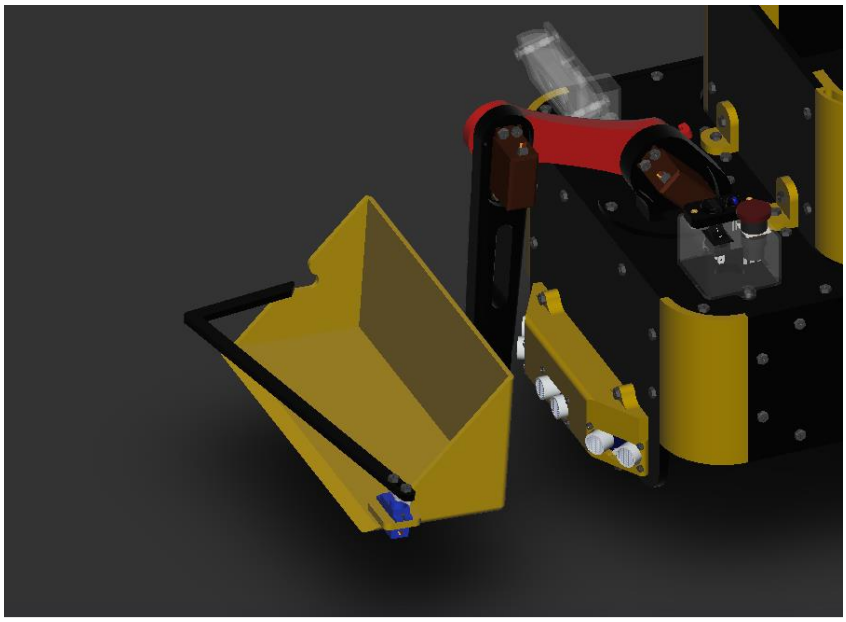


Figure 2 extra mechanism design

b) Wheel Assembly

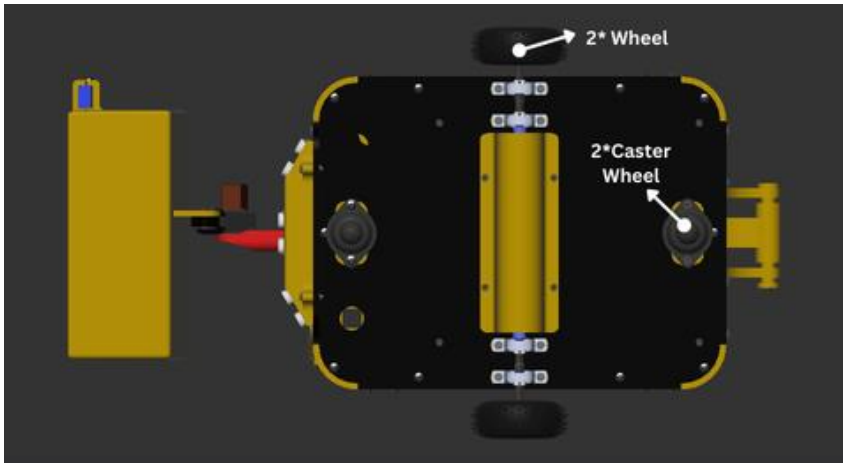


Figure 3 Wheel Assembly

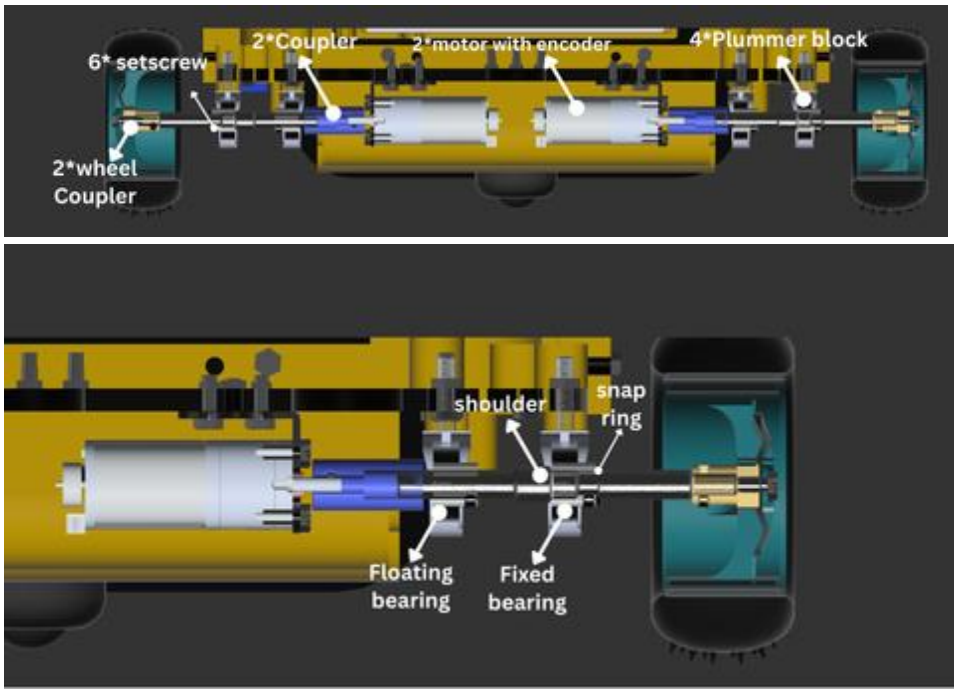


Figure 4 Shaft fixation

2. Sensors & Electronics

a) Electronics

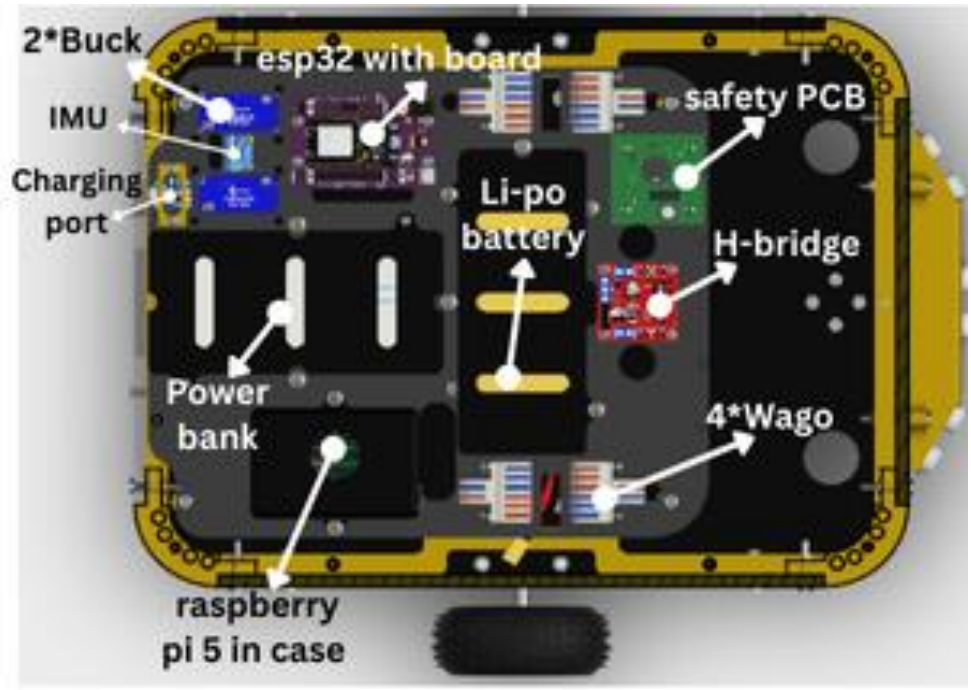


Figure 5 electronics fixation

- **Li-Po Battery**: The main high-current power source (12V) dedicated to driving the heavy DC motors and arm servos.
- **Power Bank**: A dedicated, isolated 5V power source specifically to keep the Raspberry Pi running stably without voltage drops.
- **H-Bridge Motor Driver**: Receives PWM signals from the ESP32 to control the speed and rotational direction of the drive wheels.
- **IMU (Inertial Measurement Unit)**: A mandatory sensor that tracks the robot's orientation and heading to ensure accurate autonomous navigation.
- **Safety PCB**: Fulfills the requirement for a hardware safety circuit (such as overcurrent or reverse polarity protection).

b) Sensors & indicators

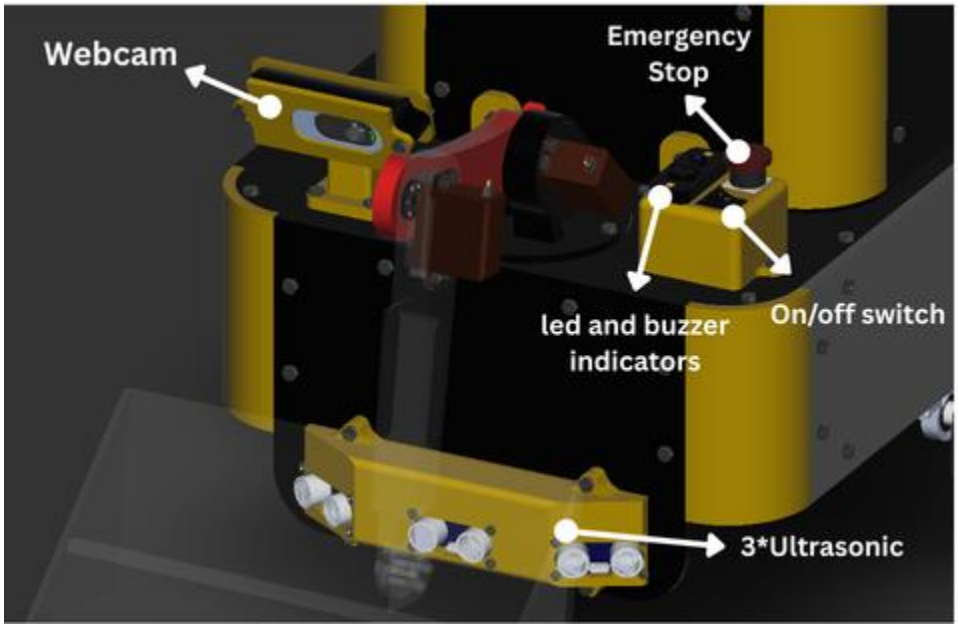


Figure 6 sensors fixation

- **Webcam:** Acts as the vision module to identify and track the specific trash objects necessary to complete the mission.
- **Emergency Stop (E-Stop):** A required safety feature that physically removes power from all actuators (motors and servos) instantly in case of a malfunction, without shutting down the main controllers.
- **LED and Buzzer Indicators:** Serve as the mandatory state indicators to communicate the robot's current status (e.g., Power-Good, Fault detected, or mission mode).
  3x Ultrasonic Sensors: Provide distance data to detect and avoid dynamic and static obstacles during autonomous navigation.

3. CAD rendered images

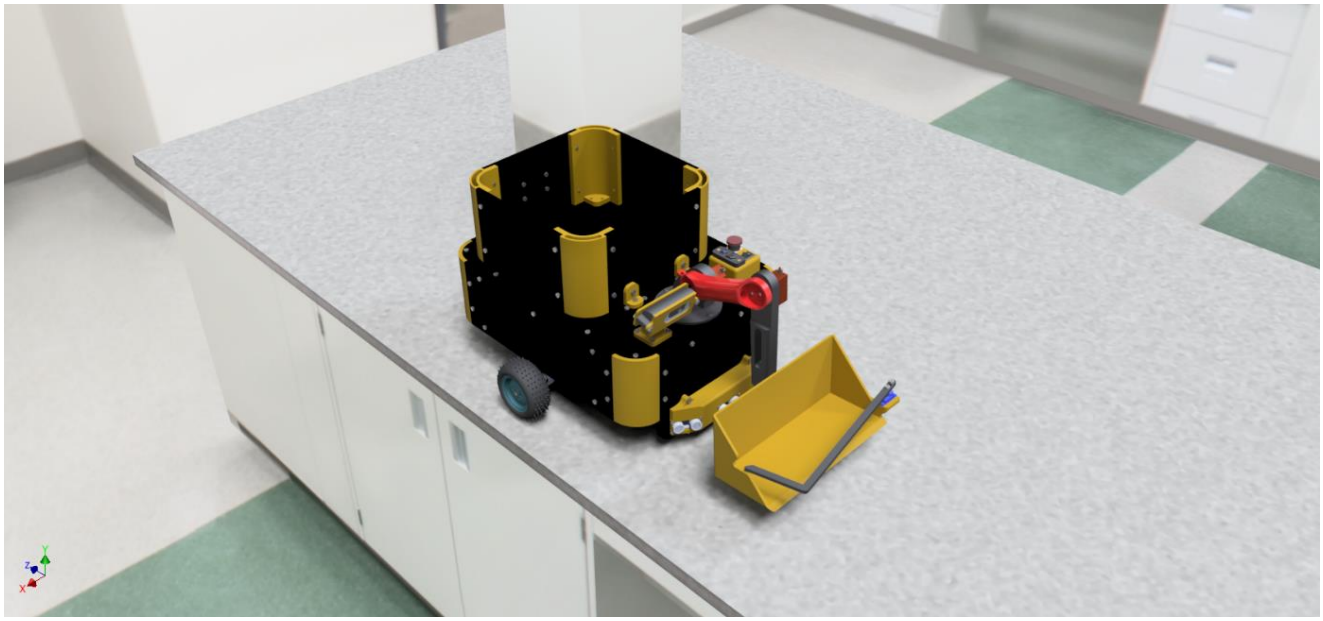
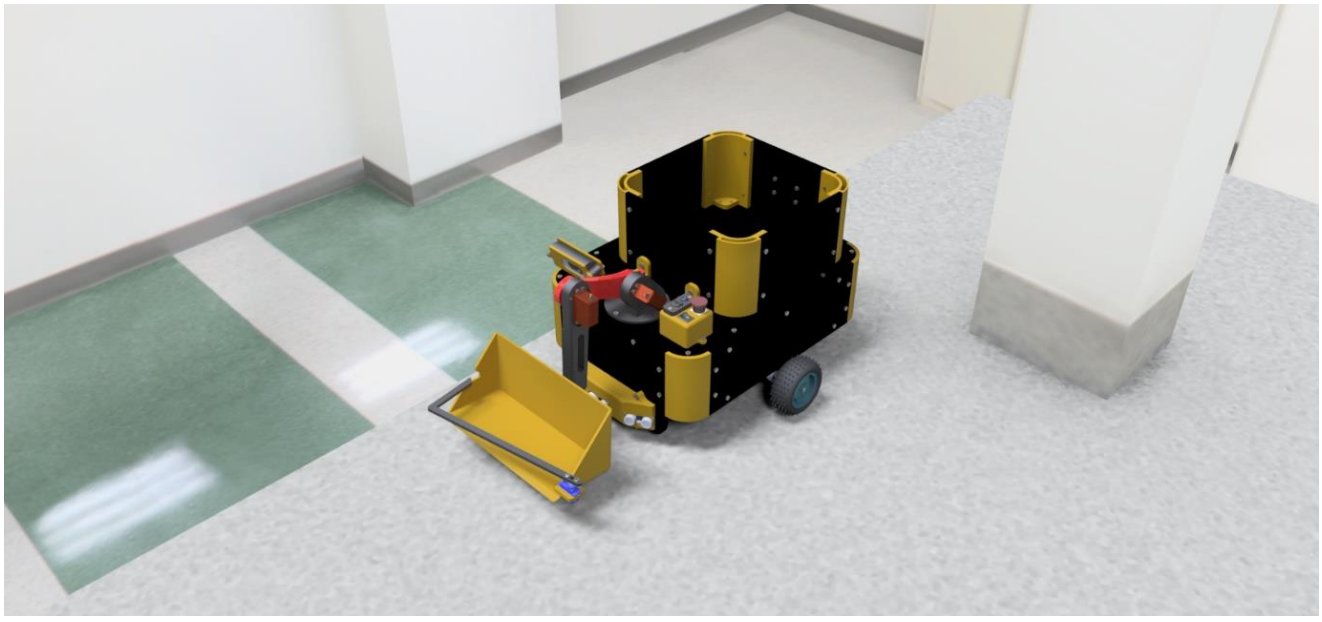
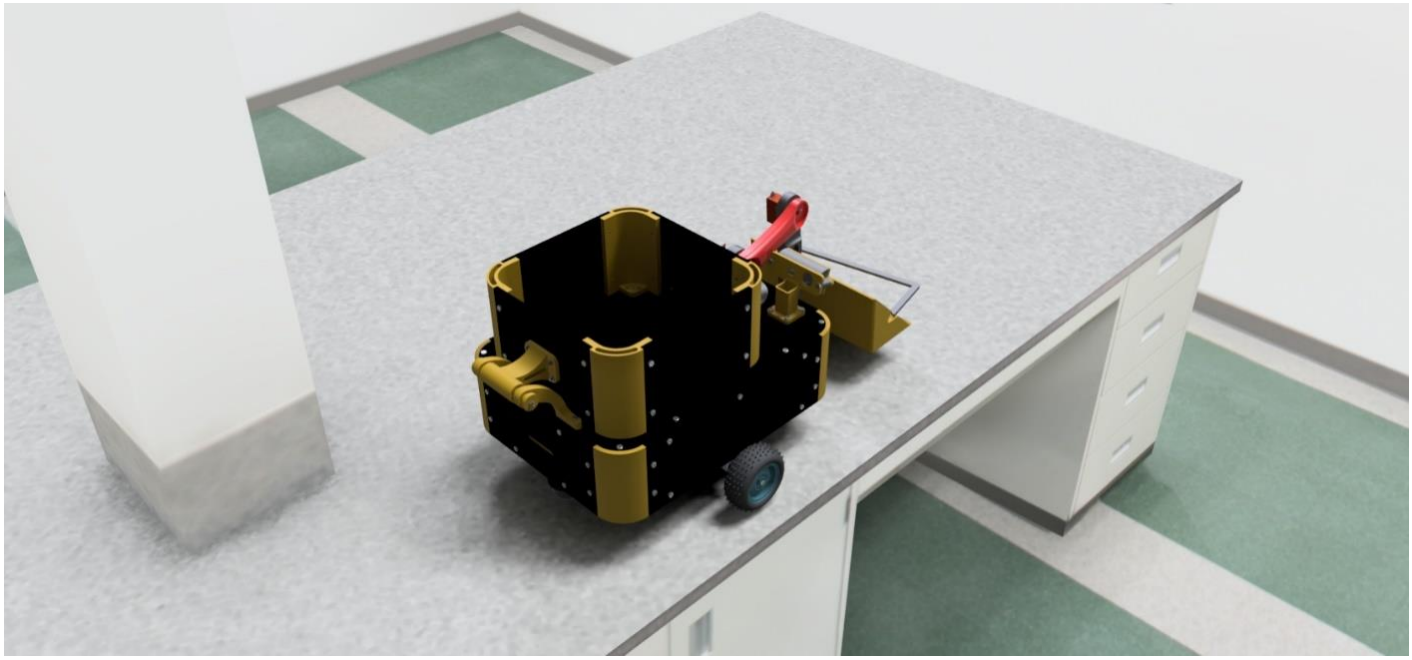
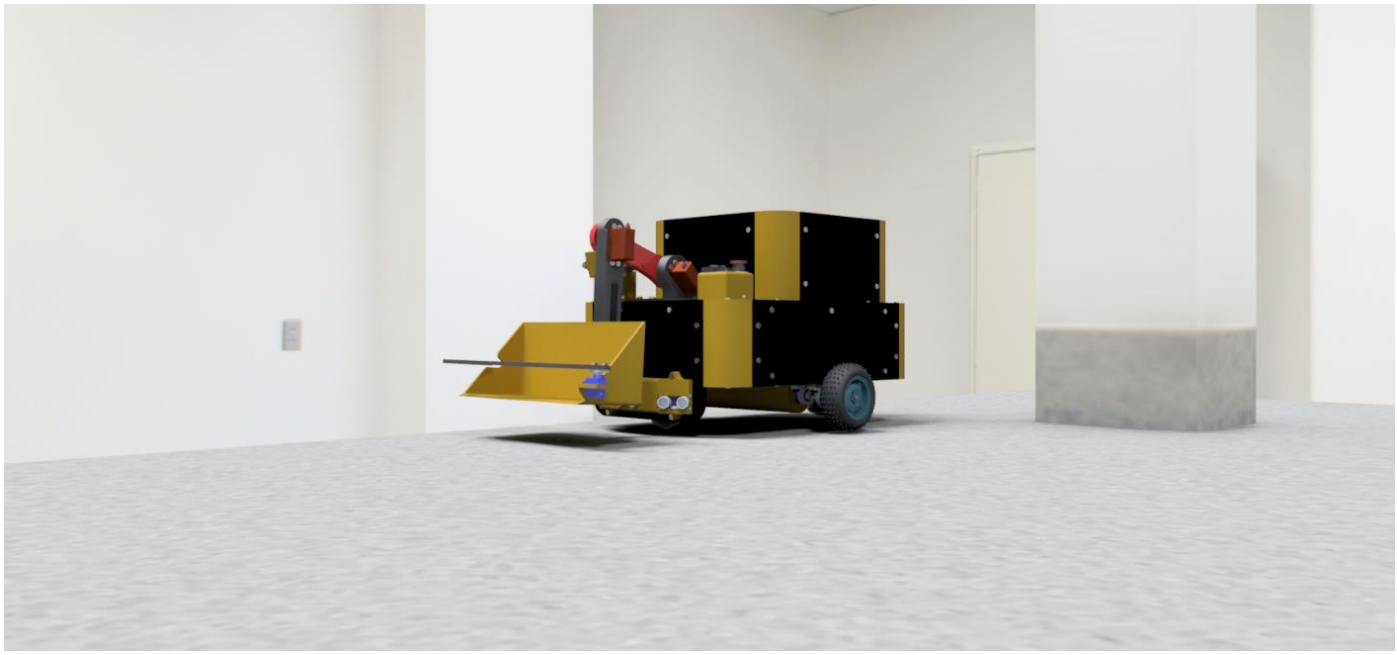
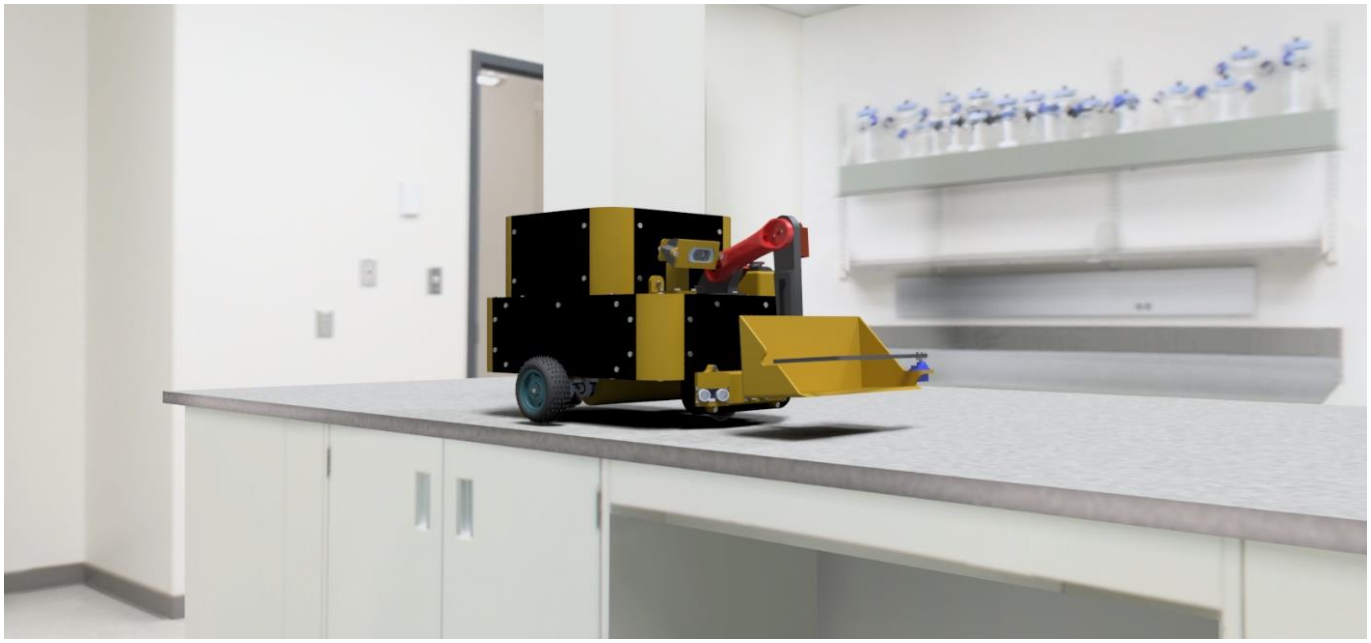


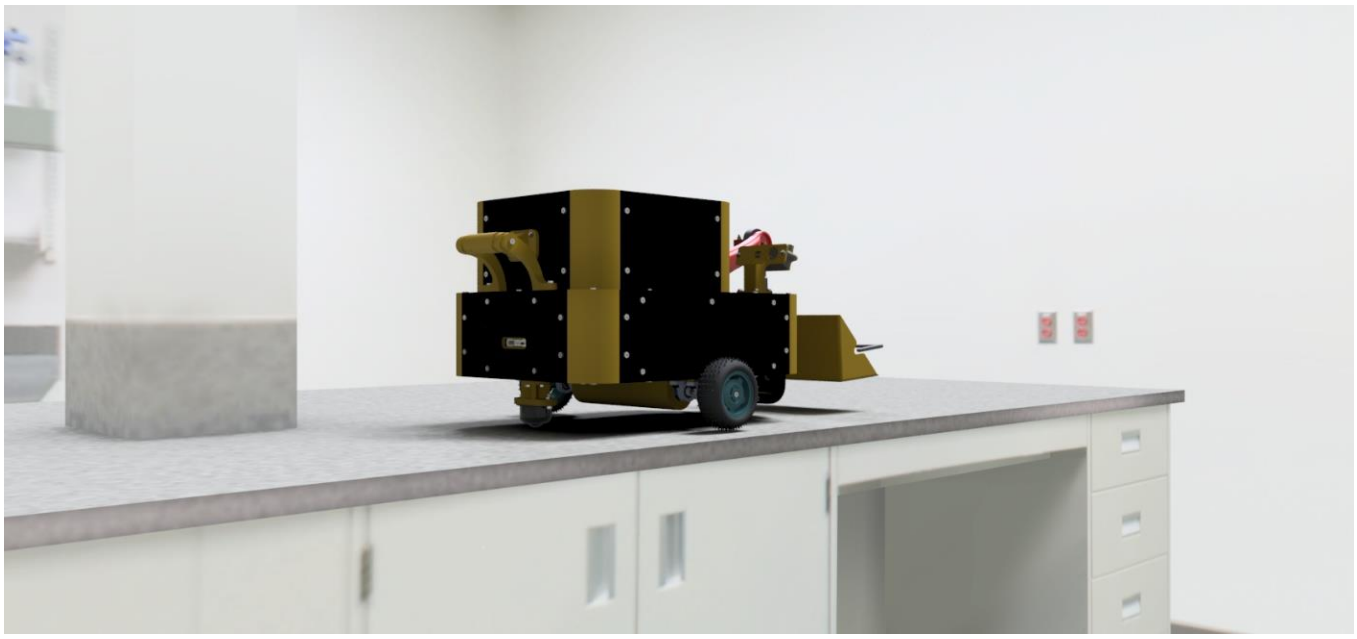
Figure 7 CAD rendered











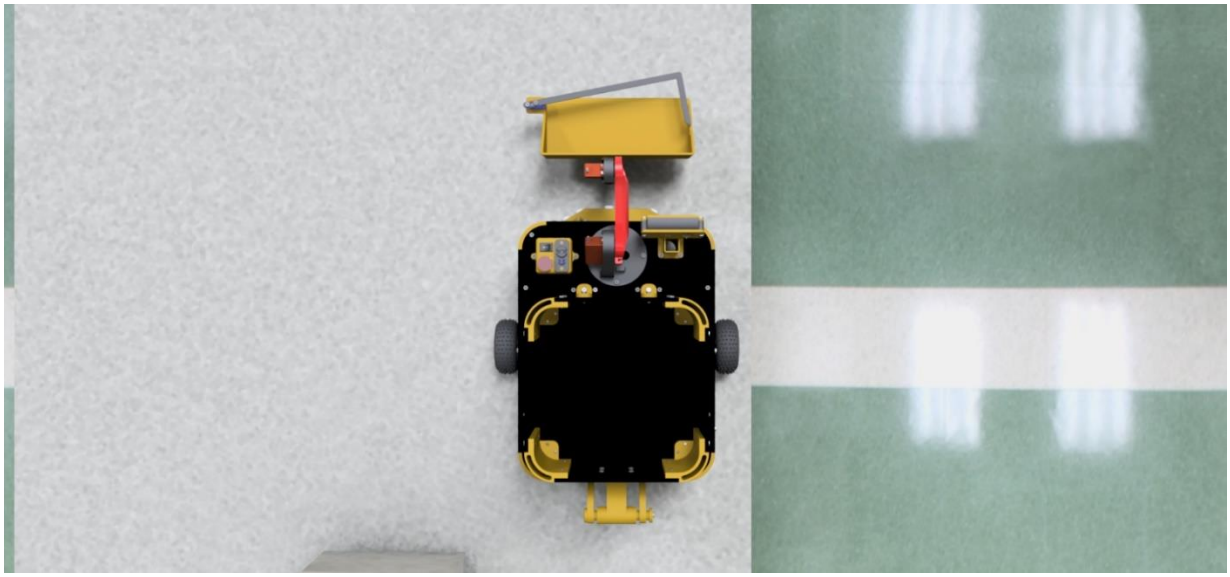


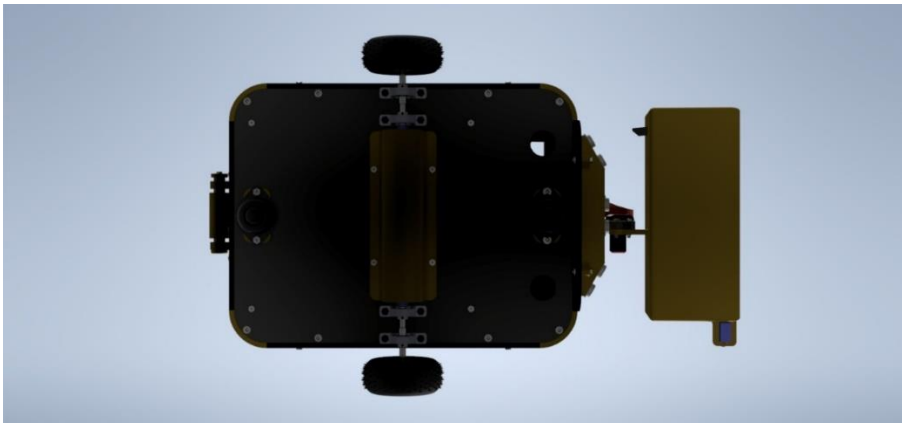












4. Working drawings

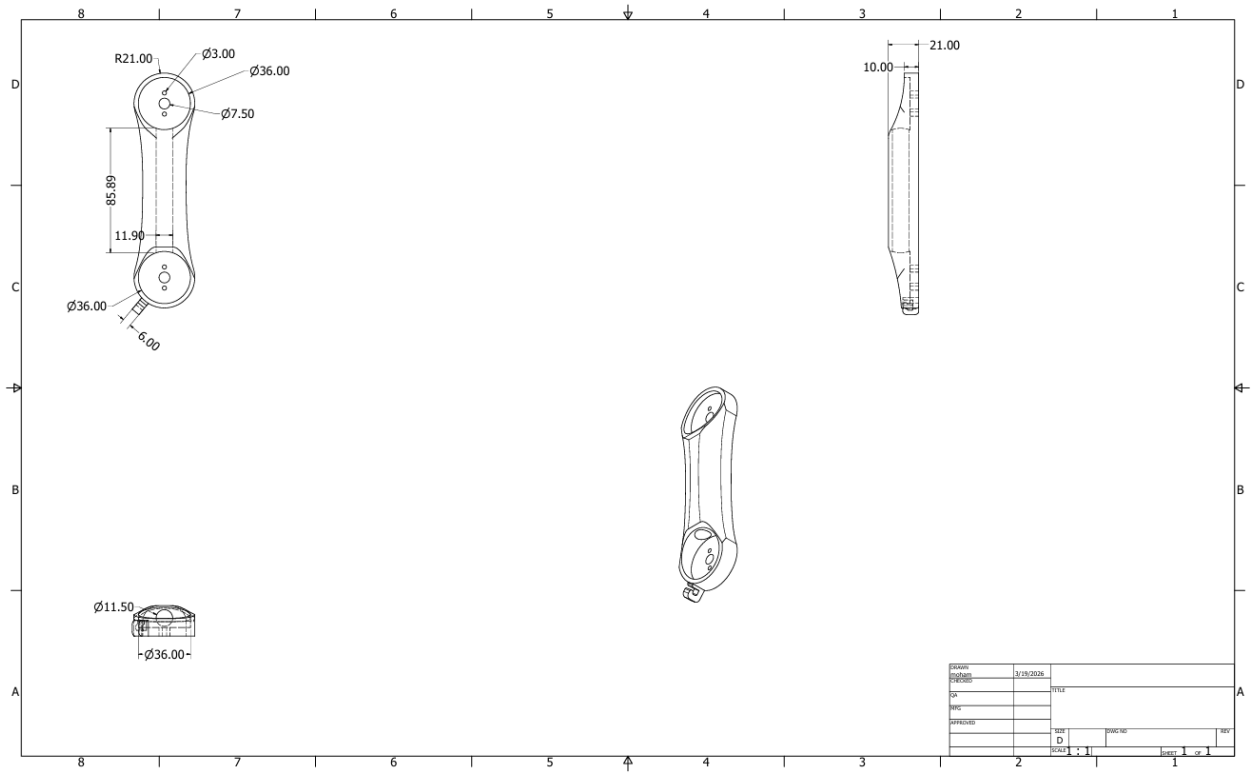
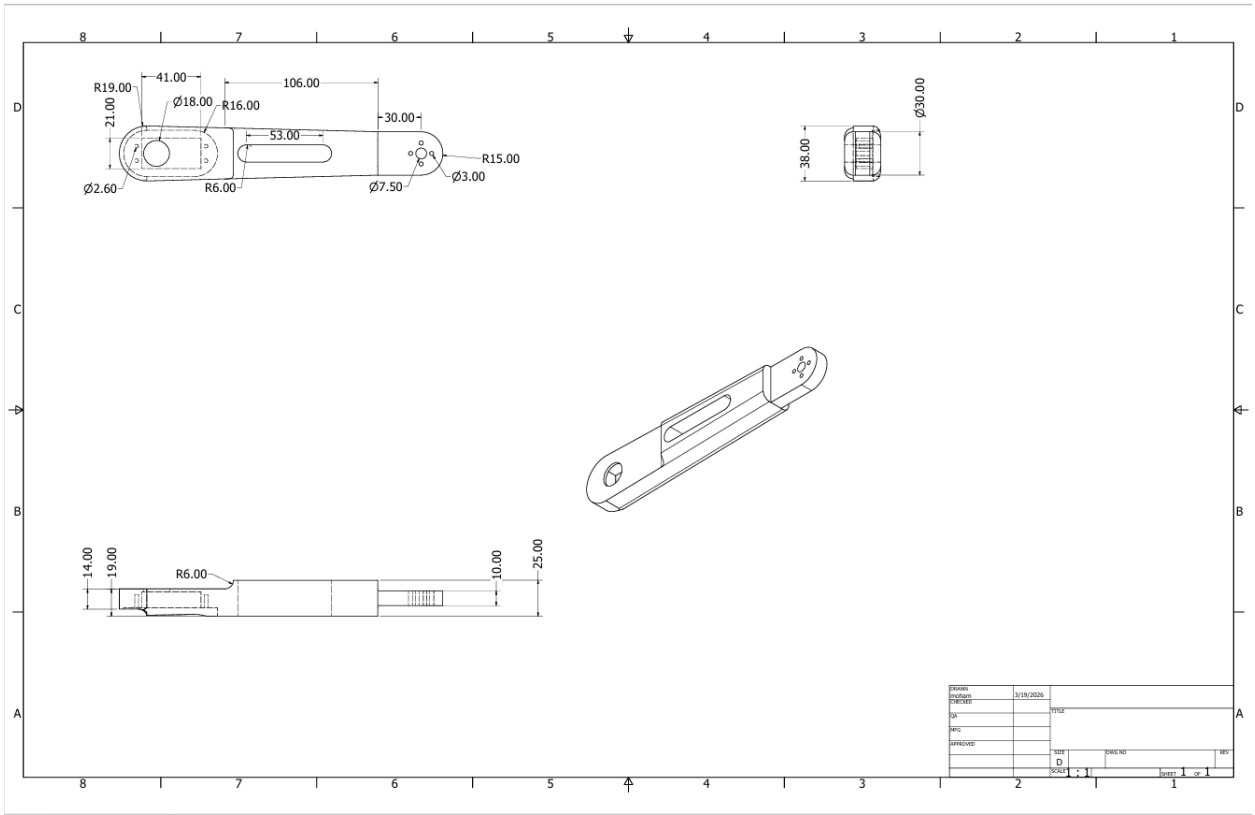
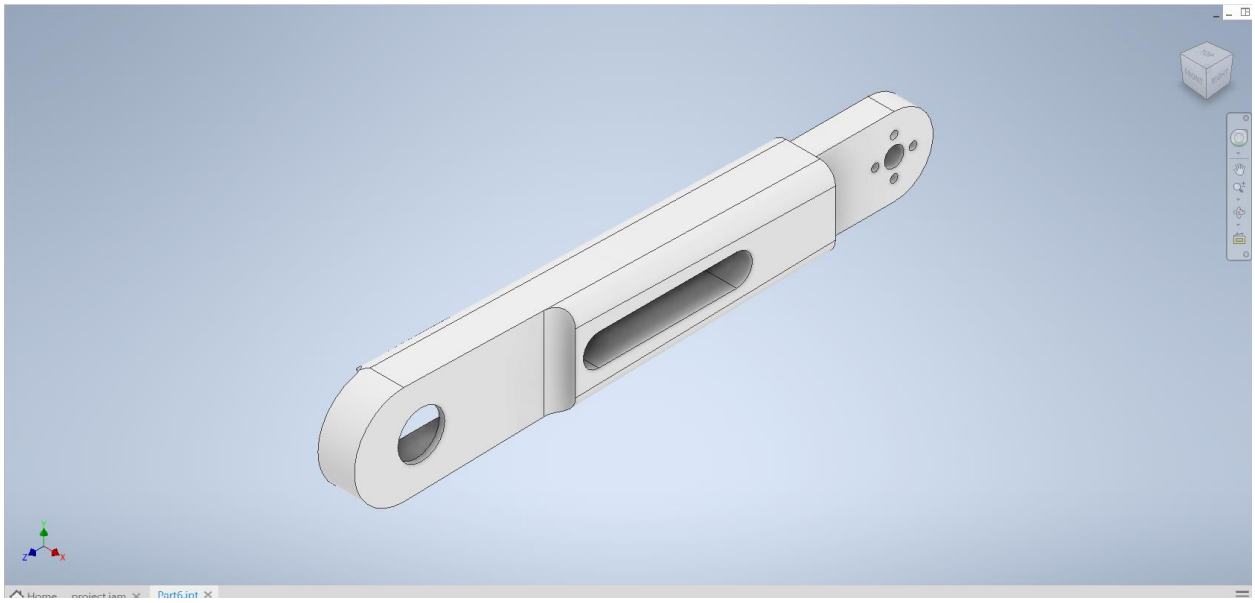
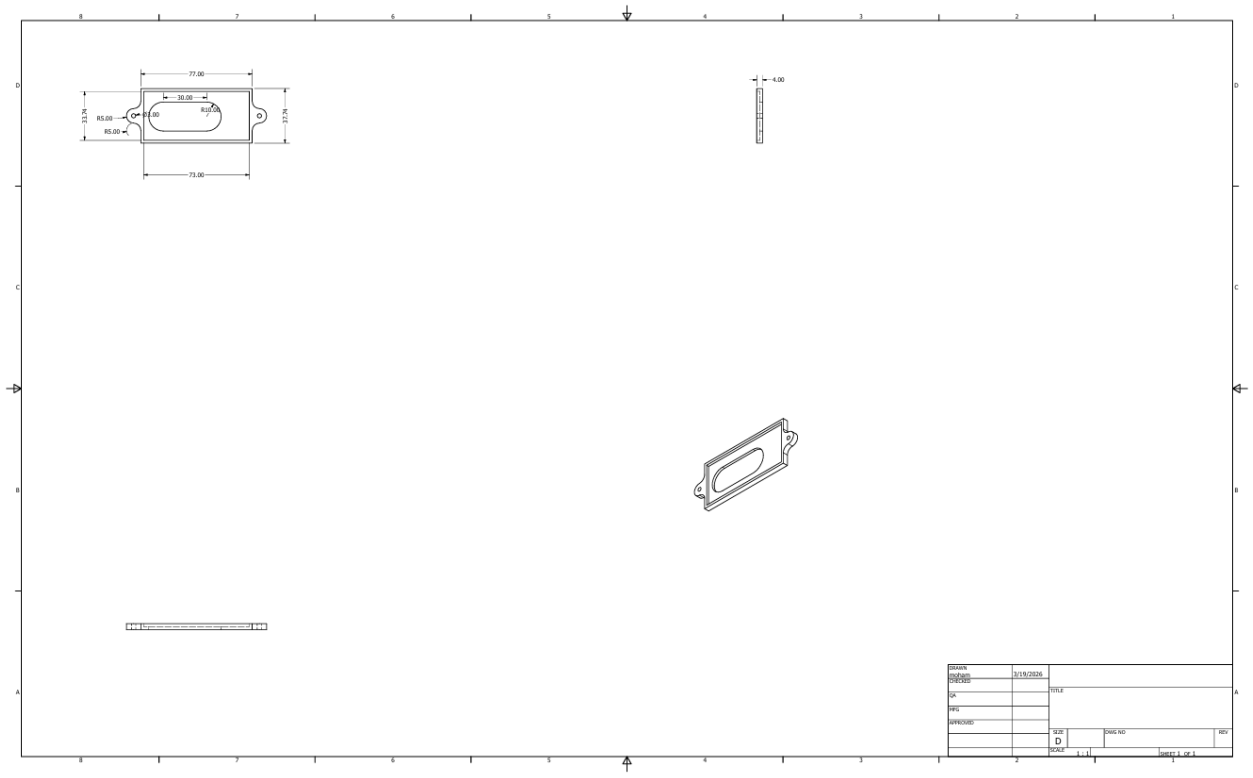


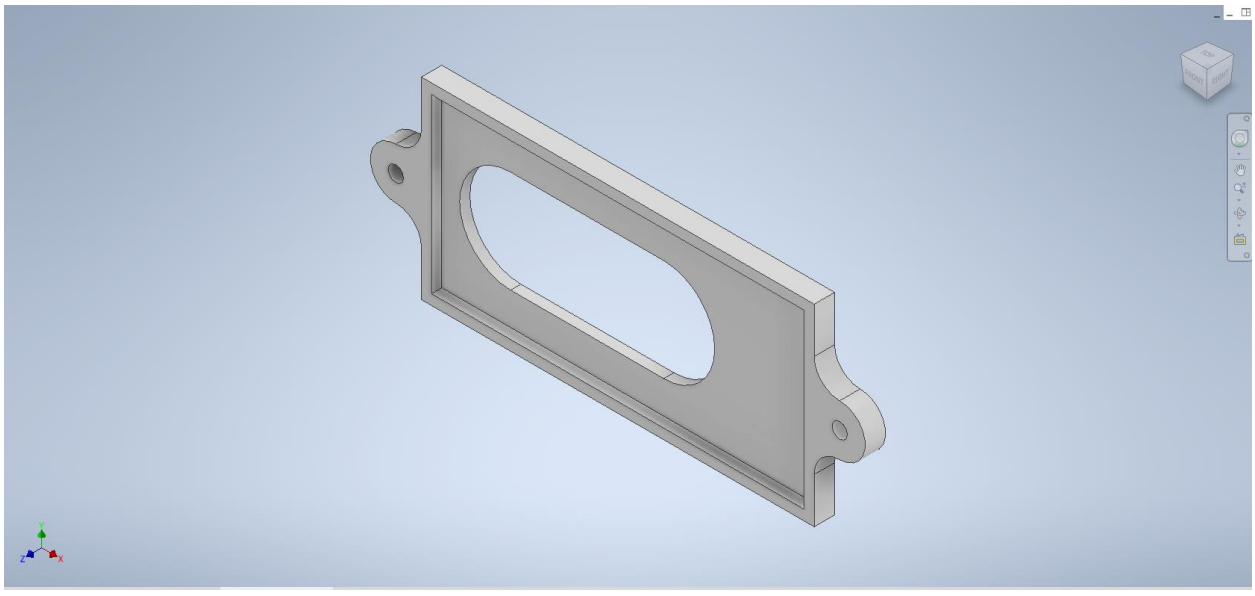
Figure 8 Working drawings

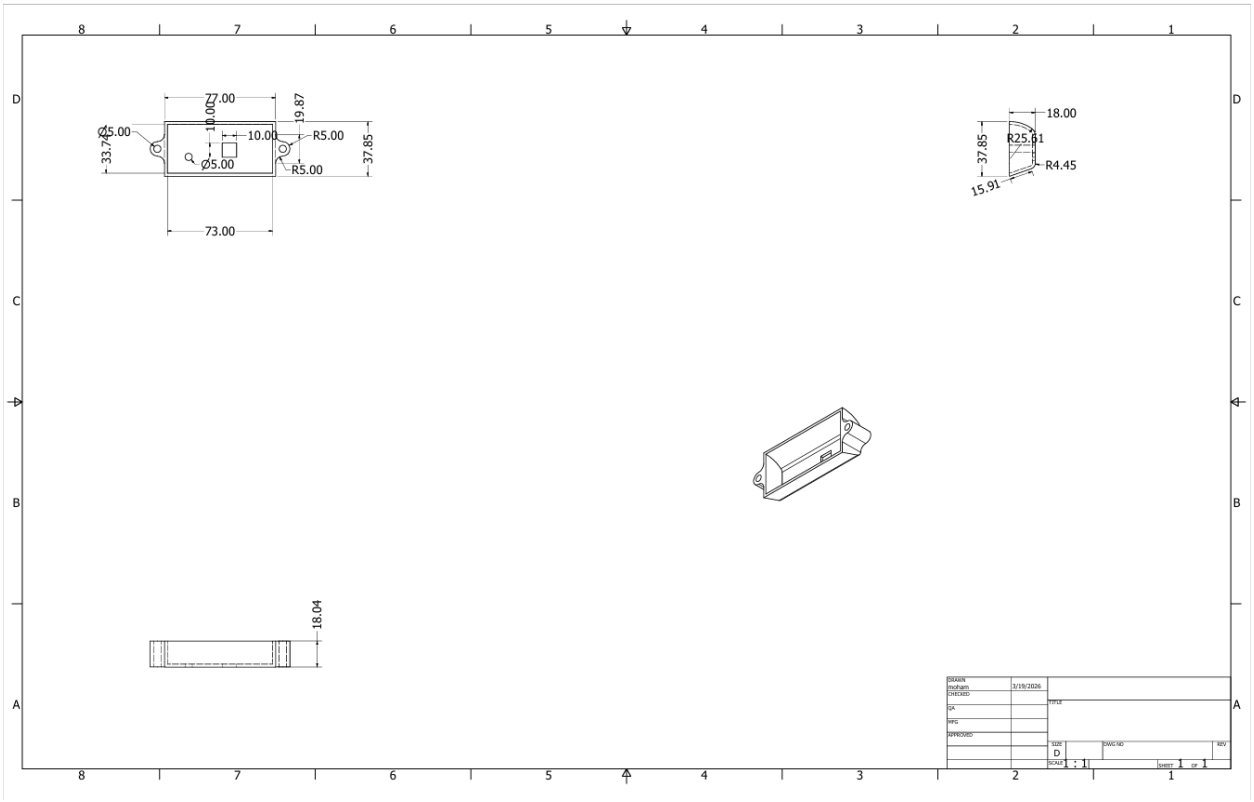


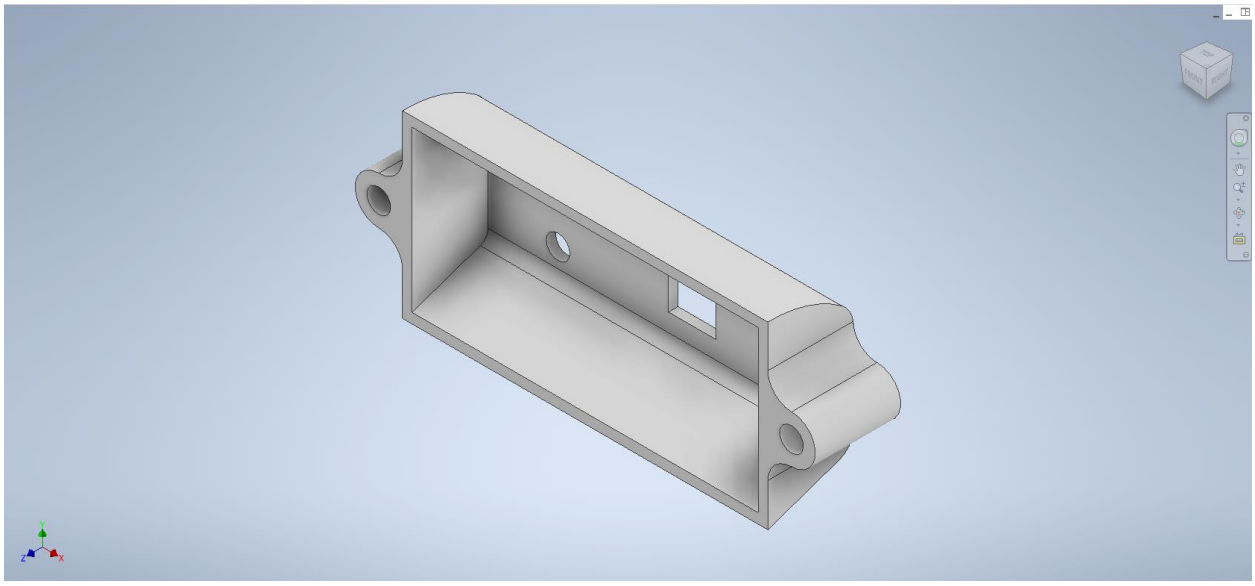

R19.00
41.00
106.00
Ø18.00
R16.00
21.00
30.00
53.00
R15.00
Ø3.00
Ø2.60
R6.00
Ø7.50
Ø30.00
38.00
14.00
19.00
R6.00
10.00
25.00

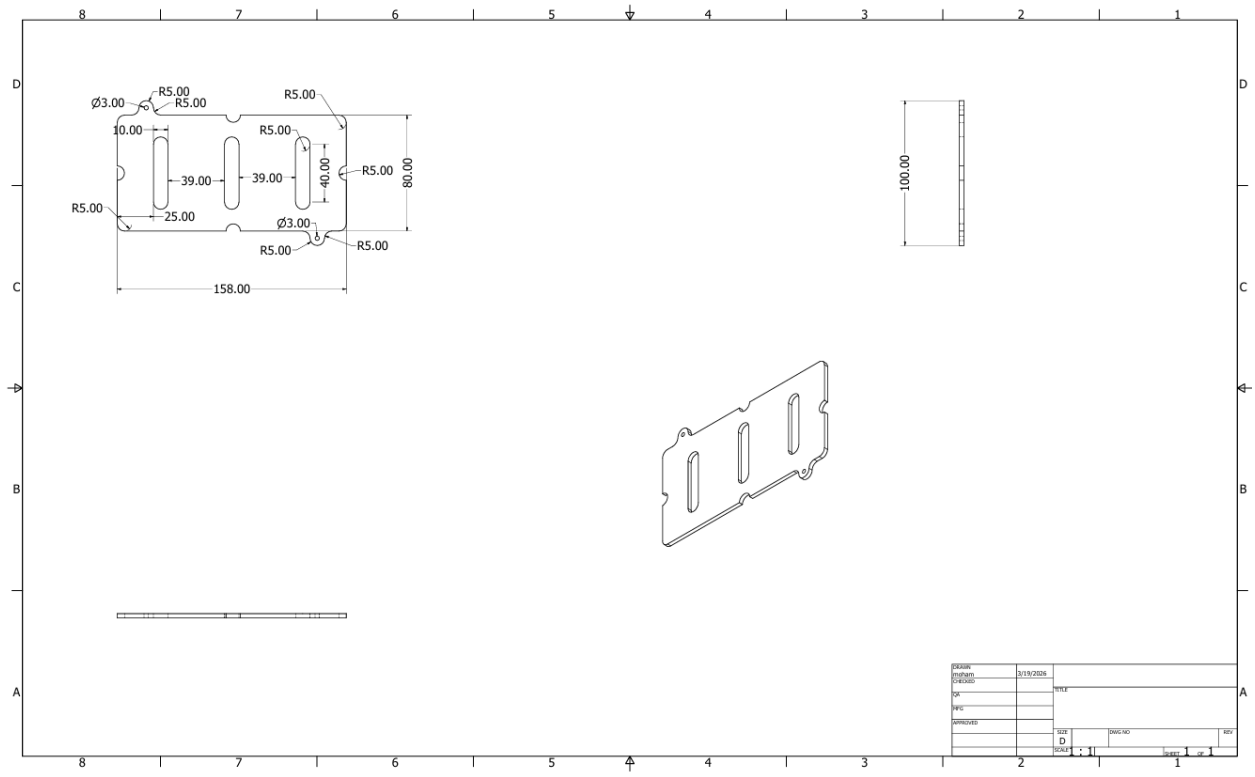


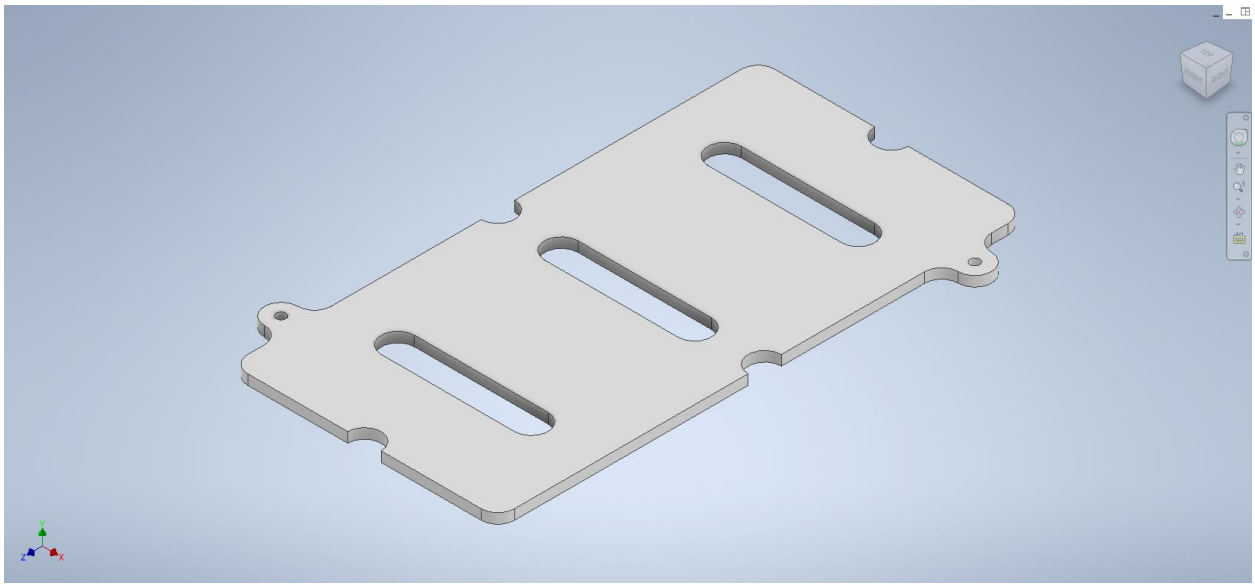

77.00
30.00
R10.00
R5.00
R5.00
37.74
37.74
73.00
4.00
3/19/2026
D

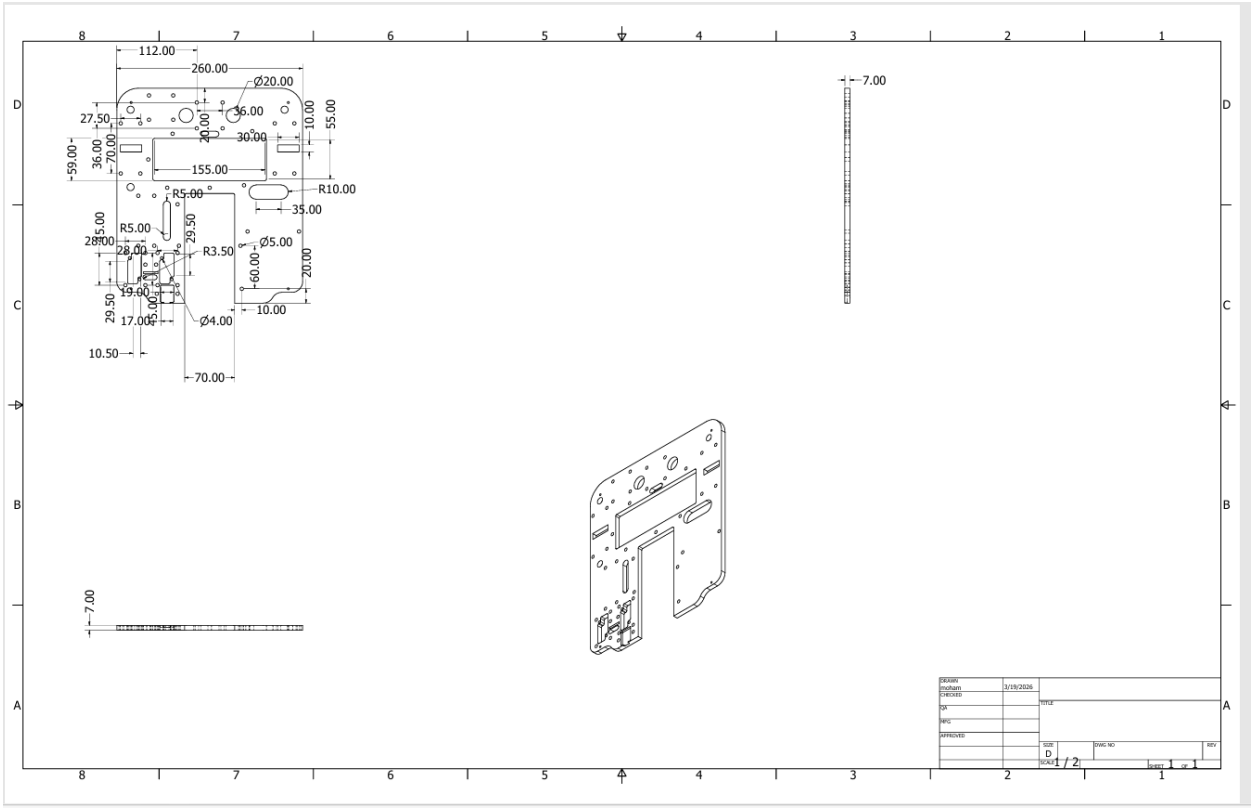


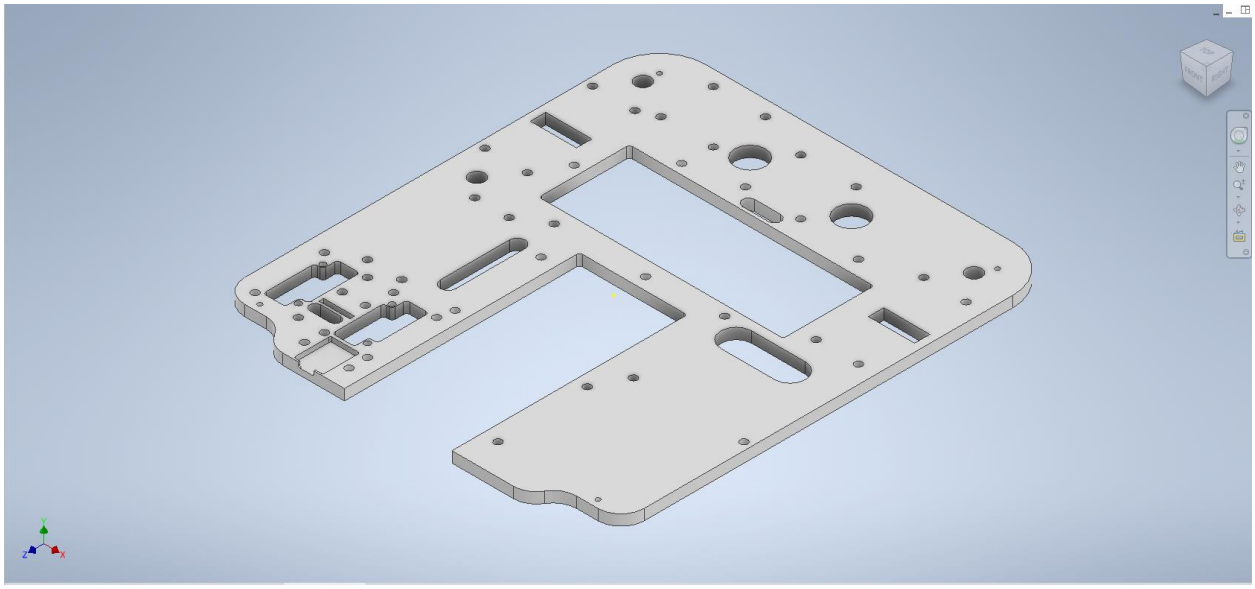

77.00
19.87
10.00
10.00
R5.00
R5.00
Ø5.00
Ø5.00
33.74
37.85
73.00
18.00
R25.61
R4.45
37.85
15.91
18.04
moham
3/19/2026
D
1 : 1
1 OF 1

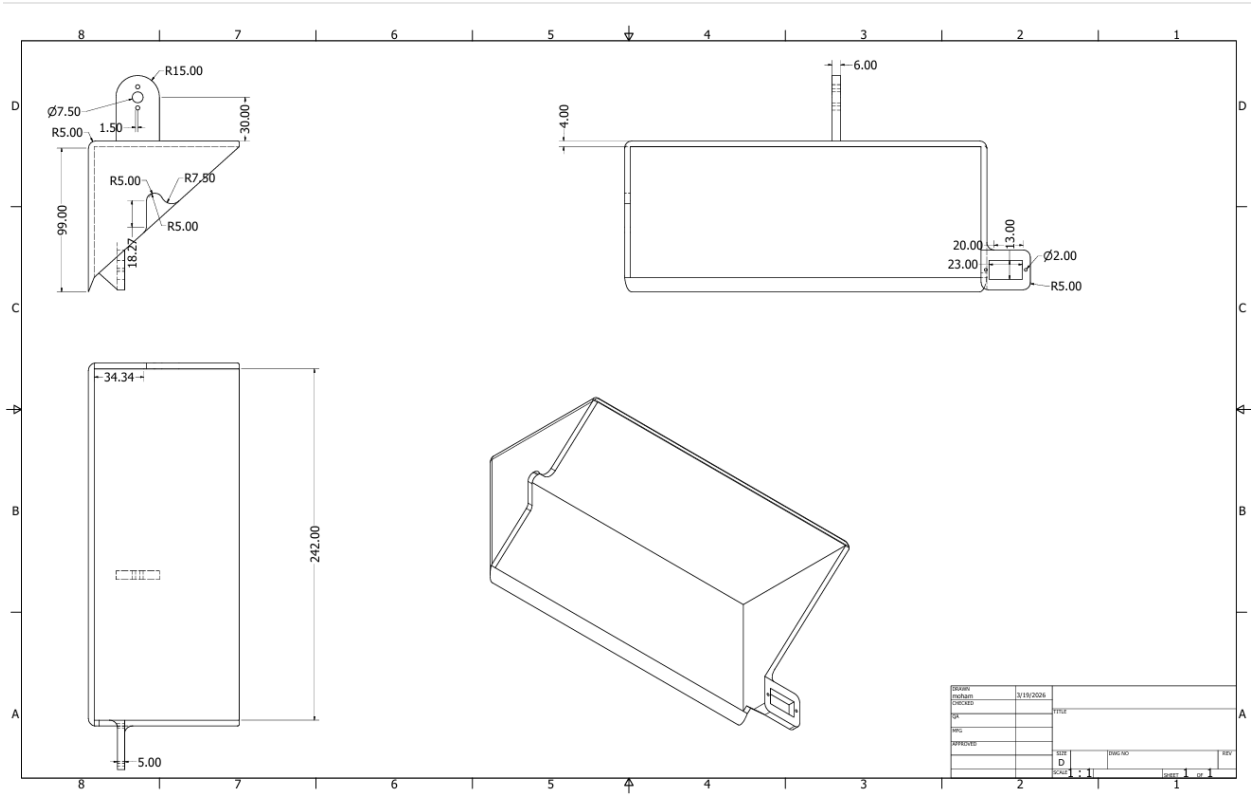


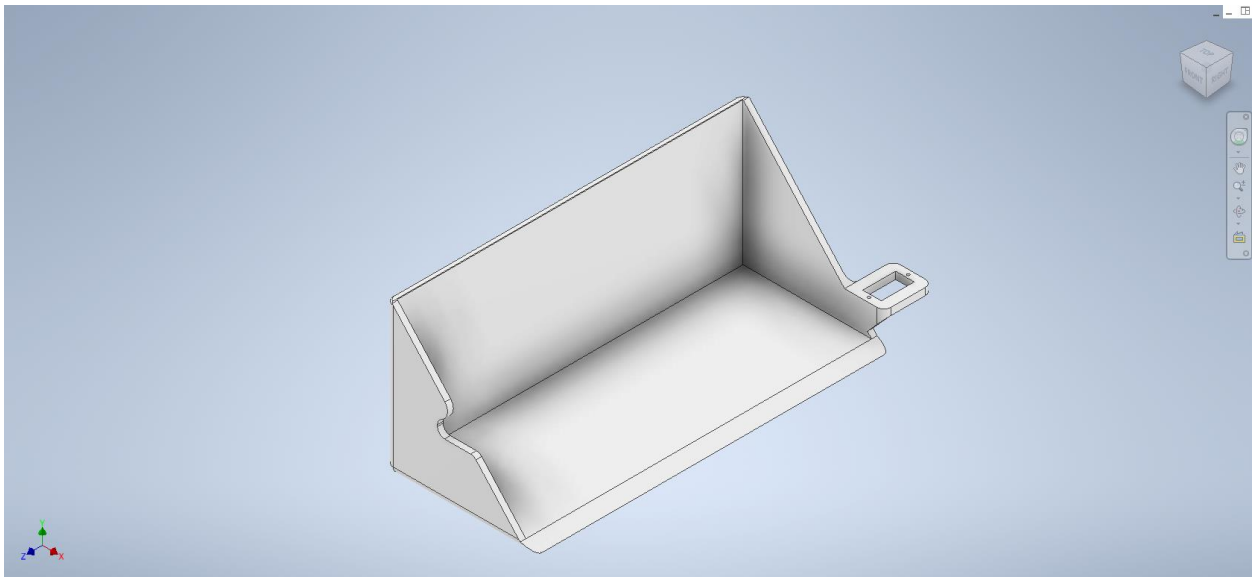

Ø3.00
R5.00
R5.00
R5.00
10.00
R5.00
39.00
39.00
40.00
R5.00
80.00
R5.00
25.00
Ø3.00
R5.00
R5.00
158.00
100.00
DRAWN
mirham
3/19/2026
CHECKED
QA
MFG
APPROVED
TITLE
SIZE
D
DWG NO
REV
SCALE 1 : 1
SHEET 1 OF 1

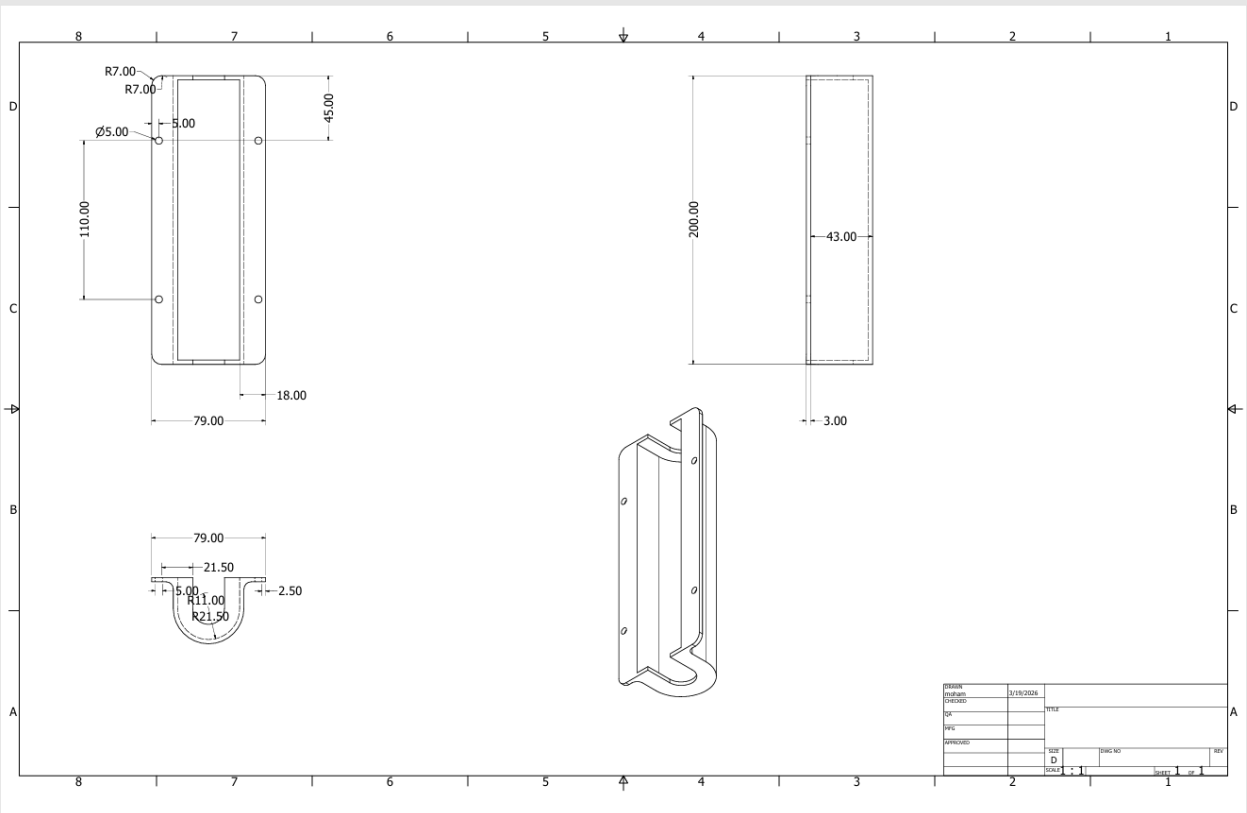


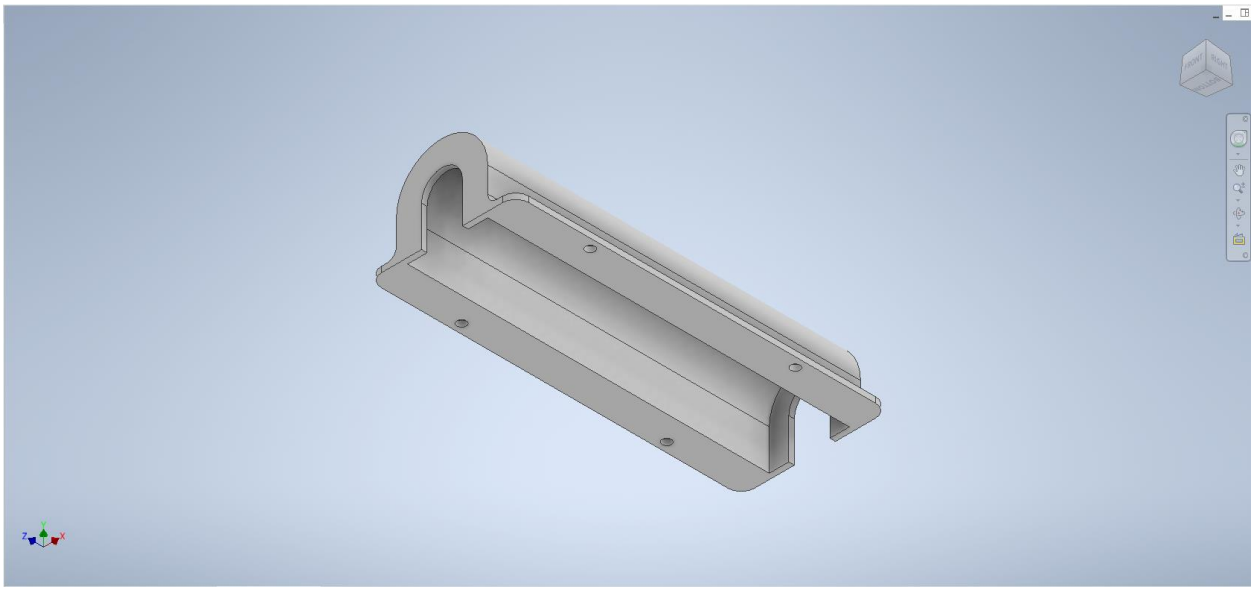

112.00
260.00
Ø20.00
27.50
20.00
36.00
10.00
55.00
59.00
36.00
70.00
30.00
155.00
R5.00
R10.00
35.00
5.00
28.00
R5.00
29.50
28.00
R3.50
Ø5.00
60.00
20.00
19.00
29.50
17.00
10.00
Ø4.00
10.50
70.00
7.00
7.00
mcham
3/19/2026
D
1 / 2
1 OF 1

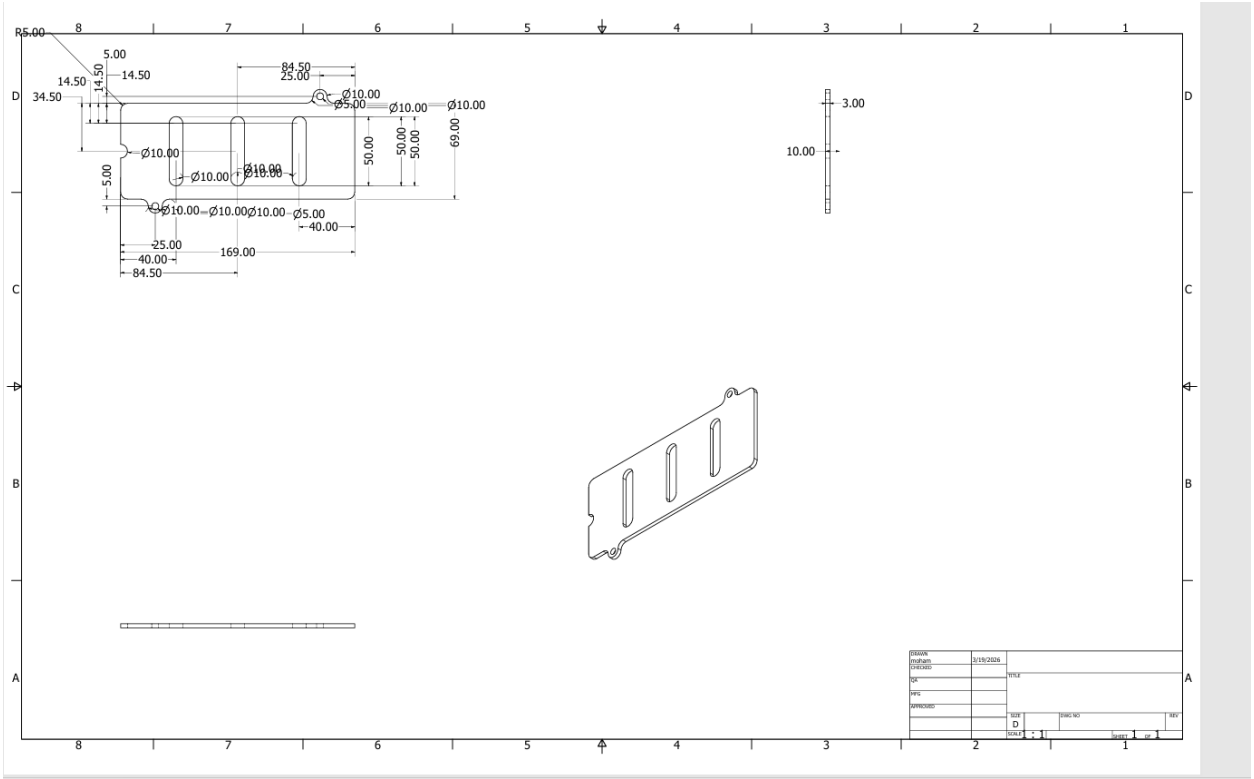


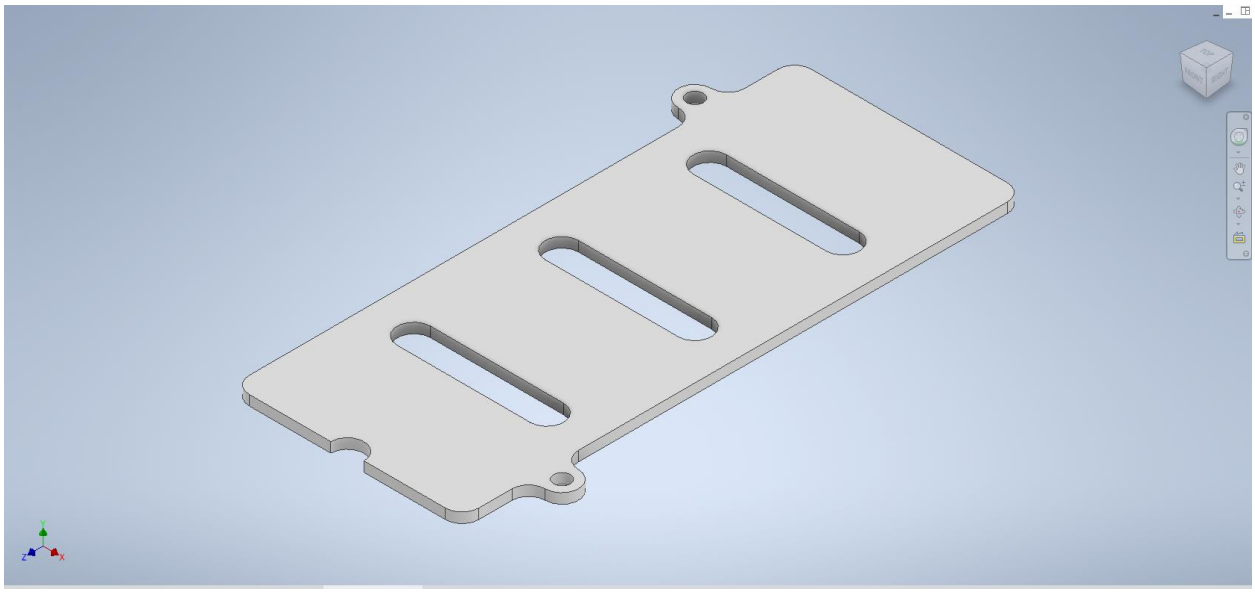

R15.00
Ø7.50
R5.00
1.50
30.00
R5.00
R7.50
R5.00
99.00
18.27
6.00
4.00
13.00
20.00
23.00
Ø2.00
R5.00
34.34
242.00
5.00

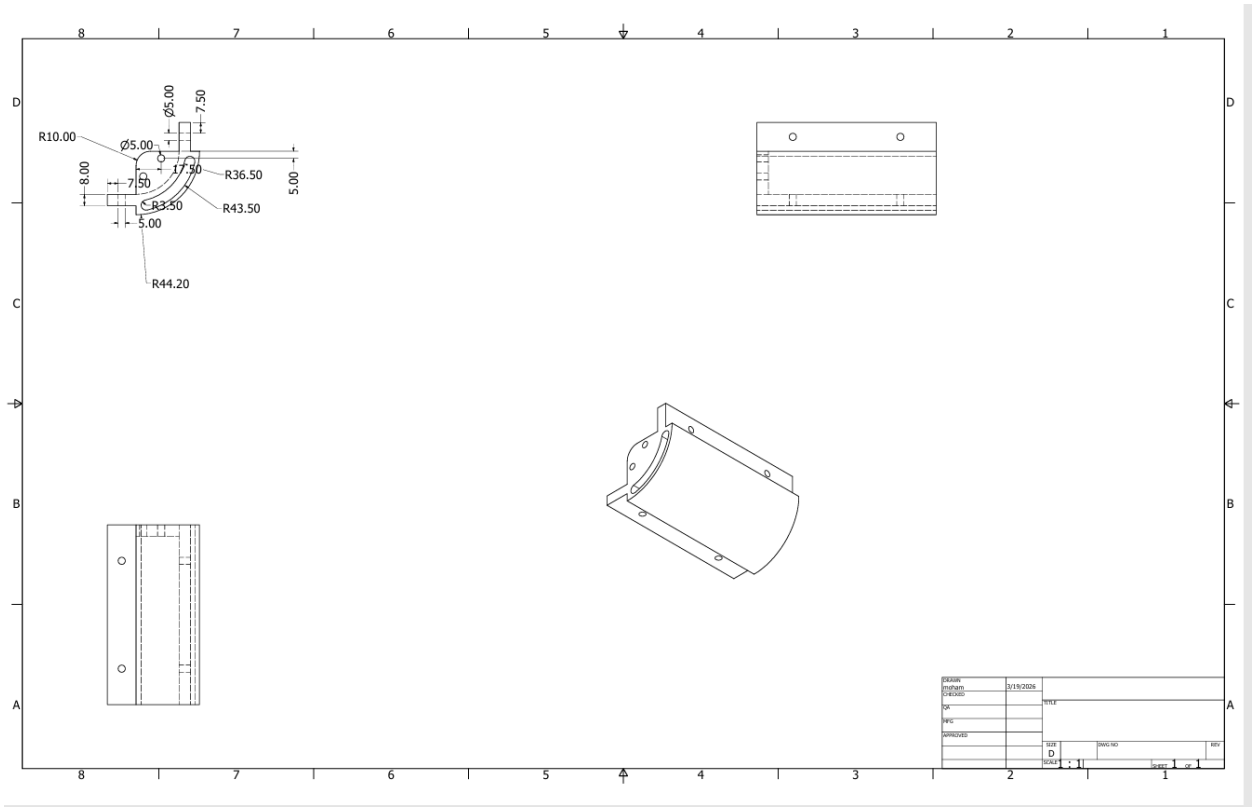


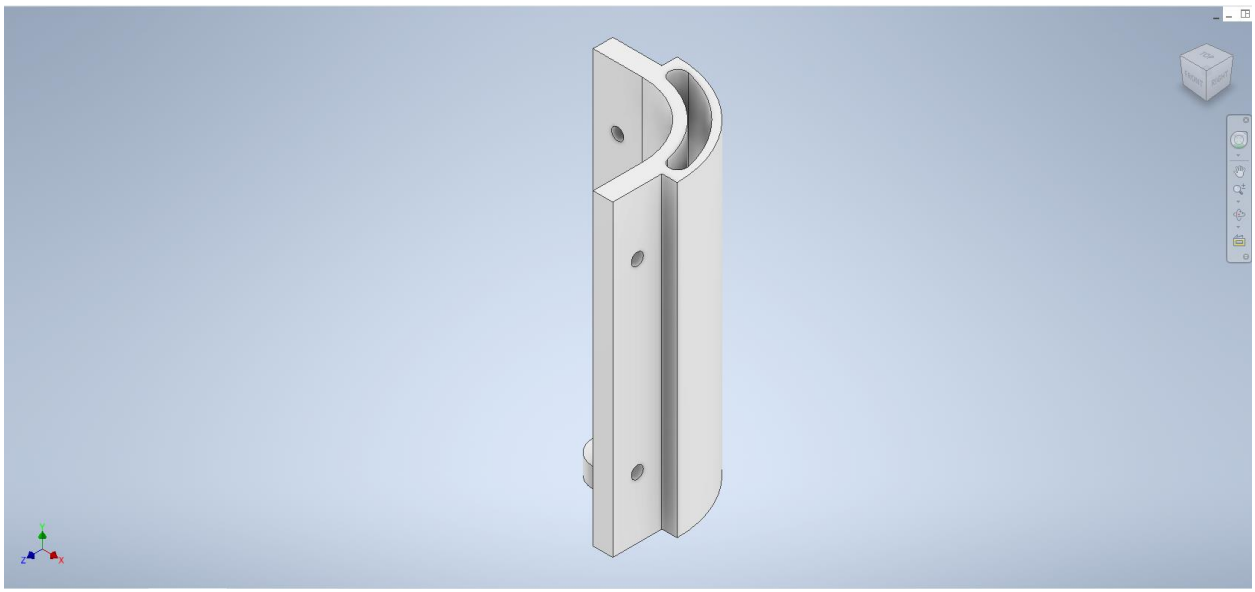

R7.00
R7.00
Ø5.00
5.00
45.00
110.00
18.00
79.00
200.00
43.00
3.00
79.00
21.50
5.00
R11.00
R21.50
2.50

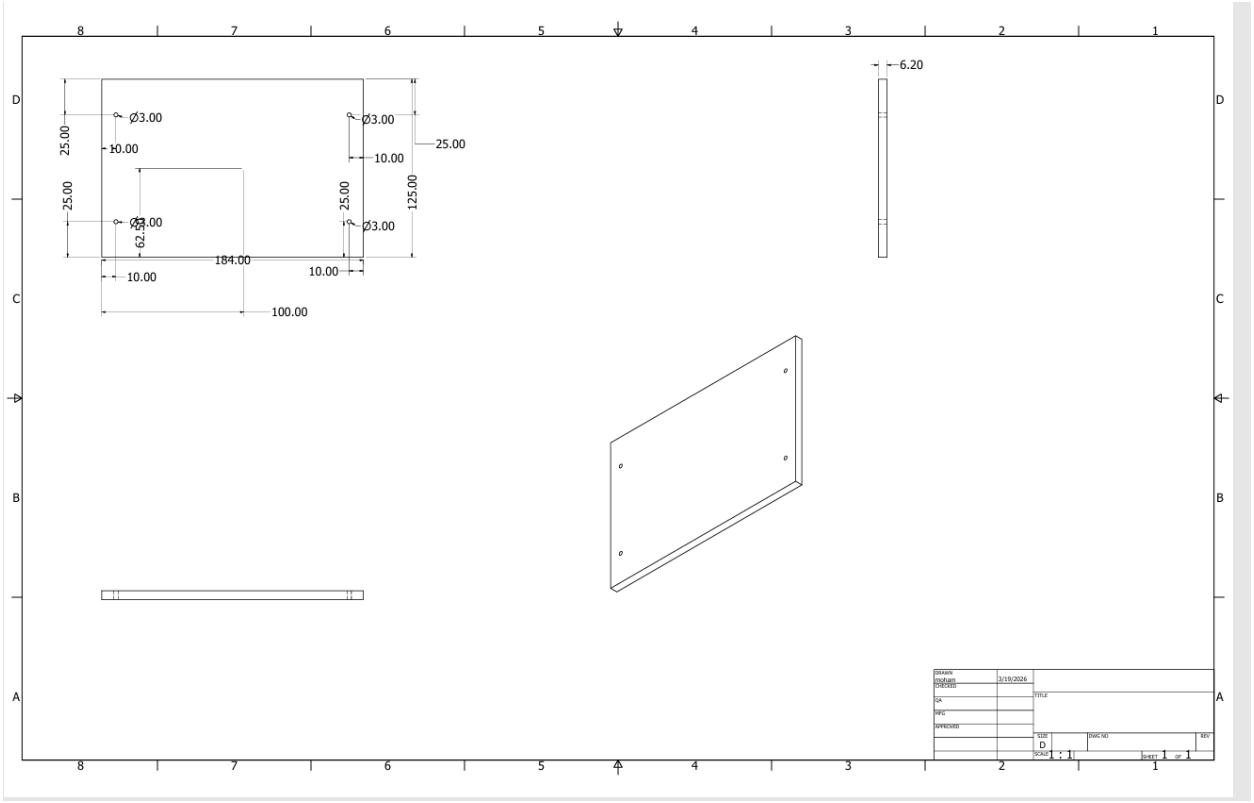


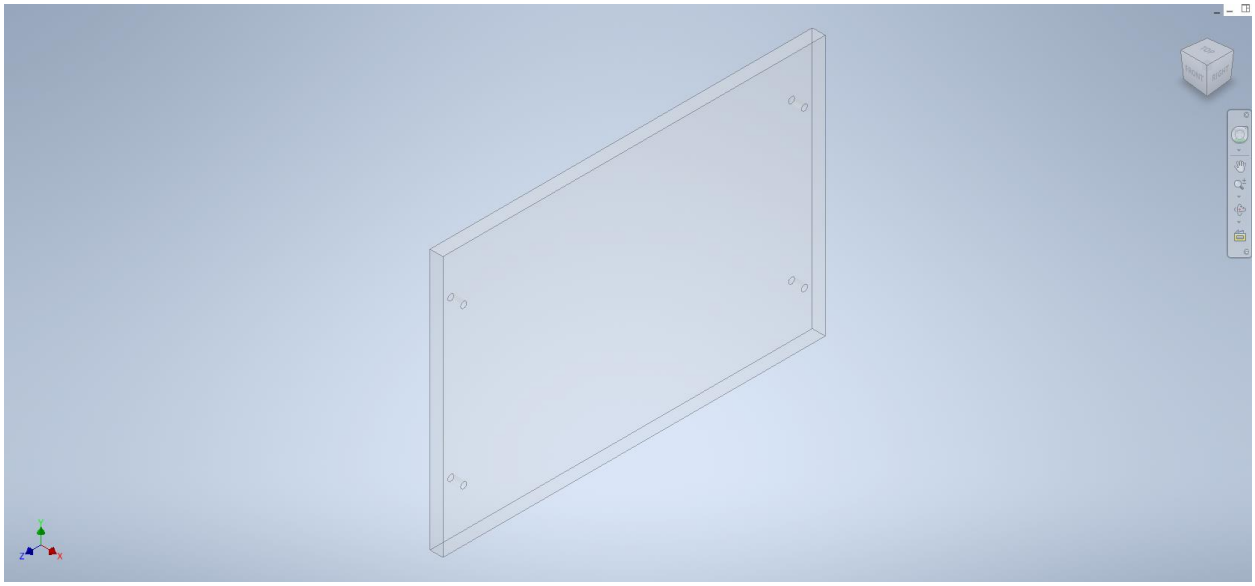

R5.00
5.00
14.50
14.50
14.50
34.50
84.50
25.00
Ø10.00
Ø5.00
Ø10.00
Ø10.00
Ø10.00
Ø10.00
Ø10.00
Ø10.00
Ø10.00
Ø10.00
Ø10.00
Ø5.00
40.00
25.00
169.00
40.00
84.50
5.00
50.00
50.00
50.00
69.00
3.00
10.00

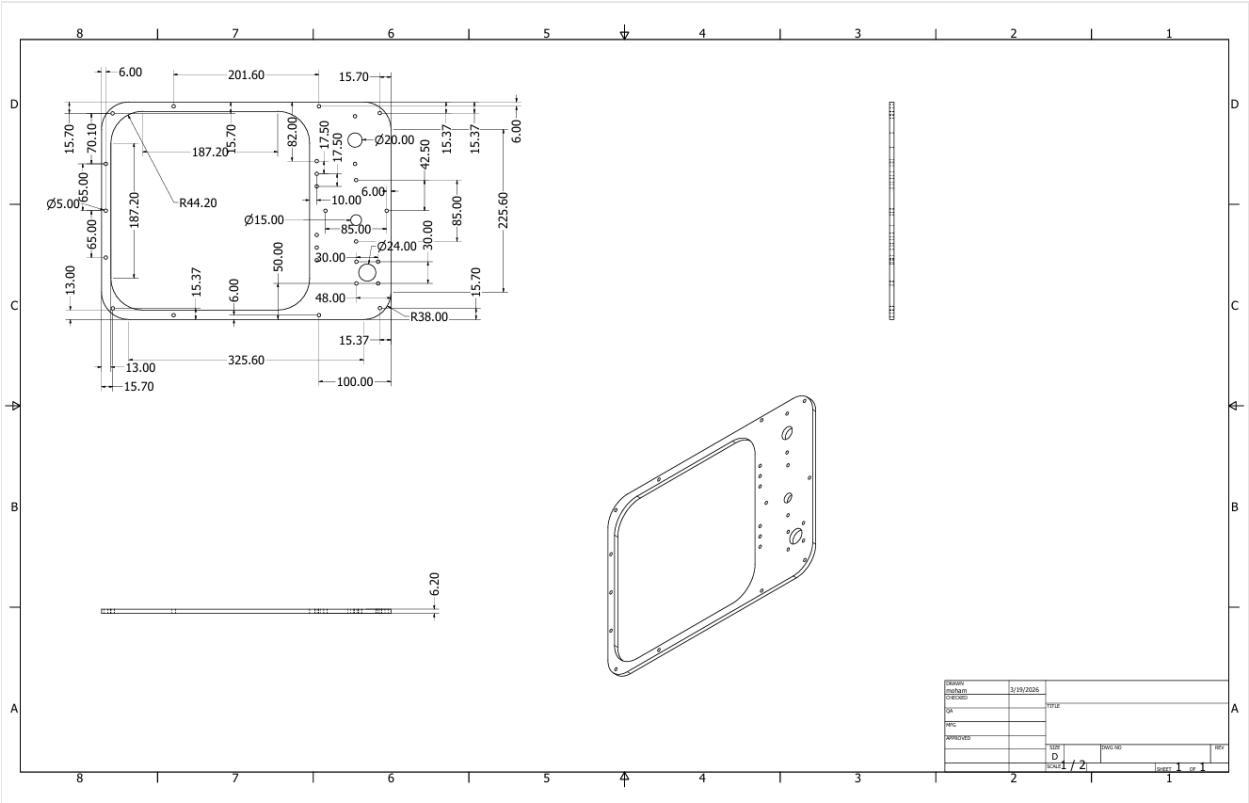


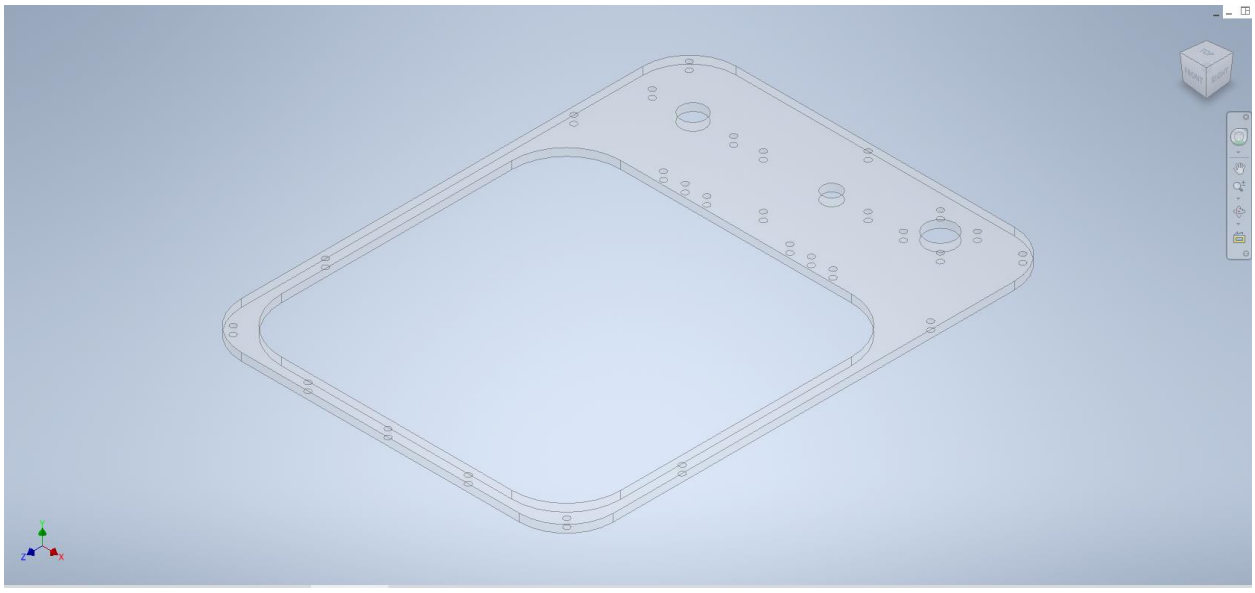

R10.00
Ø5.00
Ø5.00
7.50
8.00
7.50
17.50
R36.50
5.00
R43.50
R3.50
5.00
R44.20
3/19/2026

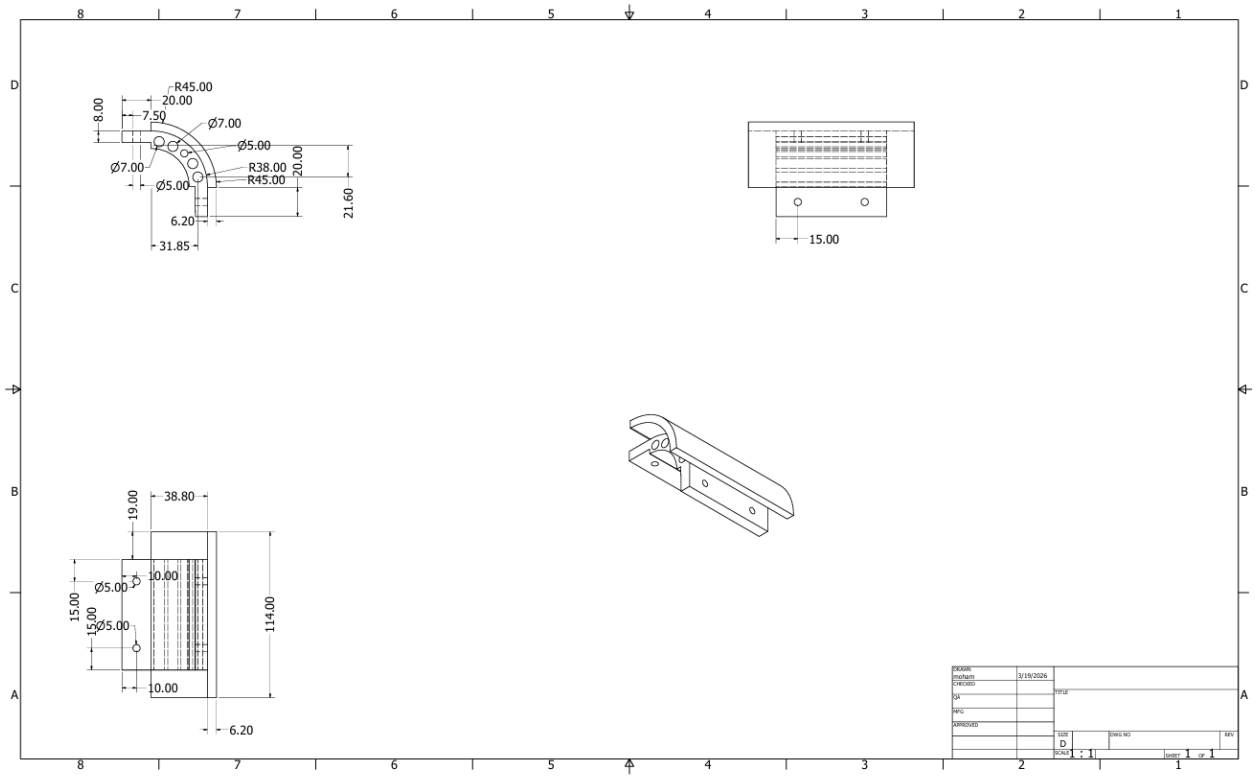


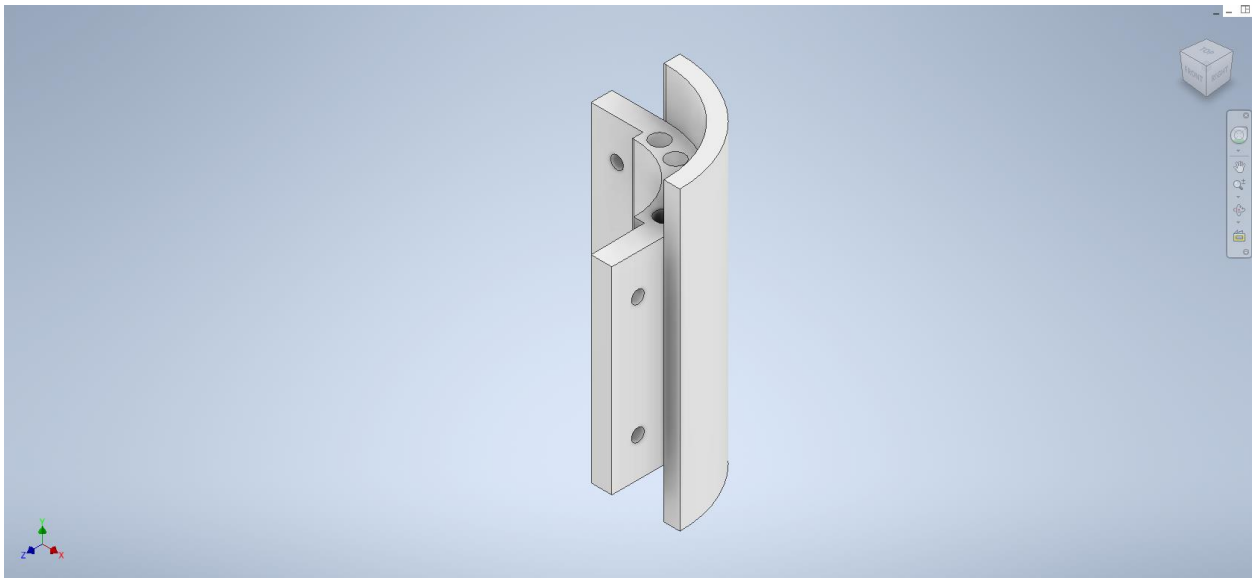

6.20
Ø3.00
25.00
10.00
125.00
62.50
184.00
100.00

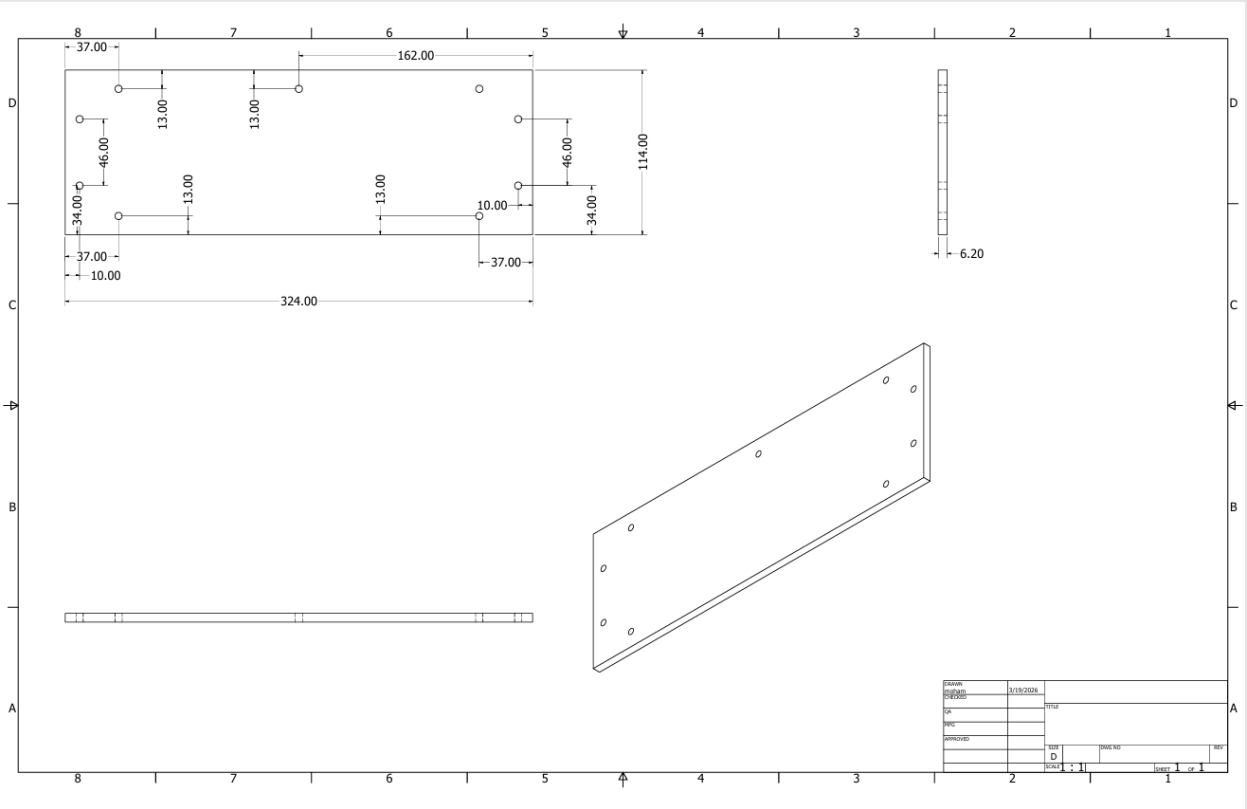


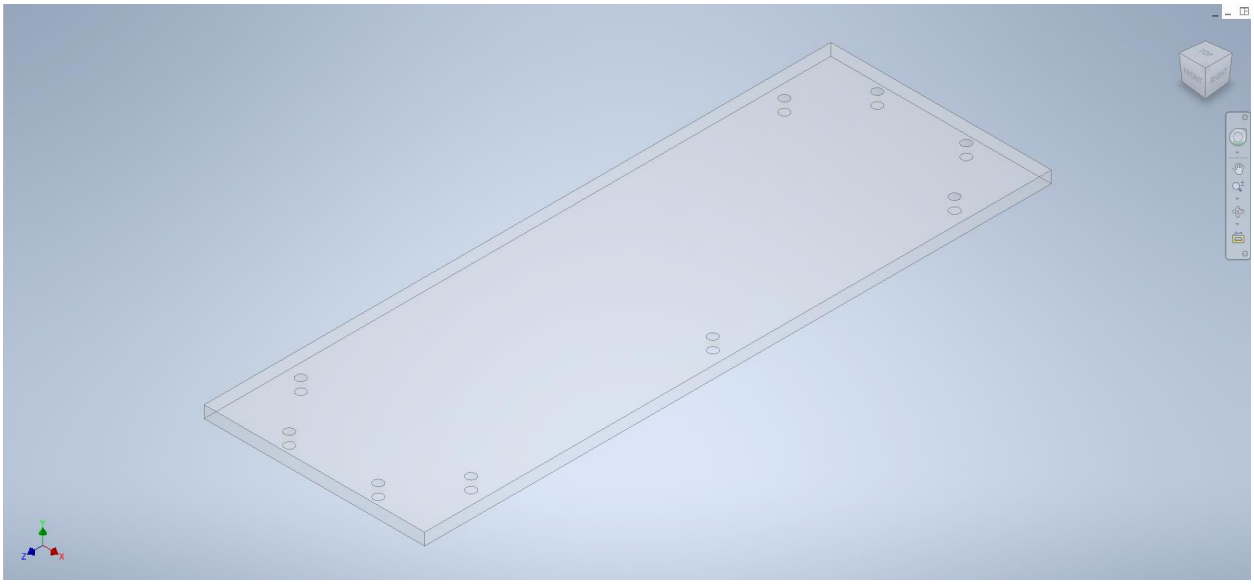

6.00
201.60
15.70
15.70
70.10
65.00
65.00
187.20
15.70
82.00
187.20
R44.20
Ø5.00
17.50
17.50
Ø20.00
15.37
15.37
6.00
42.50
6.00
10.00
85.00
225.60
Ø15.00
85.00
30.00
Ø24.00
30.00
13.00
15.37
6.00
50.00
48.00
15.70
R38.00
15.37
325.60
13.00
15.70
100.00
6.20
1 / 2

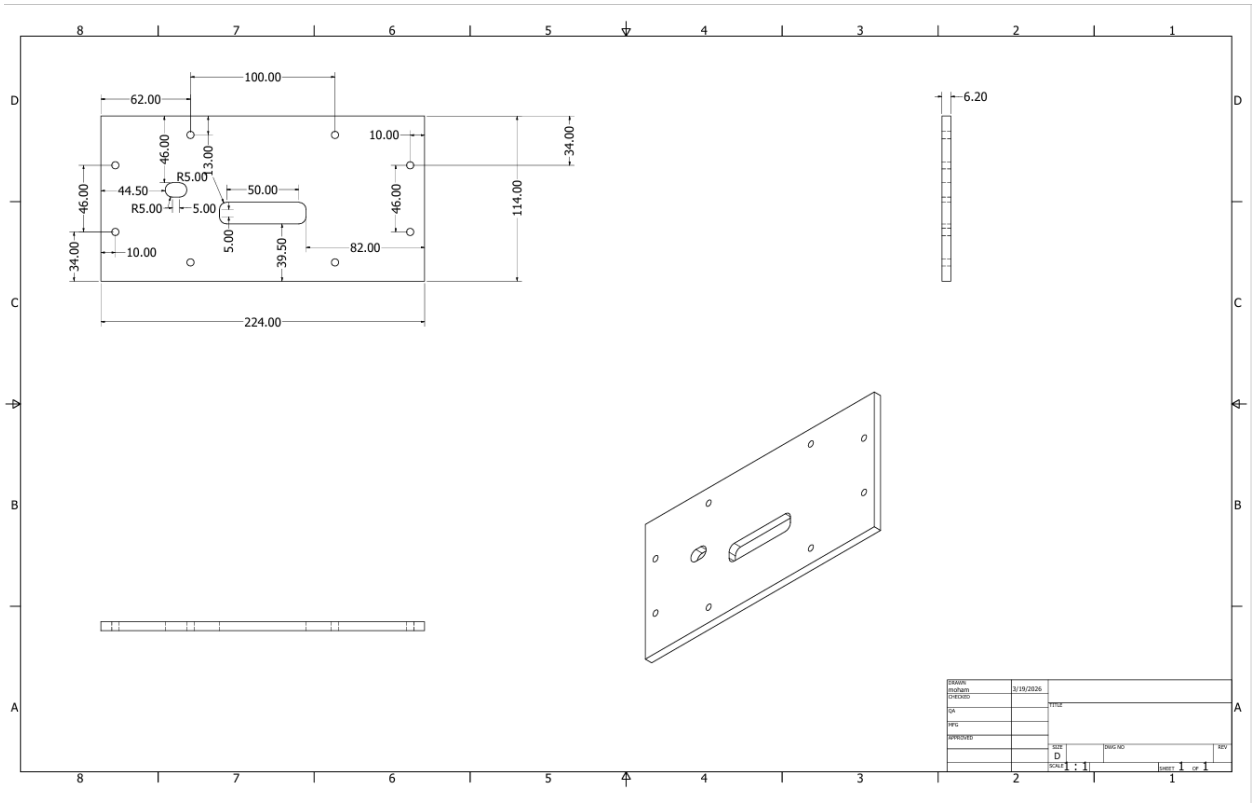


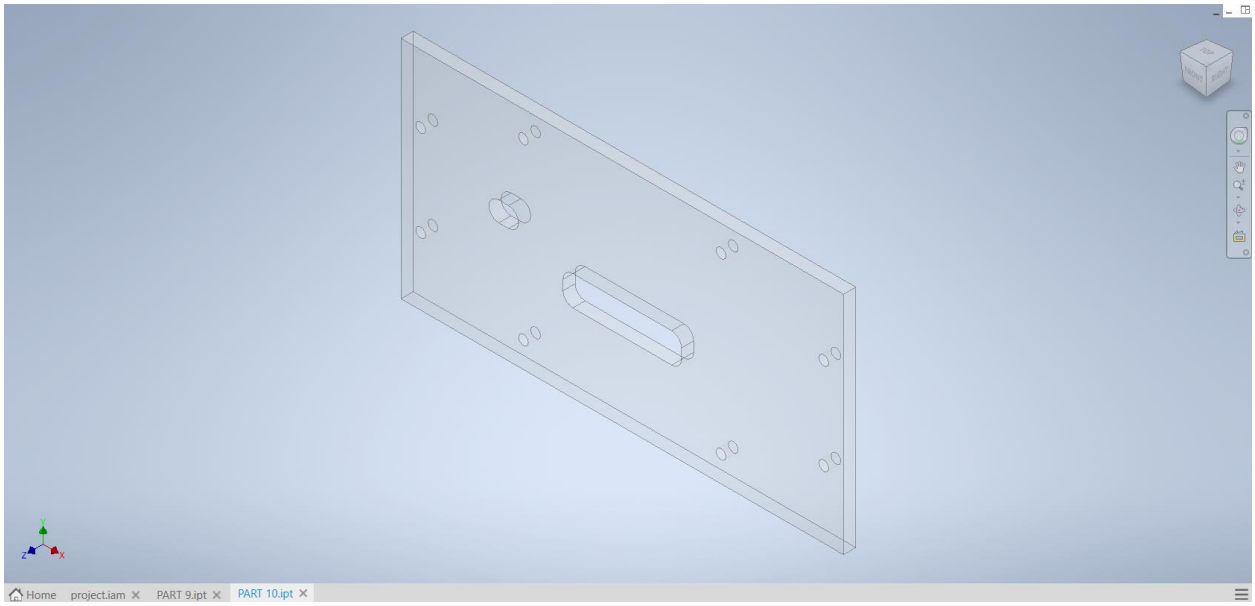

R45.00
20.00
7.50
8.00
Ø7.00
Ø5.00
Ø7.00
R38.00
R45.00
Ø5.00
20.00
21.60
6.20
31.85
15.00
38.80
19.00
10.00
Ø5.00
15.00
15.00
Ø5.00
114.00
10.00
6.20
moham
3/19/2026
D
1 : 1
1 OF 1

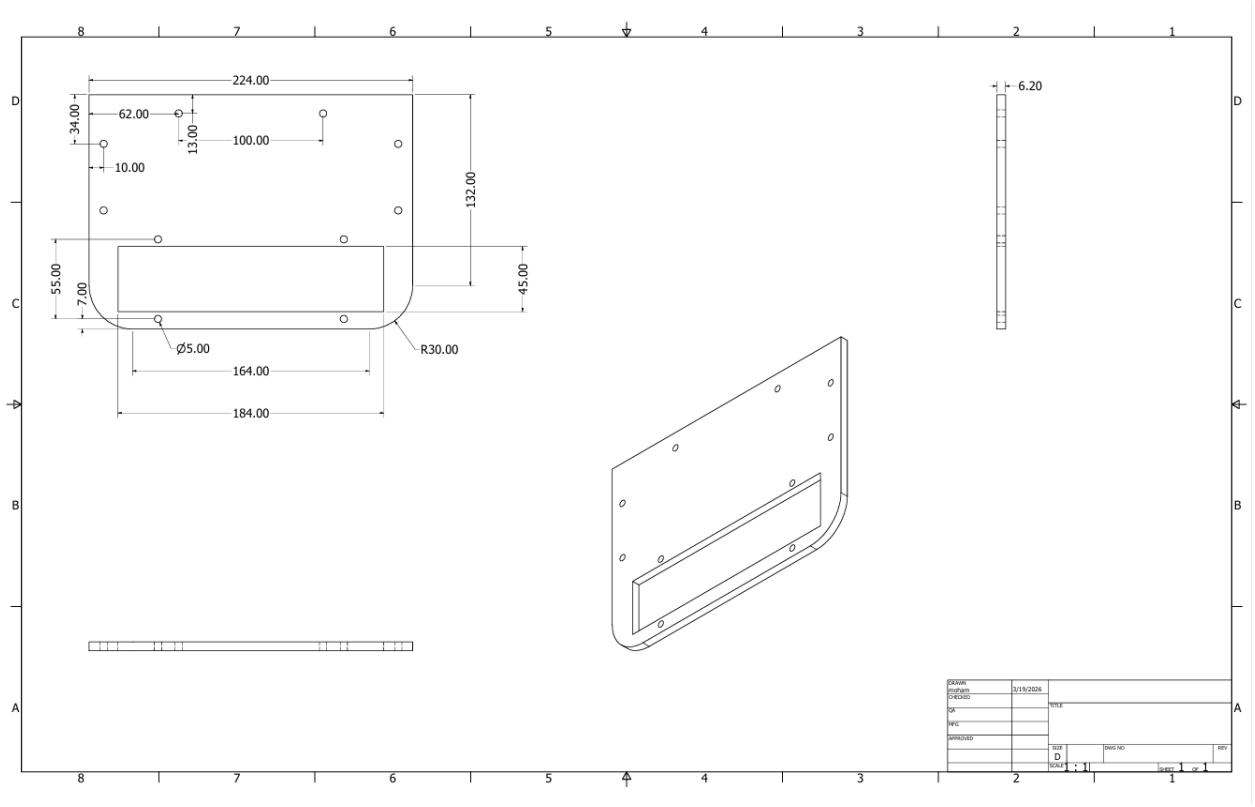


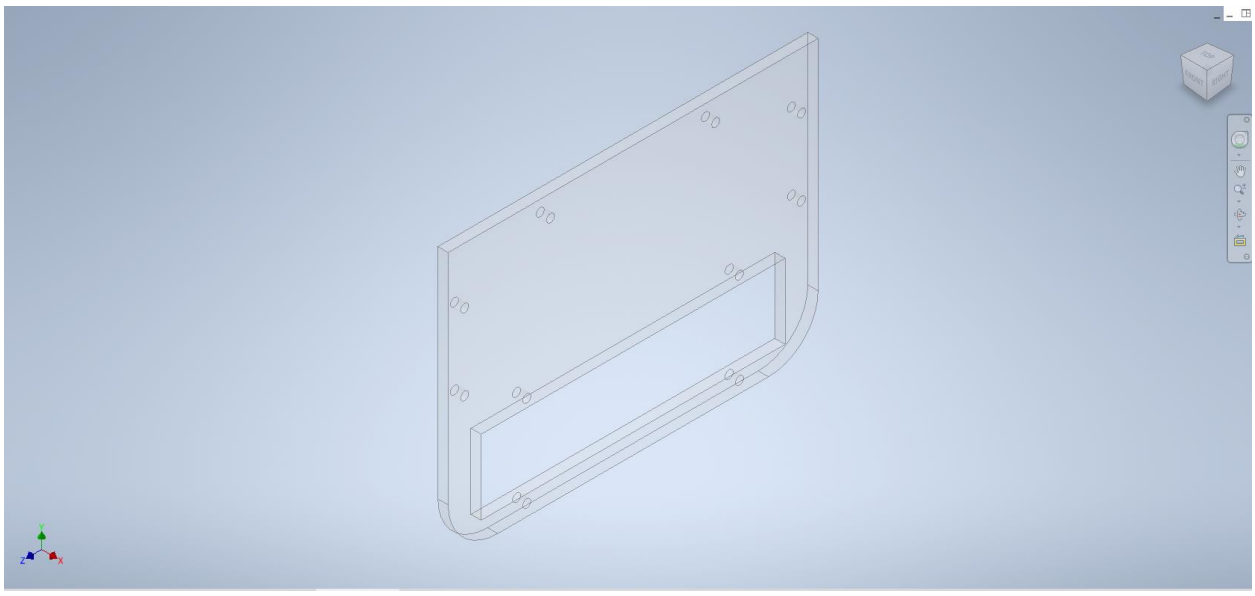

37.00
162.00
13.00
13.00
46.00
46.00
114.00
34.00
13.00
13.00
10.00
34.00
37.00
37.00
10.00
324.00
6.20

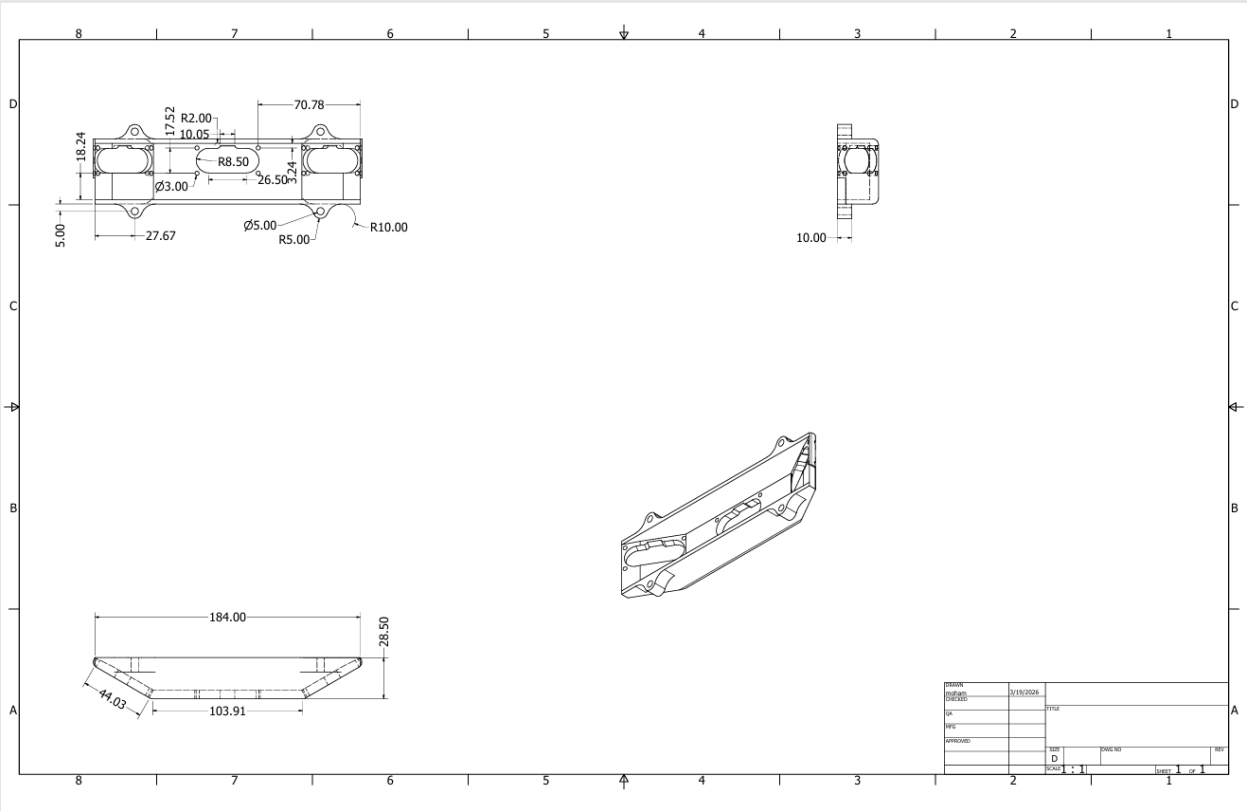


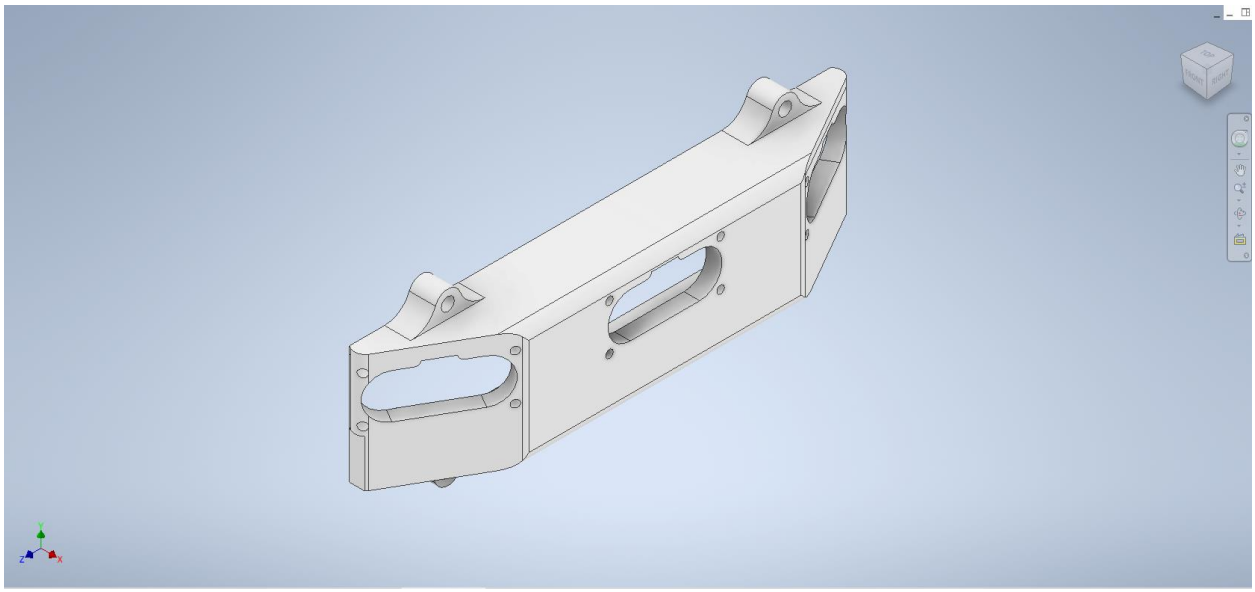

100.00
62.00
10.00
34.00
46.00
3.00
R5.00
44.50
50.00
R5.00
5.00
46.00
114.00
34.00
10.00
5.00
39.50
82.00
224.00
6.20
mohamm
3/19/2026
D
1 : 1
1 OF 1

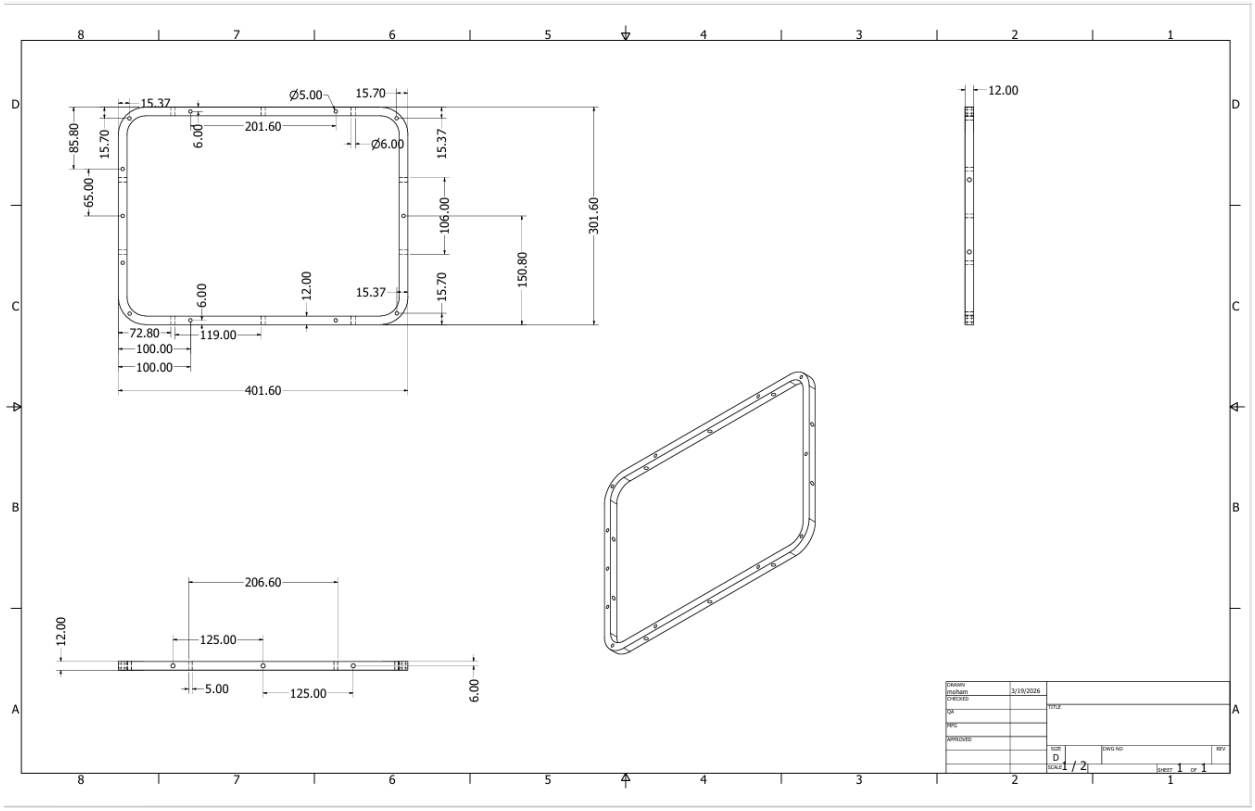

Home
project.iam
PART 9.ipt
PART 10.ipt

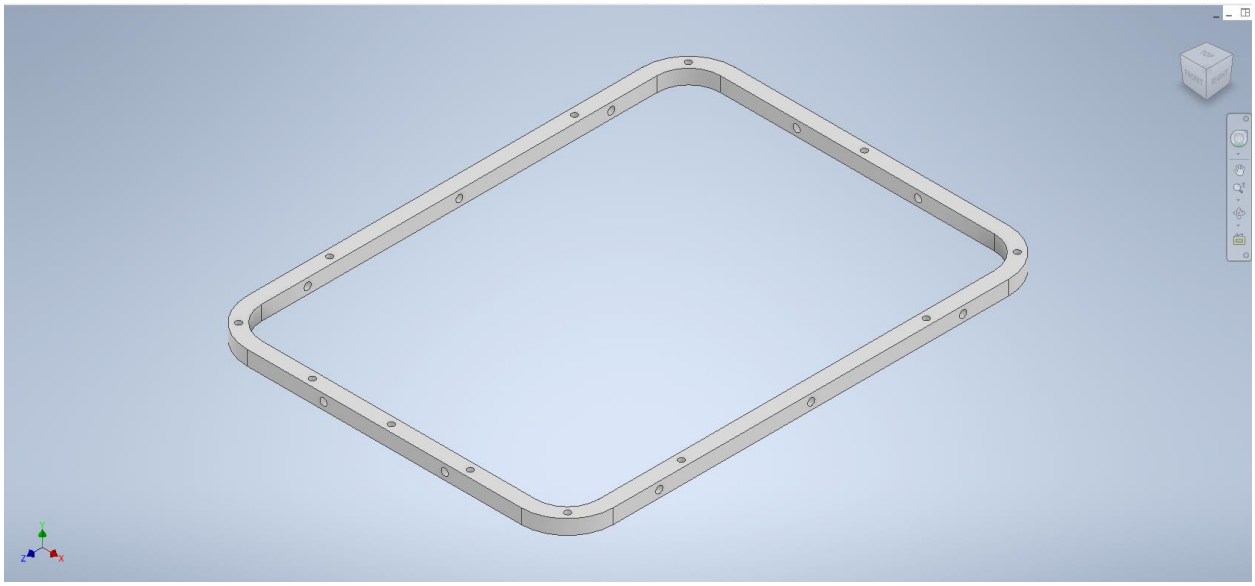

224.00
62.00
34.00
13.00
100.00
10.00
132.00
55.00
7.00
45.00
Ø5.00
R30.00
164.00
184.00
6.20

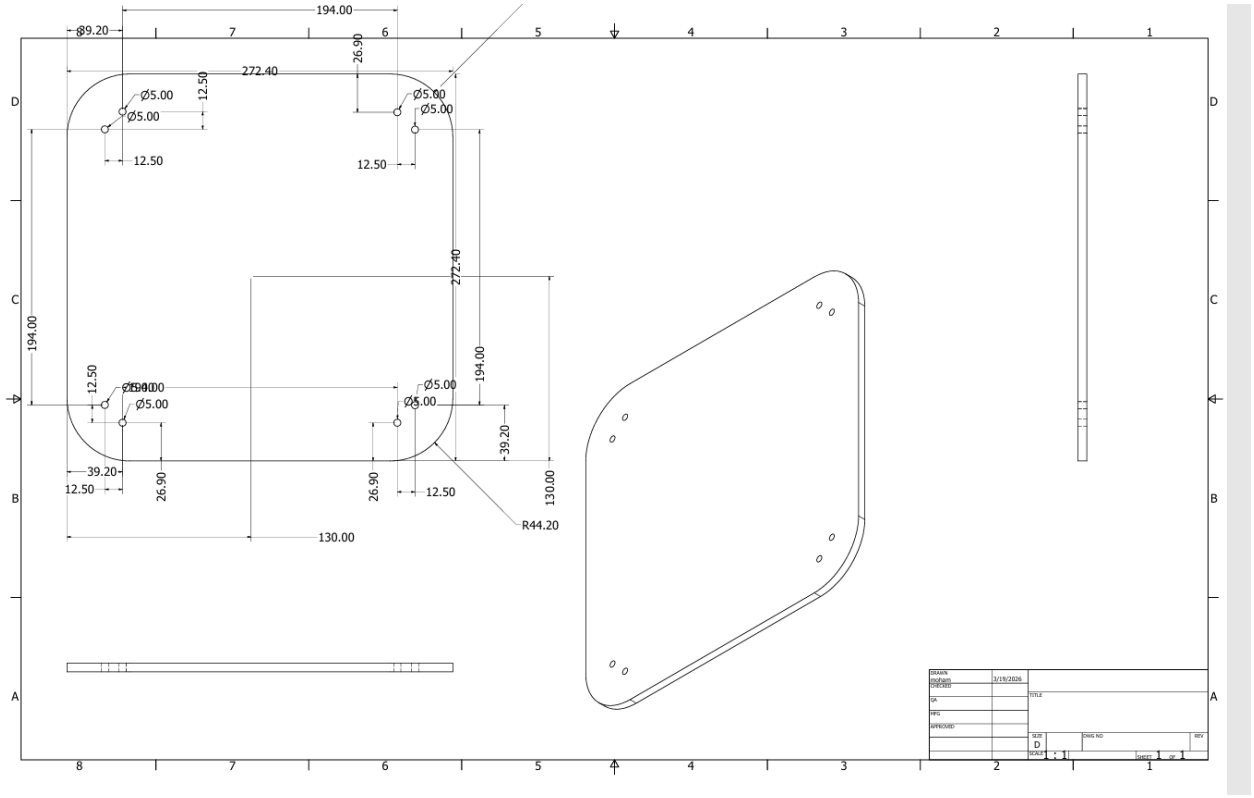


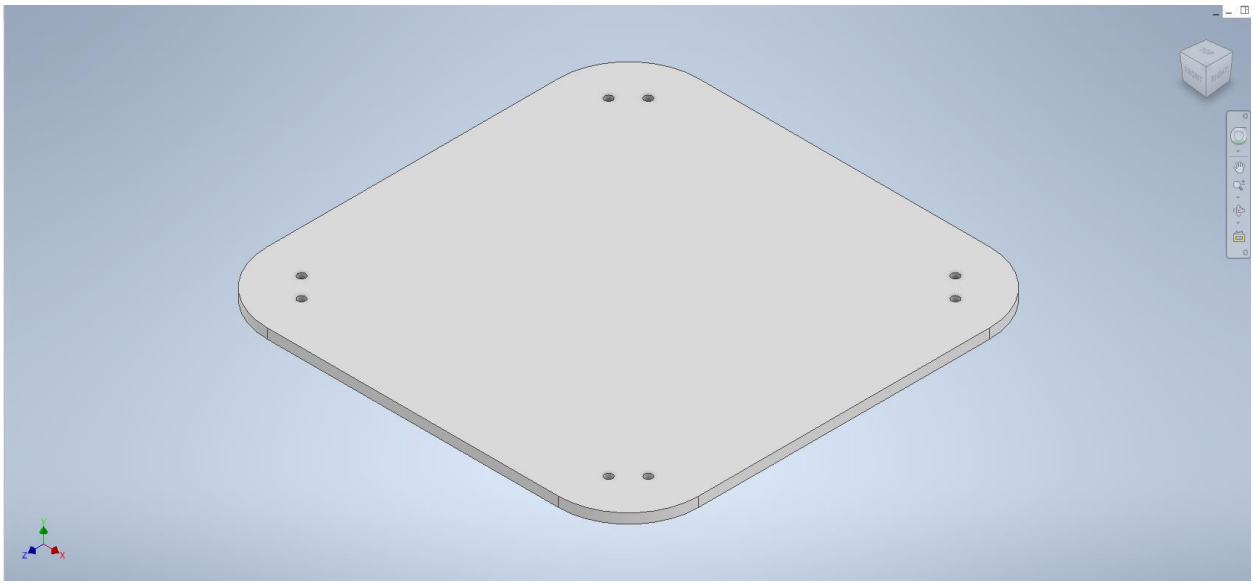

70.78
R2.00
10.05
17.52
18.24
R8.50
26.50
3.24
Ø3.00
5.00
27.67
Ø5.00
R5.00
R10.00
10.00
184.00
28.50
44.03
103.91

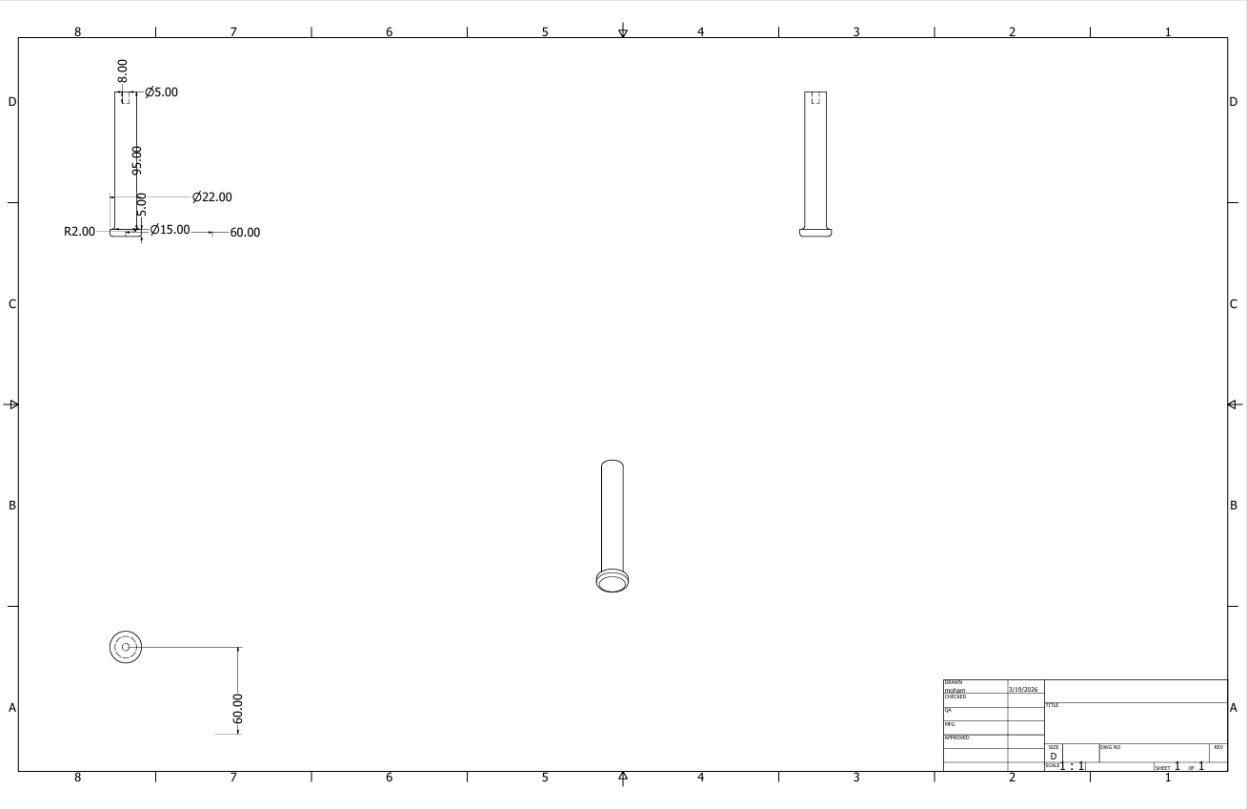



15.37
Ø5.00
15.70
85.80
15.70
6.00
201.60
Ø6.00
15.37
65.00
106.00
301.60
150.80
6.00
12.00
15.37
15.70
72.80
119.00
100.00
100.00
401.60
12.00
206.60
125.00
12.00
5.00
125.00
6.00
1 / 2

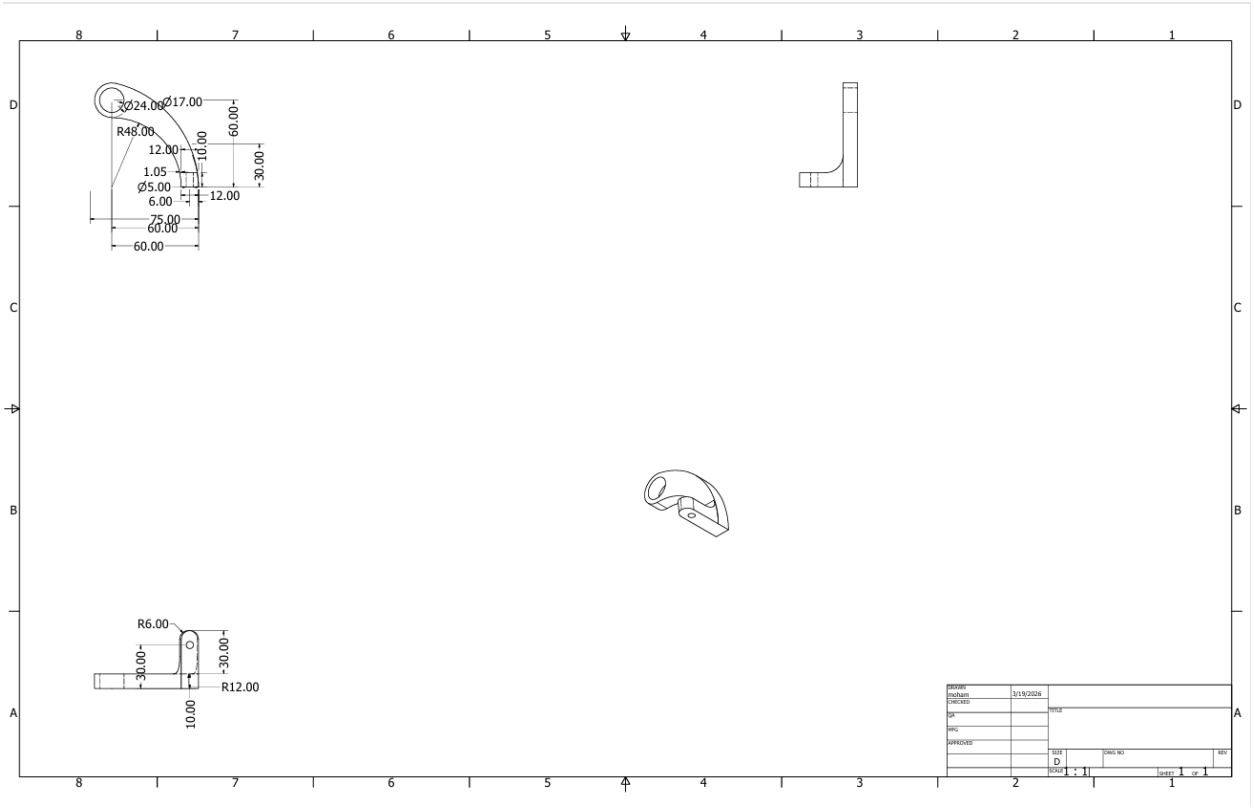


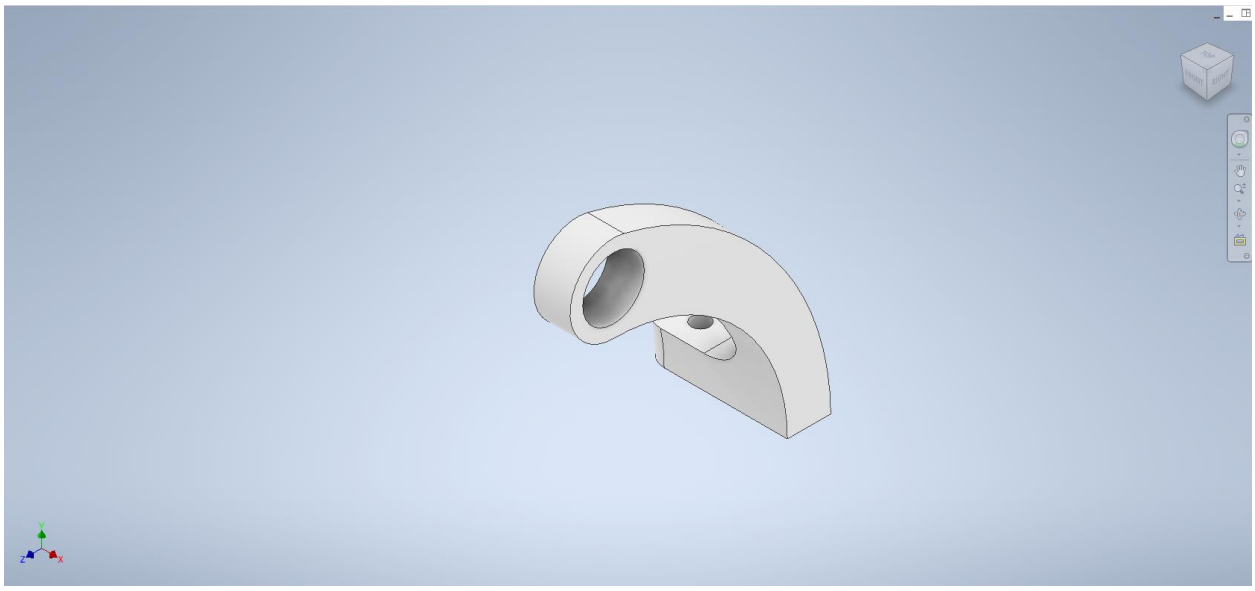

194.00
89.20
272.40
26.90
12.50
Ø5.00
Ø5.00
Ø5.00
Ø5.00
12.50
12.50
272.40
194.00
194.00
12.50
Ø5.00
Ø5.00
Ø5.00
Ø5.00
39.20
39.20
12.50
26.90
26.90
12.50
130.00
R44.20
130.00

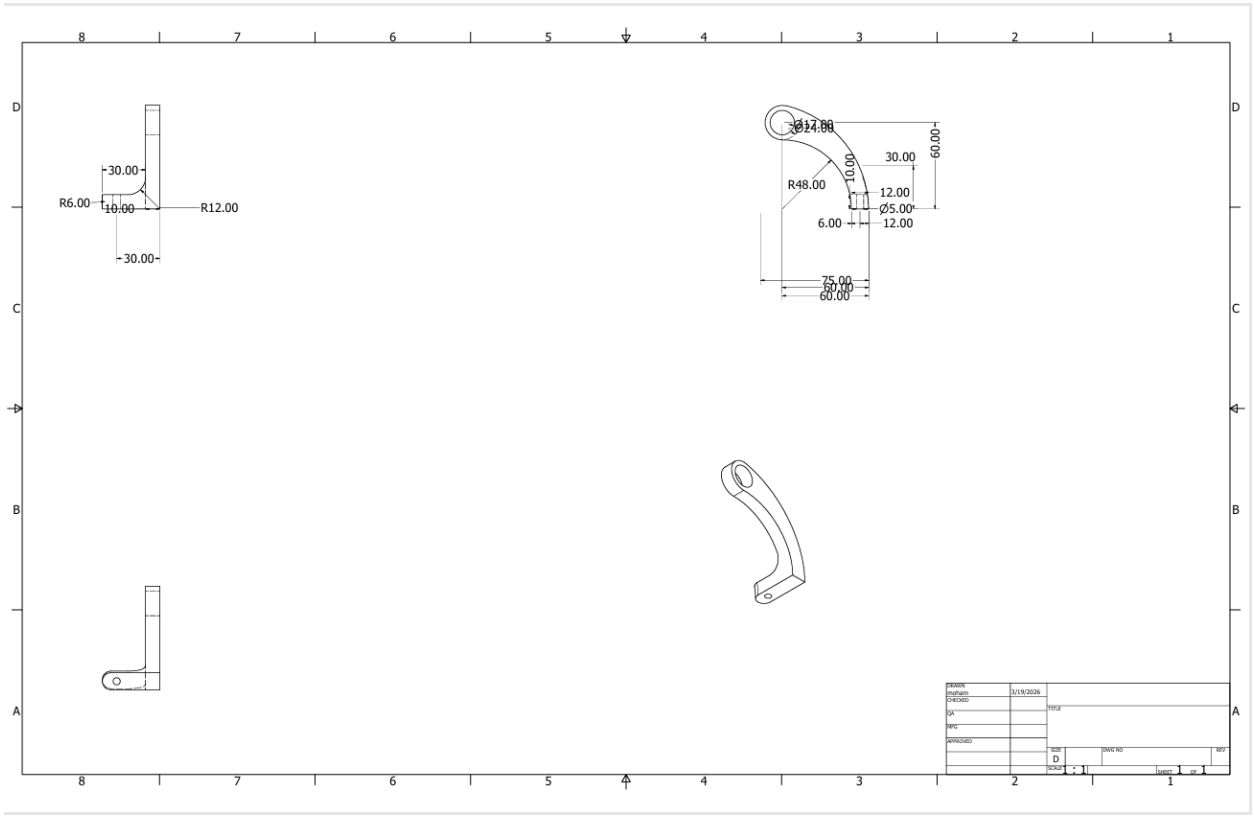


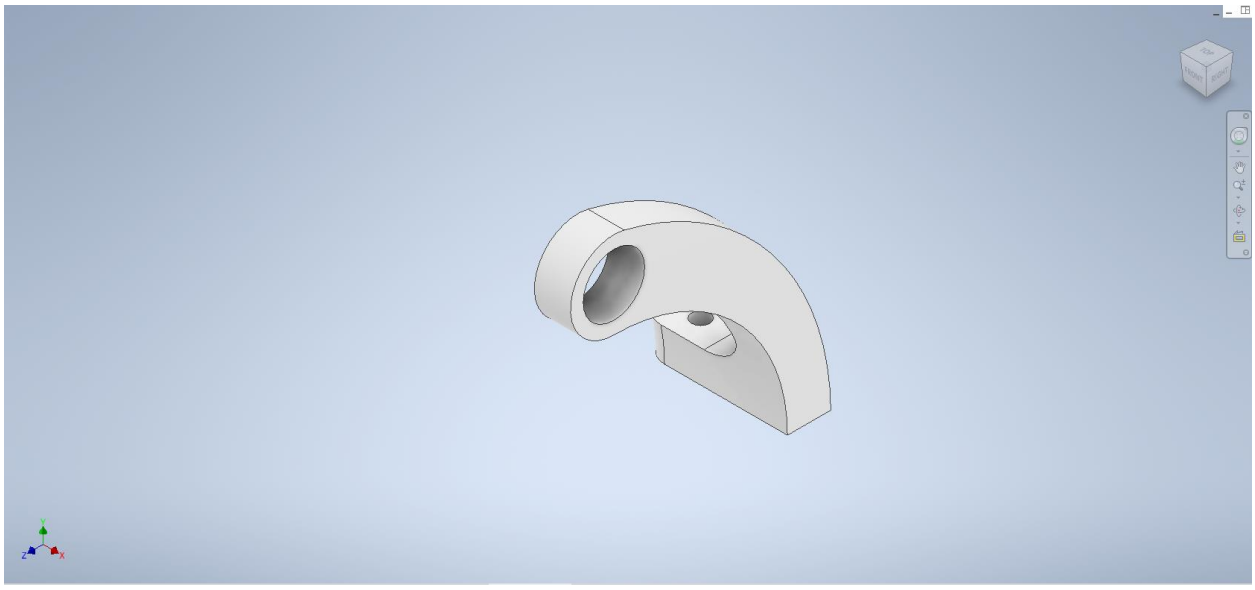

8.00
Ø5.00
95.00
5.00
Ø22.00
R2.00
Ø15.00
60.00
60.00
1 : 1

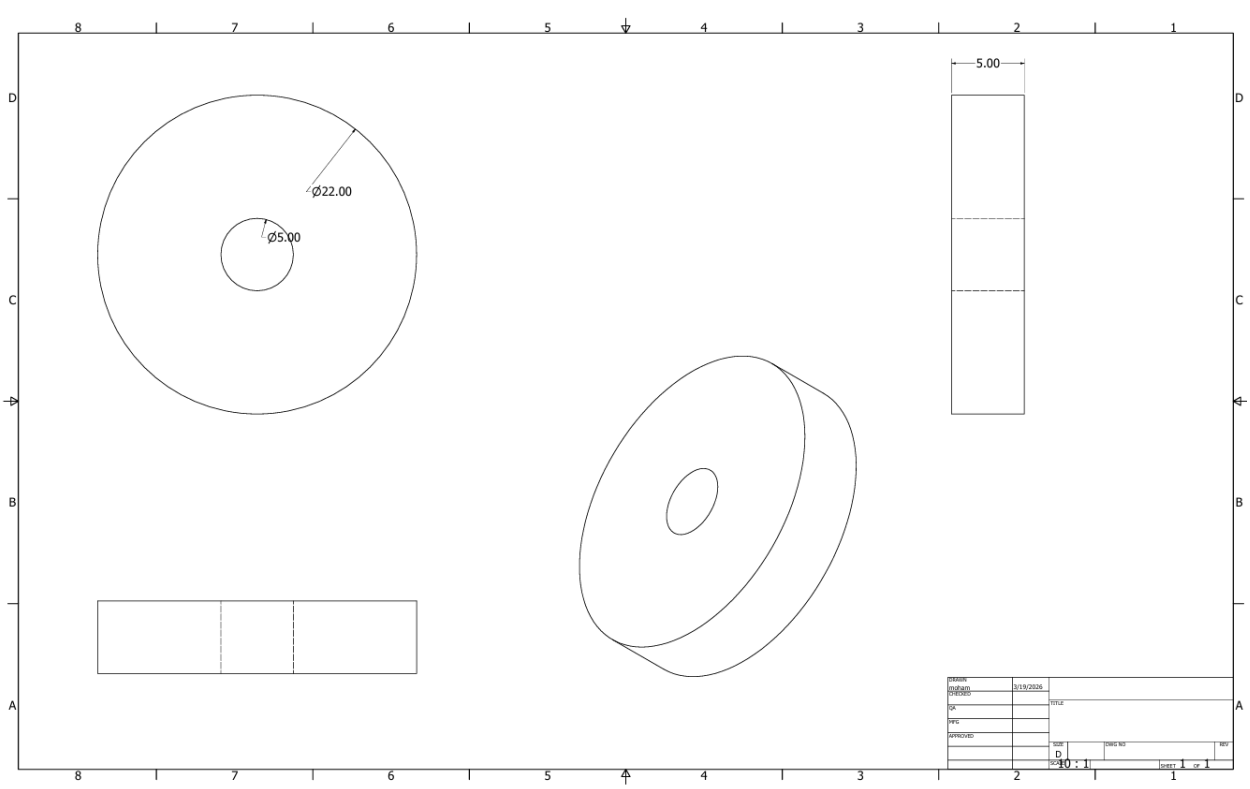


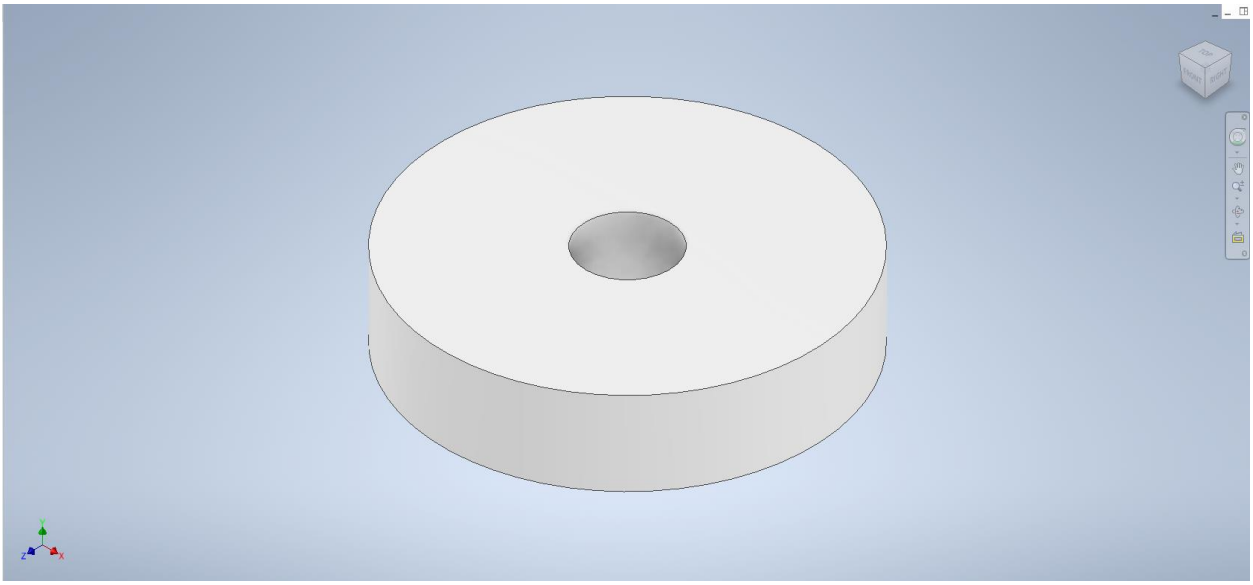

Ø24.00
Ø17.00
R48.00
12.00
10.00
60.00
30.00
1.05
Ø5.00
6.00
12.00
75.00
60.00
60.00
R6.00
30.00
30.00
R12.00
10.00
mohsm
3/19/2026
D
SCALE 1 : 1
SHEET 1 OF 1

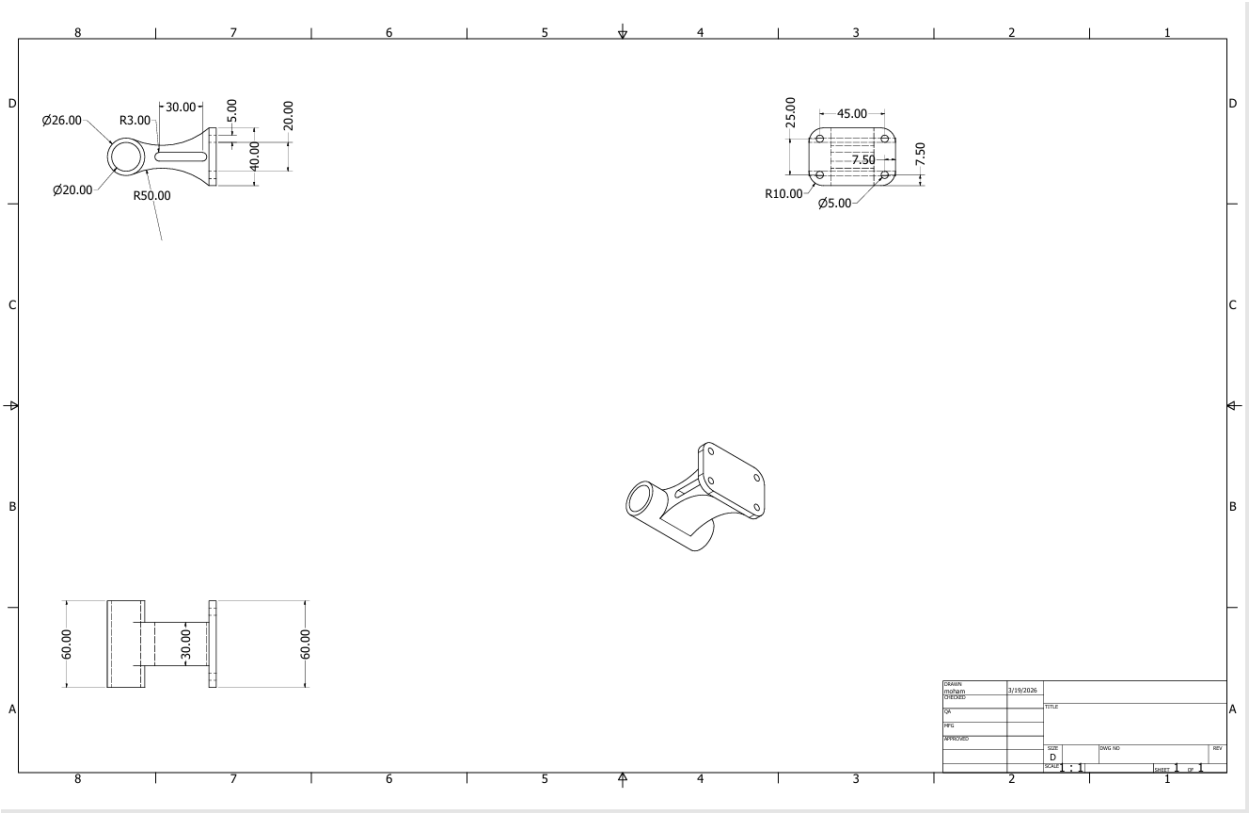


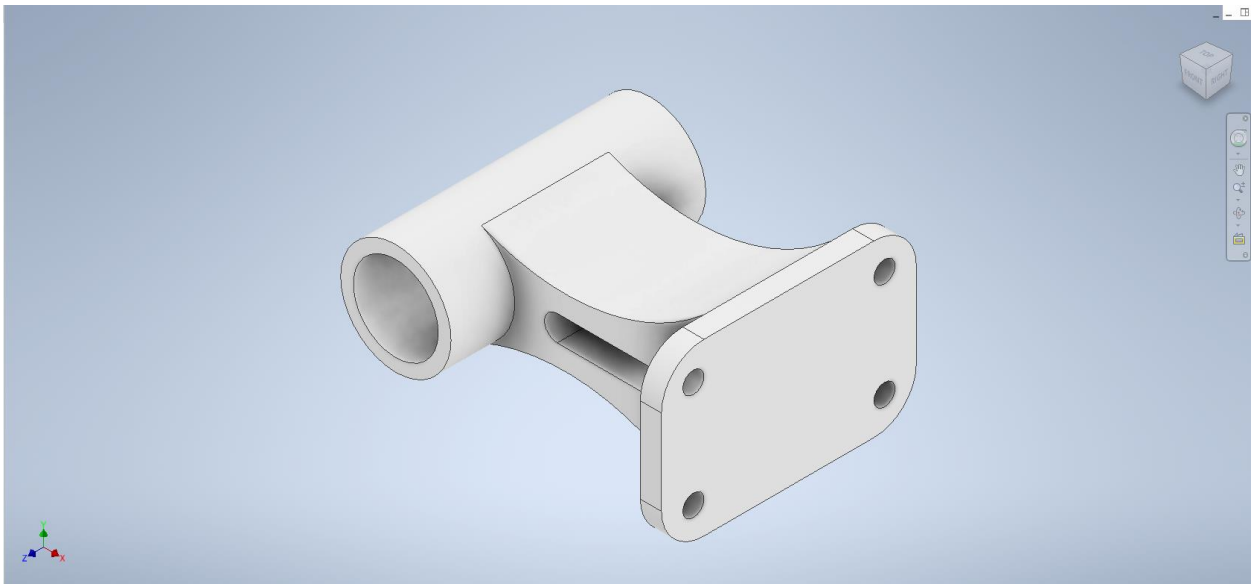

30.00
R6.00
10.00
R12.00
30.00
Ø12.00
Ø24.00
60.00
10.00
30.00
R48.00
12.00
Ø5.00
6.00
12.00
75.00
50.00
60.00
3/19/2026
SIZE
D
SCALE
1 : 1
SHEET 1 OF 1

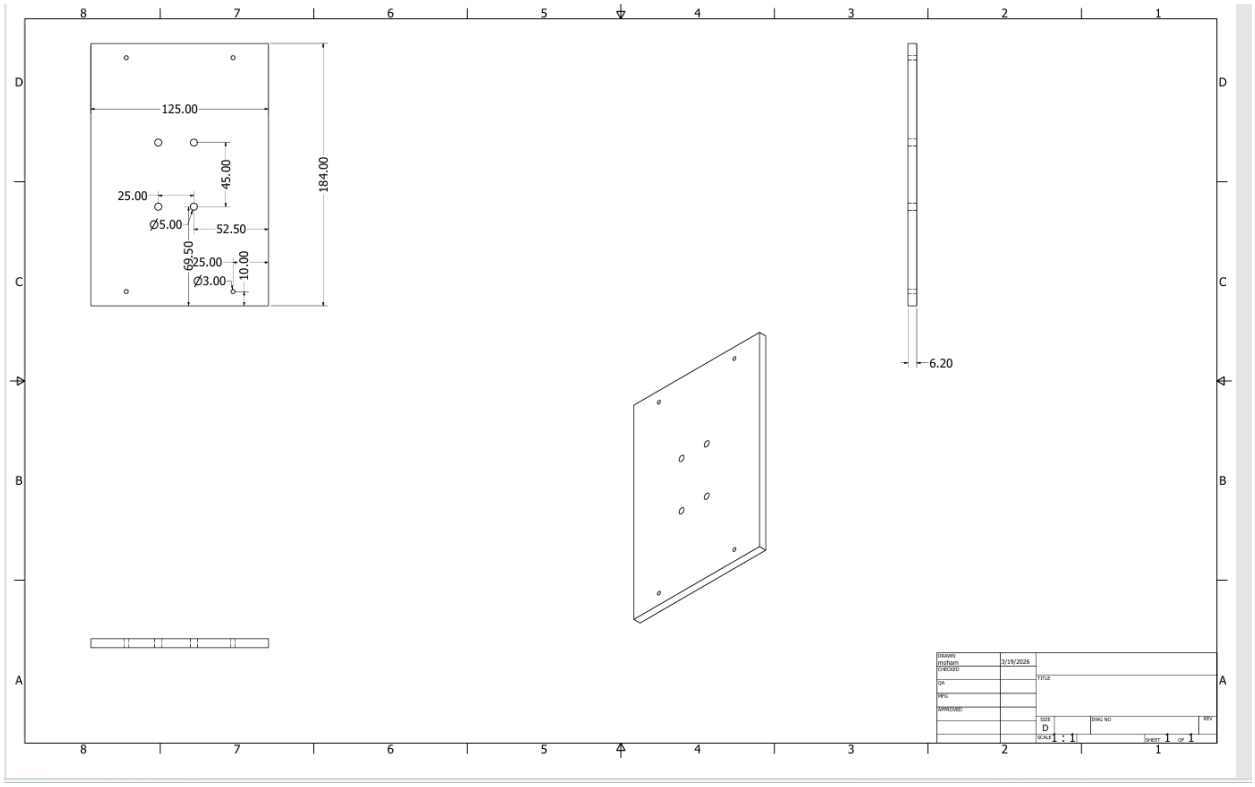


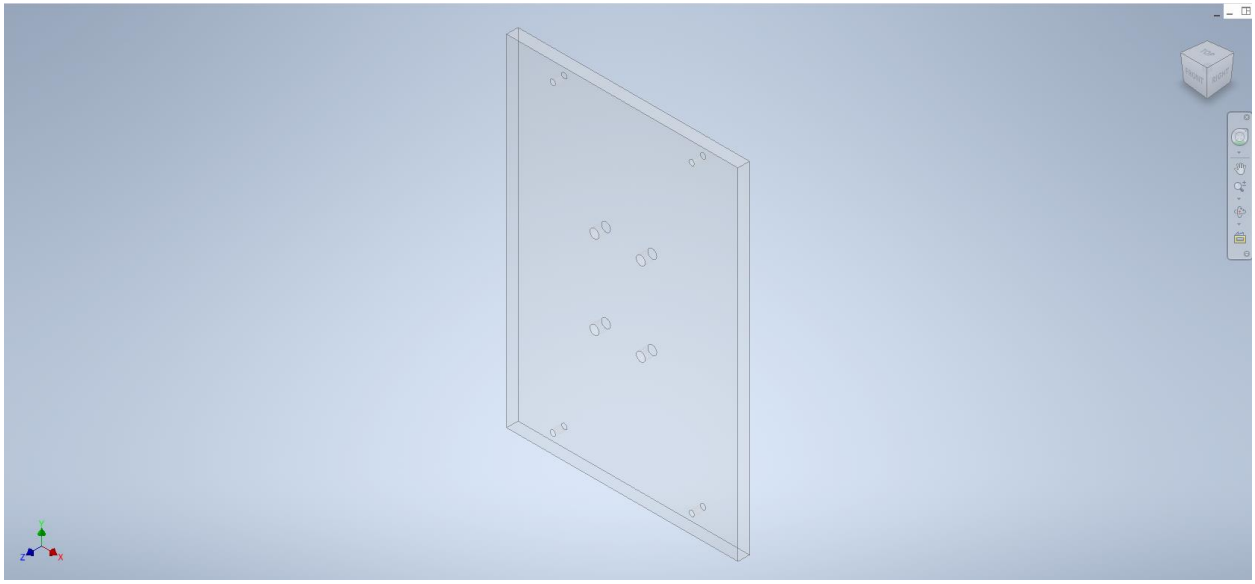

5.00
Ø22.00
Ø5.00
SCALE: 10 : 1
SHEET 1 OF 1

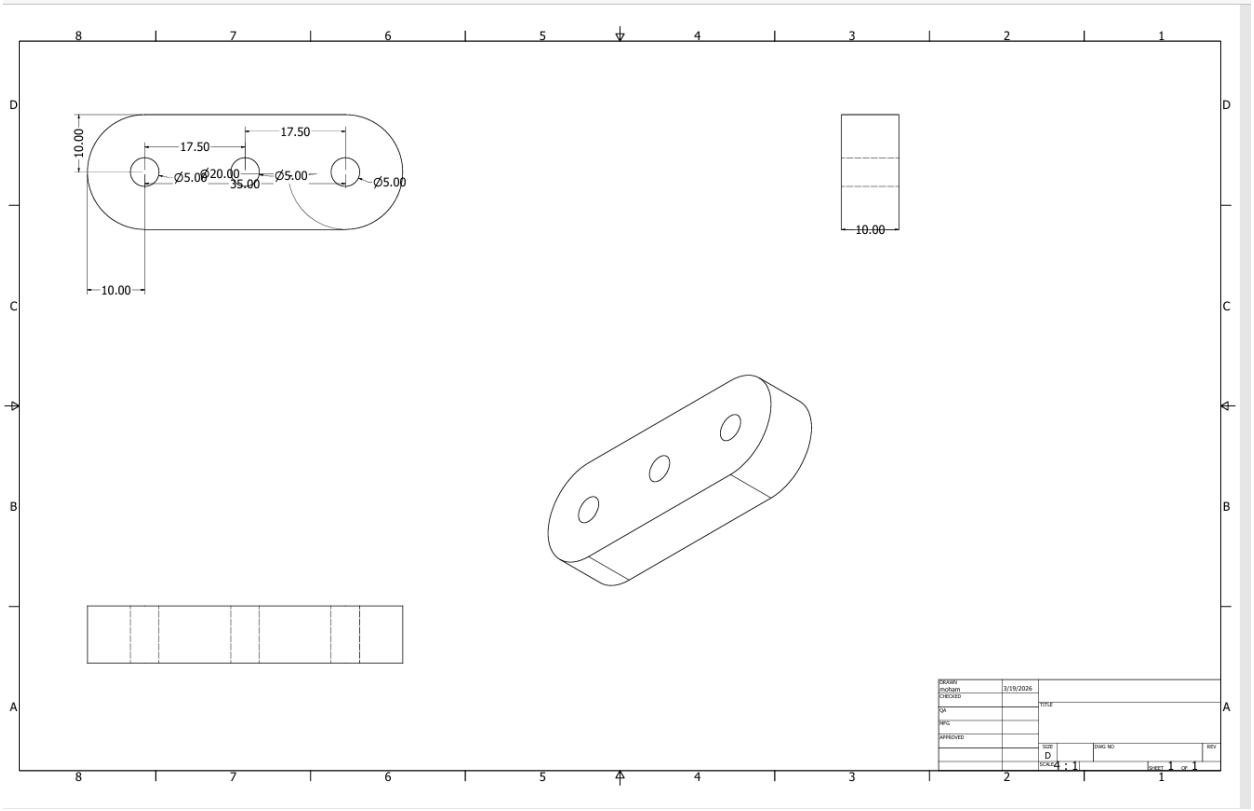


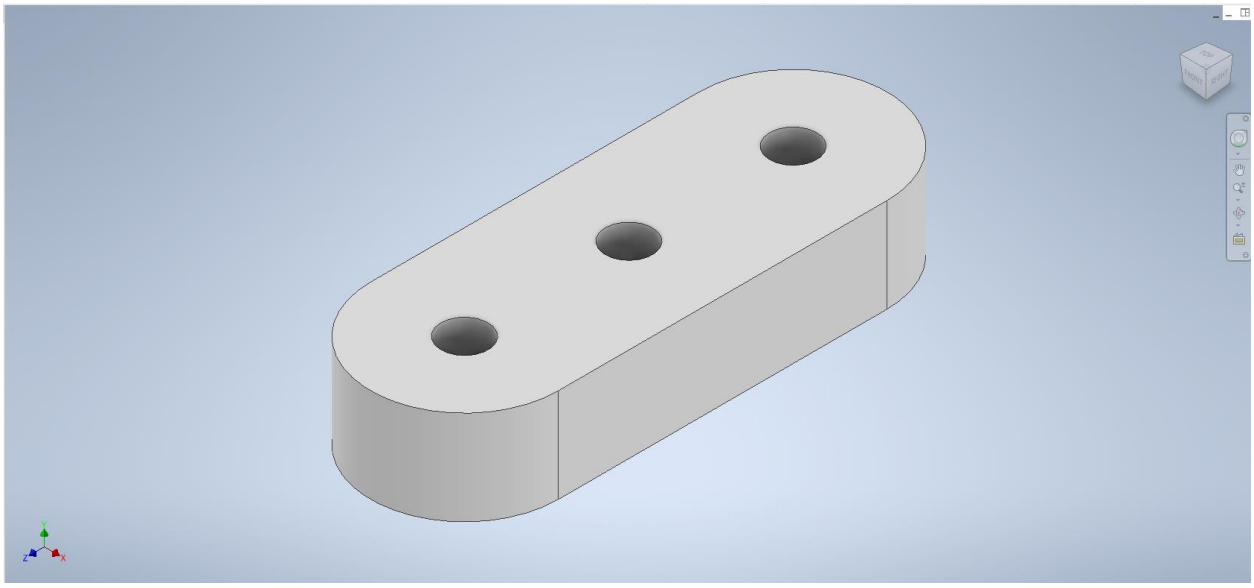

Ø26.00
R3.00
30.00
5.00
20.00
40.00
Ø20.00
R50.00
25.00
45.00
7.50
7.50
R10.00
Ø5.00
60.00
30.00
60.00

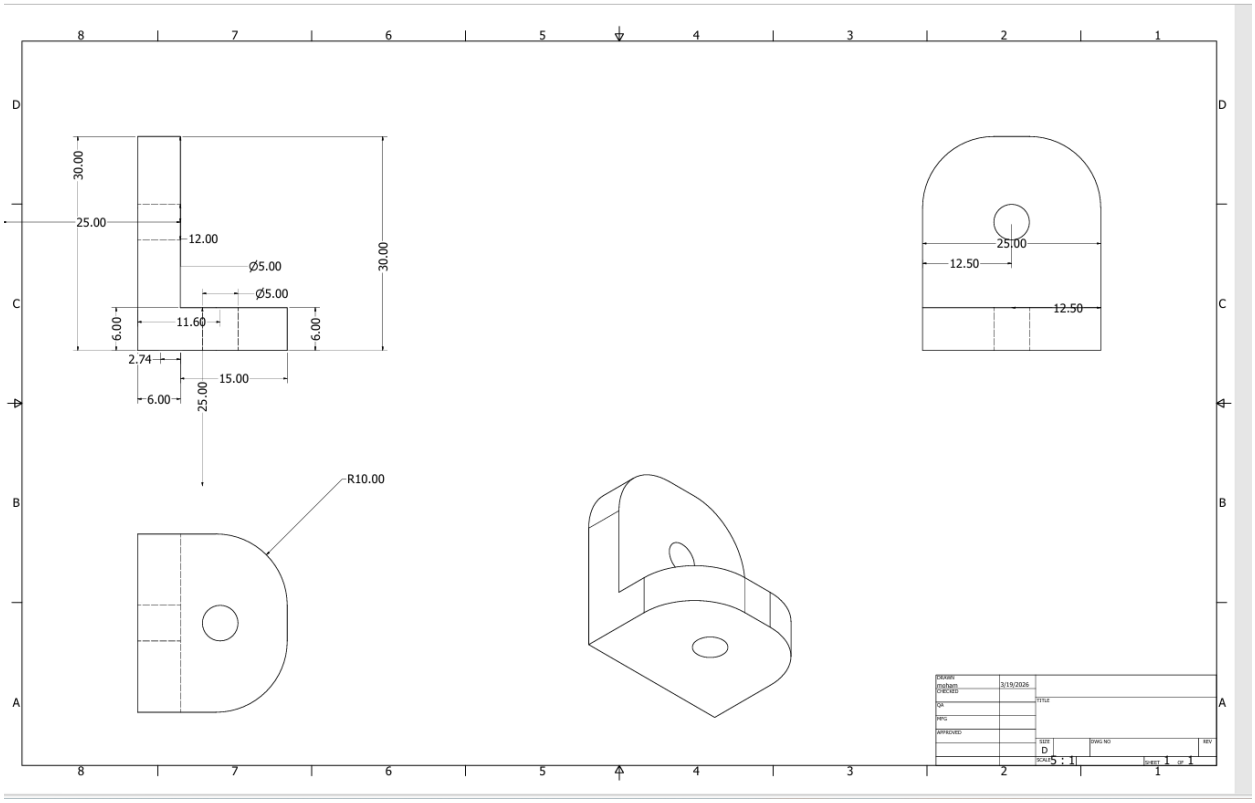


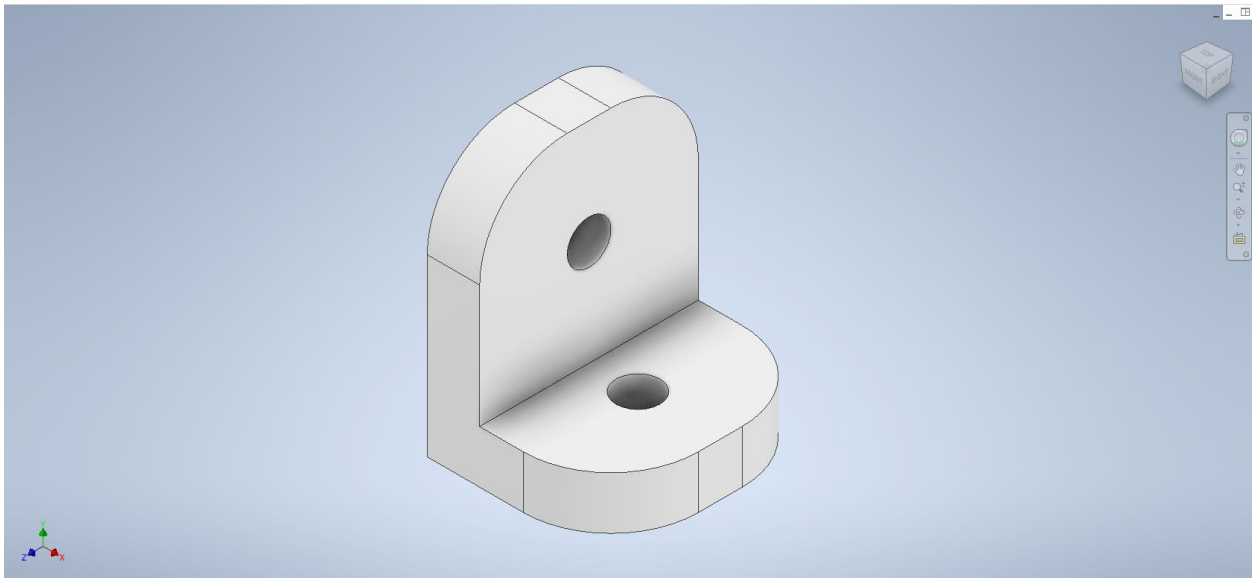

125.00
45.00
184.00
25.00
Ø5.00
52.50
69.50
25.00
10.00
Ø3.00
6.20
mohsen
3/19/2026
SCALE 1 : 1
SHEET 1 OF 1

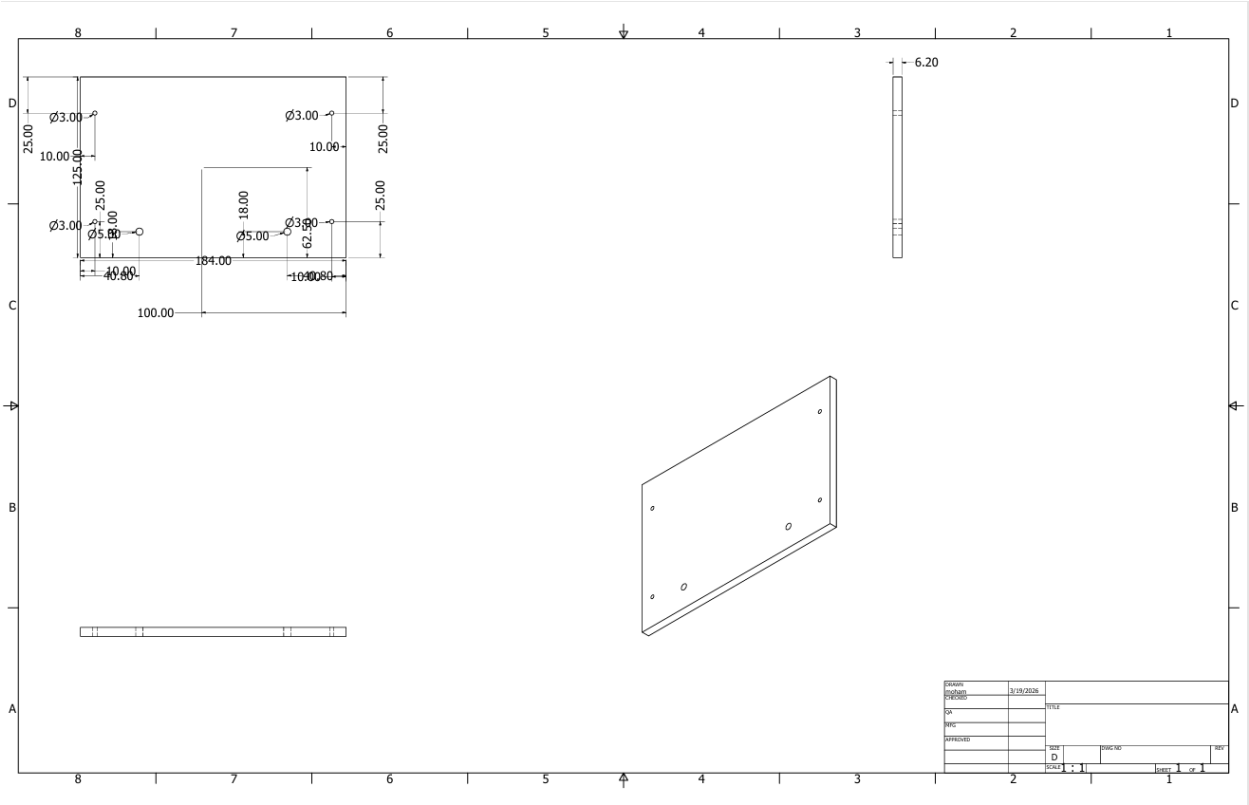


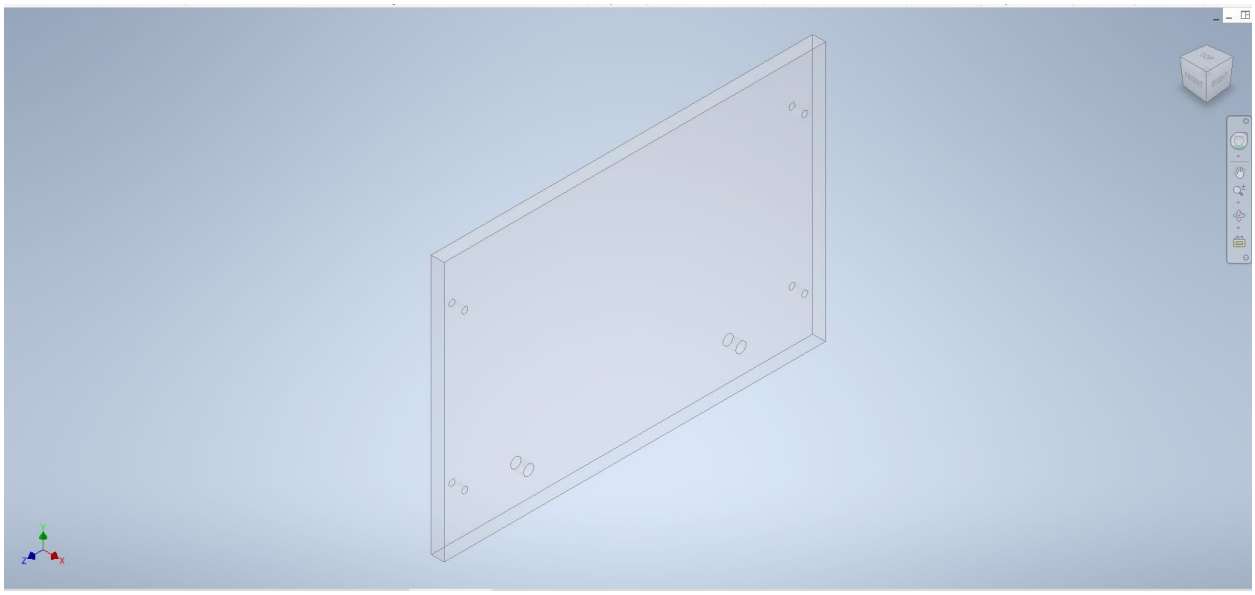

17.50
17.50
10.00
Ø5.00
Ø20.00
35.00
Ø5.00
Ø5.00
10.00
10.00
3/19/2026
1 : 1

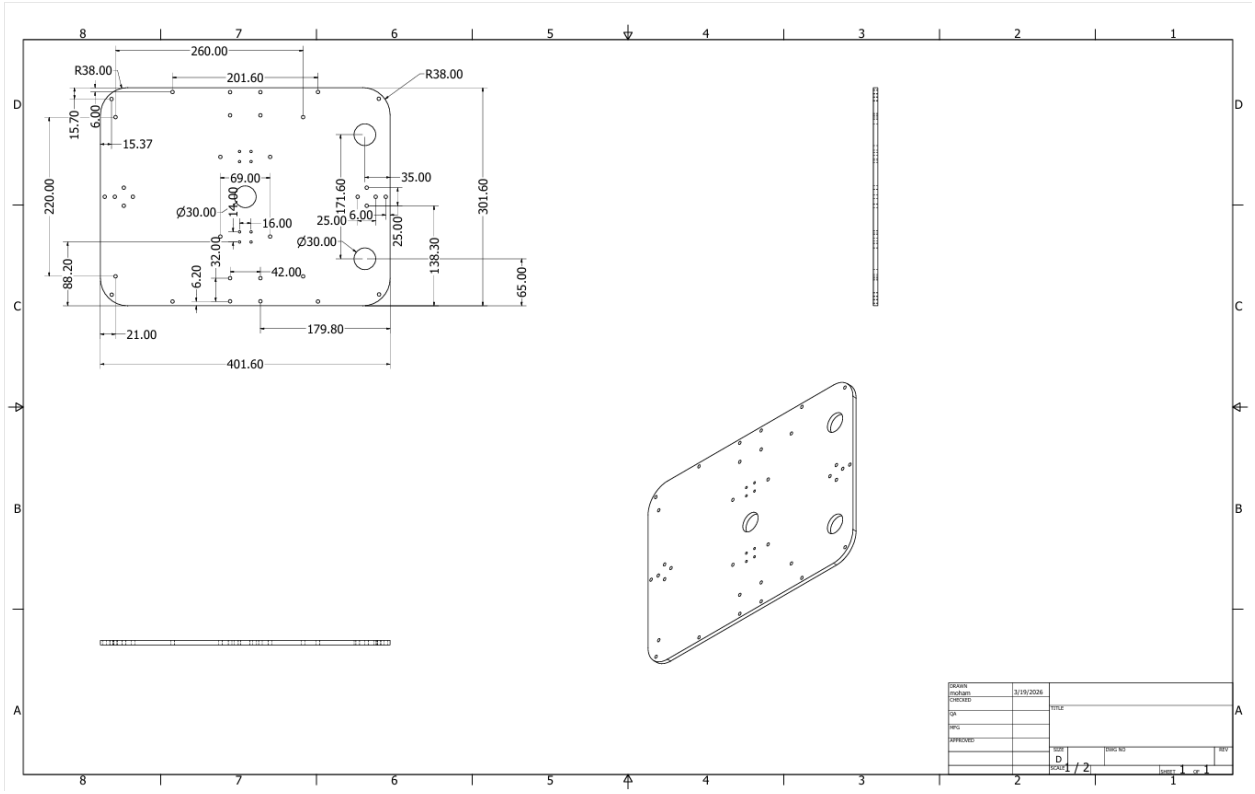


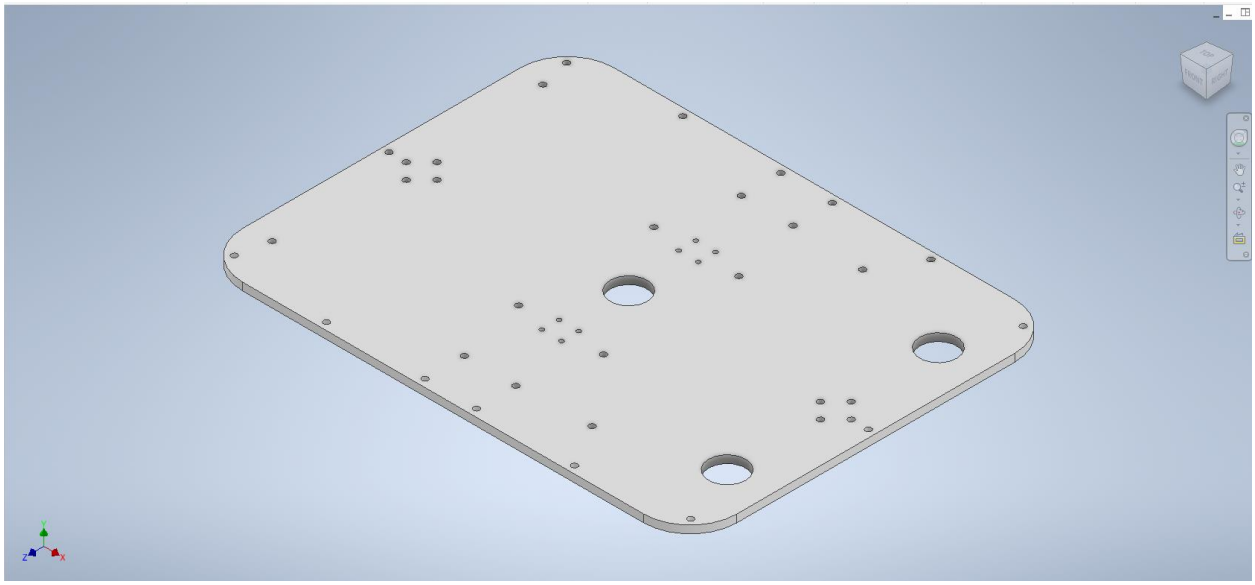

30.00
25.00
12.00
Ø5.00
Ø5.00
30.00
6.00
11.60
6.00
2.74
15.00
6.00
25.00
R10.00
25.00
12.50
12.50
3/19/2026
mahem
D
5 : 1

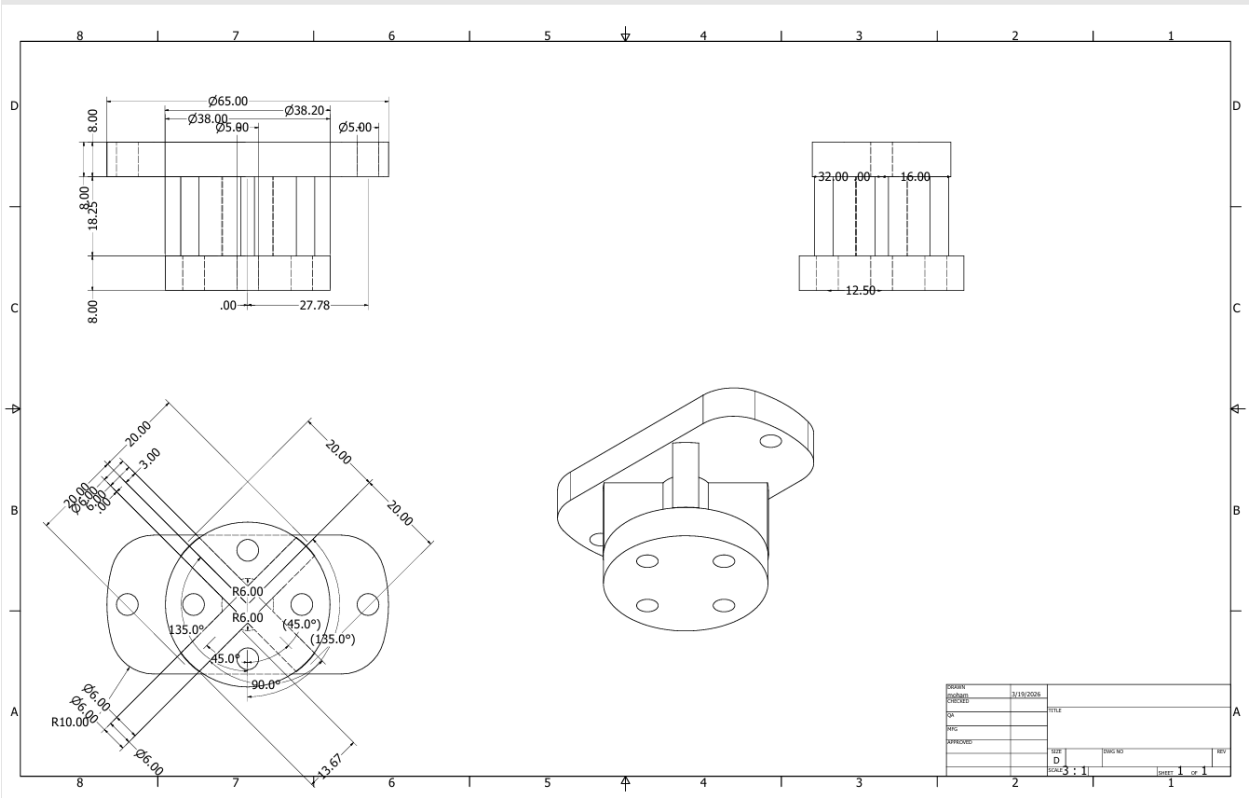


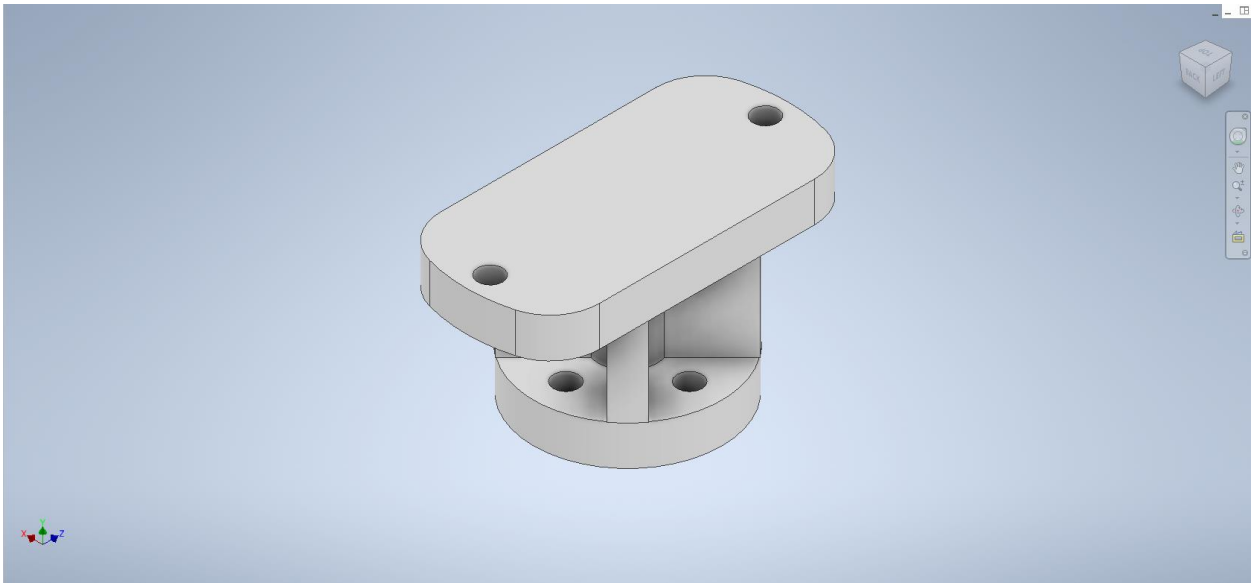

6.20
Ø3.00
Ø3.00
Ø3.00
Ø3.00
Ø5.00
Ø5.00
25.00
10.00
125.00
25.00
25.00
10.00
18.00
62.50
25.00
25.00
40.00
184.00
100.00
3/19/2026
D
1 : 1
1 OF 1

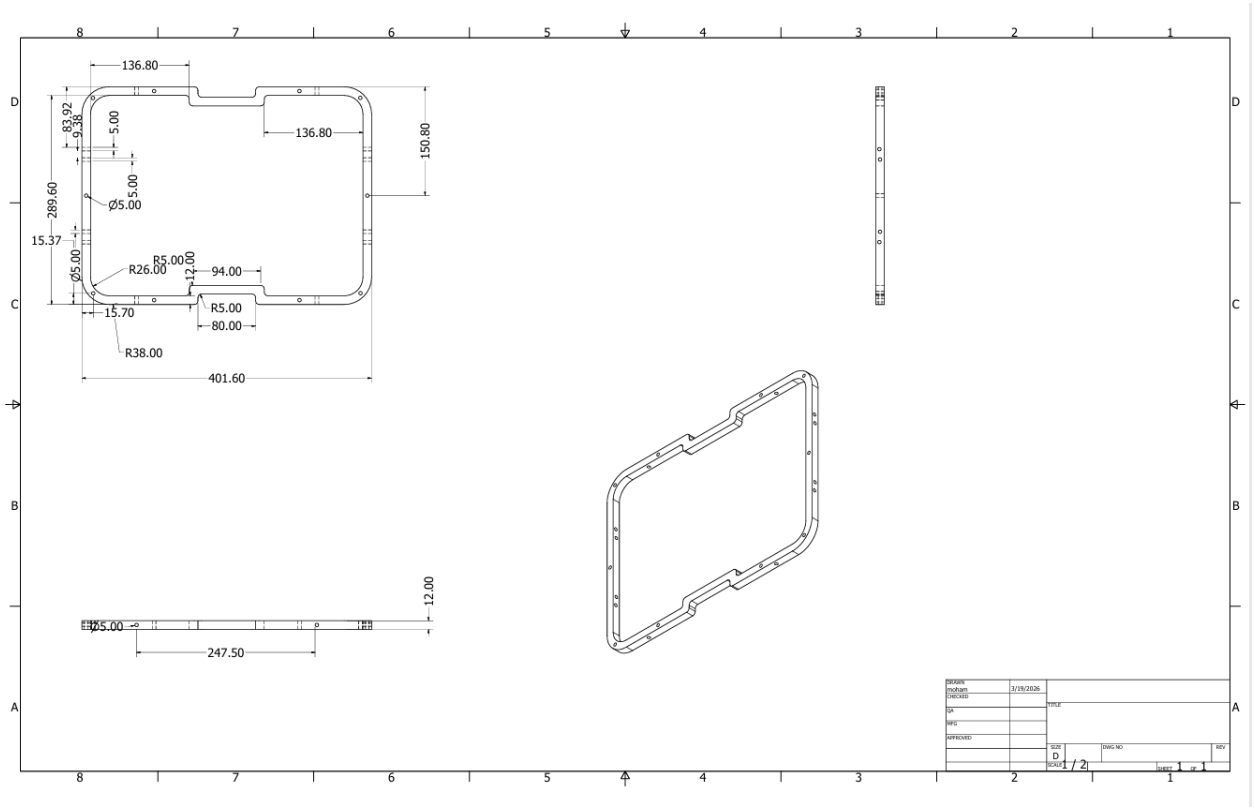


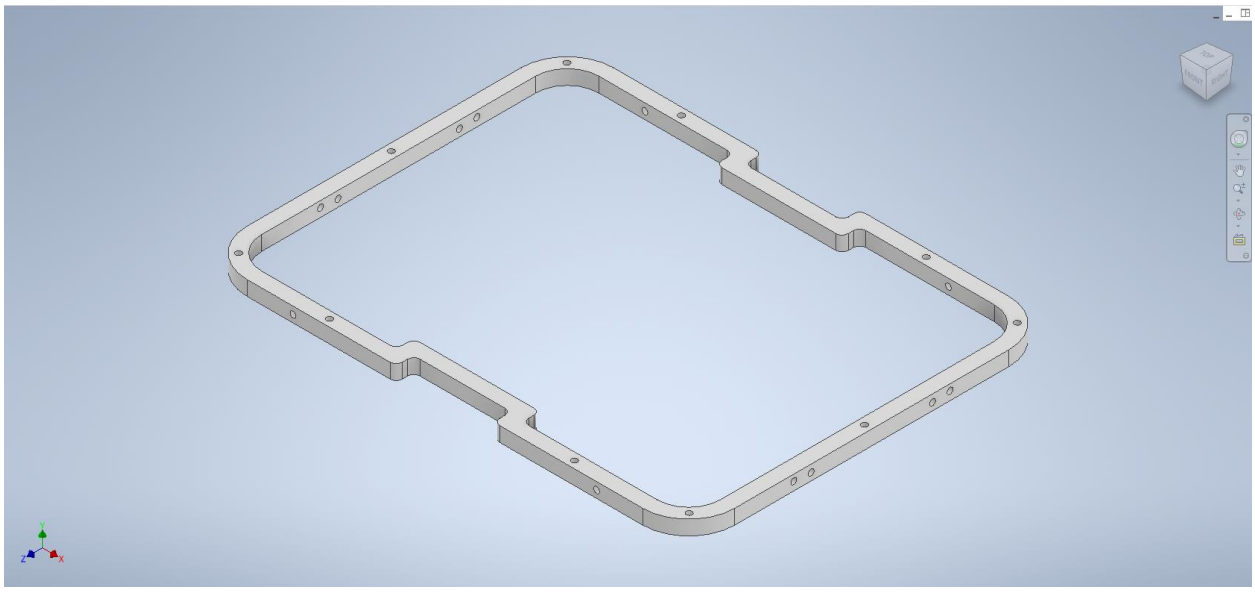

260.00
201.60
R38.00
R38.00
15.70
6.00
15.37
69.00
35.00
220.00
Ø30.00
14.00
16.00
171.60
25.00
6.00
301.60
Ø30.00
25.00
138.30
88.20
6.20
32.00
42.00
65.00
21.00
179.80
401.60
DRAWN
mohamn
3/19/2026
CHECKED
QA
MFG
APPROVED
TITLE
SIZE
D
DWG NO
REV
SCALE 1 / 2
SHEET 1 OF 1

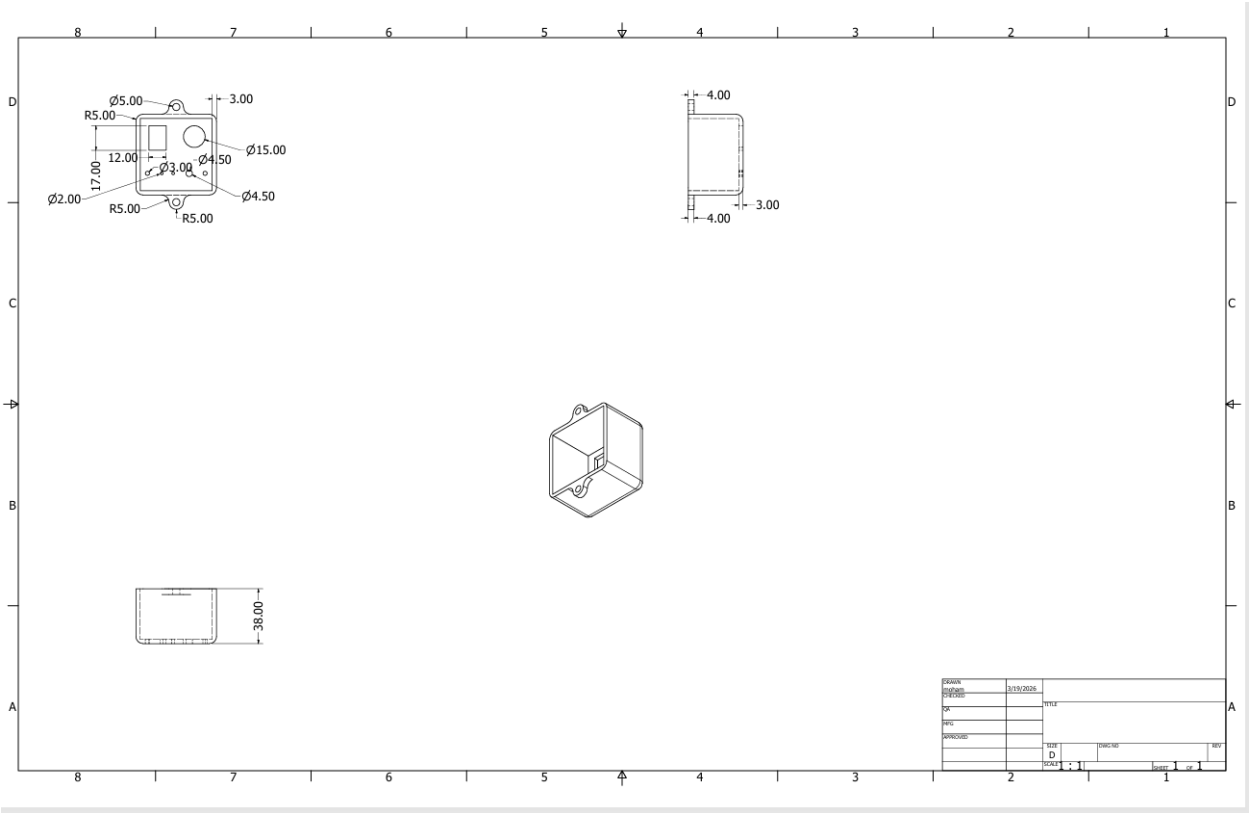


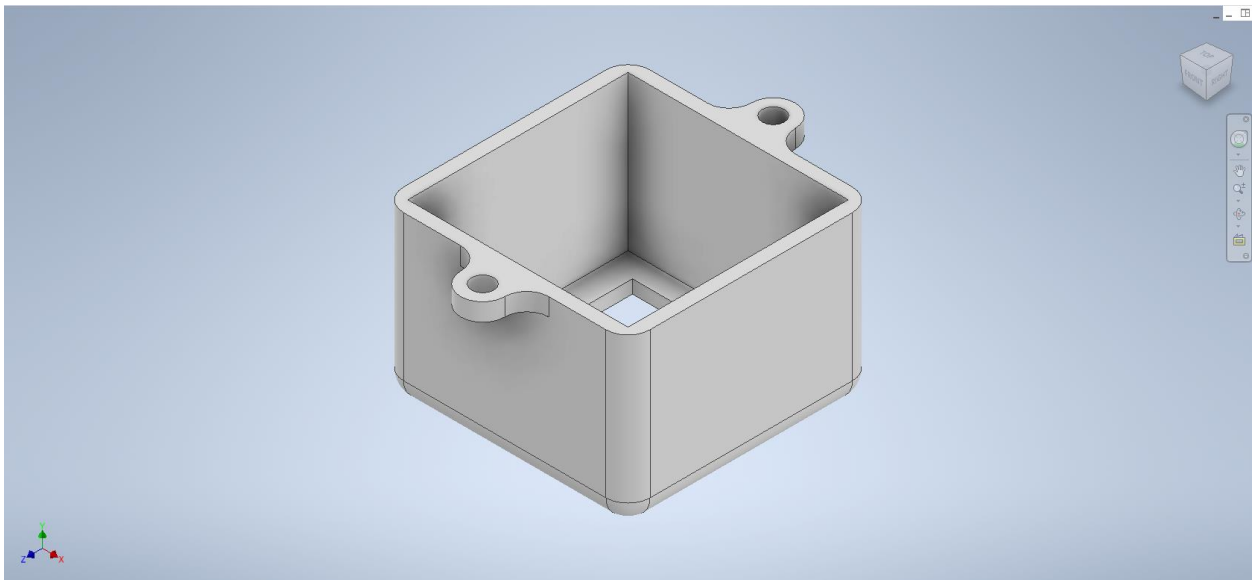

Ø65.00
Ø38.20
Ø38.00
Ø5.00
Ø5.00
8.00
8.00
18.25
8.00
.00
27.78
32.00
.00
16.00
12.50
20.00
3.00
20.00
20.00
20.00
R6.00
R6.00
135.0°
(45.0°)
(135.0°)
45.0°
90.0°
Ø6.00
Ø6.00
R10.00
Ø6.00
13.67
3 : 1
1 OF 1

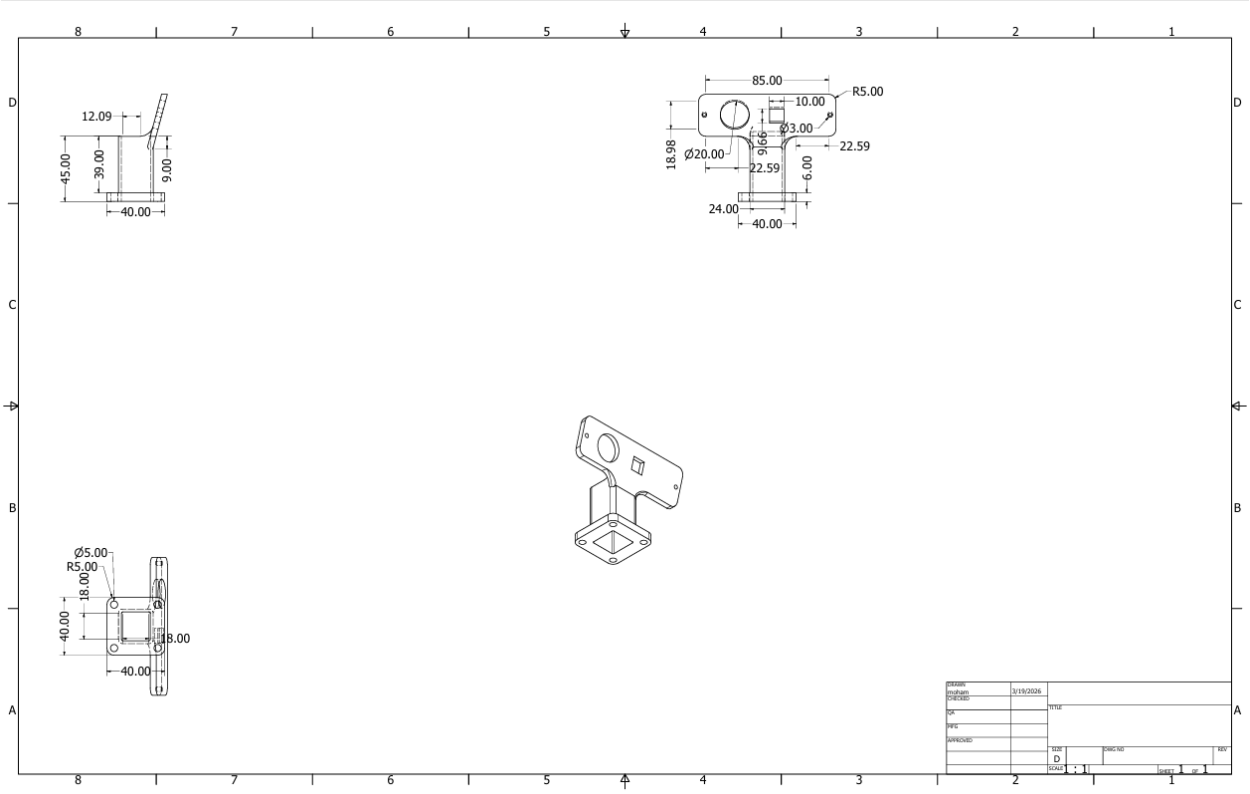


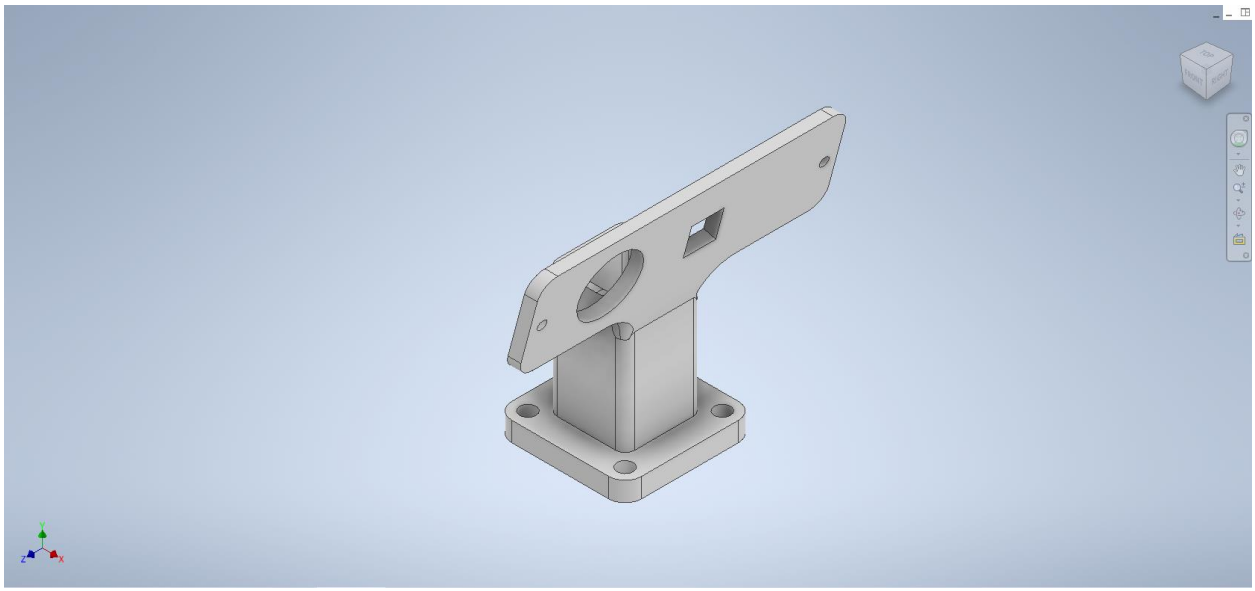

136.80
83.92
9.38
5.00
136.80
150.80
5.00
Ø5.00
289.60
15.37
Ø5.00
R26.00
R5.00
12.00
94.00
15.70
R5.00
80.00
R38.00
401.60
12.00
Ø5.00
247.50
3/19/2026
1 / 2

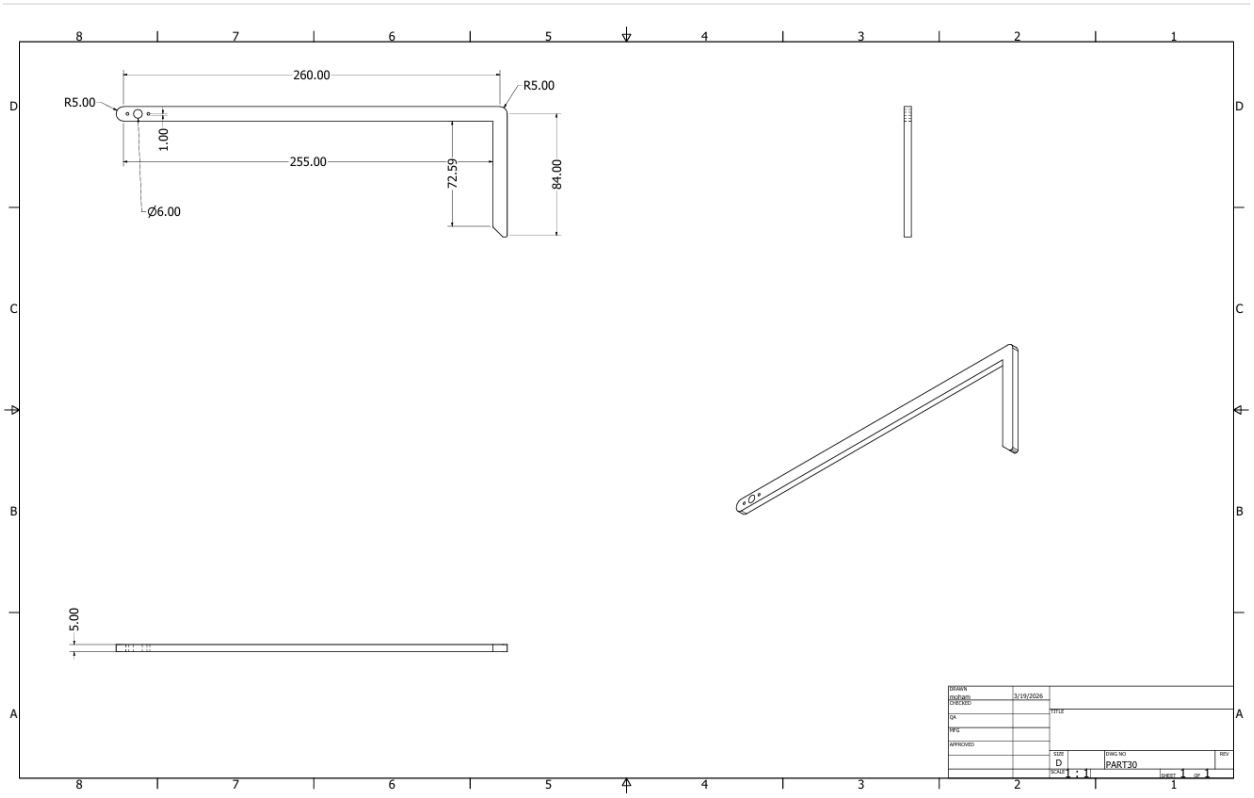


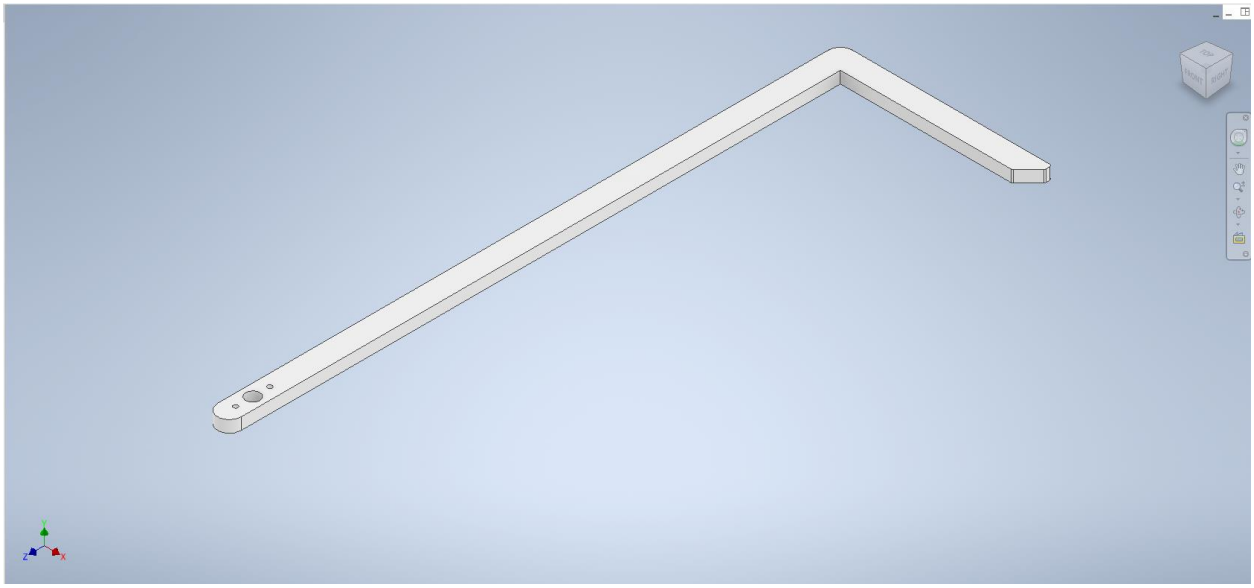

Ø5.00
3.00
R5.00
Ø15.00
12.00
Ø4.50
17.00
Ø3.00
Ø2.00
Ø4.50
R5.00
R5.00
4.00
3.00
4.00
38.00
3/19/2026

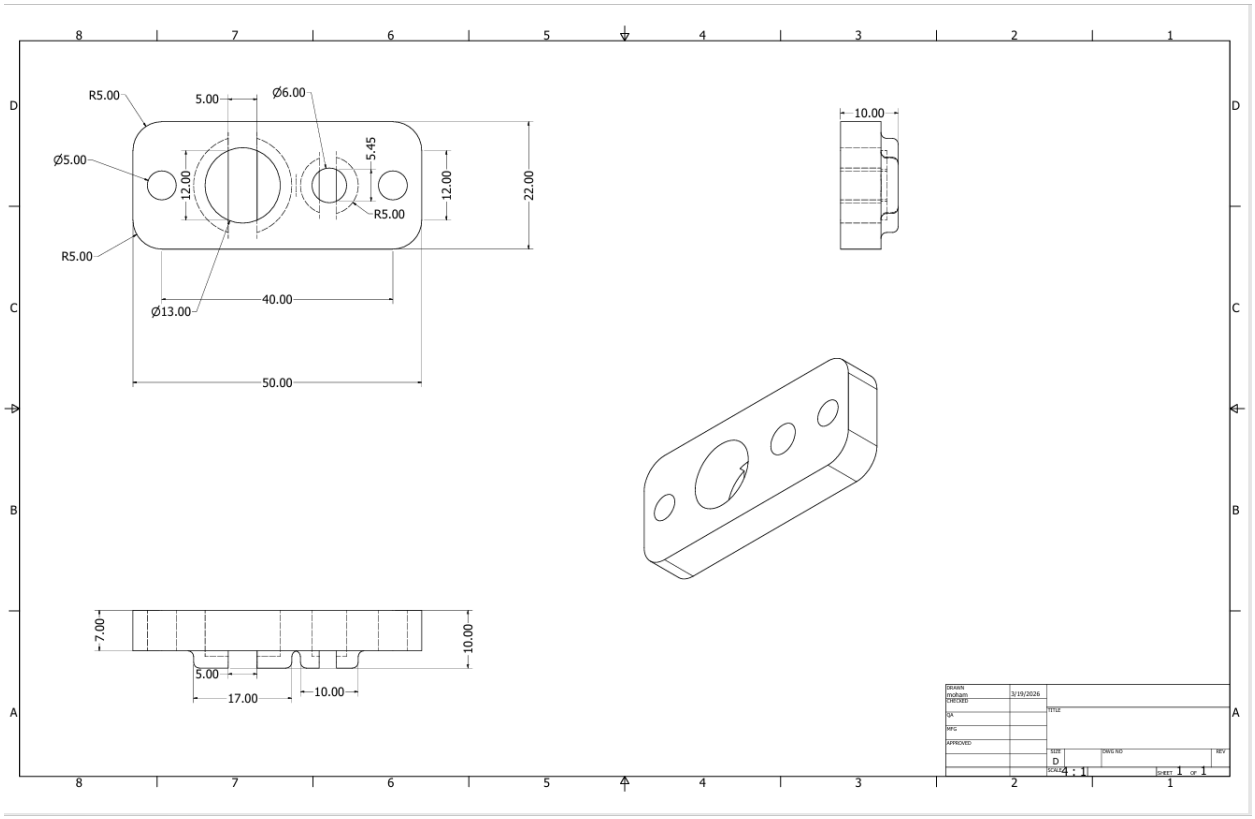


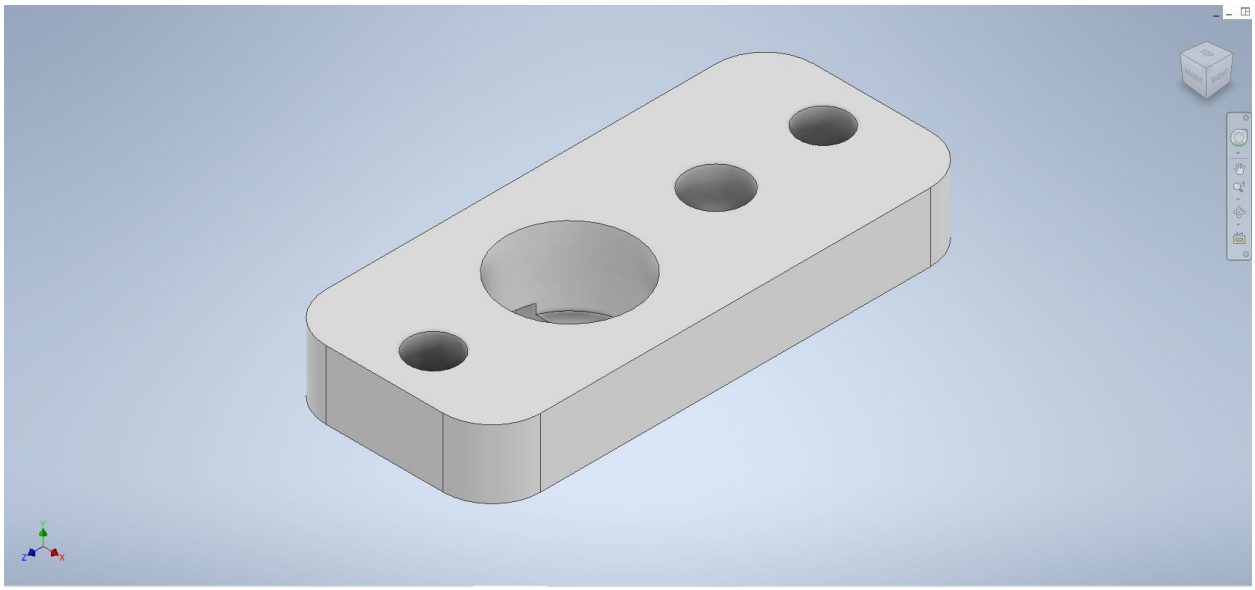

12.09
45.00
39.00
9.00
40.00
85.00
10.00
R5.00
Ø3.00
18.98
Ø20.00
9.66
22.59
22.59
6.00
24.00
40.00
Ø5.00
R5.00
18.00
40.00
18.00
40.00
3/19/2026
D
SCALE 1 : 1
SHEET 1 OF 1

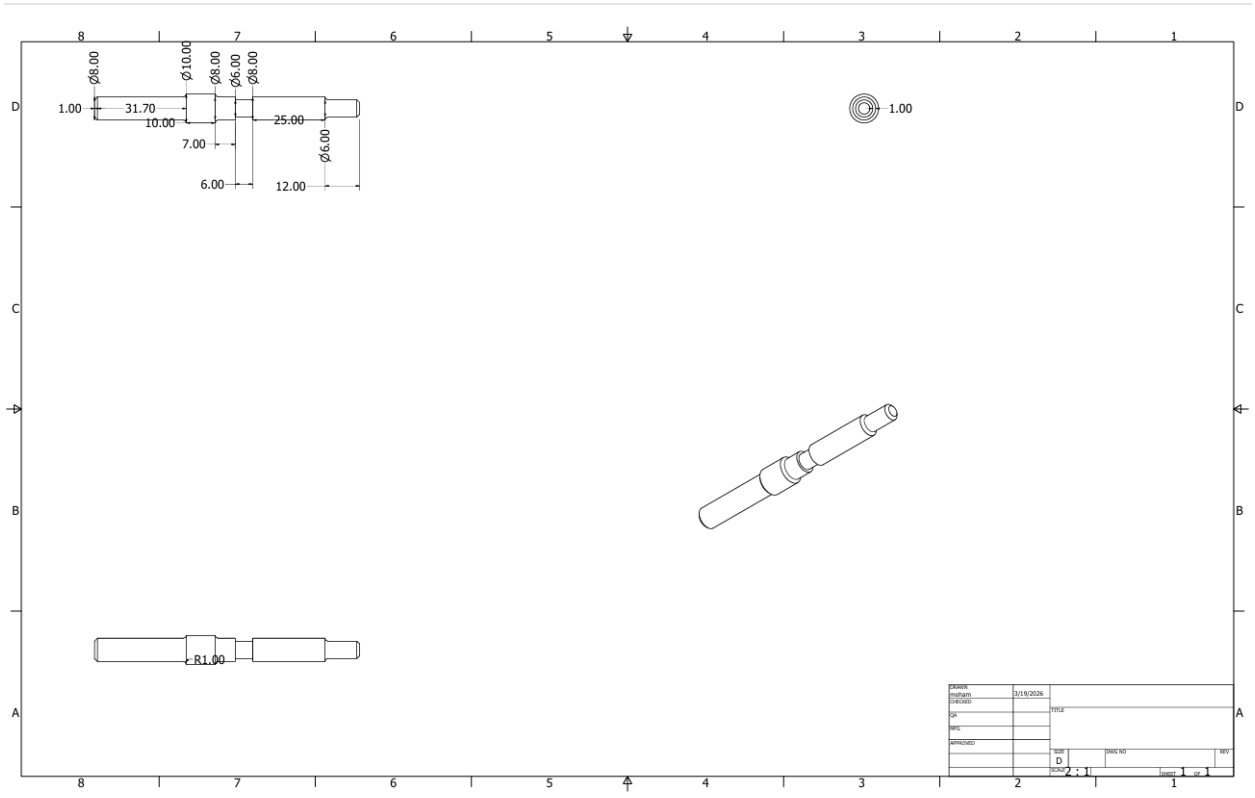


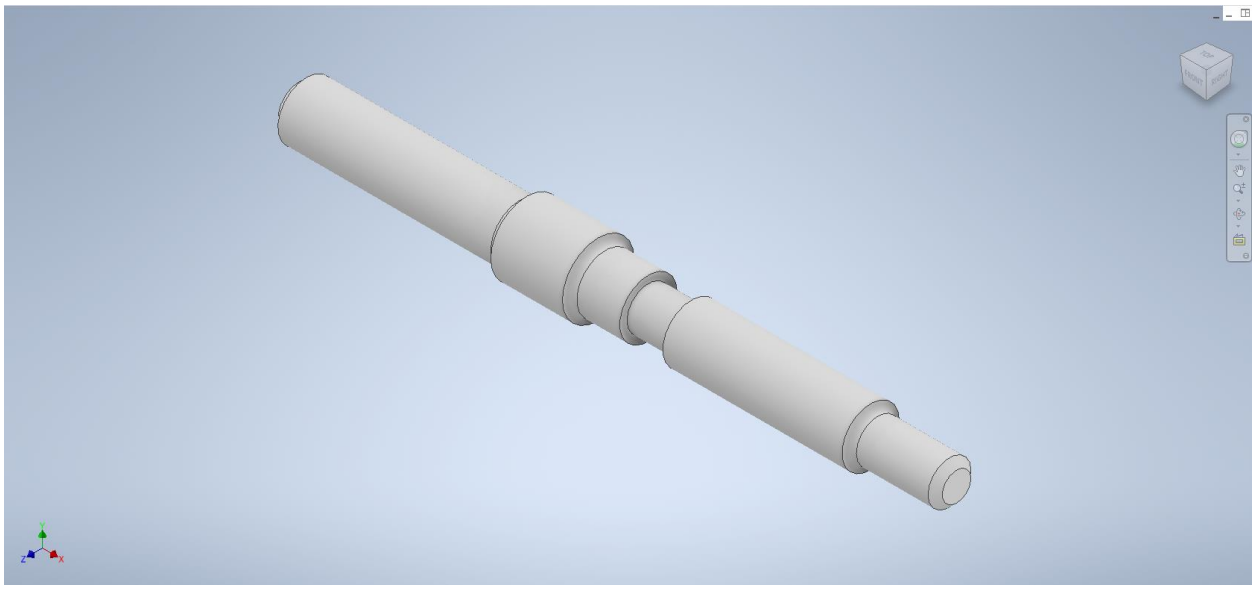

260.00
R5.00
R5.00
1.00
255.00
72.59
84.00
Ø6.00
5.00
PART30

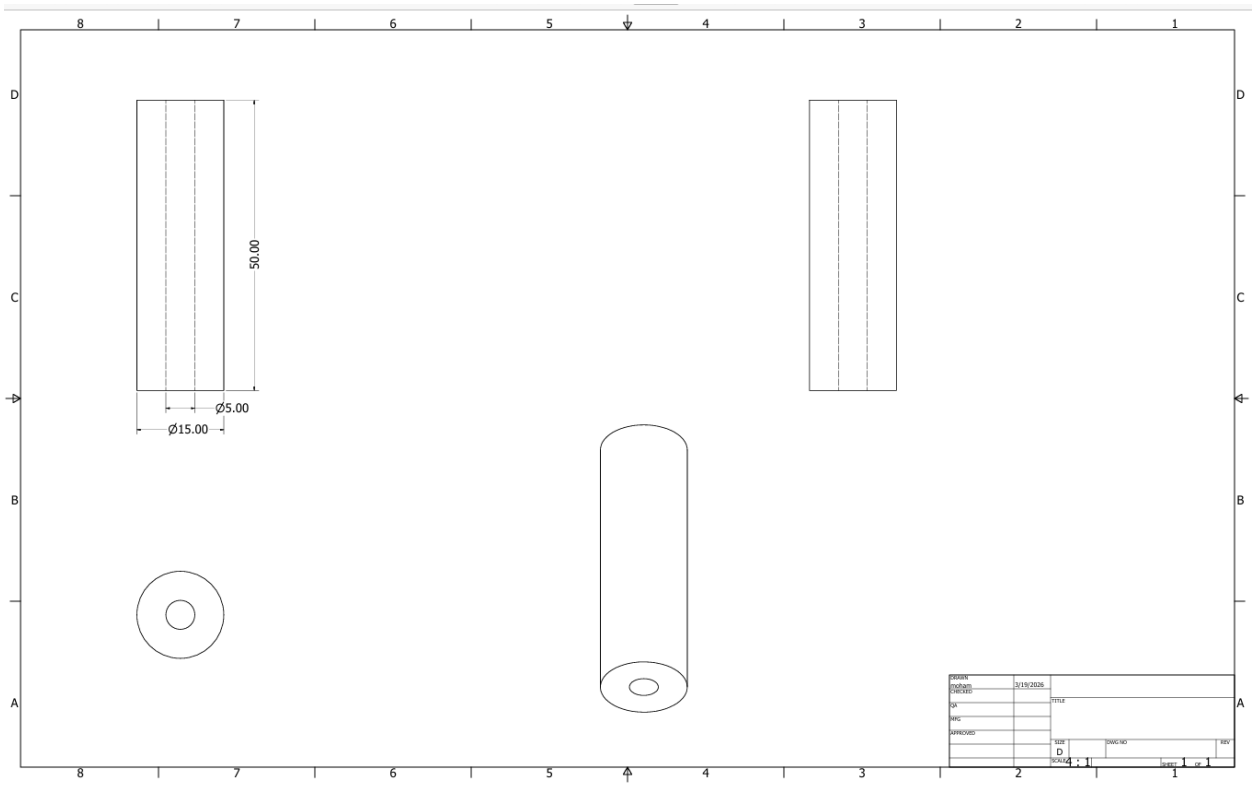


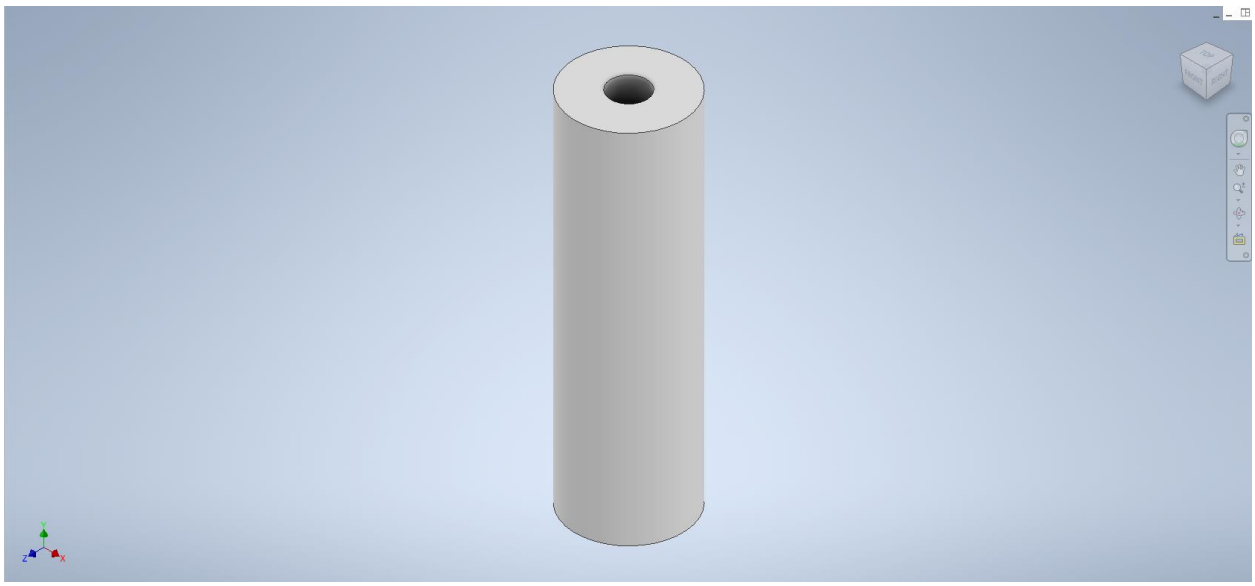

R5.00
5.00
Ø6.00
Ø5.00
12.00
5.45
12.00
22.00
R5.00
R5.00
40.00
Ø13.00
50.00
10.00
7.00
10.00
5.00
17.00
10.00
4 : 1

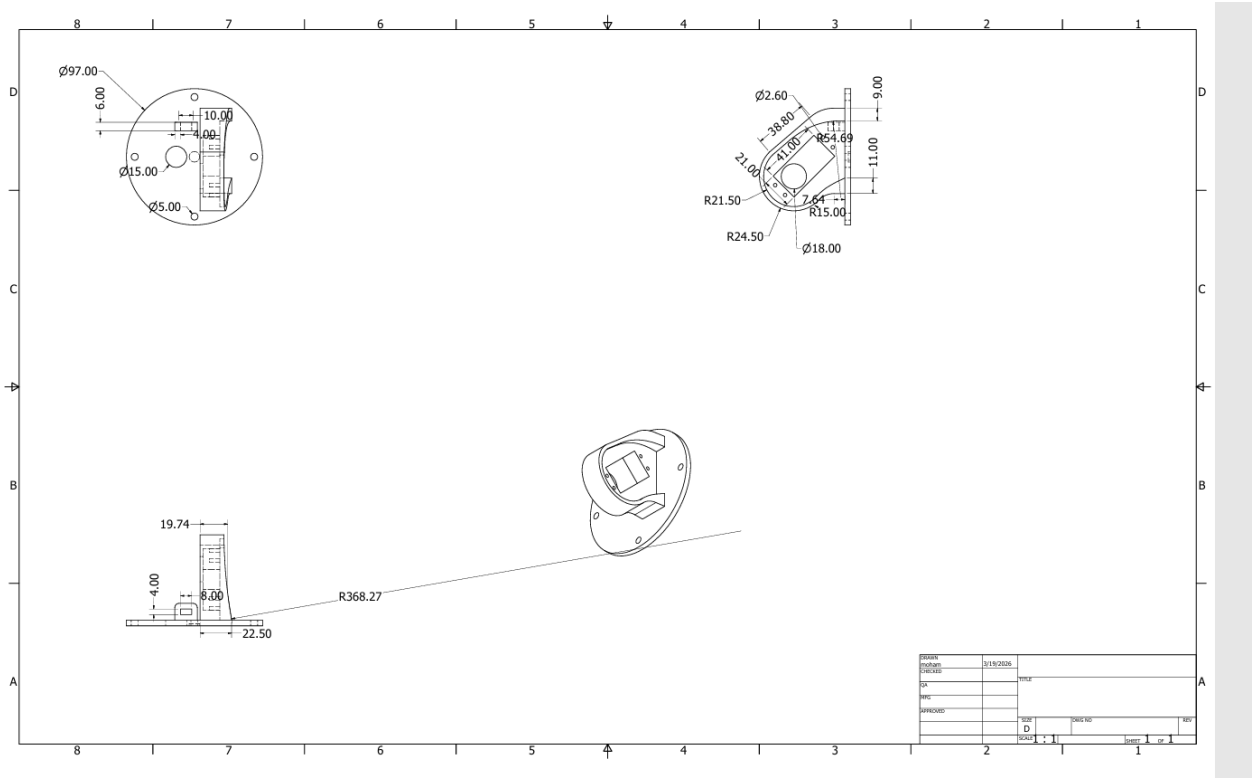


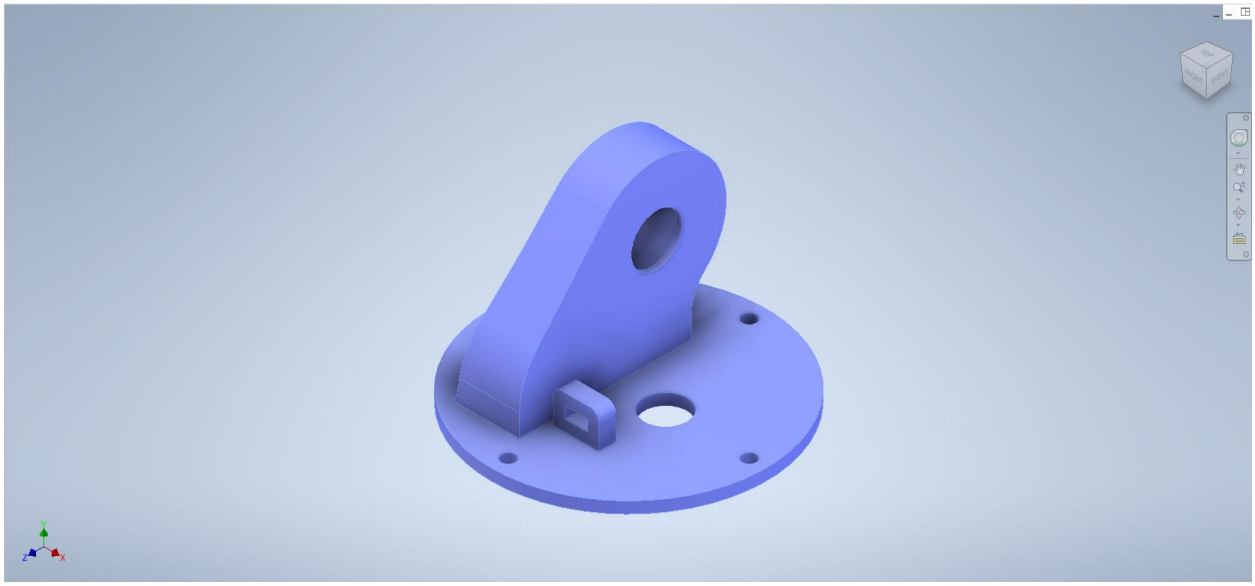

Ø8.00
Ø10.00
Ø8.00
Ø6.00
Ø8.00
1.00
31.70
10.00
25.00
7.00
6.00
12.00
Ø6.00
1.00
R1.00
3/19/2026
D
2 : 1
SHEET 1 OF 1



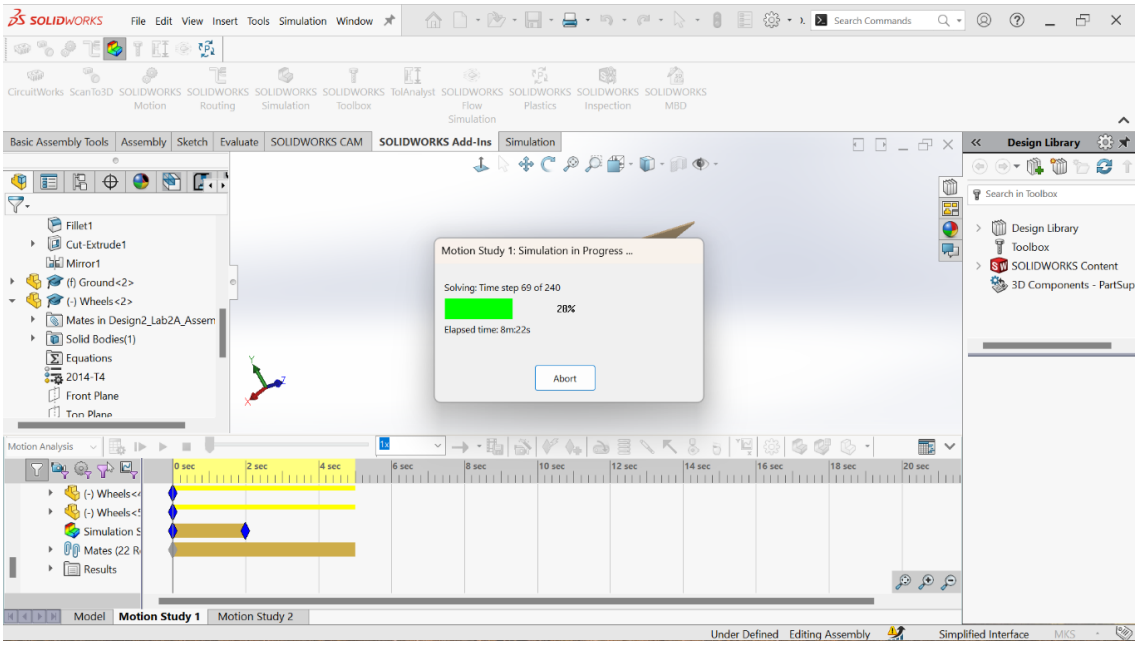

50.00
Ø5.00
Ø15.00
moham
3/19/2026
SIZE
D
SCALE
4 : 1
SHEET 1 OF 1




Ø97.00
6.00
10.00
4.00
Ø15.00
Ø5.00
Ø2.60
9.00
38.80
41.00
R54.69
21.00
11.00
R21.50
7.64
R15.00
R24.50
Ø18.00
19.74
4.00
8.00
R368.27
22.50



1. Actuator Sizing Using Solidworks



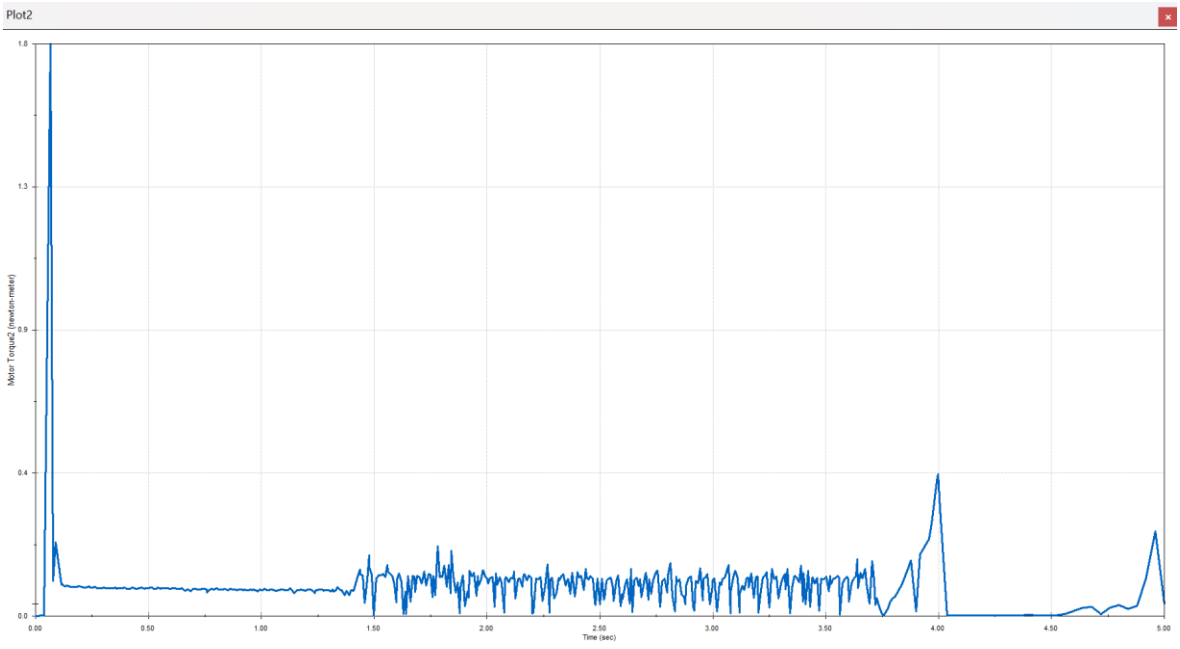
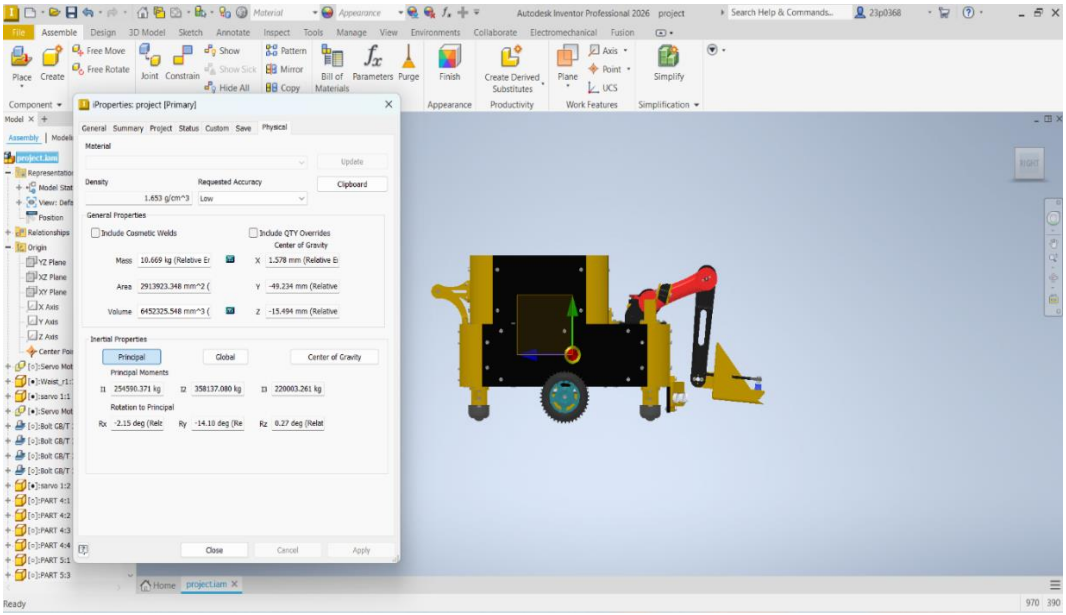
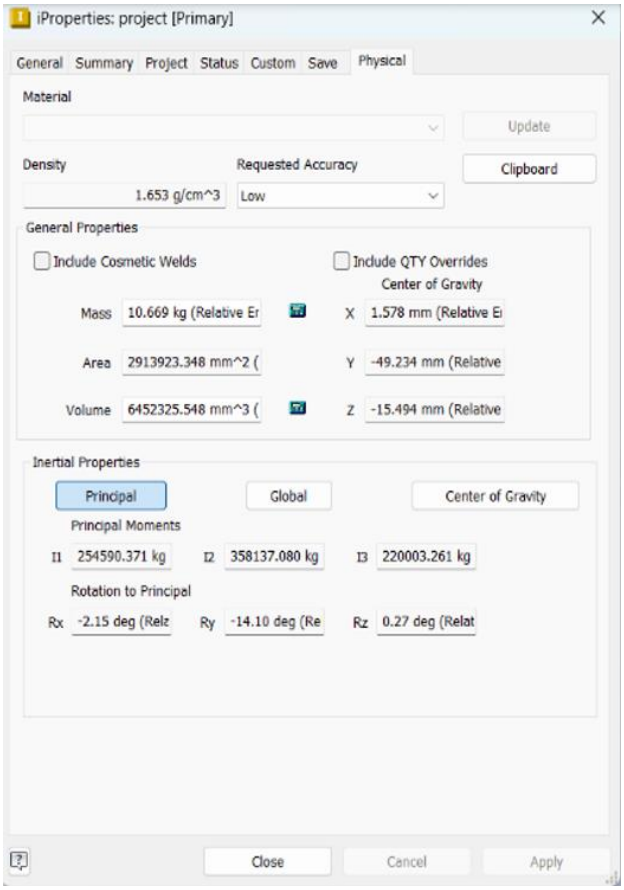


Figure 9 torque-time graph

2. Robot Dynamics and Stability Calculations





a) Stability in the Longitudinal Direction:

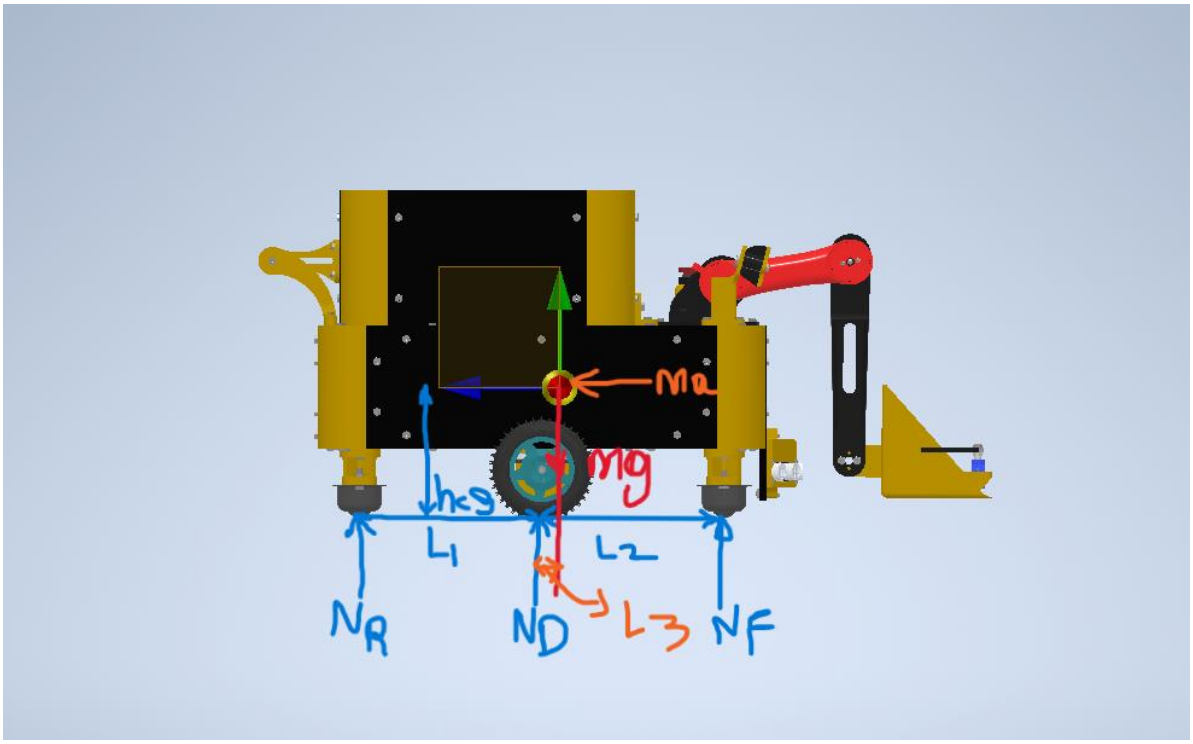


Figure 10 Stability in the Longitudinal Direction

Geometric Parameters and Physical Constants

The physical configuration of the mobile robot is defined by its mass distribution and the spatial arrangement of its components. These parameters are critical for determining the center of gravity (CG) and the resulting moments during motion.

- **Total Mass (m): 10.669 kg**
- **Distance (Rear to Drive wheel) (L1): 168.3 mm**
- **Distance (Drive to Front wheel) (L2): 168.3 mm**
- **Horizontal Offset of CG (L3): 15.494 mm**
- **Vertical Chassis Height (L5): 177.6 mm**
- **Vertical distance to base (L4): 49.234 mm**

Calculated Height of Center of Gravity (hcg):

$$h_{\{cg\}} = L_5 - L_4 = 177.6 - 49.234 = 128.366\, mm$$

b) Dynamic Force Balance (Moment Equilibrium)

To evaluate the stability of the robot under acceleration (a), we apply the principles of dynamic equilibrium. By taking the moments about the contact points of the wheels, we derive the following equations for the normal reaction forces (NR, ND, and NF):

Equation 3.1: Sum of Moments about the Rear Wheels

$$N_D \cdot L_1 + N_F \cdot (L_1 + L_2) = mg \cdot (L_1 + L_3) - ma \cdot h_{\{cg\}}$$

Equation 3.2: Sum of Moments about the Front Wheels

$$N_D \cdot L_2 + N_R \cdot (L_1 + L_2) = mg \cdot (L_2 - L_3) + ma \cdot h_{\{cg\}}$$

c) Stability Limits and Tip-over Conditions

The stability threshold is reached when the robot is on the verge of losing contact with the ground. This occurs when the normal reaction forces at certain wheels drop to zero.

Maximum Backward Acceleration ($amax$) The backward tip-over limit is reached when the normal forces at the drive and front wheels become zero (ND, NF = 0):

$$a = g * \frac{l1 + l3}{hcg} = 9.81 * \frac{168.3 + 15.494}{128.366} \approx 14.049 \ {}^{m}/_{s^2}$$

Maximum Forward Deceleration The forward tip-over limit during braking is reached when the normal forces at the drive and rear wheels become zero (ND, NR = 0):

$$a_{dec} = -g * \frac{l2 - l3}{hcg} = -9.81 * \frac{168.3 - 15.494}{128.366} \approx -11.677 \ {}^{m}/_{s^2}$$

$$\therefore a(acc) = 14.049 \ {}^{m}/_{s^2}$$

$$\therefore a(dec) = -11.677 \ {}^{m}/_{s^2}$$

d) Stability in the Lateral Direction and Roll-over Analysis:

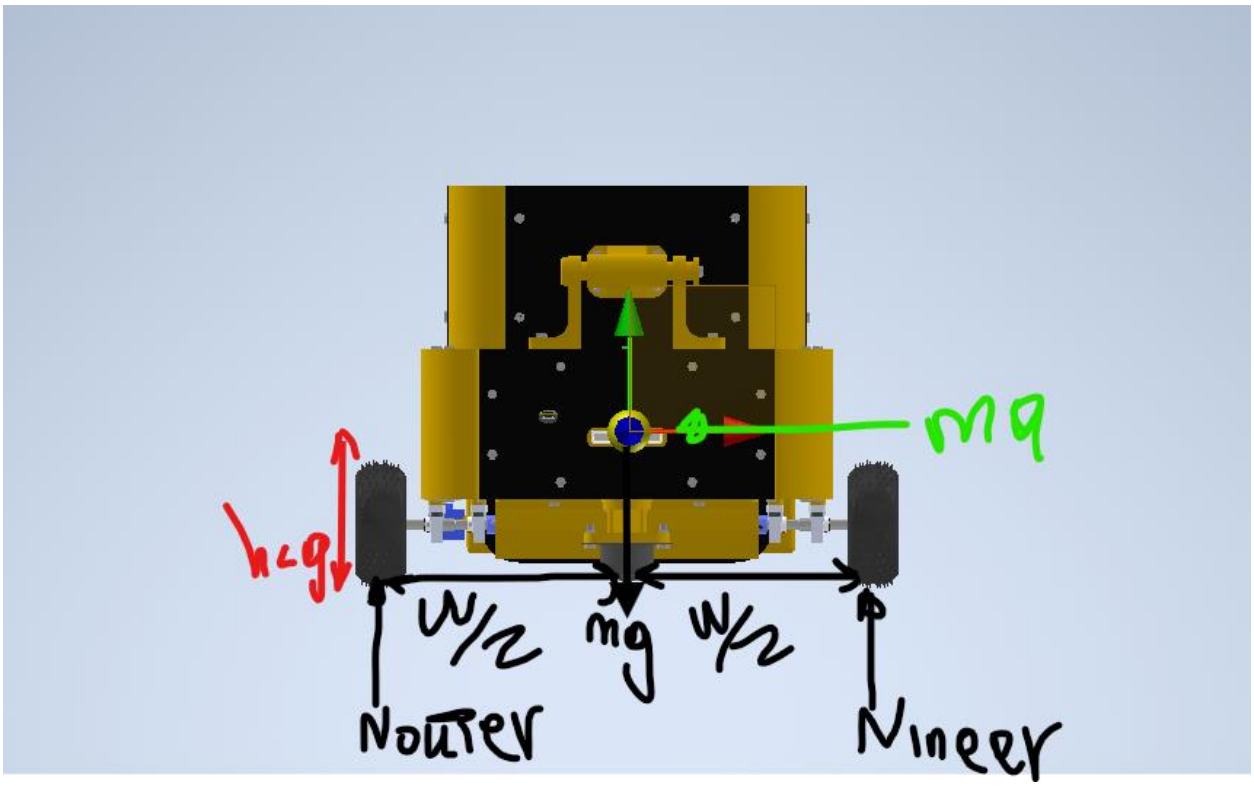


Figure 11 stability in the Lateral Direction

Static Weight Distribution (Stationary)

When the robot is stationary on flat ground, the weight is distributed equally between the inner and outer wheels:
w=374.556 mm
hcg=128.366 mm

Weight Transfer in the Lateral Direction

Taking the moment about the outer Wheel

$$N(ineerr) = \frac{m * g}{2} - \frac{m * v^2 * hcg}{w * R}$$

Longitudinal Roll-over Condition

$$N(ineerr) = \frac{m * g}{2} - \frac{m * v^2 * hcg}{w * R} = 0$$

$$V = \sqrt{\frac{g * w * R}{2 * hcg}} = \sqrt{\frac{9.81 * 374.556 * R}{2 * 128.366}}$$

$$\therefore V = \sqrt{14.31 * R}$$

Table 4 radius to safe velocity

Turning Radius (R) [m]	Max Safe Velocity (V) [m/s]
0.1	1.196
0.15	1.465
0.2	1.69
0.25	1.89
0.3	2.07
0.35	2.237
0.4	2.39
0.45	2.537
0.5	2.67
0.55	2.8
0.6	2.93

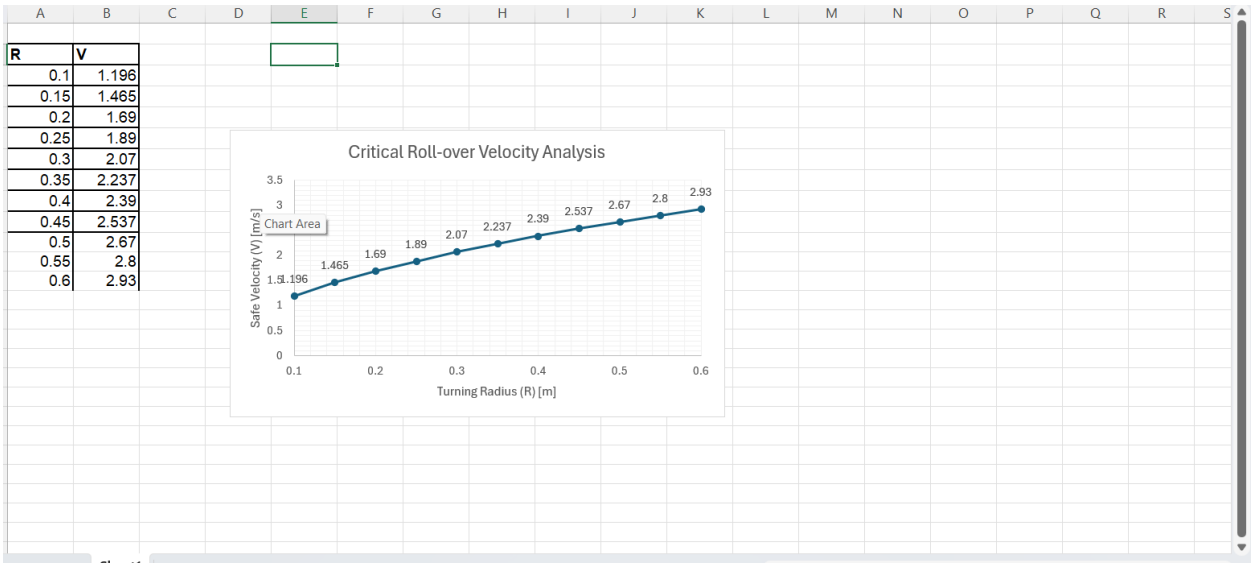


Figure 12 the corresponding velocity curves

Extra Mechanism Analysis

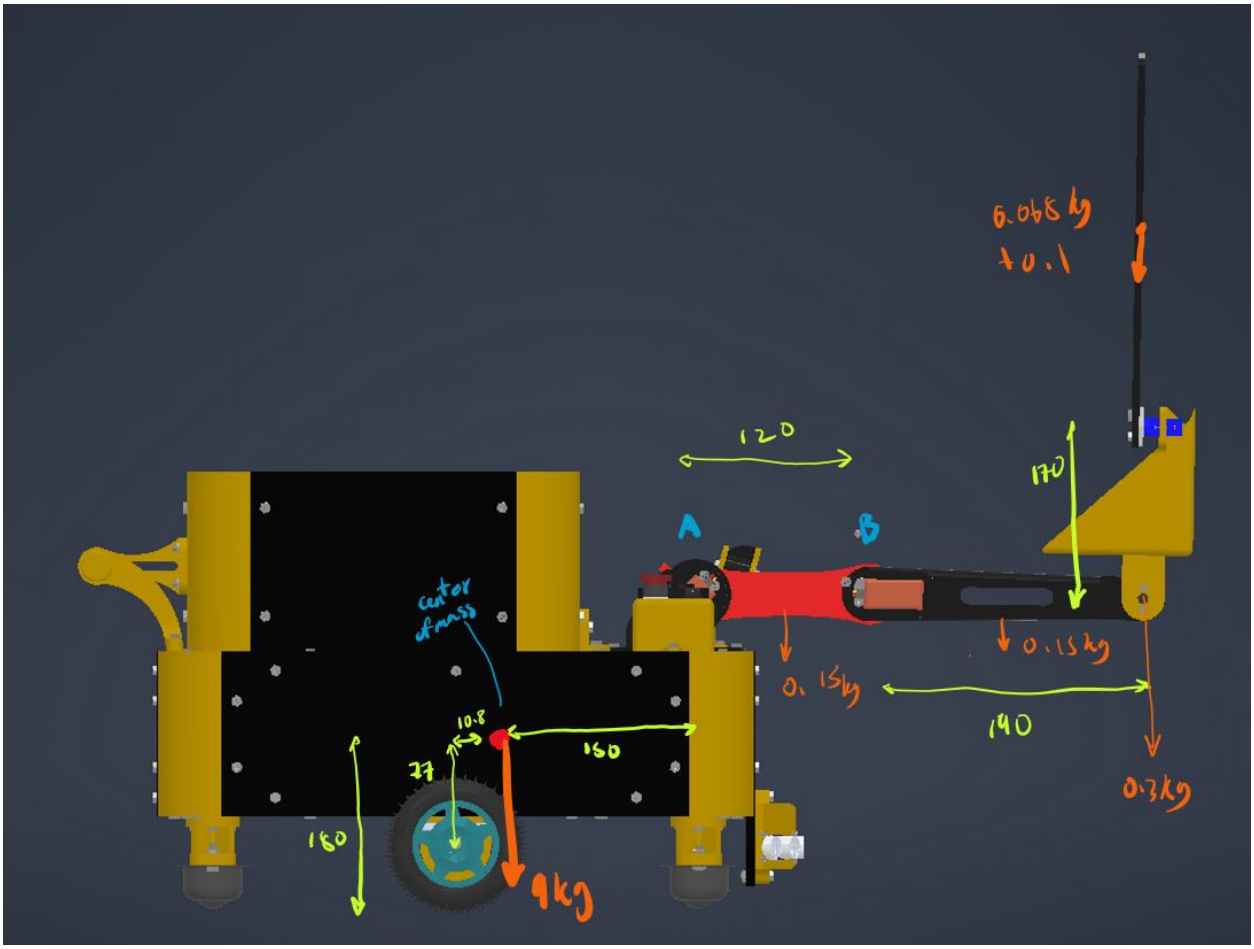


Figure 13 extra mechanism analysis

***assuming trash mass ≈ 0.1 kg**

a) Joint Torque Analysis

The amount of torque needed per joint to be able to lift the weight on the link using servo motors positioned:

$$\boldsymbol{Torque = Force\ x\ distance\ to\ joint}$$

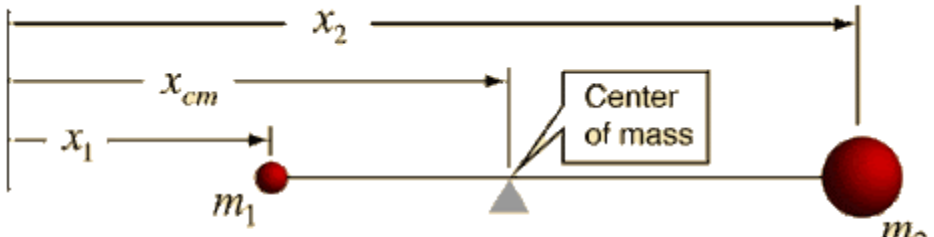
$$\tau_B = g\left(0.15 \times \frac{190}{2} + (0.169 + 0.3) \times 190\right)$$

$$= 1012.1\ Nmm$$

$$\tau_A = g\left(0.15 \times \frac{120}{2} + 0.15\left(\frac{190}{2} + 120\right) + 0.468 \times (190 + 120)\right)$$

$$= 1827.9\ Nmm$$

b) Tipping Analysis



For two masses: $x_{cm} = \dfrac{m_1 x_1 + m_2 x_2}{m_1 + m_2}$

$$x_{cm} = \frac{M_{chasis}\, x_{chasis} + M_1 x_1 + M_2 x_2 + M_3 x_3}{M_T}$$

$$= \frac{10(10.8) + 0.15\left(150 + \frac{120}{2}\right) + 0.15\left(\frac{190}{2} + 120 + 150\right) + (0.468)(190 + 150 + 120)}{10.7}$$

$$= 38.3$$

As long as the x_{cm} we calculated is less than the distance from centre of mass till the support wheels (150 mm), the robot car is stable. In this case, the safety factor we put to avoid tipping of the car equals:

$$n = \frac{150}{38.3} = 3.92$$

c) Critical Tipping Angle

The maximum angle that allows the robot to be stable preventing it from tipping backwards when inclining on ramps (y is the vertical distance between centre of gravity and the ground):

$$\theta_{max} = tan^{-1}\left(\frac{x_{cm}}{y}\right) = tan^{-1}\left(\frac{38.3}{180}\right) = 12^0$$

Typical street ramps are usually around 5^0 which means that the safety factor here equals:

$$n = \frac{12}{5} = 2.4$$

This means that the car is absolutely stable and safe due to the high safety factor.

$$X = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$

$$Y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

Position Analysis

$$X_{tip} = 0.12 \cos(0^\circ) + 0.19 \cos(0^\circ + 0^\circ)$$

$$X_{tip} = 0.12(1) + 0.19(1) = \mathbf{0.31}\ \text{m}\ \textbf{(310 mm)}$$

Velocity Analysis

$$v_x = -L_1\dot{\theta}_1 \sin(\theta_1) - L_2(\dot{\theta}_1 + \dot{\theta}_2)\sin(\theta_1 + \theta_2)$$

$$v_y = L_1\dot{\theta}_1 \cos(\theta_1) + L_2(\dot{\theta}_1 + \dot{\theta}_2)\cos(\theta_1 + \theta_2)$$

$$v_y = (0.12 \cdot 2) + 0.19(2 + 2)$$

$$v_y = 0.24 + 0.76 \qquad \mathbf{1.0}\ \text{m/s}$$

v_x =0 m/s (At 0°, the arm only moves vertically)

Acceleration Analysis

$$a_y = L_1\alpha_1 + L_2(\alpha_1 + \alpha_2)$$

$$a_y = (0.12 \cdot 20) + 0.19(20 + 20)$$

$$a_y = 2.4 + 7.6 = \mathbf{10.0}\ \text{m/s}^{\mathbf{2}}$$

3. Stress Analysis

a) Stress analysis on all critical robot components

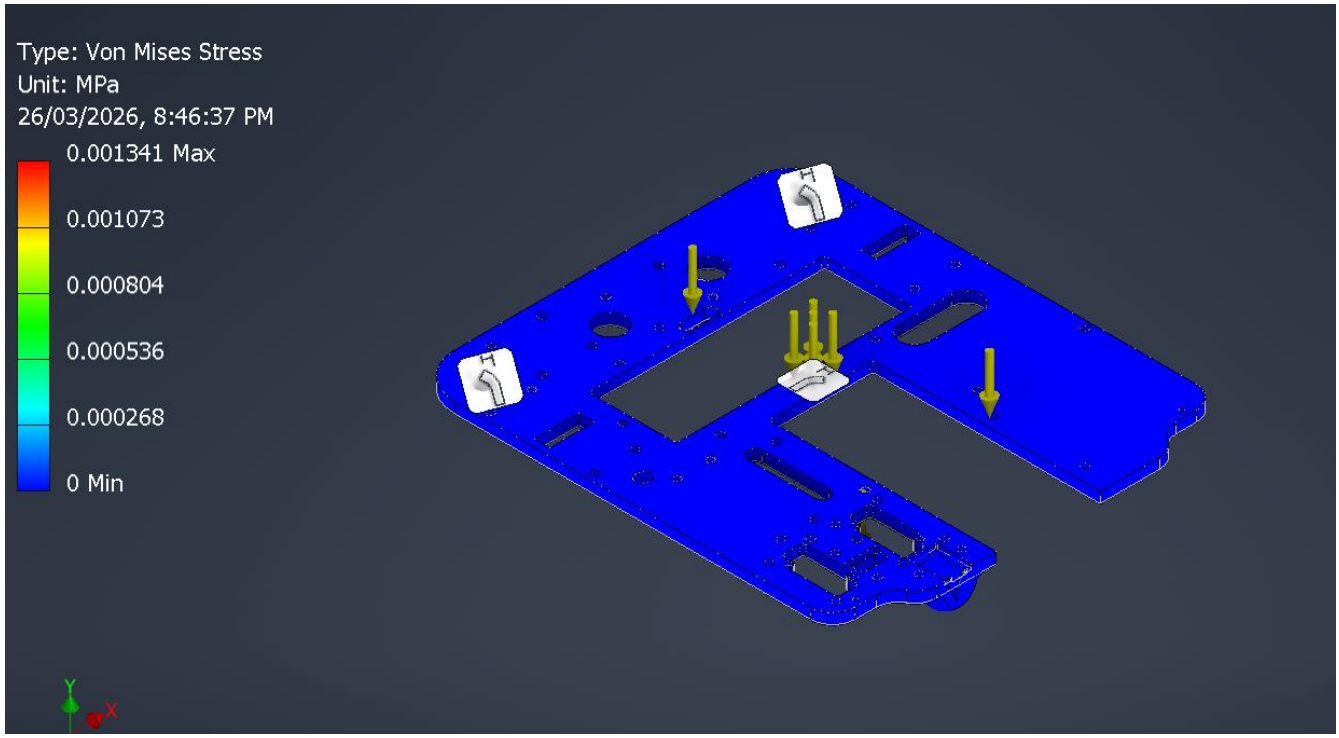
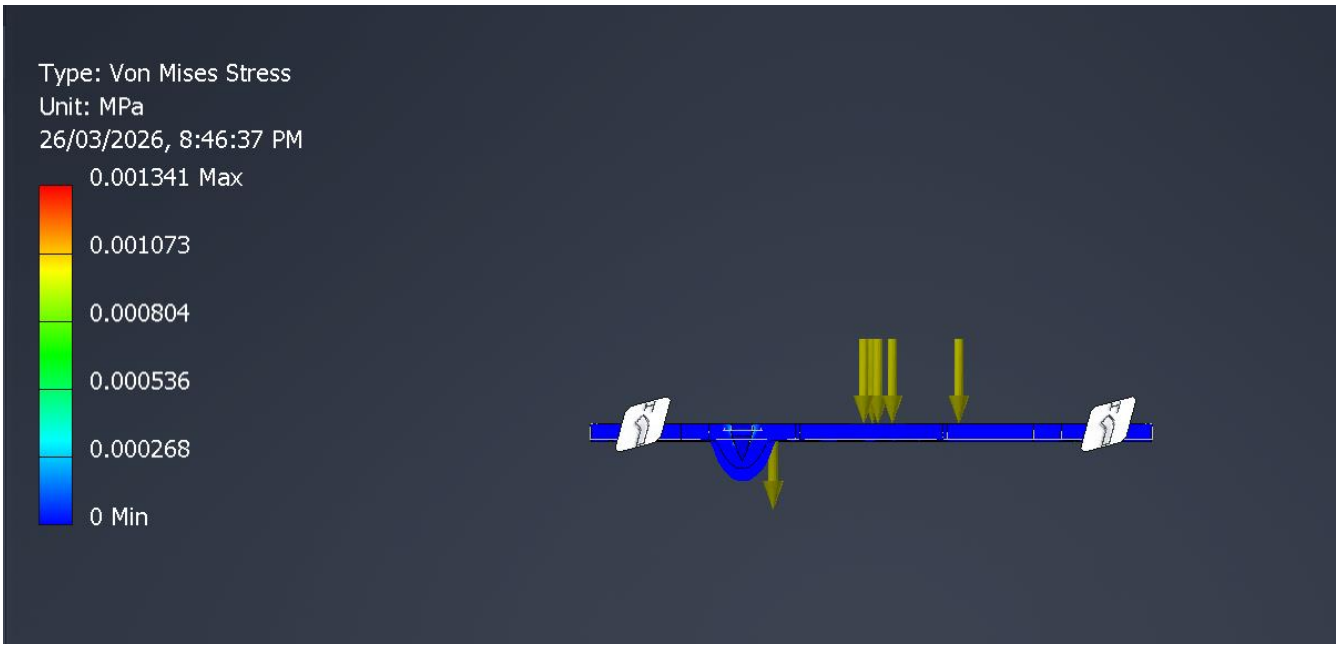
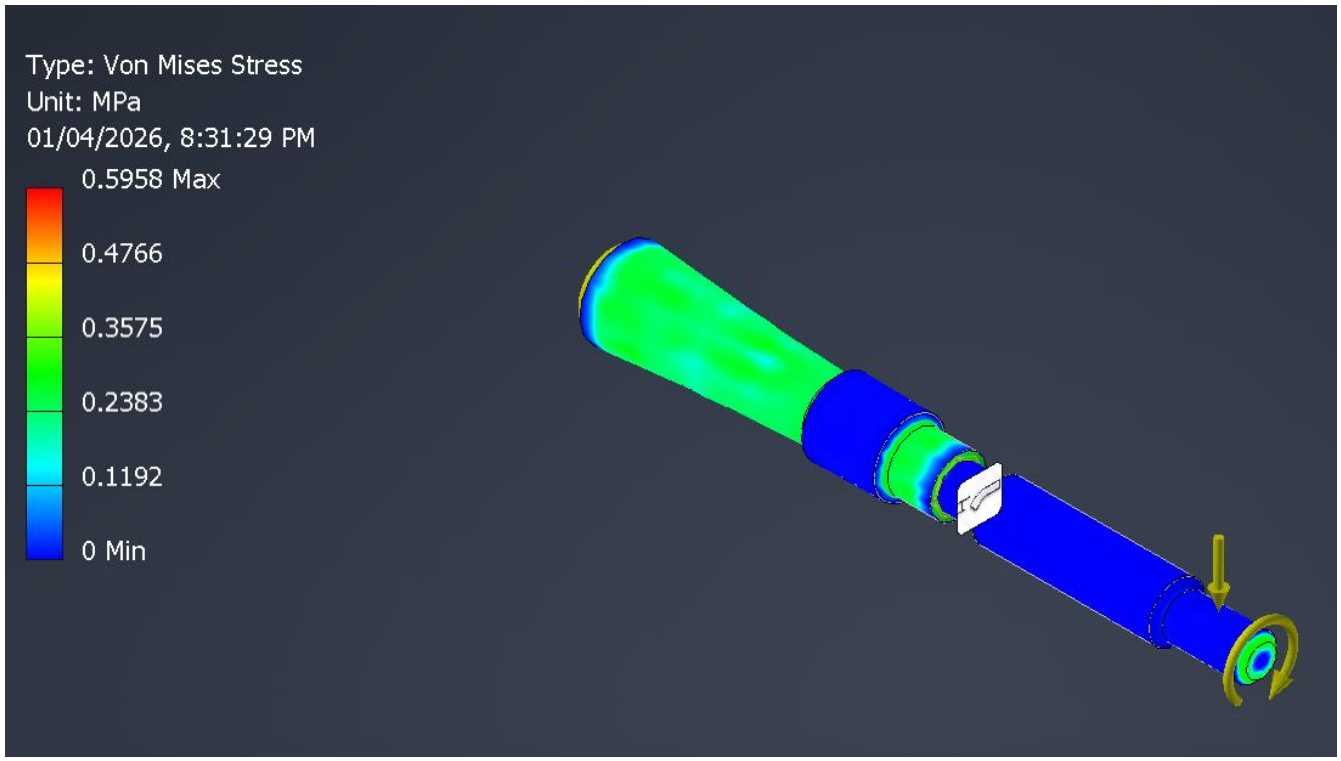
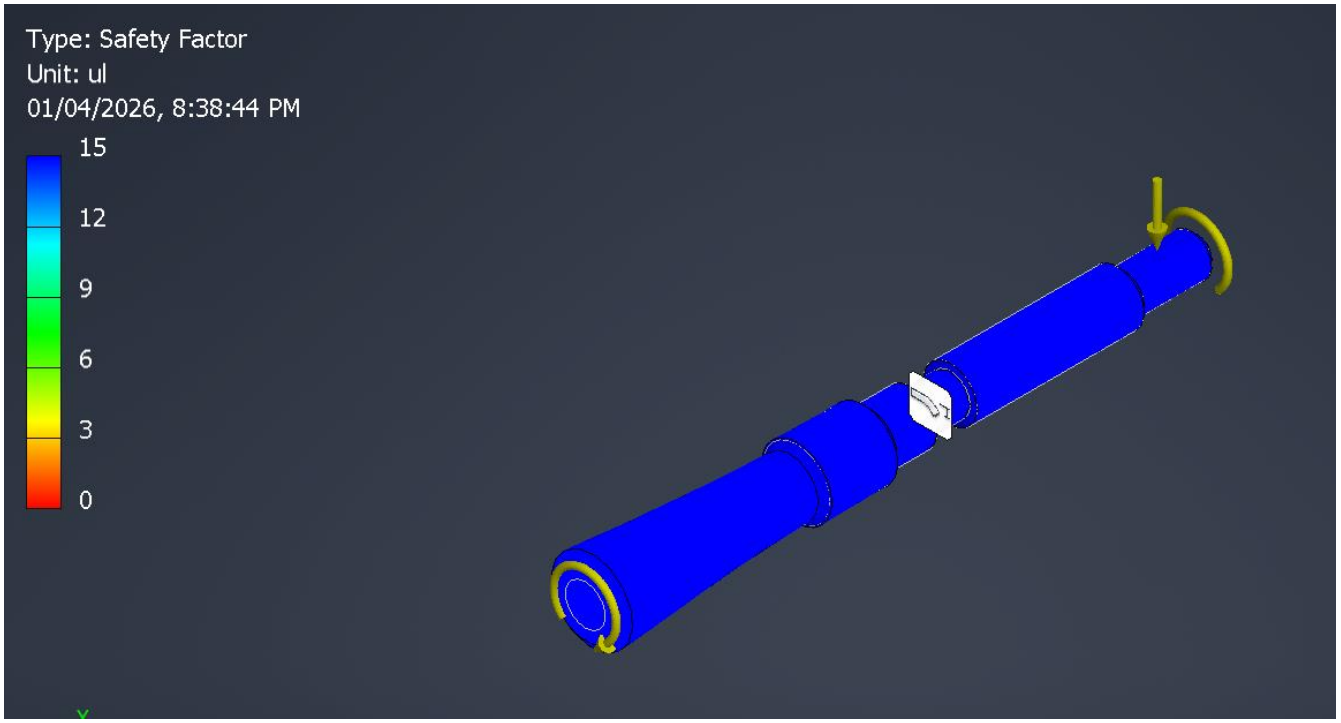
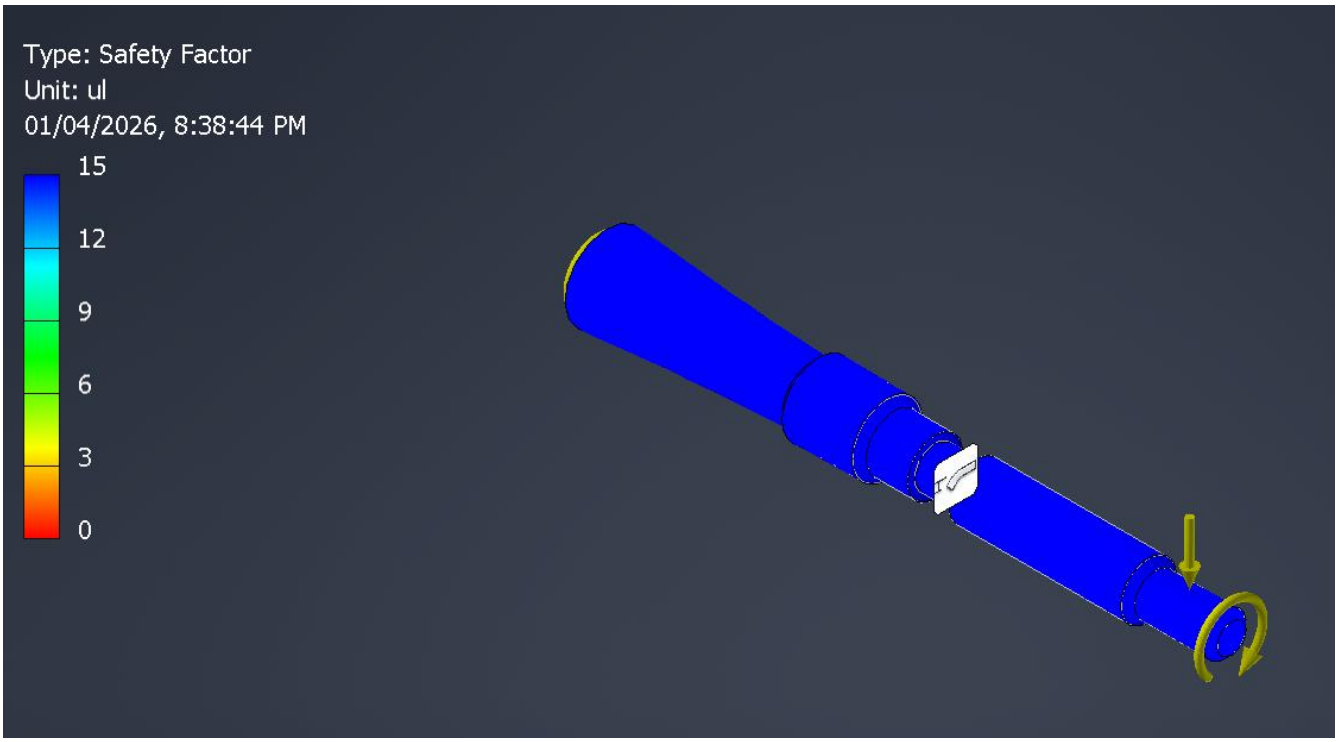


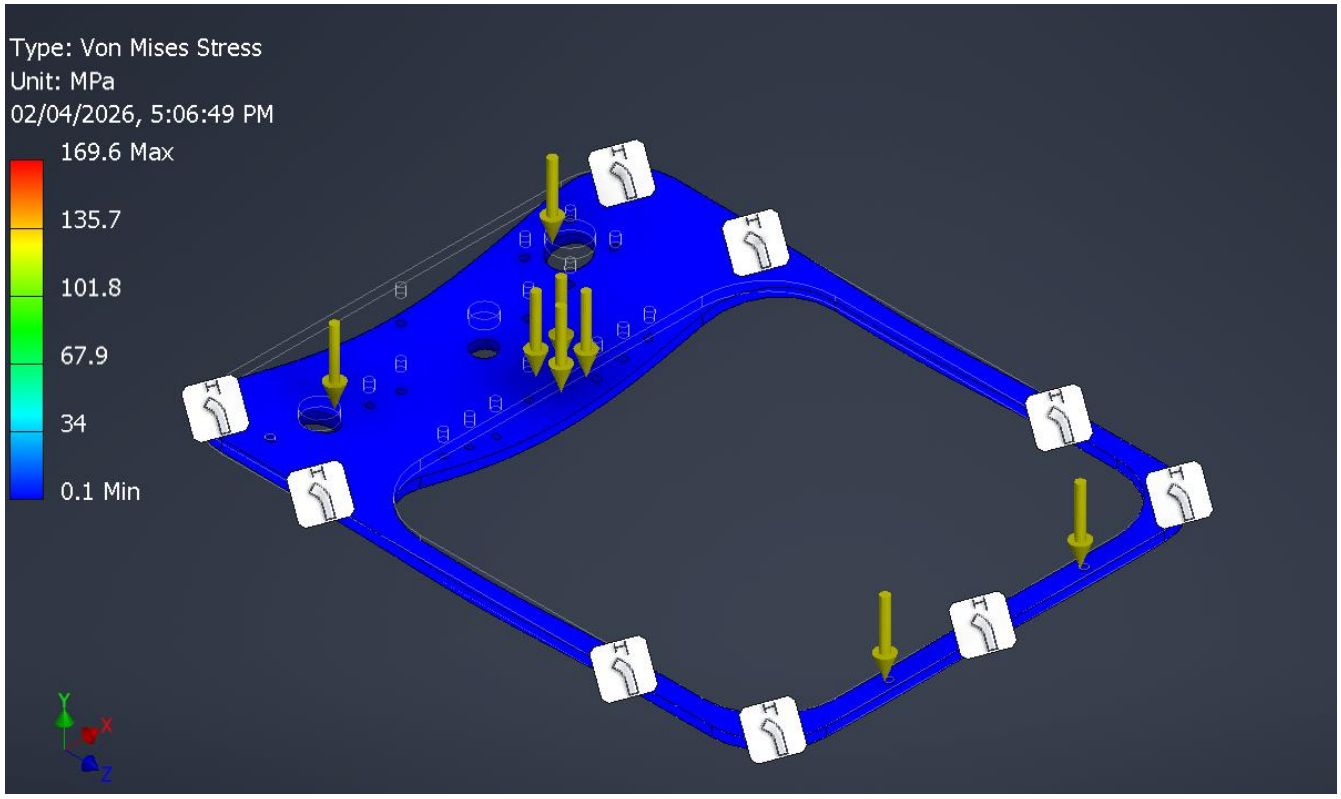
Figure 14 stress analysis

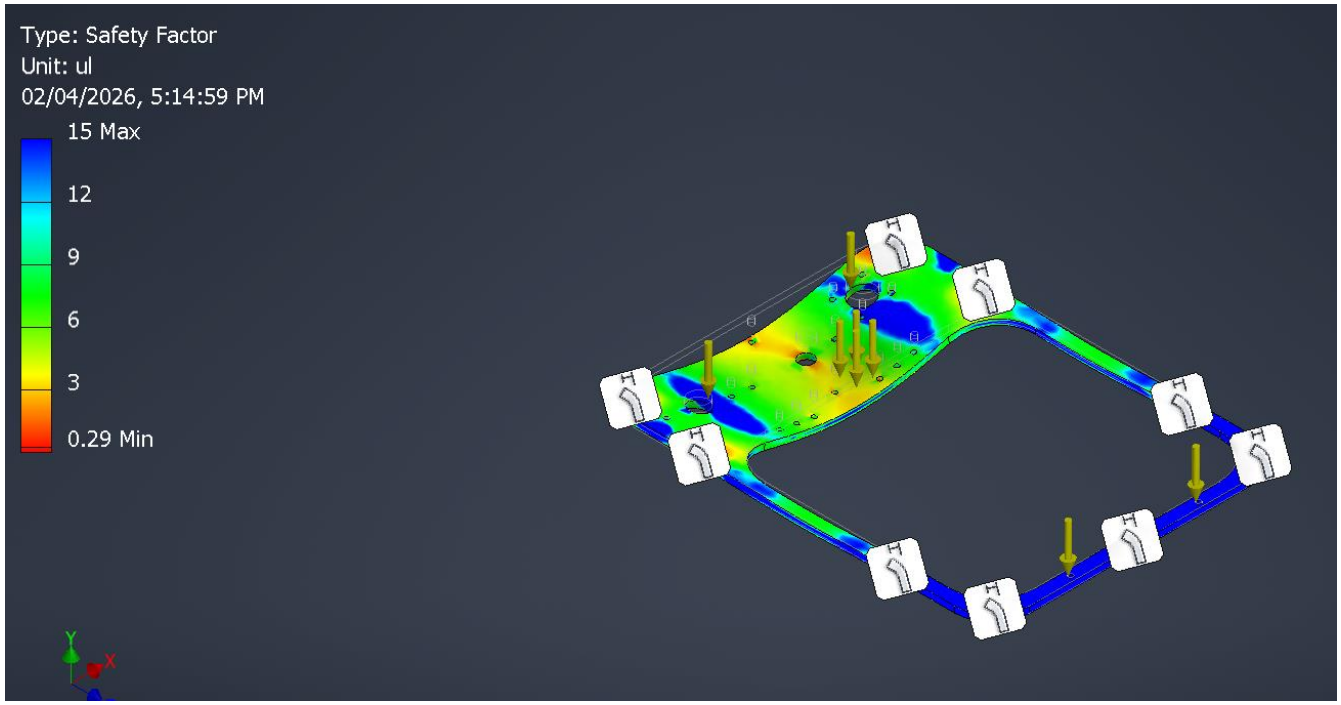


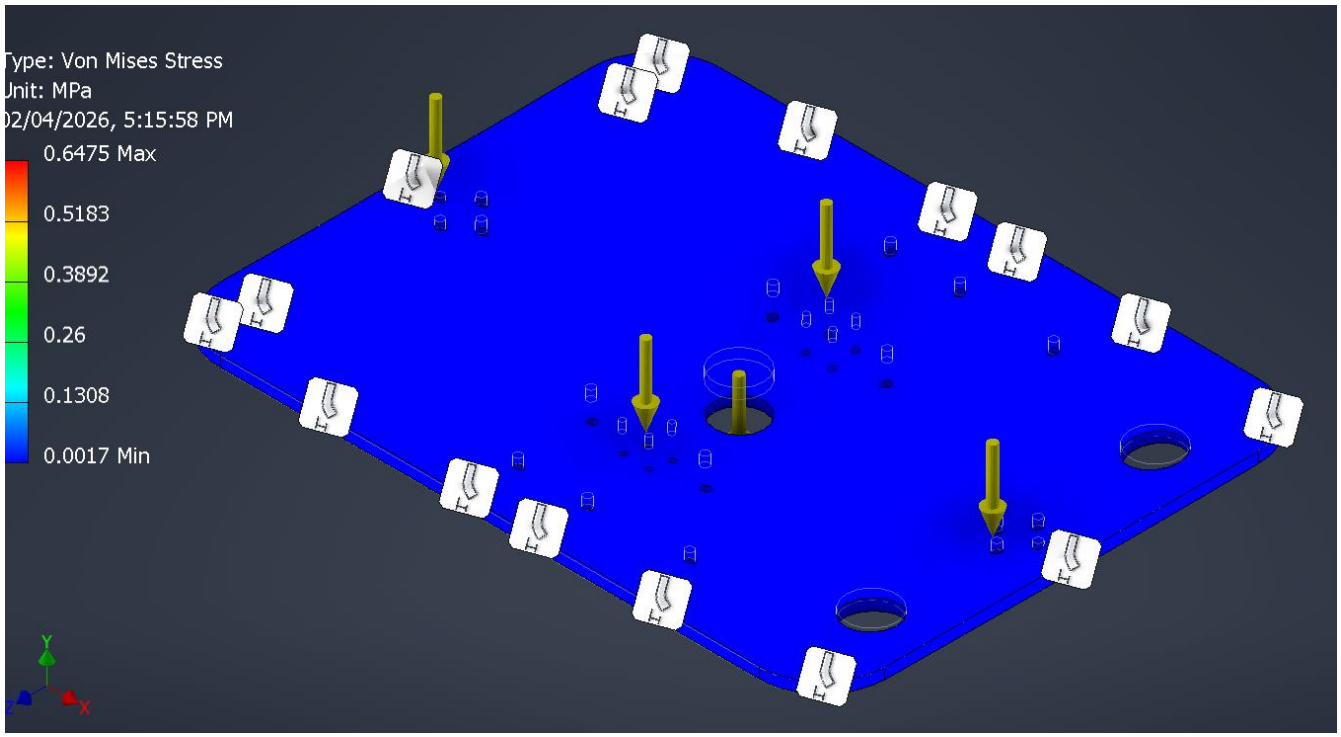

Type: Von Mises Stress
Unit: MPa
01/04/2026, 8:31:29 PM
0.5958 Max
0.4766
0.3575
0.2383
0.1192
0 Min

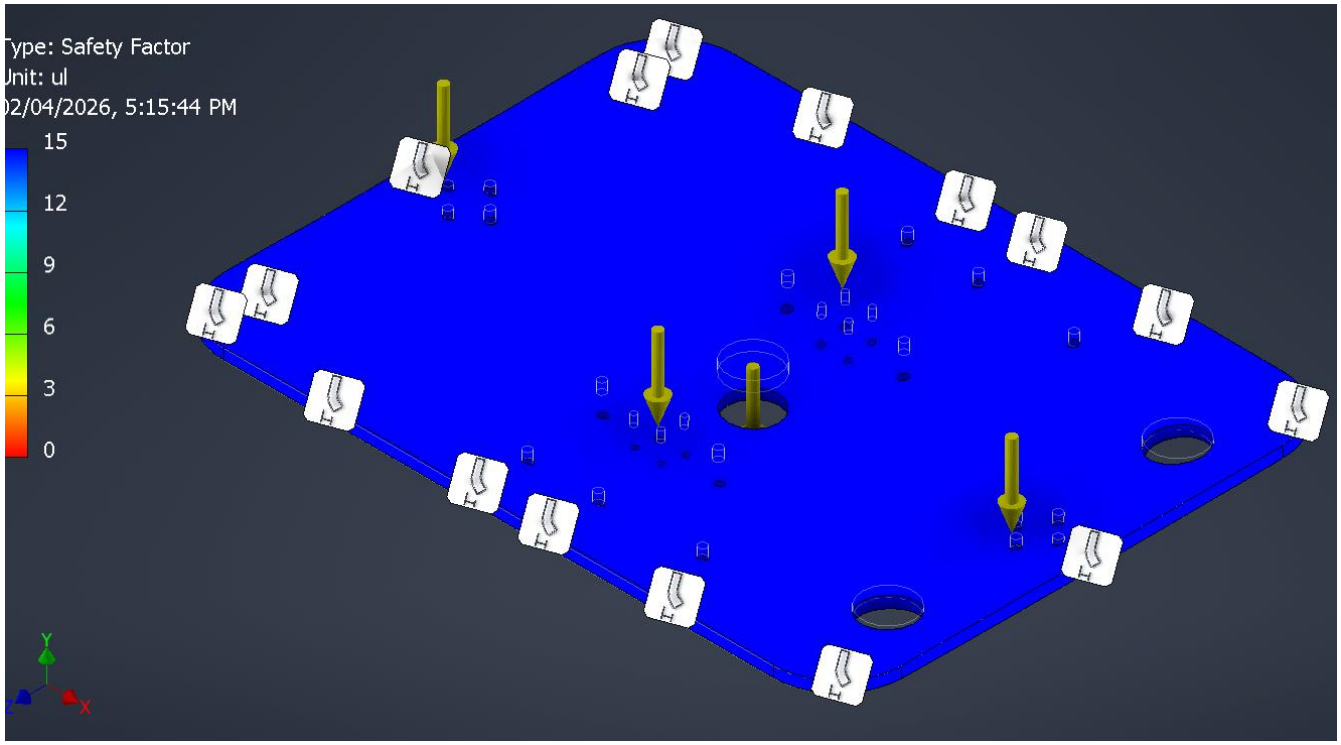

Type: Safety Factor
Unit: ul
01/04/2026, 8:38:44 PM
15
12
9
6
3
0

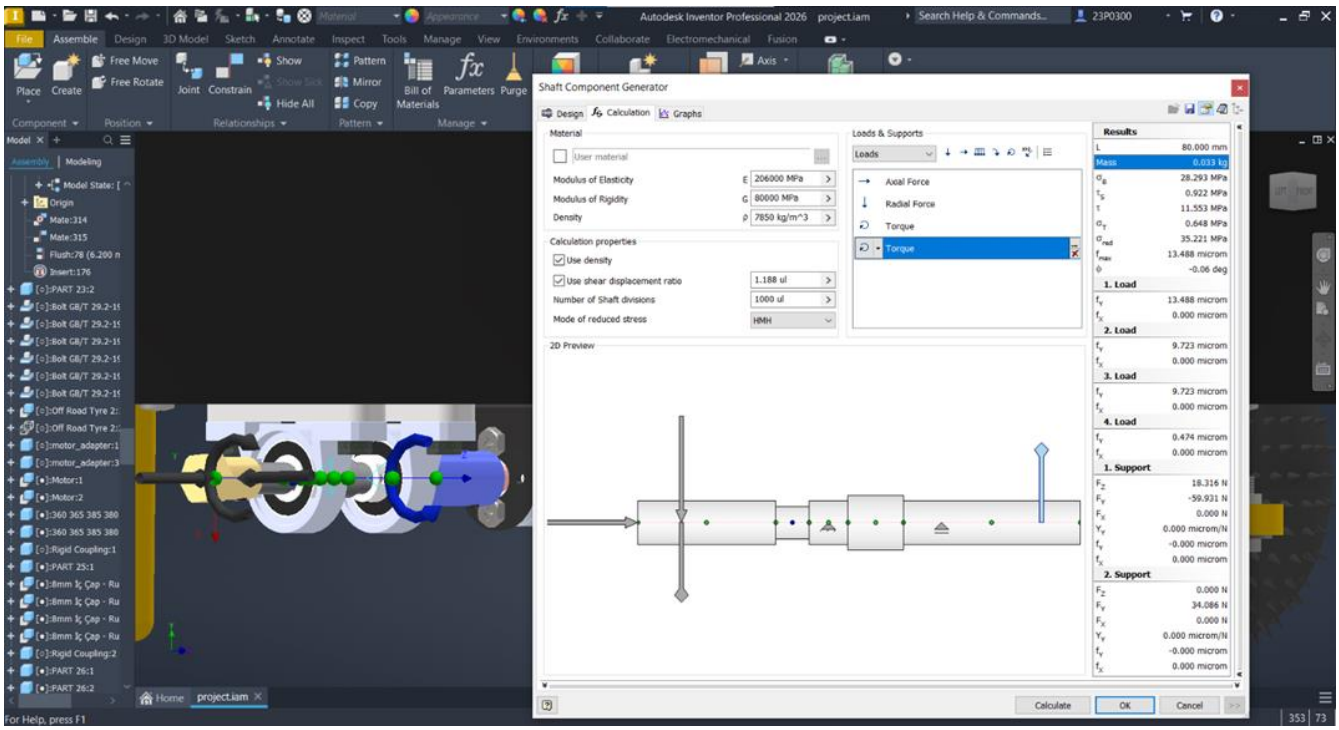

Type: Safety Factor
Unit: ul
01/04/2026, 8:38:44 PM
15
12
9
6
3
0

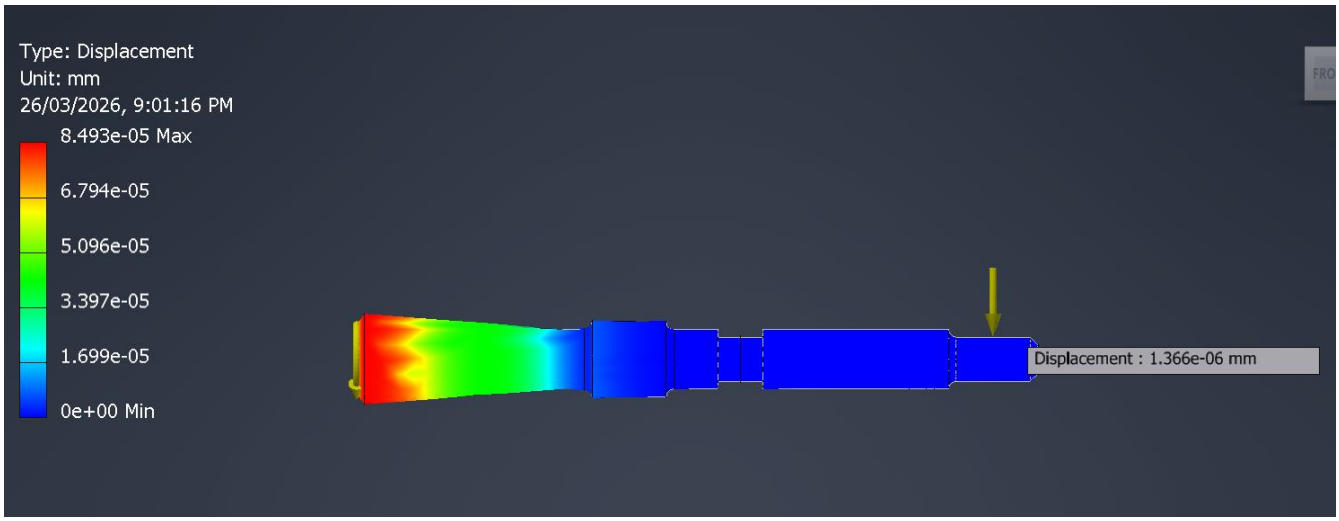

Type: Von Mises Stress
Unit: MPa
02/04/2026, 5:06:49 PM
169.6 Max
135.7
101.8
67.9
34
0.1 Min

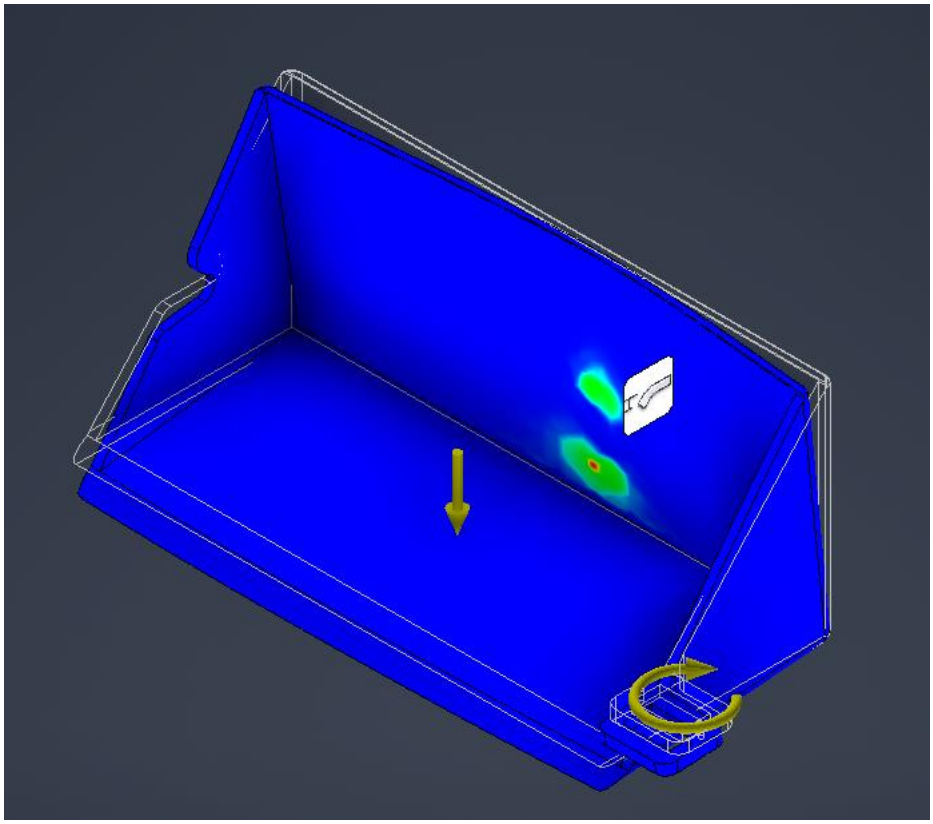

Type: Safety Factor
Unit: ul
02/04/2026, 5:14:59 PM
15 Max
12
9
6
3
0.29 Min

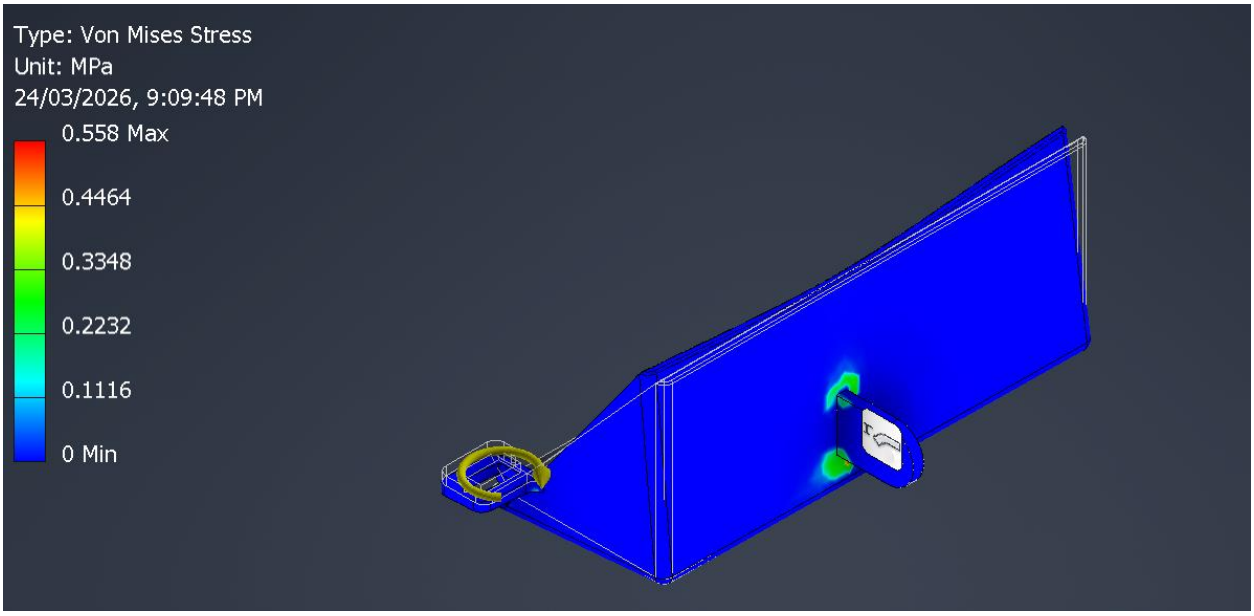

Type: Von Mises Stress
Unit: MPa
02/04/2026, 5:15:58 PM
0.6475 Max
0.5183
0.3892
0.26
0.1308
0.0017 Min
Y
X
Z

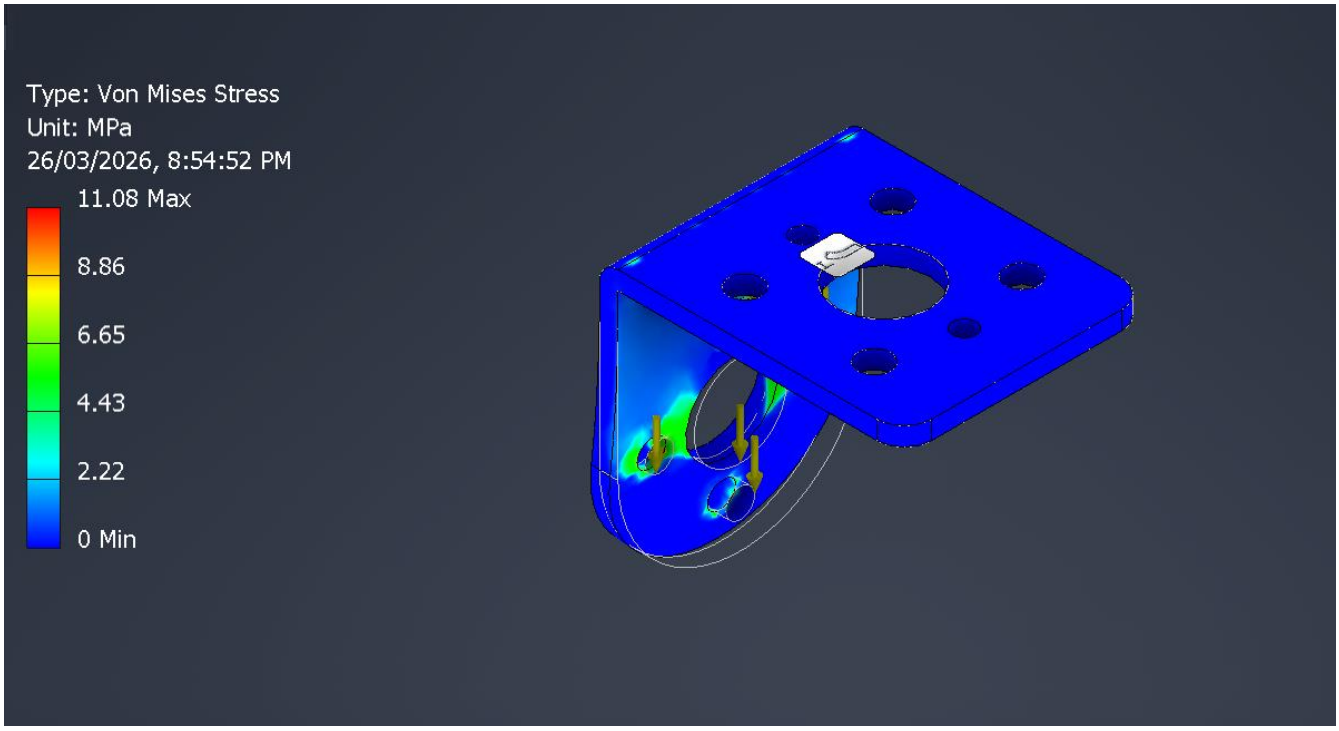

Type: Safety Factor
Unit: ul
02/04/2026, 5:15:44 PM
15
12
9
6
3
0
Y
X
Z

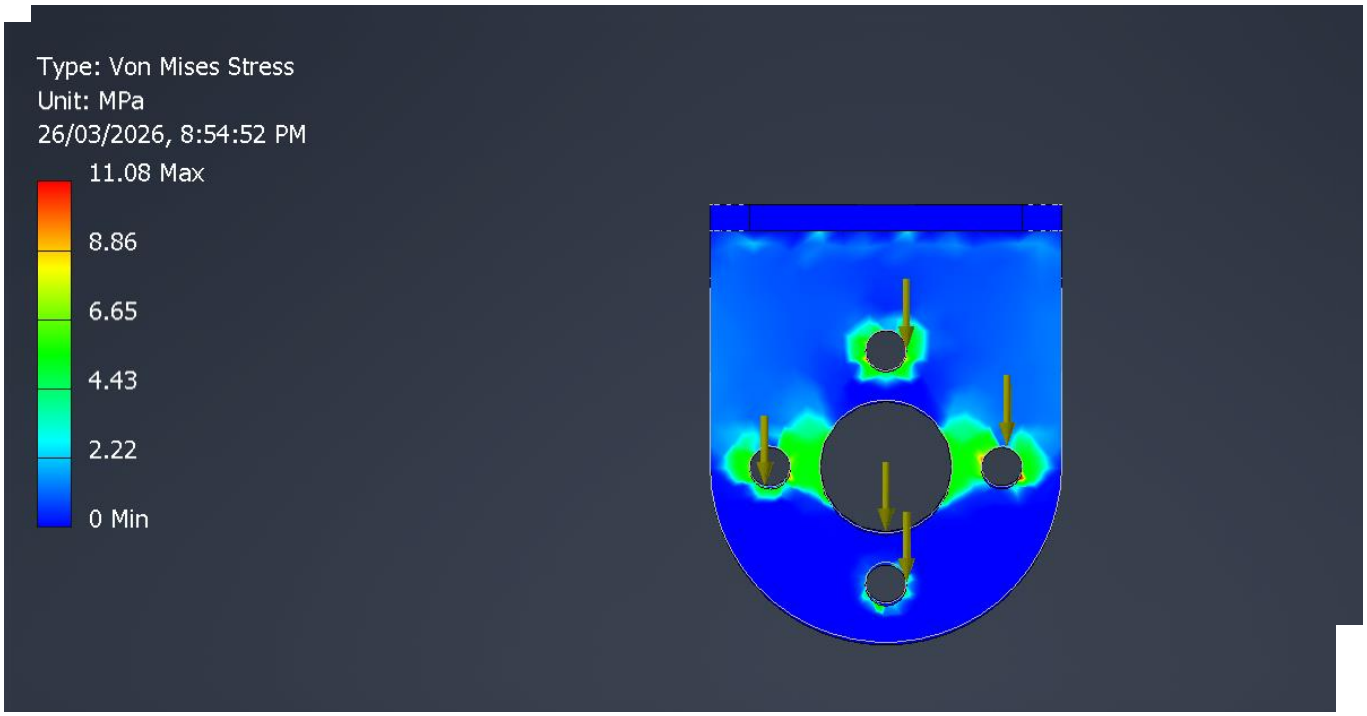

Autodesk Inventor Professional 2026
project.iam
Shaft Component Generator
Design
Calculation
Graphs
Material
Modulus of Elasticity
Modulus of Rigidity
Density
Calculation properties
Use density
Use shear displacement ratio
Number of Shaft divisions
Mode of reduced stress
Loads & Supports
Axial Force
Radial Force
Torque
2D Preview
Results
Calculate
OK
Cancel

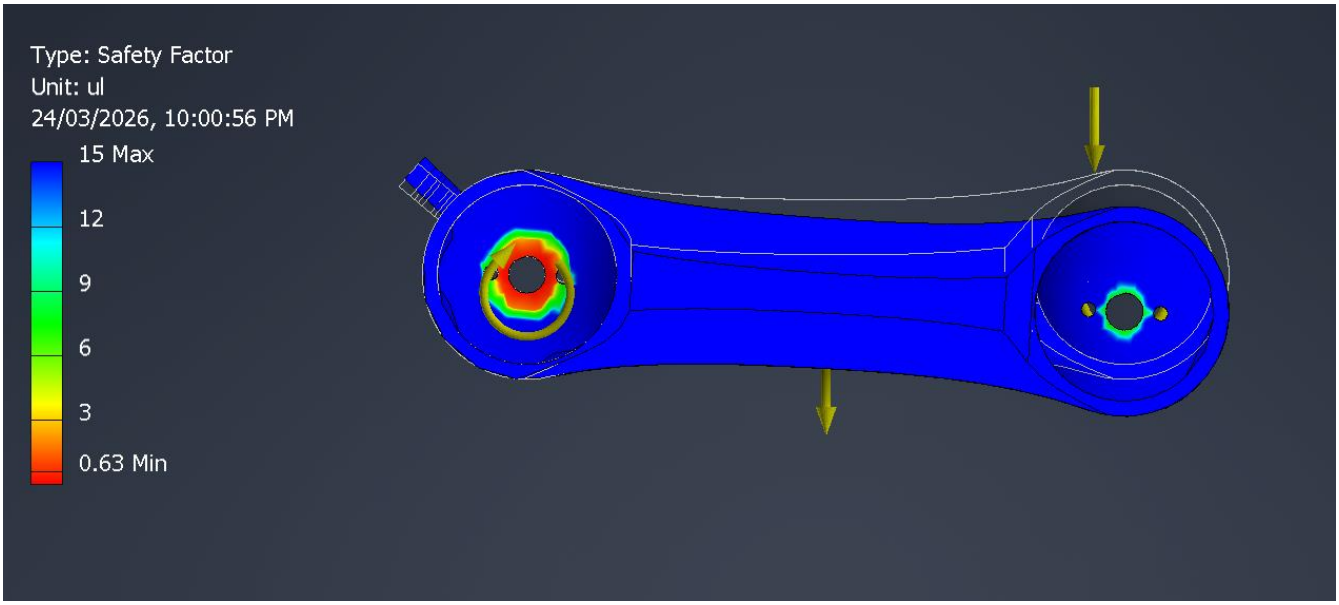

Type: Displacement
Unit: mm
26/03/2026, 9:01:16 PM
8.493e-05 Max
6.794e-05
5.096e-05
3.397e-05
1.699e-05
0e+00 Min
Displacement : 1.366e-06 mm

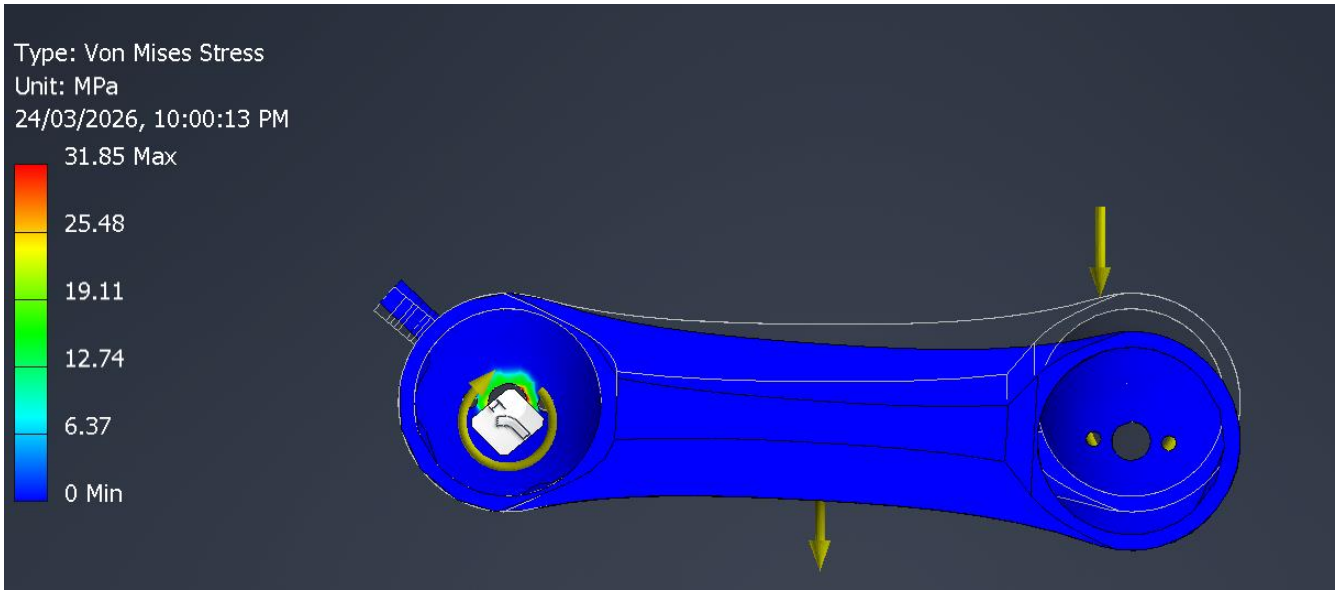


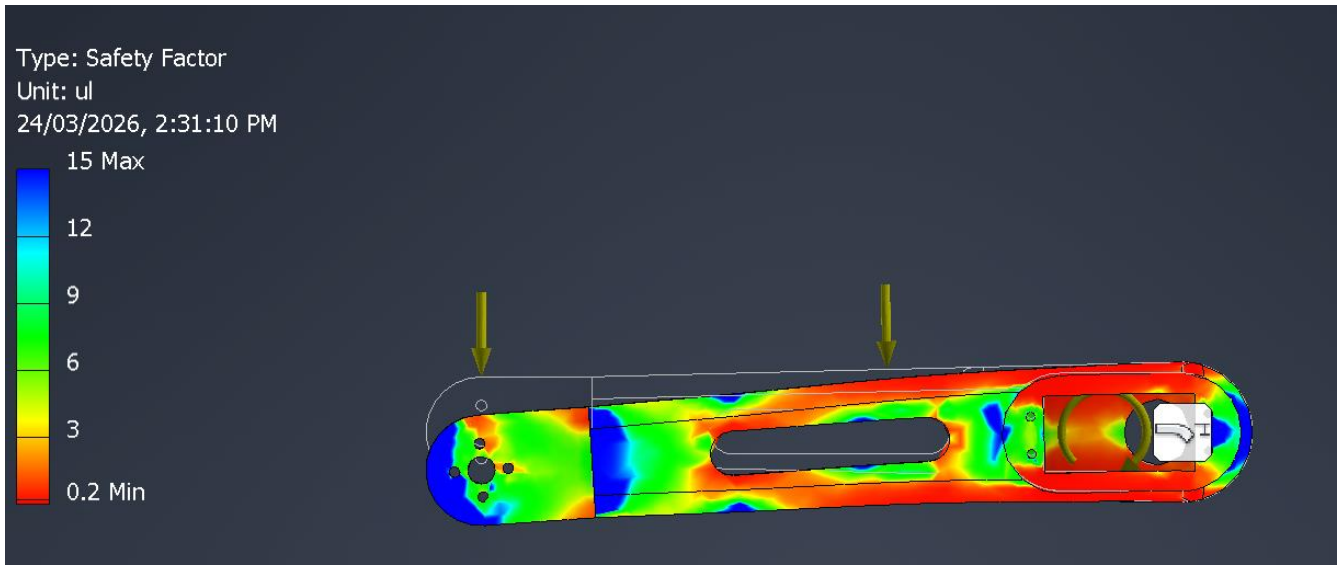

Type: Von Mises Stress
Unit: MPa
24/03/2026, 9:09:48 PM
0.558 Max
0.4464
0.3348
0.2232
0.1116
0 Min

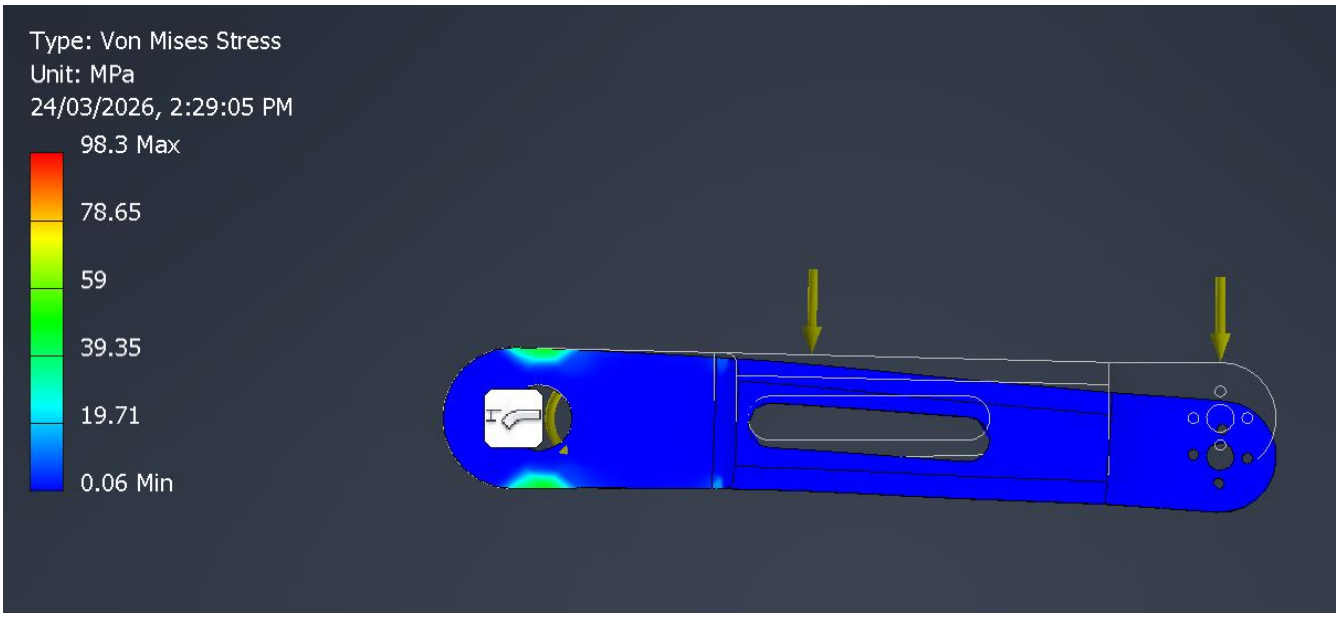

Type: Von Mises Stress
Unit: MPa
26/03/2026, 8:54:52 PM
11.08 Max
8.86
6.65
4.43
2.22
0 Min

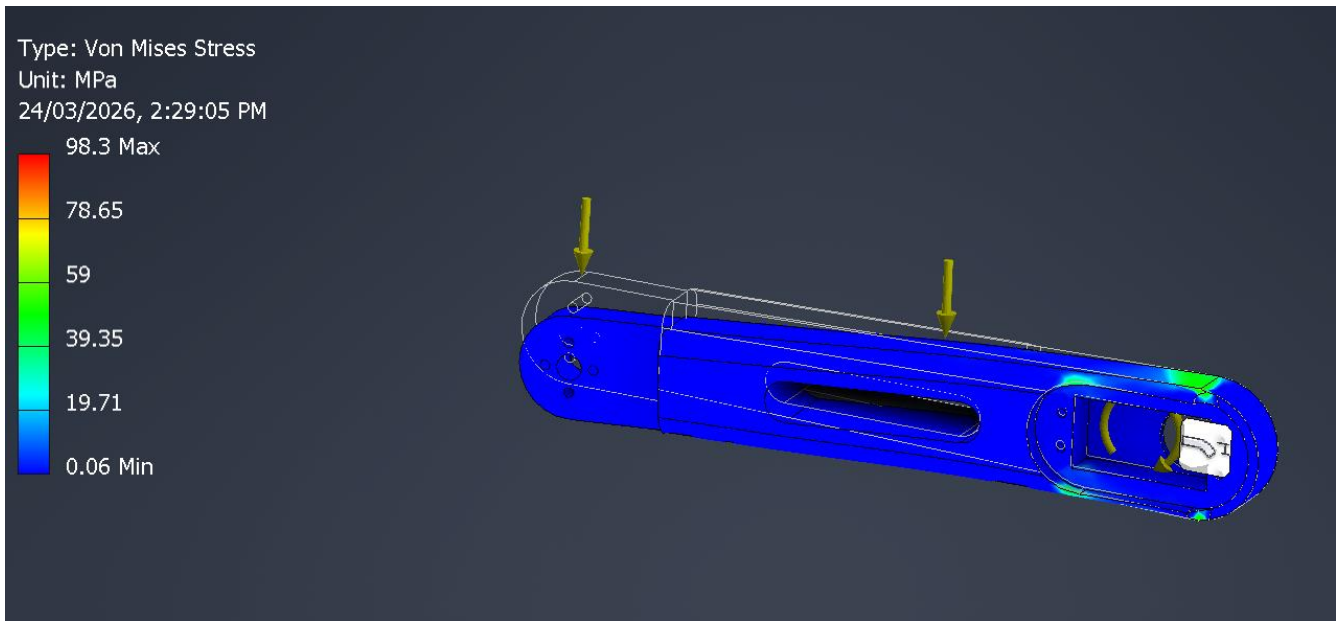

Type: Von Mises Stress
Unit: MPa
26/03/2026, 8:54:52 PM
11.08 Max
8.86
6.65
4.43
2.22
0 Min

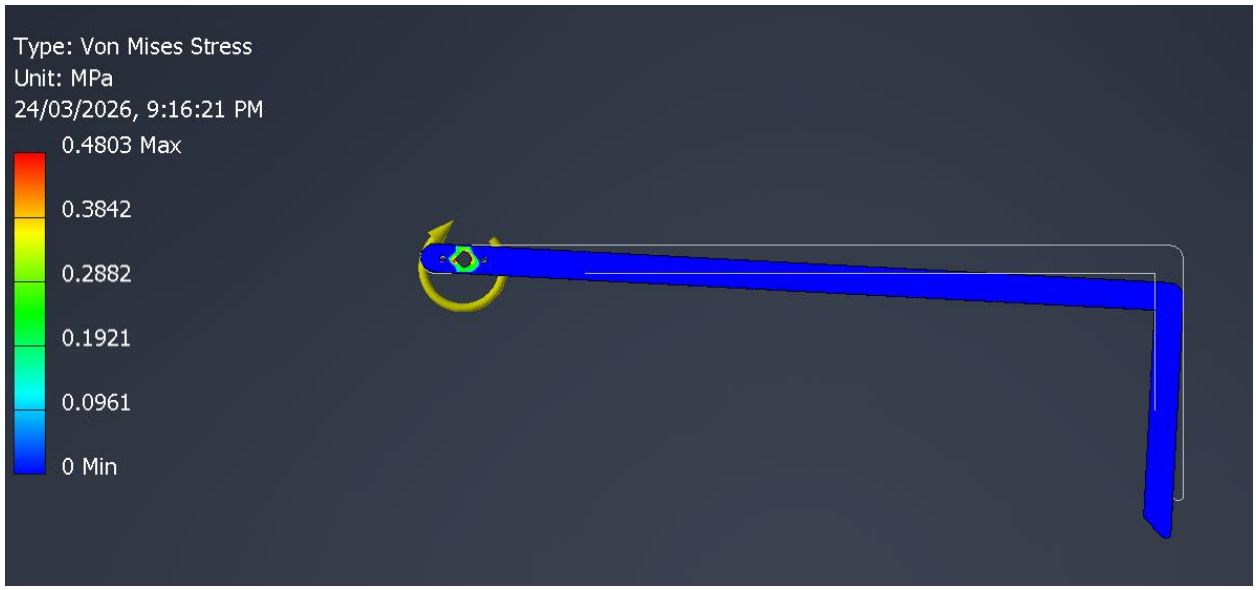

Type: Safety Factor
Unit: ul
24/03/2026, 10:00:56 PM
15 Max
12
9
6
3
0.63 Min

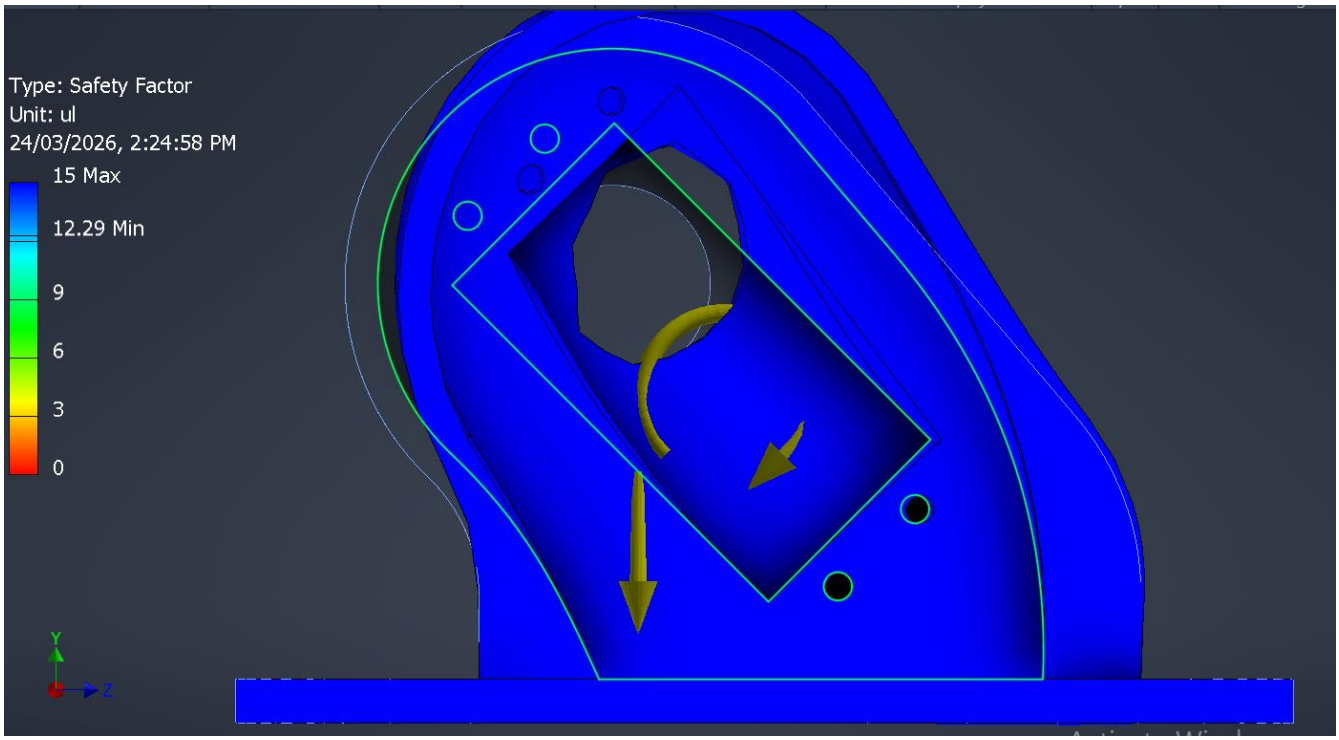

Type: Von Mises Stress
Unit: MPa
24/03/2026, 10:00:13 PM
31.85 Max
25.48
19.11
12.74
6.37
0 Min

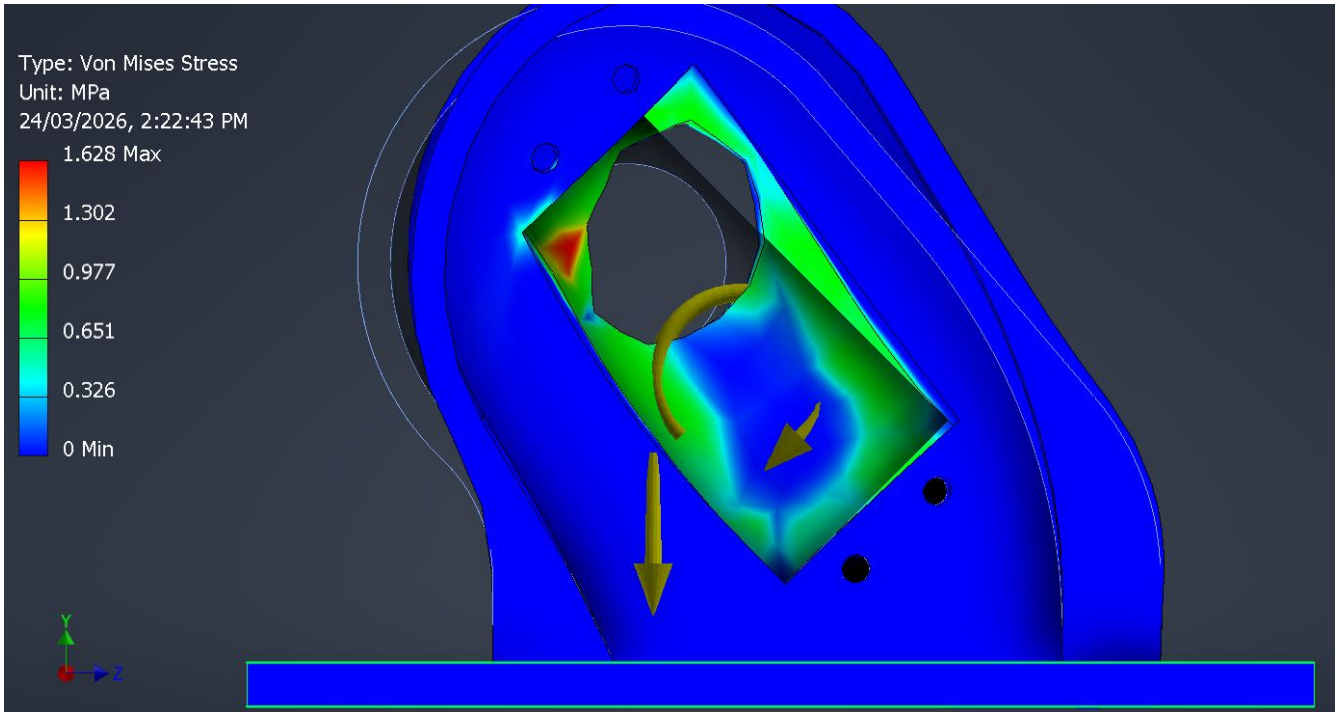

Type: Safety Factor
Unit: ul
24/03/2026, 2:31:10 PM
15 Max
12
9
6
3
0.2 Min

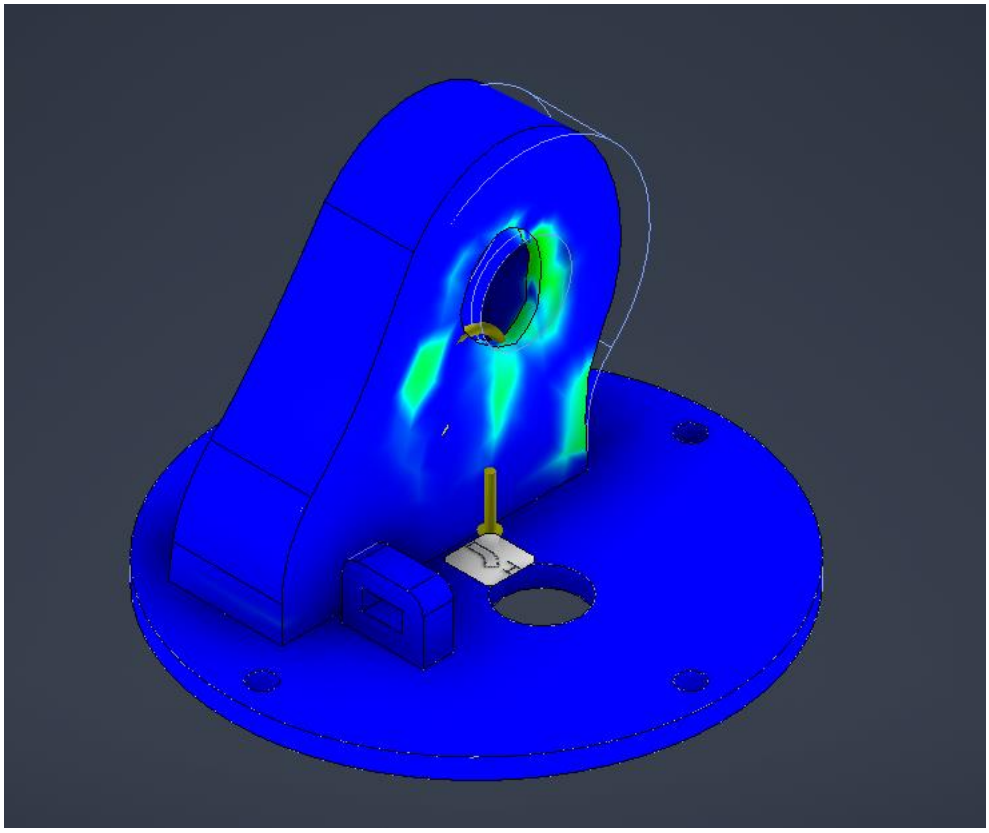

Type: Von Mises Stress
Unit: MPa
24/03/2026, 2:29:05 PM
98.3 Max
78.65
59
39.35
19.71
0.06 Min


Type: Von Mises Stress
Unit: MPa
24/03/2026, 2:29:05 PM
98.3 Max
78.65
59
39.35
19.71
0.06 Min


Type: Von Mises Stress
Unit: MPa
24/03/2026, 9:16:21 PM
0.4803 Max
0.3842
0.2882
0.1921
0.0961
0 Min


Type: Safety Factor
Unit: ul
24/03/2026, 2:24:58 PM
15 Max
12.29 Min
9
6
3
0
Y
Z


Type: Von Mises Stress
Unit: MPa
24/03/2026, 2:22:43 PM
1.628 Max
1.302
0.977
0.651
0.326
0 Min
Y
Z



b) Minimum required diameter of the wheel shafts

Step1: Construct the shaft.

First cylinder is chamfered it breaks the sharp edge, allowing the wheel hub to slide on easily without scratching the shaft or binding.

Second cylinder is 6mm diameter with undercut at the beginning to be suitable for snap ring to fit inside also the fillet at the end is smooth out the "flow" of stress, drastically lowering the peak stress and extending the life of the shaft.

Third cylinder of 8mm ensures a precision "transition fit" (like a j6 or k6 tolerance) with the inner race of an 8mm bearing.

Fourth cylinder stepping the diameter up to 10mm between the two bearings, you created a shoulder for the bearing as When the wheel is hit from the side and generates an Axial Force, this 10mm shoulder transfers thrust directly into the steel bearing races.

Fifth cylinder 8mm allows it to fit through the bearing and perfectly match the bore of your blue coupling.

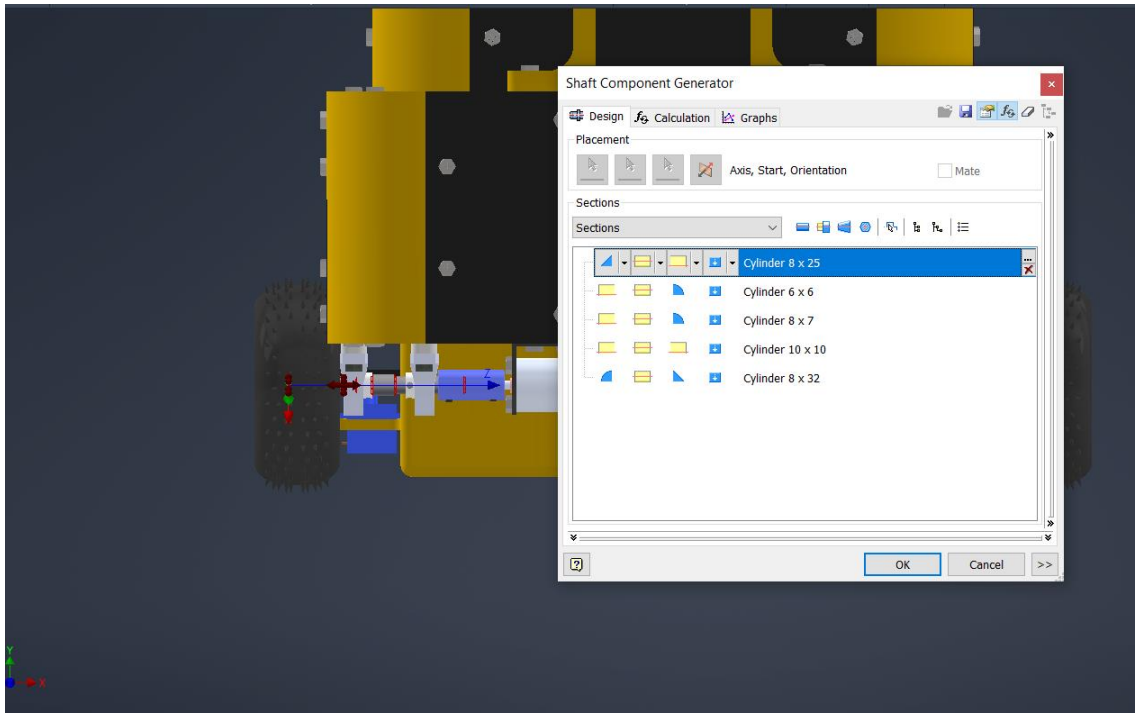


Figure 15 create the shaft and place it

Step2: calculate the acting forces

Table 5 shaft forces calculations

Parameter	Symbol	Value	Unit	Excel Formula
Total Robot Mass	m	10.669	kg	total mass
Number of Wheels	n	4		number of load-bearing wheels
Gravity	g	9.81	m/s^2	
Friction Coefficient	mu	0.6		friction coefficient (e.g. 0.6 to 0.8)
Wheel Radius	R_w	0.065	m	radius in meters
CALCULATED FORCES				
Vertical Bending	F_r	26.166	N	(m*g)/n
Horizontal Bending	F_t	15.699	N	mu*F_r
Torque	T	1.7008	N.m	R_w*F_r
Axial Force	F_a	15.699	N	mu*F_r

Step 3 add forces, supports and material data then calculate



Physical Properties	Metric	English	Comments
Density	7.85 g/cc	0.284 lb/in³	

Mechanical Properties	Metric	English	Comments
Hardness, Brinell	179	179	
Hardness, Knoop	200	200	Converted from Brinell
Hardness, Rockwell B	88	88	Converted from Brinell
Hardness, Vickers	188	188	Converted from Brinell
Tensile Strength, Ultimate	625 MPa	90600 psi	
Tensile Strength, Yield	530 MPa	76900 psi	
Elongation at Break	12 %	12 %	In 50 mm
Reduction of Area	35 %	35 %	
Modulus of Elasticity	206 GPa	29900 ksi	
Bulk Modulus	163 GPa	23600 ksi	Estimated from elastic modulus
Poissons Ratio	0.29	0.29	Typical for steel
Machinability	55 %	55 %	Based on AISI 1212 steel = 100%
Shear Modulus	80.0 GPa	11600 ksi	Estimated from elastic modulus

Figure 16 shaft datasheet

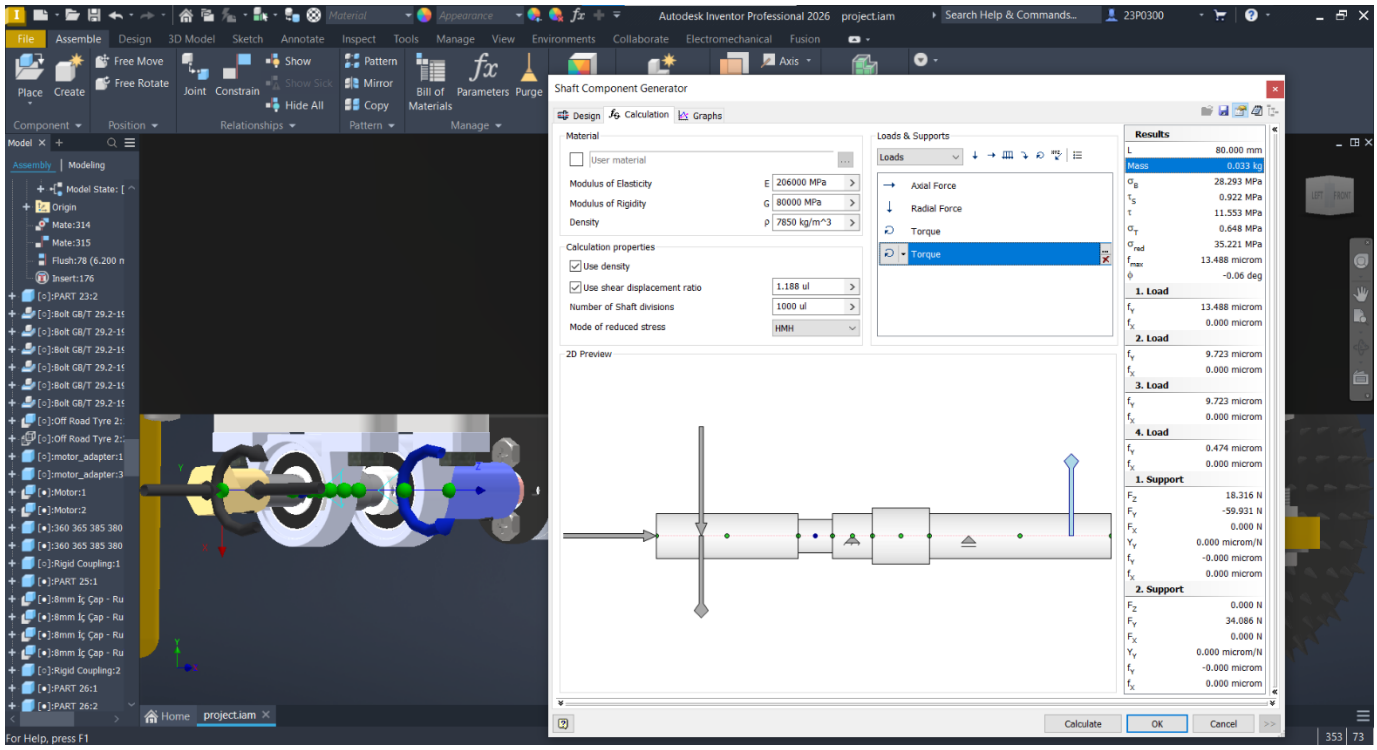


Figure 17 add forces, and material data

Results

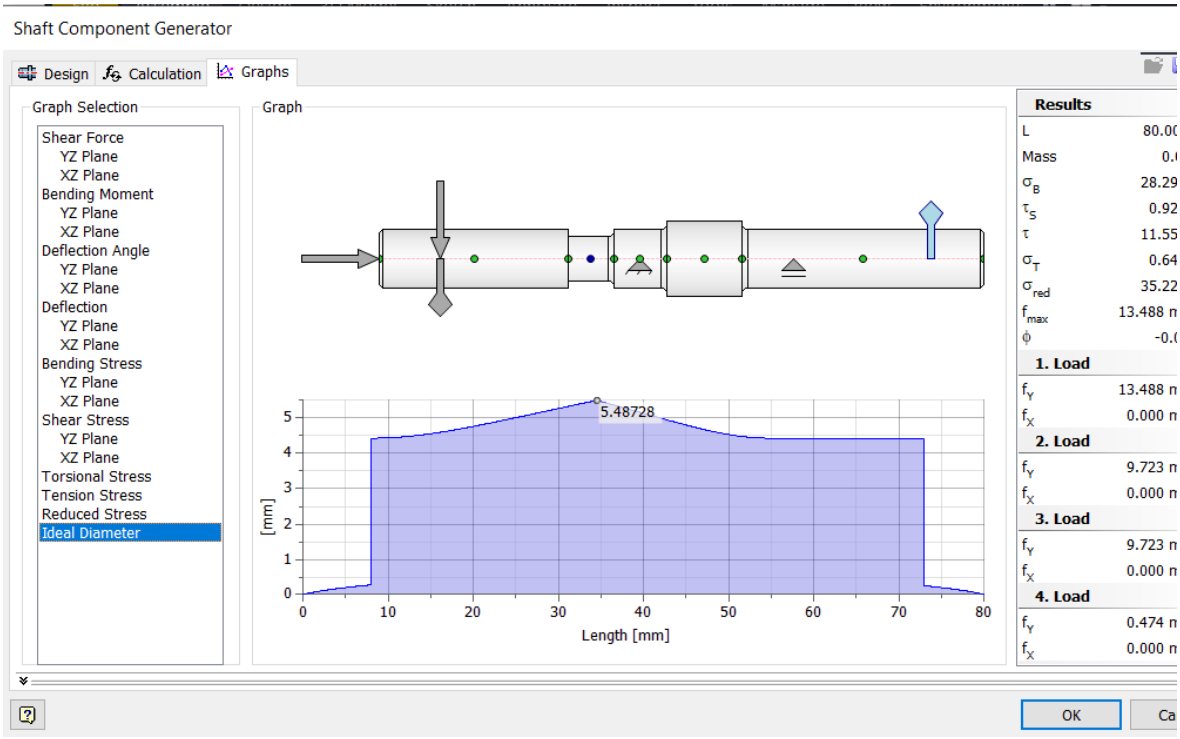


Figure 18 minimum diameter

This graph serves as the ultimate mathematical proof for dimensions. The absolute thinnest section of design is the standard snap ring groove, which sits at roughly 6 mm in diameter. Because your smallest physical diameter (6 mm) is strictly larger than the theoretical required maximum (5.48 mm), the shaft is mathematically guaranteed not to yield under the specified loads.

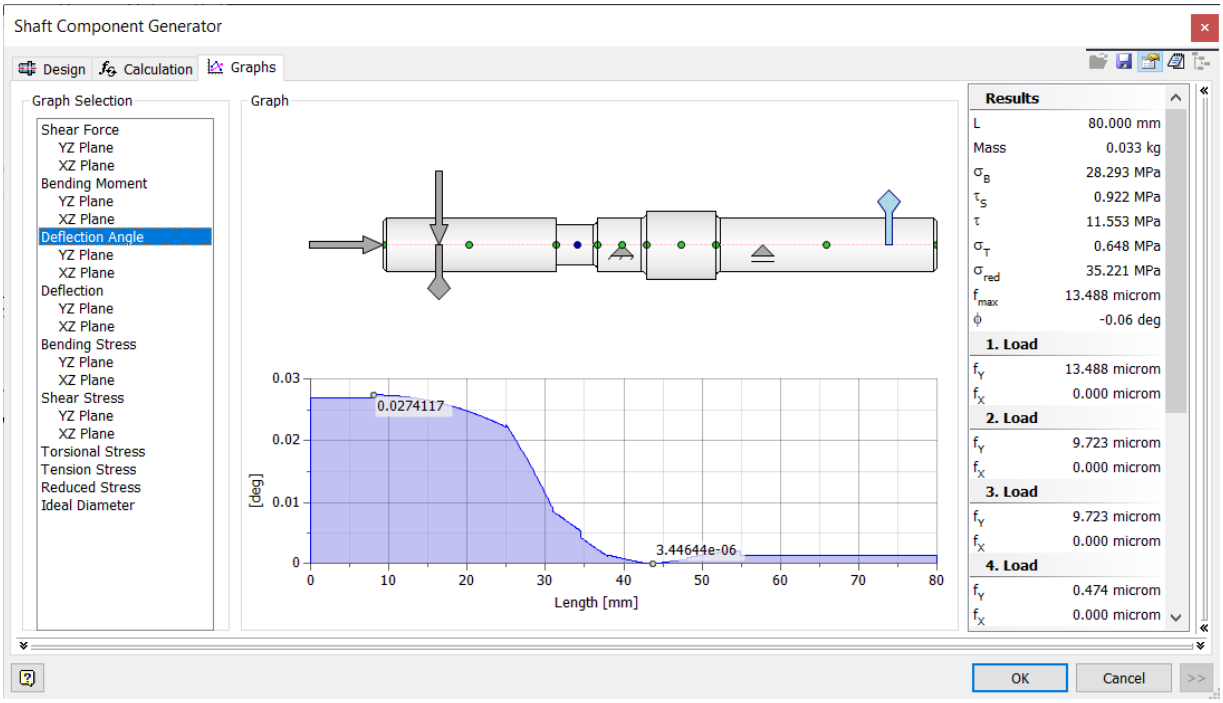


Figure 19 deflection angle

The motor's torque will be transmitted to the wheels instantaneously without any "spring-up" lag, ensuring precise kinematic control of the robot.

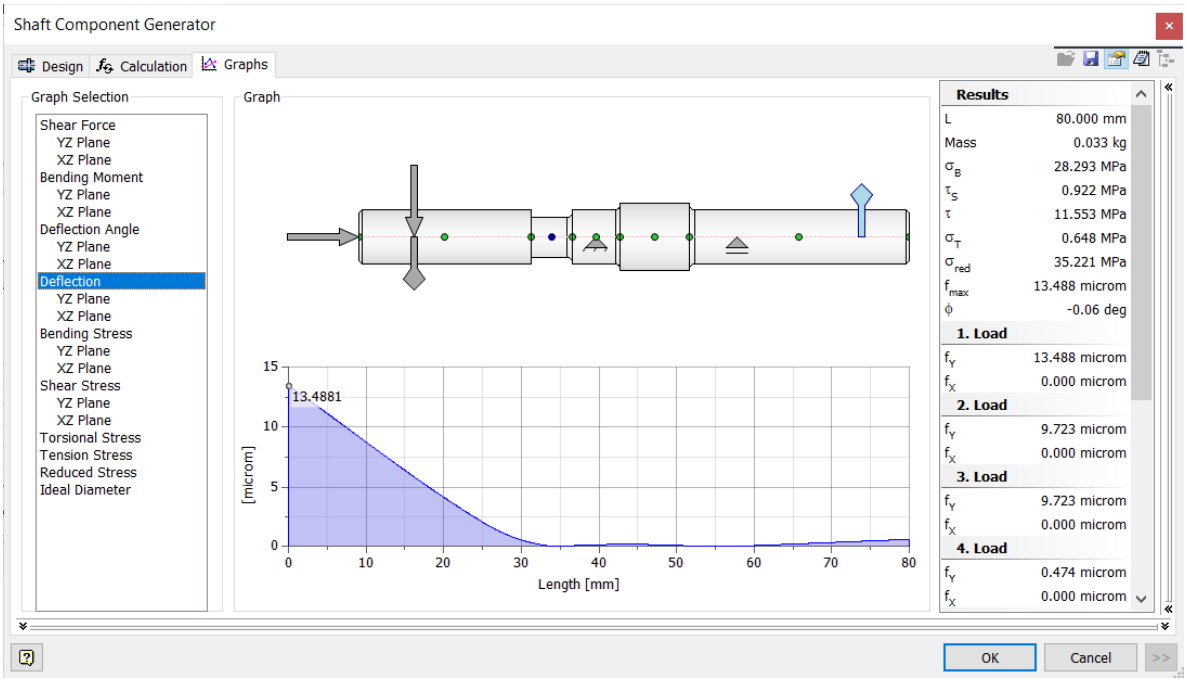


Figure 20 deflection

a deflection is entirely microscopic. It falls well within the permissible misalignment tolerances for standard radial ball bearings, guaranteeing that the shaft will not bind or cause premature bearing failure.

Safety Factor = $\frac{Yield\ Strength}{\text{Maximum Reduced Stress}(\sigma_{red})} = \frac{530}{16.126}$ = 32.9 (overdesigned)

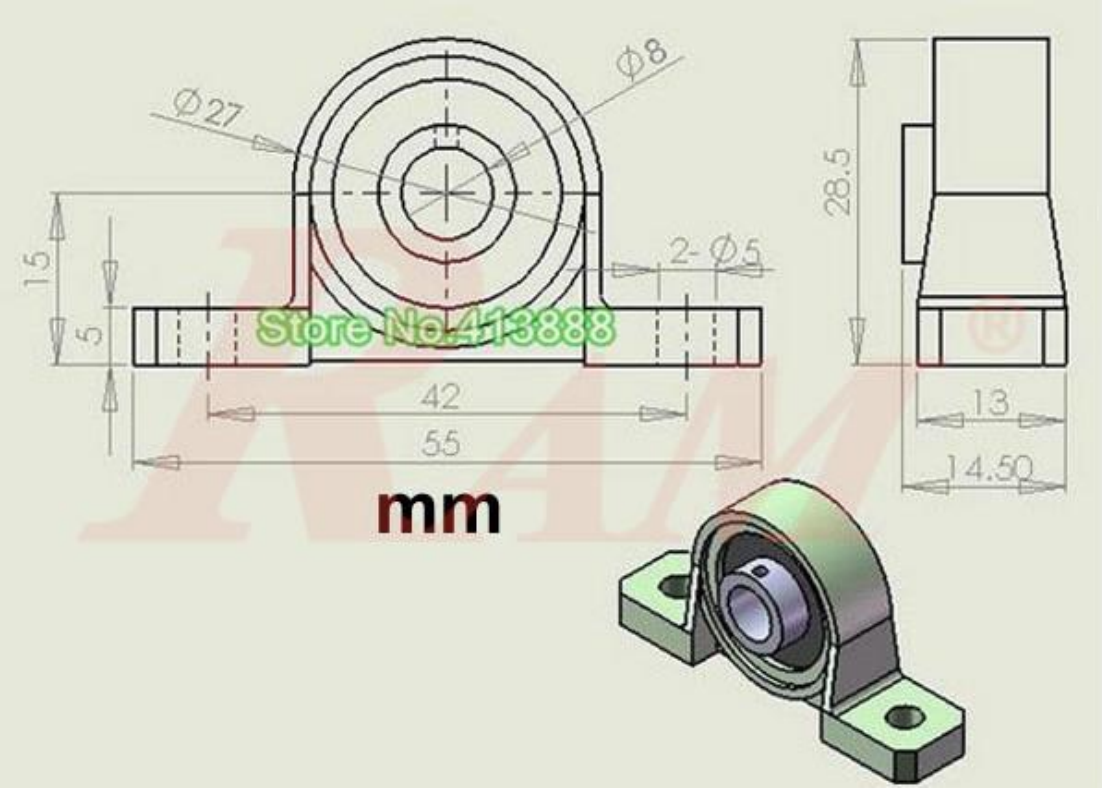
Conclusion

These values mathematically verify that the selected 10 mm linear AISI 1045 Steel is exceptionally rigid and will comfortably withstand all dynamic operational stresses without yielding or compromising the robot's mobility.

4. Bearing Selection and Placement Justification

The bearings used in this design are Lead Screw T8 KP08(self-A;igning Flange) End Support Pillow Bearings with an 8 mm bore diameter. The shafts that fit into the bearings also have a diameter of 8 mm. Four pillow block bearings are used in total, with two KP08 bearings supporting each wheel on separate shafts.

The selected KP08 bearings feature a zinc alloy housing pillow block and are suitable for lead screw applications, providing easy installation, precise alignment, and efficient radial load support.



Bearing Type

- KP08 - 8 mm diameter zinc alloy pillow block bearing
- Supports small diameter shafts (8 mm)
- Mounted housing for easy installation
- Suitable for lead screw and wheel-shaft applications

Loads on Bearings

- Radial load from wheel weight and shaft weight
- Minor dynamic loads due to movement and small shocks
- Mostly light loads since the vehicle mass is 10.669kg

Bearing Life

- Load is mostly constant
- Standard industrial bearing life assumed
- Proper installation and small load result in long service life

Justification for KP08 Pillow Block Bearings

- Able to carry radial loads efficiently
- Supports small diameter shafts
- Simple installation and alignment
- Low maintenance required
- Reduces vibration and ensures smooth wheel rotation

Load Calculation

Load Calculation During Operation

The total mass of the vehicle during operation includes:

- Vehicle mass: 9.669 kg
- Additional payload (garbage): approximately 0.5 kg
- Robotic arm weight when stationary: 1 kg
- **Total operational mass:**

$$m_{total} = 9.669 + 0.5 + 1 = 11.169 \text{ kg}$$

The total weight acting on the system is:

$$W = m_{total} \times g = 11.169 \times 9.81 \approx 109.56\text{N}$$

The vehicle is supported by four wheels, but only two wheels are mounted on driven shafts with pillow block bearings. Therefore, the load per driven wheel is approximately:

$$\text{Load per driven wheel} = \frac{109.56}{4} \approx 27.4 \text{ N}$$

Since each driven shaft is supported by **two KP08 pillow block bearings**, the radial load on each bearing is approximately:

$$\text{Load per bearing} = \frac{27.4}{2} \approx 13.7 \text{ N}$$

Additional considerations:

- Dynamic effects due to motion and acceleration
- Frictional forces in the wheels and bearings
- Small shocks from surface irregularities

Even with these factors, the **radial load on each bearing** is still relatively low and well within the safe operating range of the KP08 8 mm pillow block bearings.

Load Types Supported:

1. Radial Load (perpendicular to the shaft axis)

- Primary load includes shaft weight, wheel weight, vehicle payload, and robotic arm weight.
- KP08 handles these efficiently at light to moderate speeds.

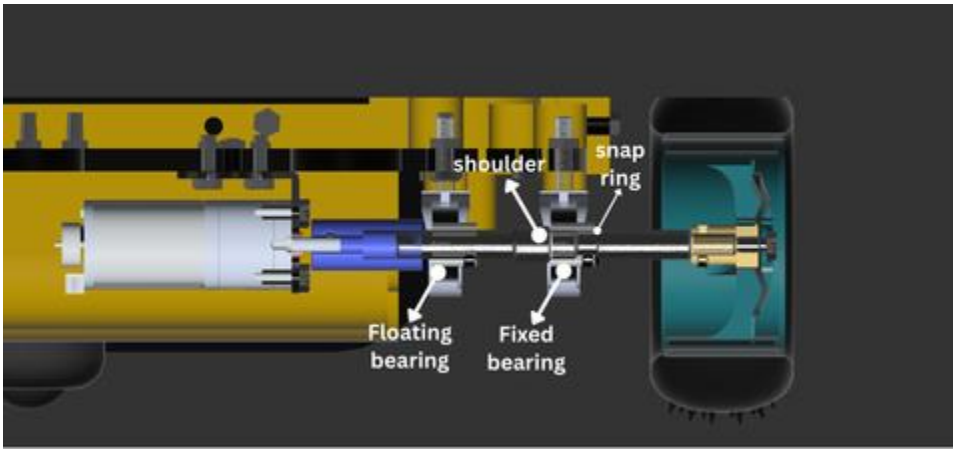
2. Axial Load (along with the shaft axis / thrust load)

- Can withstand minor axial loads due to friction or slight misalignment.
- Not suitable for heavy axial forces.

3. Bending / Moment Load

- Single KP08 bearings cannot support large bending moments.
- Using two bearings per shaft reduces deflection and increases stability.

Bearing Placement Justification (with Calculations)



Each drive shaft in the vehicle is supported by **two 8 mm pillow block bearings**, and each shaft carries **one wheel**. The same configuration is used for the second wheel shaft.

Shaft and bearing dimensions:

- Shaft length: **91.7 mm**
- First bearing: **40.6 mm from the shaft end**
- Second bearing: **73.5 mm from the same reference end**
- Distance between bearings: **25.8 mm**

Bearing Reaction Forces

Assuming the radial load from the wheel is applied at the shaft end, the reaction forces on the bearings can be calculated using static equilibrium:

$$R_1 + R_2 = F$$
$$R_1 \cdot L_2 = R_2 \cdot L_1$$

Where:

- R_1, R_2= reaction forces at bearing 1 and bearing 2
- L_1, L_2= distances from the load to each bearing

For the given positions, the load distribution ensures that **both bearings share the radial load**, reducing the stress on each bearing.

Maximum Shaft Bending

The bending moment between the bearings can be estimated as:

$$M_{max} = R \cdot a$$

- R= bearing reaction force
- a= distance between the bearing and the load

With a **bearing spacing of 25.8 mm**, the bending moment and shaft deflection are minimized

Step 1: Calculate Bearing Reaction Forces

Assuming the load is applied at the shaft end (73.5 mm from reference):

$$R_1 + R_2 = F = 15.9\,\text{N}$$
$$R_1 \cdot L = R_2 \cdot (L_2)$$

- Distance between bearings $L_2 = 25.8\,\text{mm} = 0.0258\,\text{m}$
- Distance from load to Bearing 1 $a = 0.0329\,\text{m}$

$$R_2 = F \cdot \frac{a}{L_2 + a} = 15.9 \cdot \frac{0.0329}{0.0258 + 0.0329}$$
$$R_2 = 15.9 \cdot \frac{0.0329}{0.0587} \approx 15.9 \cdot 0.5606 \approx 8.91\,\text{N}$$
$$R_1 = F - R_2 = 15.9 - 8.91 \approx 6.99\,\text{N}$$

Bearing reactions:

- $R_1 \approx 7.0\,\text{N}$
- $R_2 \approx 8.9\,\text{N}$

Step 2: Maximum Bending Moment

$$M_{max} = R \cdot a$$

- Take **the larger reaction** $R_2 = 8.91$ N
- Distance from bearing to load $a = 0.0258$ m

$M_{max} = 8.91 \cdot 0.0258 \approx 0.23$ N.mm

Step 3: Convert to N·mm

$$M_{max} = 0.23 \times 1000 \approx 230 \text{ N.mm}$$

Result:

- **Bearing Reactions:** $R_1 \approx 7$ N, $R_2 \approx 8.9$ N
- **Maximum Bending Moment on Shaft:** $M_{max} \approx 0.23$ N.m $= 230$ N.mm

Bearing Load Analysis and Selection Justification

KP08 Pillow Block Bearing 8 mm Load Ratings:

- Dynamic load rating (C): ~170 N
- Static load rating (C0): ~85–90 N
- Axial load capacity: light to moderate (< 20 N)
- Radial load capacity: primary load, excellent
- Moment / Bending: weak for large bending; two bearings per shaft recommended

Calculated Loads on Shaft / Bearings in This Project:

- Vehicle mass: 9.669 kg
- Additional payload (garbage): 0.5 kg
- Robotic arm (stationary): 1 kg
- Total operational mass: 11.169 kg → total weight $W = 11.169 \cdot 9.81 \approx 109.56$N
- Each driven wheel supports ~27.4 N
- Each shaft has 2 KP08 bearings → radial load per bearing: ~13.7 N
- Maximum bending moment on shaft between bearings: 0.23 N·m (230 N·mm)

Comparison and Justification:

Parameter	Shaft/Bearing Load	KP08 Capacity	Status
Radial load per bearing	13.7N	170 N	Safe
Static load	13.7 N	85–90 N	Safe
Axial load	< 1 N	< 20 N	Safe
Maximum bending moment	0.23 N·m	low (shaft supported by 2 bearings)	Acceptable

Conclusion:
The calculated loads on the shaft and bearings are well below the KP08 pillow block bearing limits. Radial and axial loads are safely supported, and the maximum bending moment is minimized by using two bearings per shaft. Based on these numbers, KP08 Pillow Block Bearing 8 mm is the most suitable choice for this application, ensuring smooth rotation, low deflection, and reliable operation.

ELECTRICAL DESIGN

1. Block Diagram

The system block diagram illustrates the overall architecture of the project and how the different hardware components are interconnected. The communication between the Raspberry Pi and the ESP32 microcontroller is implemented using UART communication, while Wi-Fi is used for wireless connectivity and remote interaction.

The Raspberry Pi is responsible for processing data from the sensors. It is connected to an Ultrasonic sensor for distance measurement and an IMU sensor for motion and orientation detection.

The ESP32 microcontroller is responsible for controlling the actuators. It is connected to two DC motors with encoders through an H-Bridge motor driver for motion control. In addition, it controls three servo motors, a buzzer, and an LED indicator.

The power system is also illustrated in the block diagram. Each microcontroller is powered using a separate 12V battery, and a buck converter is used to step down the voltage to the appropriate levels required by the electronic components.

For safety and control, an emergency switch is included to immediately stop the motors, while a separate ON/OFF switch is used to control the ESP32 power supply.

All these components and their connections are clearly represented in the block diagram to provide a complete overview of the system design.

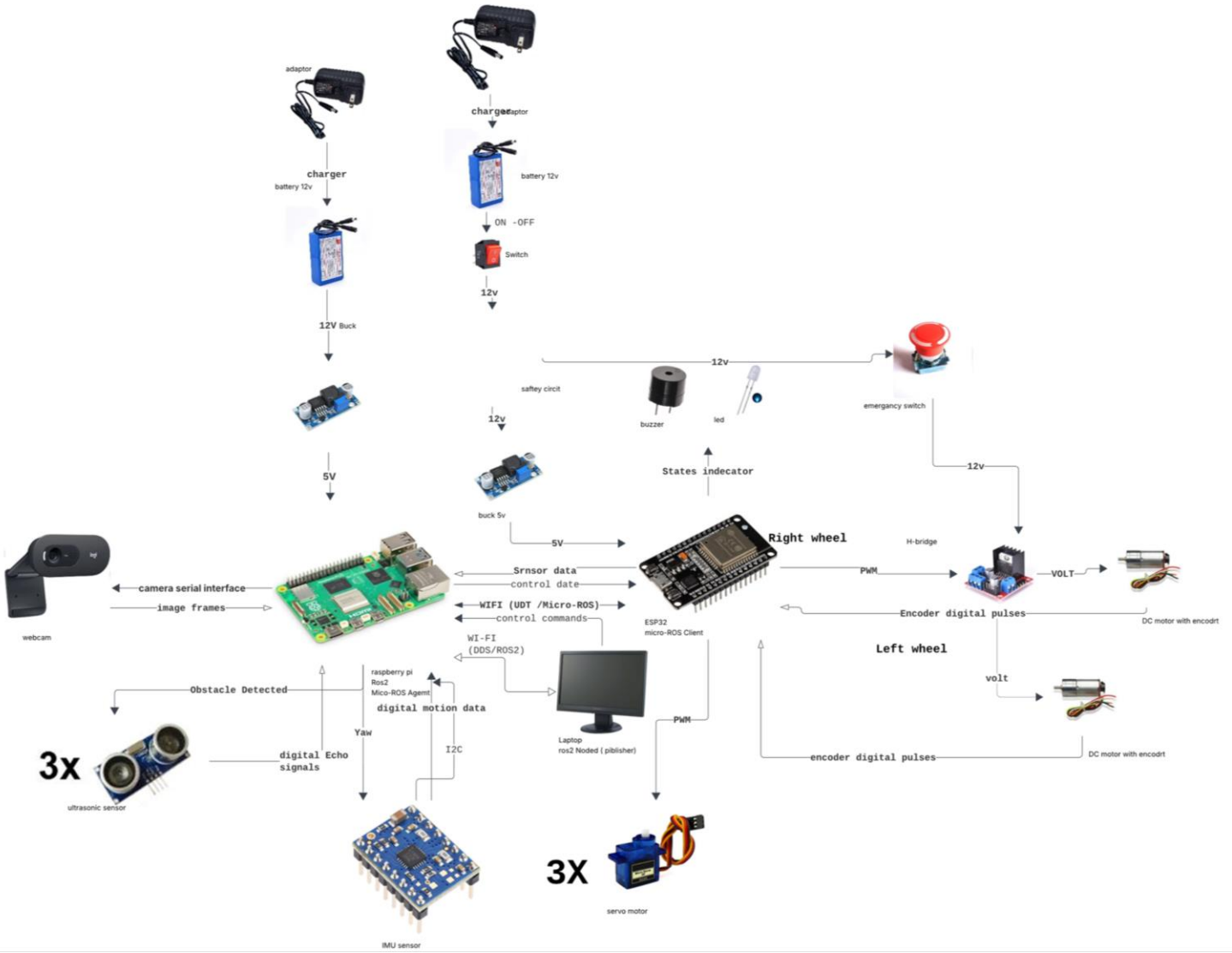


Figure 21 Block Diagram

2. Wiring Diagram

Description

The wiring diagram illustrates the electrical connections of a robotic system composed of two main control sections: a microcontroller-based motion control unit and a Raspberry Pi–based sensing and vision unit. The system integrates power management modules, motor drivers, sensors, and actuators.

a) Power Supply Section

The system is powered by two separate sources to ensure stable operation. A 12V battery is connected to wagos to supply the primary control circuit. Voltage regulation modules (DC–DC buck converters) are used to step down and stabilize the voltage before supplying it to the microcontroller and other components. These regulators ensure that sensitive electronics receive appropriate voltage levels.

A separate 5v battery pack is used to power the Raspberry Pi and the camera module to avoid power fluctuations caused by motors and servos.

b) Microcontroller and Motion Control

The central controller for the motion system is an ESP32 microcontroller board. It receives regulated power from the voltage converter and acts as the main controller for motors and servos.

The ESP32 is connected to an L298N motor driver module, which controls two DC motors. The motor driver receives power from the regulated supply and allows the ESP32 to control motor direction and speed through digital control signals.

Three servo motors are also connected directly to the ESP32. These servos are powered from the regulated supply and controlled using PWM signals from the microcontroller. They are likely used for mechanical movements such as steering, camera positioning, or robotic arm control.

c) Sensor and Vision Processing Unit

A Raspberry Pi serves as the high-level processing unit responsible for sensor data processing and computer vision tasks.

Three ultrasonic distance sensors are connected to the Raspberry Pi through GPIO pins. Each sensor uses trigger and echo pins to measure distance by sending ultrasonic waves and detecting the reflected signal. These sensors provide obstacle detection and distance measurement for navigation.

An MPU6050 inertial measurement unit (IMU) is also connected to the Raspberry Pi using the I2C communication protocol. This sensor provides acceleration and gyroscope data, which can be used for orientation detection and motion analysis.

Additionally, a camera module is connected to the Raspberry Pi through the camera interface. The camera enables image capture and computer vision processing, which can be used for object detection, navigation, or monitoring.

d) Control and System Integration

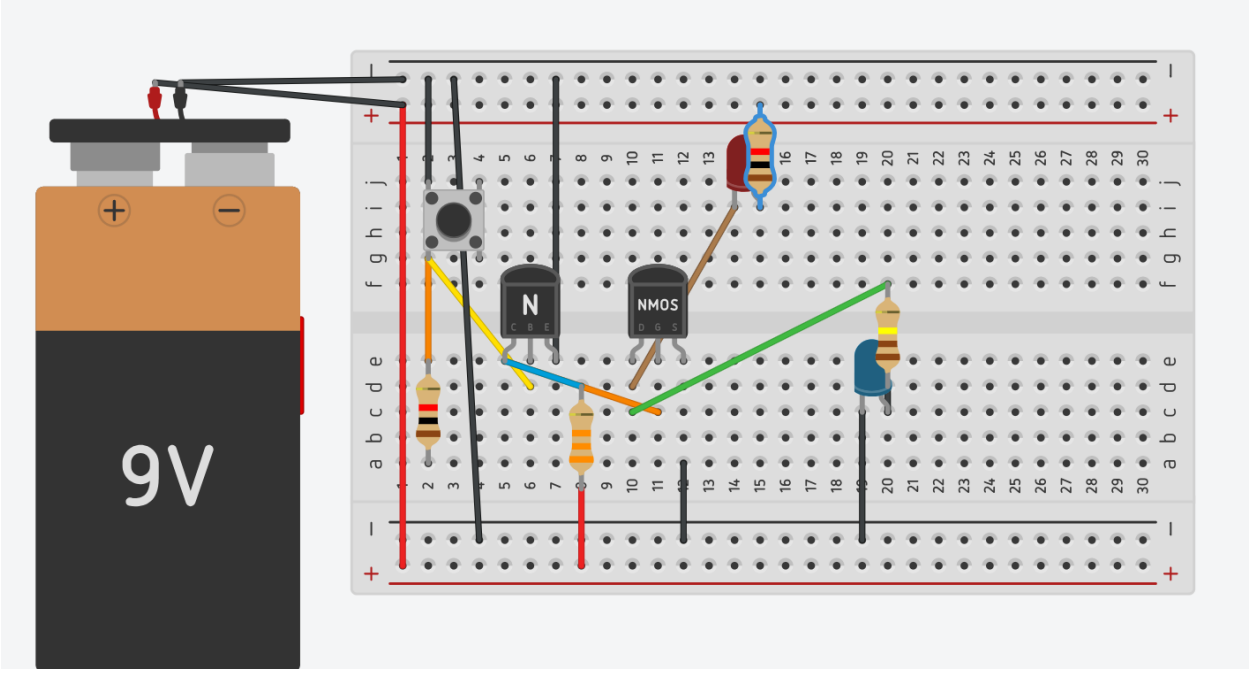
In this design, the ESP32 handles low-level hardware control such as motor driving and servo actuation, while the Raspberry Pi manages high-level processing tasks including sensor fusion, distance analysis, and visual data processing.

Separating the control tasks between the ESP32 and the Raspberry Pi improves system performance and ensures that time-sensitive motor control operations are not interrupted by computationally intensive vision processing.

e) Safety Circuit

The breadboard is used to distribute power and connect supporting components such as resistors, switches, and voltage regulators which will be replaced with PCB board. A push button switch is included for manual control, reset, or activation of the system.

Overall, the wiring configuration ensures proper power regulation, organized signal routing, and reliable communication between sensors, actuators, and processing units within the robotic system.



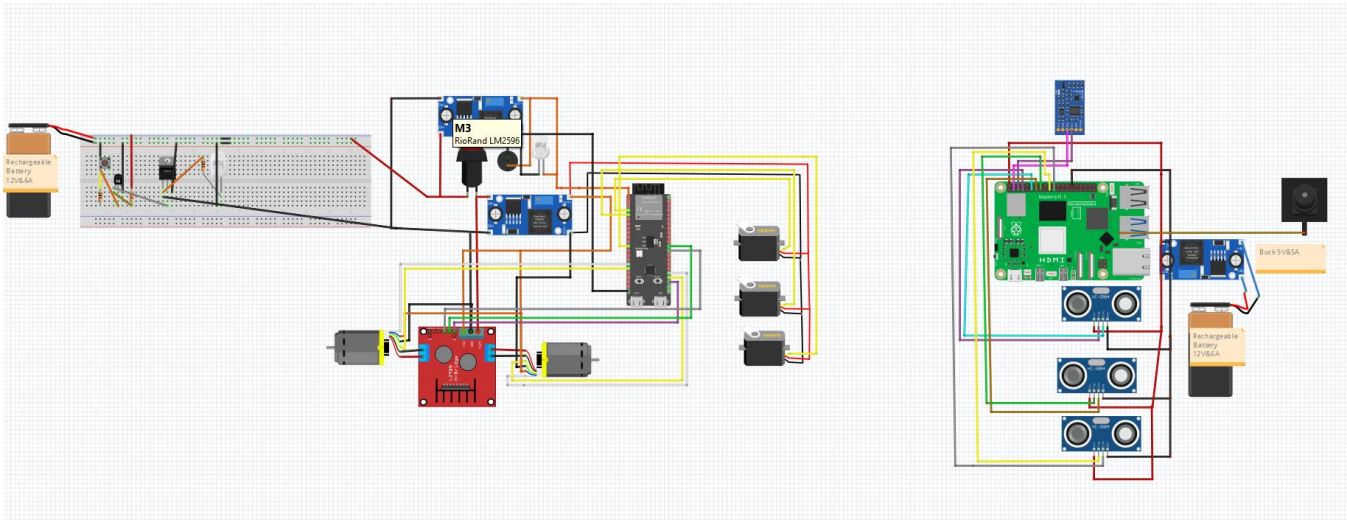


Figure 22 wiring diagram

3. Battery Sizing Calculations

a) Current analysis

All Loads in the System

Computing

- Raspberry Pi 5
- ESP32

Sensors

- IMU
- Ultrasonic Sensors (×3)

Vision

- Webcam

Motion

- DC Motors with Encoders (×2)
- Servo Motors (MG ×2)
- Servo motor (SG90 ×1)

Drivers & Power Electronics

- H-Bridge Motor Driver
- Buck Converter

Indicators & Alerts

- LED
- Buzzer

Safety & Control

- Over-Current Protection Circuit
- Emergency Stop Switch
- ON/OFF Switch

Communication

- UART link between Raspberry Pi and ESP32

Table 6 current and voltage consumption

Component	Voltage	Current
Esp 32	3.3v	100mA
Raspberry Pi 5	5v	5 A
IMU	3.3-5v	10mA
Ultrasonic Sensors	5v	20 mA
Webcam	5v	500mA
DC Motors with Encoders	12v	1.1A
Servo mg 90	5v	250 mA
Servo motor sg 90	5v	250mA
LED	3.3v	35mA
Buzzer	5v	50 mA
H-bridge	12v & 5 v	90 mA

ESP32 Peripheral Components

Esp32+ (2*dc motor with encoder) +(2* servo motor mg 90)+servo motor sg 90 +led +buzzer +H-bridge=

100mA+10mA+(2*1.1A) +(2*250mA) +250mA+35mA+50mA+90mA=3.3A

Raspberry Pi 5 Peripheral Components

Raspberry Pi 5+IMU+ webcam+(3* ultrasonic) = 5A+10mA+500m+(3*20mA)=5.57 A

b) Calculate battery capacity

Define number of hours needed for operation

Operating time = 2 hours

Calculate battery capacity (mAh) for Esp 32 *(I_{max} * operating time)*

3.3*2=6600 mAh

Add safety margin (125% – 200%)

6600 × 1.75 ≈ **11550 mAh**

Calculate battery capacity (mAh) for raspberry pi = I_{max} * *operating time*

5.57*1.5=8355 mAh

Add safety margin (125% – 200%)

8355× 1.25 ≈ **10444 mAh**

c) power consumption

ESP32 System

Specifications:

- Voltage: 12V
- Maximum Current: 3.3 A

Power Calculation:

$$P = V \times I$$
$$P = 12 \times 3.3 = 39.6\,\text{W} \approx 40\,\text{W}$$

Battery Energy:

- Battery capacity: 11,550 mAh = 11.55 Ah
- Energy stored in battery (Wh):

$$E = V \times Ah = 12 \times 11.55 = 138.6\,\text{Wh}$$

Theoretical Runtime at 3.3 A:

$$t = \frac{\text{Battery Capacity (Ah)}}{\text{Load Current (A)}} = \frac{11.55}{3.3} \approx 3.5\,\text{hours}$$

The battery can safely power the ESP32 system and peripherals for 2 hours with margin.

Raspberry Pi 5 System

Specifications:

- **Voltage:** 12V
- **Maximum Current:** 6 A

Power Calculation:

$$P = V \times I$$
$$P = 12 \times 6 = 72\,\text{W}$$

Battery Energy:

- Battery capacity: 10,444 mAh = 10.444 Ah

Energy stored in battery (Wh):

$$E = V \times Ah = 12 \times 10.444 \approx 125.33 \text{ Wh}$$

Theoretical Runtime at 6 A:

$$t = \frac{\text{Battery Capacity (Ah)}}{\text{Load Current (A)}} = \frac{10.444}{6} \approx 1.74 \text{ hours}$$

The battery can safely power the Raspberry Pi 5 system and peripherals for 2 hours with a small safety margin

System	Voltage (V)	Max Current (A)	Power (W)	Battery Capacity (mAh)	Energy (Wh)	Notes
ESP32 + Peripherals	12	3.3	39.6	11,550	138.6	Reliable for 2 hours
Raspberry Pi 5 + Peripherals	12	6	72	10,444	125.3	Reliable for 2 hours

The system requires:

- ESP32 system: a 12V battery with a maximum output current of 3.3A, a total capacity of 11,550 mAh, providing about 39.6 W of power, enough to reliably operate all components for 2 hours with a safety margin.
- Raspberry Pi 5 system: a 12V battery with a maximum output current of 6A, a total capacity of 10,444 mAh, providing about 72 W of power, enough to reliably operate all components for 2 hours with a safety margin.

These battery specifications ensure stable operation and sufficient energy for all connected peripherals.

4. Safety Circuit

a) Simulation

A safety circuit was designed to protect the system from overcurrent and short circuit conditions. This circuit ensures that the electronic components, especially the ESP32 and the motors, are protected from excessive current that may damage the system.

The protection circuit is implemented using a MOSFET (Z44) as the main switching element, along with a BC547BP transistor, resistors, an LED indicator, and a push button for resetting the circuit. The resistors were carefully selected to provide the appropriate biasing and current limiting for the components.

The circuit operates with an input voltage of 12V, which represents the main power supply of the system. The output of the circuit is connected to the ESP32 section and the motor drivers. Under normal operating conditions, the circuit allows current to pass to the load.

In the event of a short circuit or overcurrent, the protection circuit automatically disconnects the load to prevent damage. At the same time, the LED indicator turns on to signal that a fault condition has occurred. The load remains disconnected until the short circuit is removed, after which the system can be reset using the push button.

Before fabrication, the circuit was tested through simulation using LTspice and Proteus to verify its behavior under fault conditions. After successful simulation, the circuit was implemented and tested on a breadboard to ensure correct operation before designing and manufacturing the PCB.

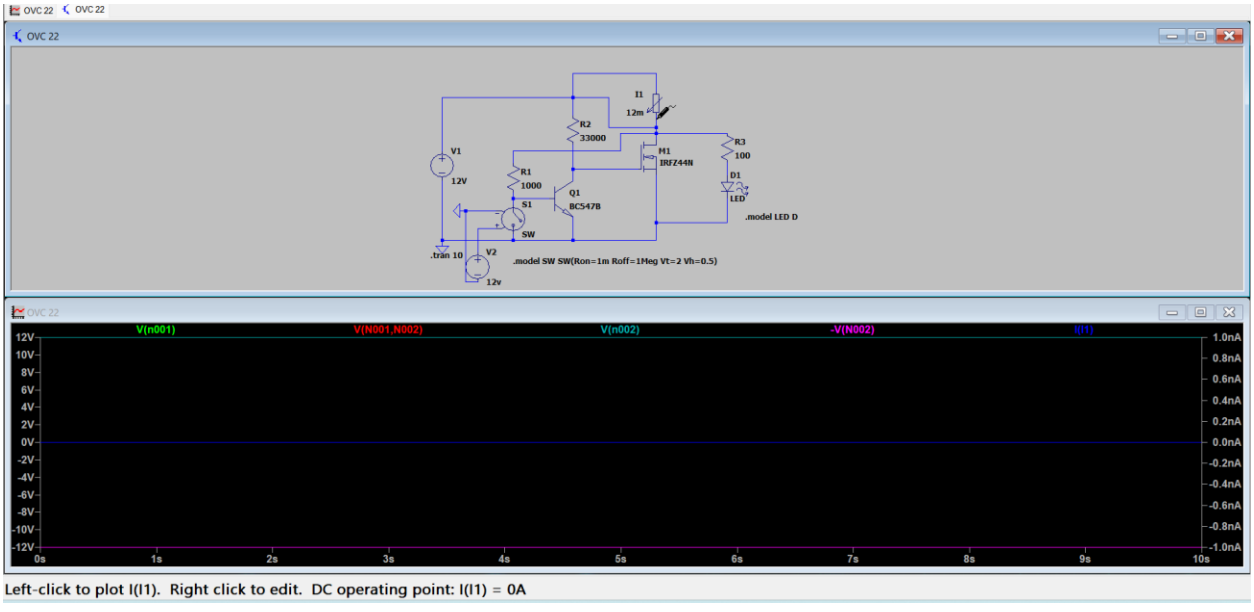


Figure 23 Safety circuit during Short circuit

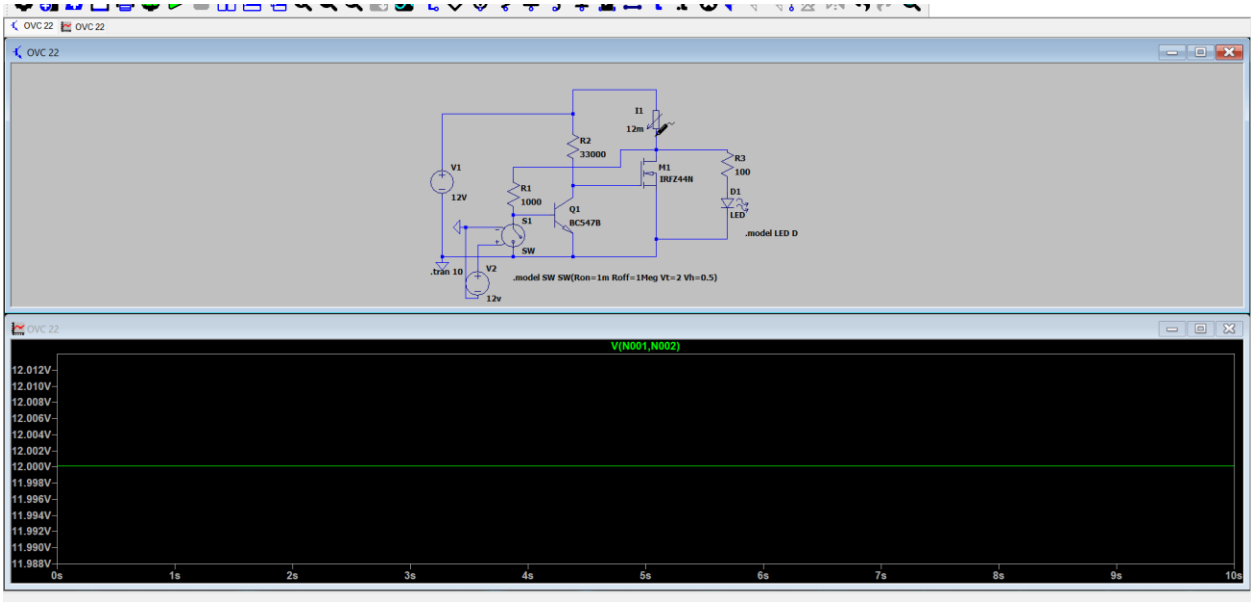


Figure 24 Safety circuit during normal

b) PCB Design

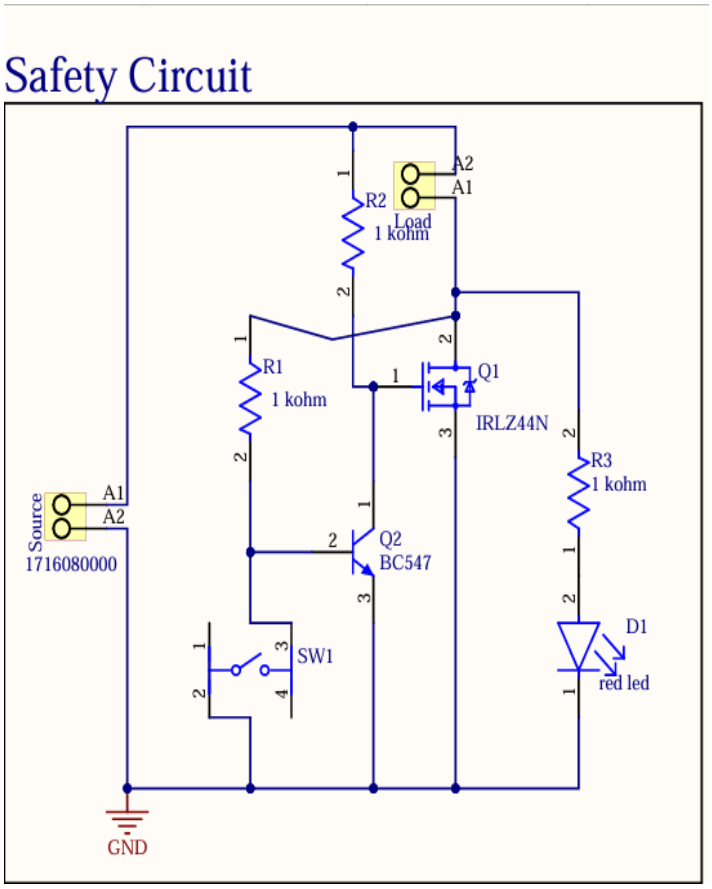


Figure 25 Safety circuit schematic

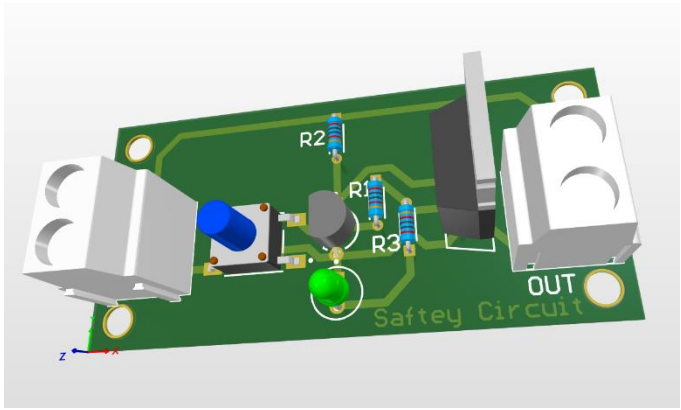


Figure 26 Safety circuit PCB

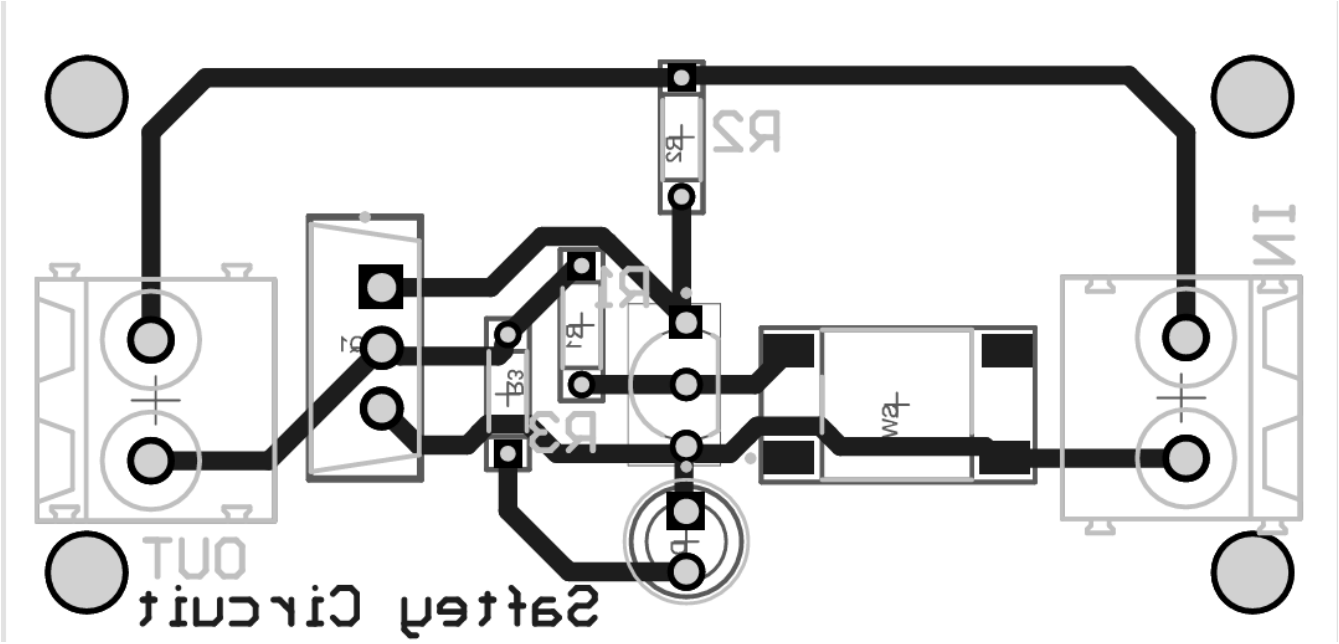


Figure 27 PCB Layout View

This PCB layout view are mirrored to be printed as I want. Also the track width is 0.8mm as this is the minimum size the manufacturer can do.

c) Validation

At normal state read 12.178 V when connected to a power supply 12V

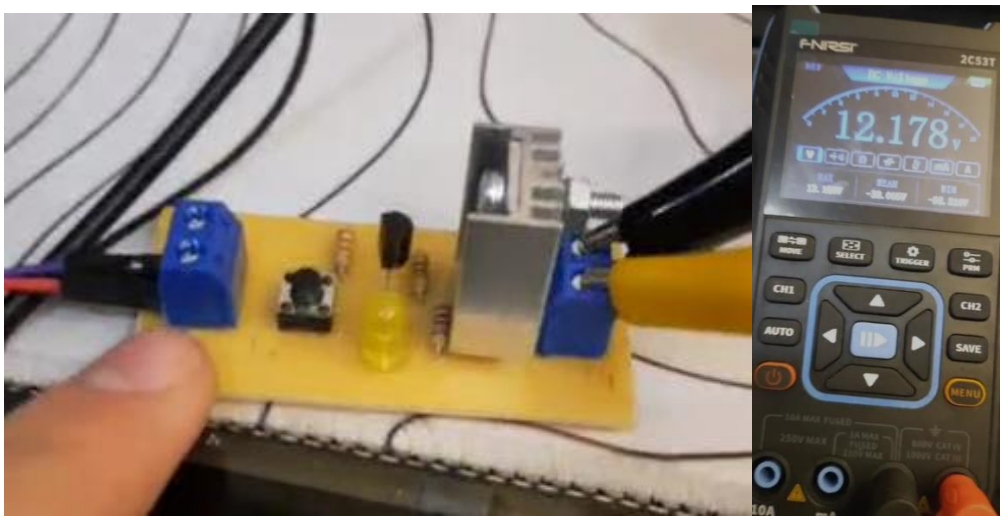


Figure 28 Normal case

At short circuit the indicator led will turn on and the multimeter reads 0.00 V

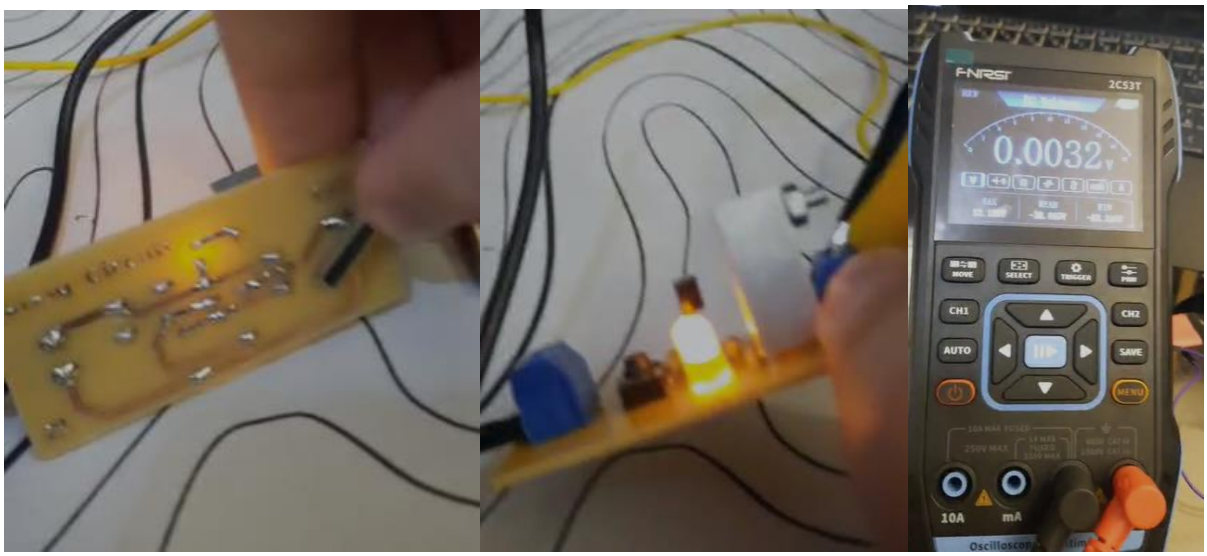


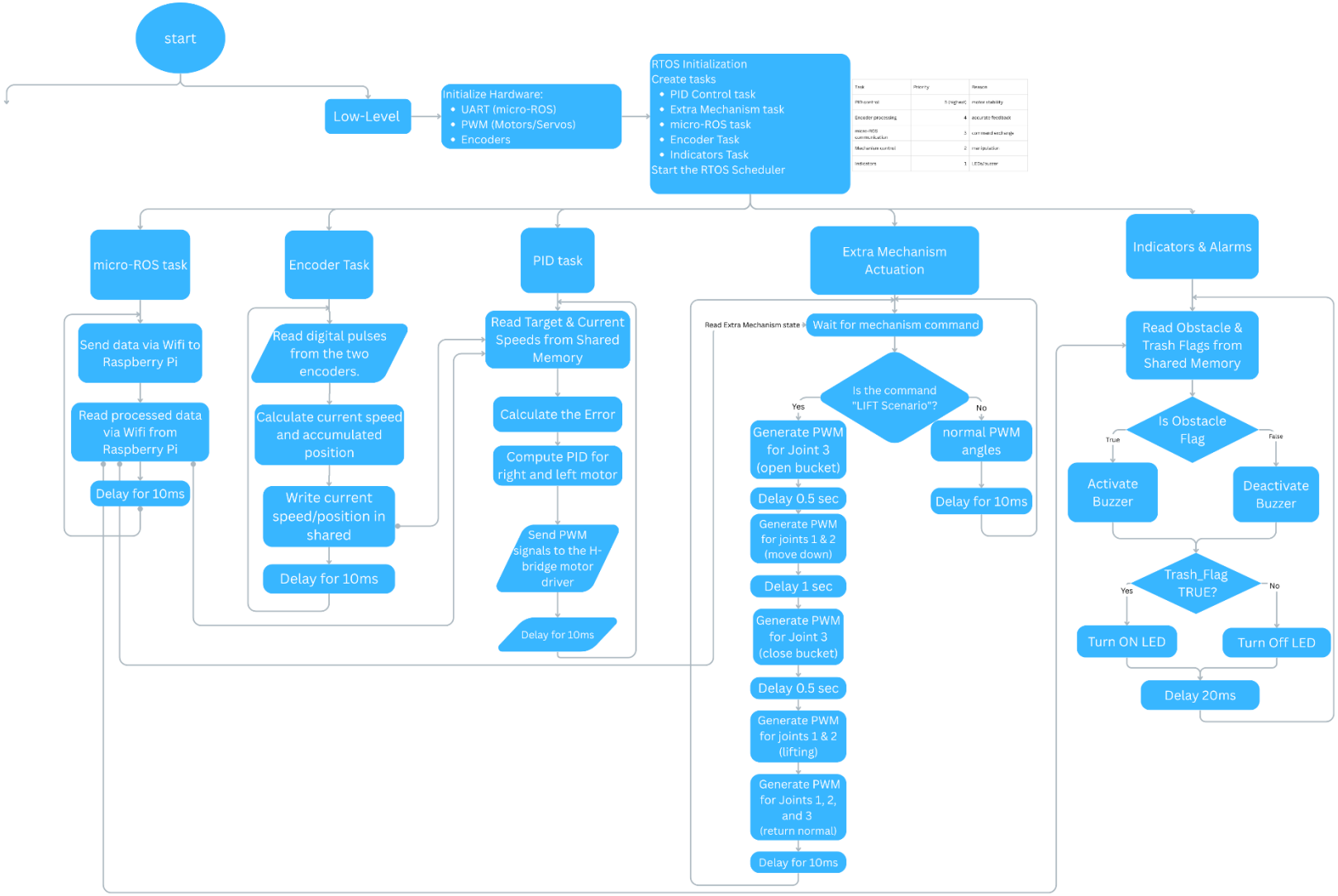
Figure 29 Short circuit case

SOFTWARE & CONTROL

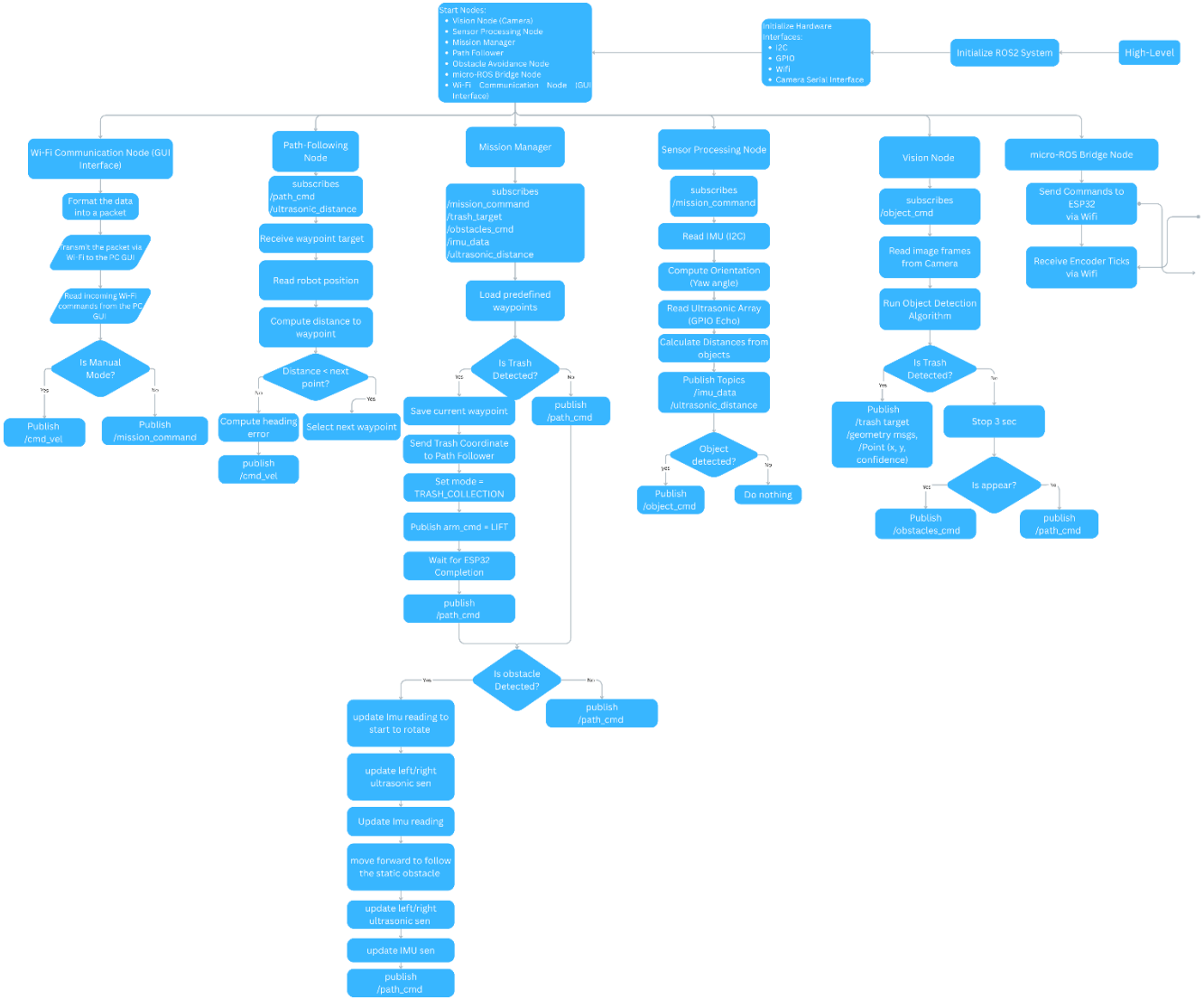
1. Software Flowchart



a) Low level Flowchart



b) High level Flowchart



2. Motor Control and PID

The robot navigation system is based on a real-time control architecture implemented using **FreeRTOS** on the ESP32 platform. Multiple tasks execute simultaneously for motor control, sensor acquisition, communication, and system monitoring, ensuring deterministic performance and efficient resource utilization.

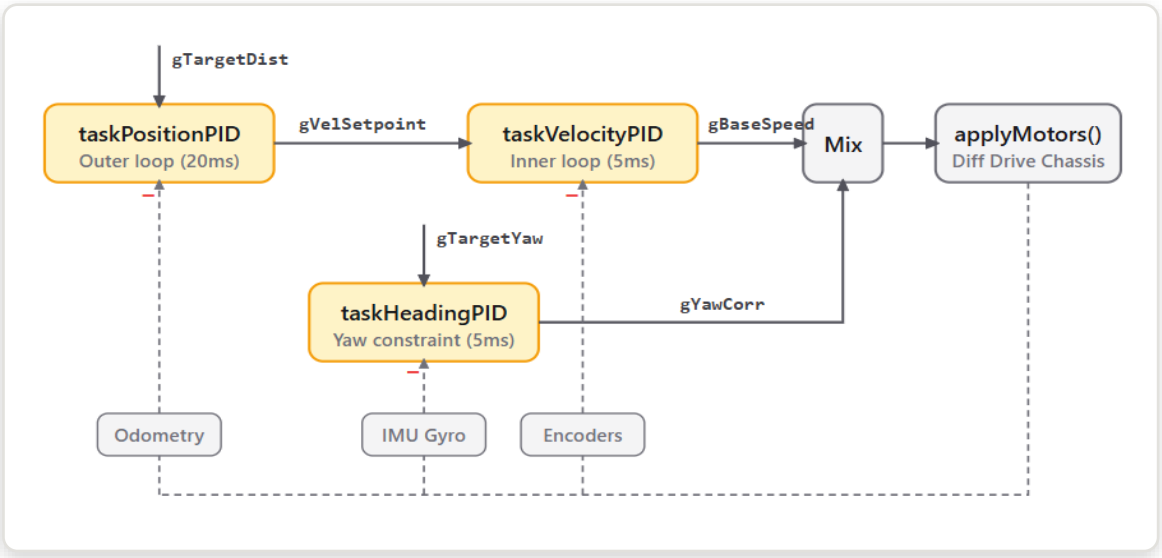


Figure 30 cascaded PID loop

A **PID (Proportional–Integral–Derivative) controller** was implemented to regulate both linear motion and robot orientation. The controller continuously calculates the error between the desired trajectory and the actual robot position and generates appropriate motor commands to minimize this error.

Feedback information is obtained from two primary sensing systems:

- Wheel encoders provide accurate measurements of wheel displacement and rotational speed.
- The IMU supplies orientation and heading information used for direction correction.

By combining encoder and IMU feedback, the robot maintains straight-line motion, performs accurate turns, and compensates for external disturbances or wheel slippage.

The PID parameters were experimentally tuned through multiple testing sessions to achieve fast response, smooth movement, minimal overshoot, and stable operation. The robot successfully demonstrated accurate forward motion, controlled rotation, and repeatable navigation performance during practical testing.

The integration of RTOS scheduling, sensor feedback, and PID control resulted in a robust real-time motion control system capable of supporting autonomous operation with high precision.

a) The Outer Loop

taskPositionPID (Runs every 20ms): It looks at how far the robot has traveled compared to gTargetDist.

- **The Math:** It calculates the *position error* (Target Distance − Actual Distance).
- **The Output:** It does **not** output motor power. Instead, it outputs a *desired speed* (gVelSetpoint).
- **Behavior:** If the robot is 2 meters away from the target, this block requests maximum speed (VEL_MAX). As the robot gets within 30cm (DECEL_DIST), this block smoothly lowers the gVelSetpoint down to a crawl (CREEP_VEL) so the robot stops exactly on the mark without overshooting.

b) The Inner Loops

These loops run much faster (every 5ms) to react to immediate physical disturbances (like bumps, varying battery voltage, or wheel slip) before the outer loop even notices them.

- taskVelocityPID. It tries to make the wheels spin exactly as fast as the taskPositionPID requested.
  - **The Math:** It calculates the *velocity error* (gVelSetpoint − Actual Wheel Speed from Encoders).
  - **The Output:** It outputs the actual base PWM motor power (gBaseSpeed).
  - **Behavior:** If the robot hits a thick patch of carpet and slows down, this PID loop immediately spikes the motor PWM to maintain the requested speed, acting like cruise control in a car.
- taskHeadingPID: It runs completely parallel to velocity, solely responsible for keeping the robot's nose pointed at the target.
  - **The Math:** It calculates the *heading error* (gTargetYaw − Actual Gyro Heading).
  - **The Output:** It outputs the rotational steering correction (gYawCorr).
  - **Behavior:** If one wheel drags or a bump knocks the robot slightly off course, this loop generates an immediate counter-steering PWM value to snap the heading back straight.

- Mix: This is the applyMotors() differential drive mixer we just discussed. It mathematically combines the forward thrust (gBaseSpeed) from the Velocity PID with the steering command (gYawCorr) from the Heading PID.
- applyMotors() / Diff Drive Chassis: This represents the physical L298N/BTS7960 motor drivers, the DC motors, and the physical mass of the ECOBee chassis moving through the real world.

c) The Feedback Layer (The "Senses")

Sensors measure the real-world results of the motors and feed that data backward (the dashed lines) into the negative (−) terminals of the PID loops to calculate the new errors.

- Odometry (taskOdometry): Runs at 20ms. It uses trigonometry to estimate the robot's X/Y position and total distance driven. It feeds this distance back to the **Position PID**.
- Encoders (Hardware ISRs): Real-time hardware interrupts that count wheel clicks. The Velocity PID reads these every 5ms to calculate the exact current speed in meters per second, feeding back to the **Velocity PID**.
- IMU Gyro (taskIMU): Runs exceptionally fast (2ms). It integrates the MPU-6050's Z-axis rotation into a continuous compass heading. It feeds this heading back to the **Heading PID**.

SIMULATION

1. Environment Setup

The simulation runs in **Gazebo Harmonic (GZ Sim)** using a custom world file (my_world.sdf). The environment replicates a restaurant floor enclosed by four boundary walls (wall_north, wall_south, wall_east, wall_west). The interior is furnished with dining tables, chairs, decorative plants, a counter, and a water bottle all representing realistic clutter a service robot would encounter in an indoor restaurant setting. Two tables (`obs_table_1`, `obs_table_2`) are deliberately placed along the robot's navigation path to serve as static obstacles. The robot model is a custom differential-drive platform equipped with front and left ultrasonic sensors, an IMU, wheel encoders (odometry), and a 3-DOF robotic arm, spawned at the home position within the restaurant floor.



Figure 31 Gazebo Environment

2. Static Obstacle Avoidance

The robot uses a **Finite State Machine (FSM)** to handle static obstacles detected by the front ultrasonic sensor (`/ultrasonic`). When the sensor reading falls below the safety threshold (0.5 m), the robot stops and enters a confirmation phase — it waits for 3 seconds to confirm the obstacle is genuinely blocking the path and not a transient reading. Once confirmed, the FSM executes a structured avoidance maneuver:

Brake → the robot applies a 0.6-second brake to stabilize.

Turn right 90° → using IMU-based yaw tracking.

Follow left wall → the robot drives forward while the left sensor detects the obstacle beside it.

Turn left 90° → once the left sensor loses the obstacle (end of the object).

Drive past → continues forward for a minimum of 2 seconds to fully clear the obstacle.

Turn left 90° → heads back toward the original line.

Return to line → drives a distance equal to the sideways offset recorded earlier.

Turn right 90° → realigns with the original heading and resumes the path.

This sequence ensures the robot navigates around the obstacle and returns precisely to its planned trajectory without collision.

Front sensor detection and confirmation logic:

```
def front_callback(self, msg: LaserScan):
    r = np.array(msg.ranges, dtype=float)
    r = np.nan_to_num(r, nan=10.0, posinf=10.0, neginf=10.0)
    self.front_distance = float(np.min(r))

# Inside SQUARE_FWD state:
if self.front_detected():
    if self.obstacle_confirm_start is None:
        self.stop()
        self.obstacle_confirm_start = self.get_clock().now()
    elif self.confirm_elapsed_sec() >= OBSTACLE_CONFIRM_SEC:
        self.obstacle_confirm_start = None
        self.start_brake()
        self.state = "BRAKE_OBS"
    else:
        self.stop()
```

Avoidance maneuver — follow left wall, then return to original line:

```
elif self.state == "FOLLOW_LEFT_WALL":
    if self.left_detected():
        self.publish_cmd(self.LIN_SPEED, 0.0)
    else:
        self.stop()
        self.sideways_dist_m = self.leg_distance()
        self.start_brake()
        self.state = "BRAKE_LEFT_1"

elif self.state == "RETURN_TO_LINE":
    dist = self.leg_distance()
    if dist < self.sideways_dist_m:
        self.publish_cmd(self.LIN_SPEED, 0.0)
    else:
        self.stop()
        self.start_brake()
        self.state = "BRAKE_RIGHT_FINAL"
```

3. Dynamic Obstacle

A dynamic obstacle is introduced at **92.2 seconds** of simulation time. The node spawns a colored box (0.5 m × 0.5 m × 0.5 m) at coordinates (−2.0, −2.5) directly in the robot's path using the `gz service` API to call `/world/restaurant_world/create` with a runtime-generated SDF. The box remains for **2 seconds**, then is automatically removed via the `/world/restaurant_world/remove` service.

When the robot encounters this box, the same front-sensor detection triggers the FSM's obstacle confirmation phase. However, because the box disappears within the 3-second confirmation window, the sensor readings clear before the timer expires. The robot logs "Obstacle cleared – resuming" and continues forward without initiating the full avoidance maneuver. This demonstrates the system's ability to distinguish dynamic obstacles from static ones — it only commits to avoidance after sustained confirmation, and resumes immediately once the path is free.

Spawn and remove logic:

```
 BOX_SPAWN_SEC    = 92.2
 BOX_LIFETIME_SEC = 2.0
 BOX_X, BOX_Y     = -2.0, -2.5
 BOX_SIZE         = 0.5

 def spawn_box(self):
     sdf_content = f"""<?xml version="1.0" ?>
     <sdf version="1.7">
       <model name="dynamic_box">
         <static>1</static>
         <pose>{BOX_X} {BOX_Y} 0 0 0 0</pose>
         <link name="link">
           <collision name="col">
             <pose>0 0 {BOX_SIZE/2} 0 0 0</pose>
             <geometry><box><size>{BOX_SIZE} {BOX_SIZE}
{BOX_SIZE}</size></box></geometry>
           </collision>
           <visual name="vis">
             <pose>0 0 {BOX_SIZE/2} 0 0 0</pose>
             <geometry><box><size>{BOX_SIZE} {BOX_SIZE}
{BOX_SIZE}</size></box></geometry>
           </visual>
         </link>
       </model>
     </sdf>"""
```

```
    subprocess.Popen(['gz', 'service', '-s',
        '/world/restaurant_world/create', ...])

def remove_box(self):
    subprocess.Popen(['gz', 'service', '-s',
        '/world/restaurant_world/remove', ...])
```

Timed spawning inside the control loop:

```
elapsed_since_start = sim_now - self.node_start_time

if not self.box_spawned and elapsed_since_start >= BOX_SPAWN_SEC:
    self.box_spawned    = True
    self.box_spawn_time = sim_now
    self.spawn_box()

elif self.box_spawned and not self.box_removed:
    if sim_now - self.box_spawn_time >= BOX_LIFETIME_SEC:
        self.box_removed = True
        self.remove_box()
```

4. IMU and Encoders Integration

Both IMU and encoder data are actively used throughout navigation:

IMU (Inertial Measurement Unit): The robot subscribes to `/imu` and extracts yaw from the orientation quaternion. To handle angle wrapping (±π discontinuities), the node maintains an **unwrapped yaw** accumulator that tracks continuous rotation. Every 90° turn — whether at a square corner or during obstacle avoidance — is executed by computing the delta between the current unwrapped yaw and the yaw recorded at turn start, rotating until the accumulated angle reaches 90° within a 5° tolerance. This provides precise, drift-free angular control.

```
 def imu_callback(self, msg: Imu): q = msg.orientation siny_cosp = 2.0 * (q.w * q.z + q.x * q.y) cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z) raw_yaw = math.atan2(siny_cosp, cosy_cosp) self.current_yaw = raw_yaw if self.last_raw_yaw is None: self.last_raw_yaw = raw_yaw self.yaw_unwrapped = raw_yaw return delta = self.normalize_angle(raw_yaw - self.last_raw_yaw) self.yaw_unwrapped += delta self.last_raw_yaw = raw_yaw
```

IMU-based turning execution:

```
def execute_turn(self) -> bool:
    delta_yaw       = self.yaw_unwrapped - self.turn_start_yaw
    accumulated_deg = math.degrees(-self.turn_direction * delta_yaw)
    remaining_deg   = 90.0 - accumulated_deg

    if abs(remaining_deg) <= TURN_TOLERANCE_DEG:
        self.stop()
        return True

    if remaining_deg > 0:
        self.publish_cmd(0.0, self.turn_direction * ANG_SPEED)
    else:
        self.publish_cmd(0.0, -self.turn_direction * ANG_SPEED)
    return False
```

Encoders / Odometry: The robot subscribes to `/diff_drive_controller/odom` and reads the (x, y) position from the odometry frame. At the start of each path leg, the current position is

saved as a reference point. The node continuously computes the Euclidean distance traveled from that reference. When the distance reaches the target (3.0 m per leg), the robot transitions to the next FSM state. During obstacle avoidance, the odometry also records the sideways offset so the robot can return to the exact original line after clearing the obstacle.

```
 def odom_callback(self, msg: Odometry): self.odom_x = msg.pose.pose.position.x
self.odom_y = msg.pose.pose.position.y
 def set_leg_start(self): self.leg_start_x = self.odom_x self.leg_start_y =
self.odom_y
 def leg_distance(self) -> float: dx = self.odom_x - self.leg_start_x dy = self.odom_y
- self.leg_start_y return math.sqrt(dx * dx + dy * dy)
```

Usage inside the FSM — leg completion check:

```
 dist = self.leg_distance()
 if dist >= SQUARE_SIDE_M:       # SQUARE_SIDE_M = 3.0
     self.stop()
     self.square_leg += 1
     if self.square_leg >= TOTAL_LEGS:
         self.state = "SQUARE_DONE"
     else:
         self.start_brake()
         self.state = "BRAKE_CORNER"
```

5. Goal Communication

All navigation goals are managed within a **ROS 2 node** (`obstacle_avoidance`), not by manually publishing to topics from the terminal. The node defines the square path parameters internally (side length = 3.0 m, 4 legs, right-turn corners) and autonomously sequences through each waypoint using the FSM.

```
class ObstacleAvoidance(Node): def init(self): super().init('obstacle_avoidance')
   # Subscriptions
   self.create_subscription(LaserScan, '/ultrasonic',      self.front_callback, 10)
   self.create_subscription(LaserScan, '/ultrasonic_left', self.left_callback,  10)
   self.create_subscription(Imu,       '/imu',             self.imu_callback,   10)
   self.create_subscription(Odometry,
               '/diff_drive_controller/odom', self.odom_callback, 10)

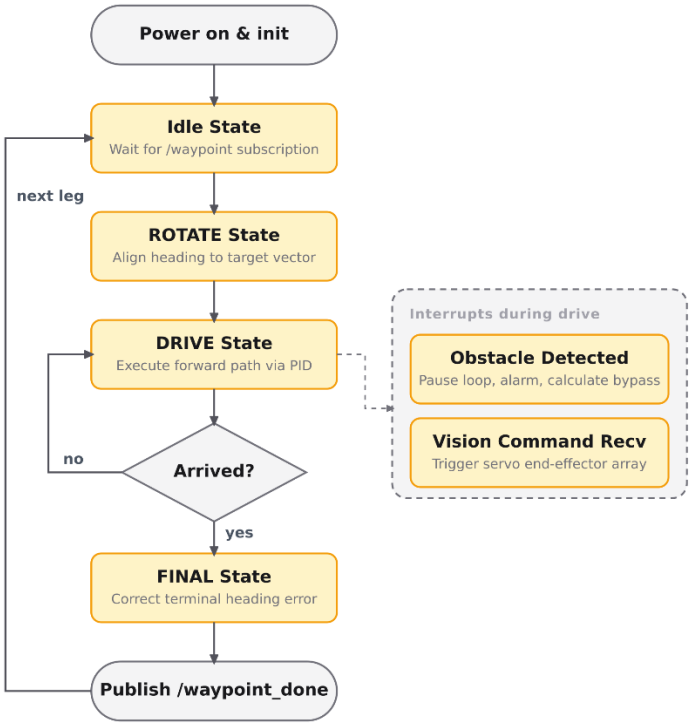
   # Publishers
   self.publisher = self.create_publisher(
       TwistStamped, '/diff_drive_controller/cmd_vel', 10)
   self.arm_publisher = self.create_publisher(
       JointTrajectory, '/arm_controller/joint_trajectory', 10)

   # Control loop timer
   self.create_timer(0.05, self.control_loop)
```

The system is launched via a **ROS 2 launch file** that starts all required components in a single command: the Gazebo simulation, robot state publisher, `ros_gz_bridge` nodes for sensor topics (`/ultrasonic`, `/ultrasonic_left`, `/imu`), the diff-drive controller, the arm controller, and the `obstacle_avoidance` navigation node. This ensures the entire system is reproducible and self-contained.

6. Path Completion

The robot completes a **square path** consisting of 4 legs of 3.0 m each, with a **90° right turn** at each corner. This produces **three distinct waypoint orientations** (excluding the home position):



Waypoint	Leg	Approximate Heading	Description
Home	—	0°	Starting position (not counted)
Corner 1	After Leg 1	90° (right)	First turn — new heading
Corner 2	After Leg 2	180°	Second turn — facing opposite
Corner 3	After Leg 3	270°	Third turn — perpendicular
Home	After Leg 4	360° / 0°	Returns to start

```
SQUARE_SIDE_M  = 3.0
TOTAL_LEGS     = 4

elif self.state == "BRAKE_CORNER":
    if self.execute_brake():
        self.begin_turn(+1)          # +1 = right turn
        self.state = "SQUARE_TURN"

elif self.state == "SQUARE_TURN":
    if self.execute_turn():
        self.set_leg_start()
        self.state = "SQUARE_FWD"
```

Each corner involves braking, executing a 90° IMU-tracked turn, and resetting the odometry leg reference before proceeding. The FSM transitions to `SQUARE_DONE` after all four legs are completed, and the robot halts at the home position. The three corners provide three different target orientations as required.

7. Extra Mechanism Simulation

The robot is equipped with a **3-DOF robotic arm** (`link_1_joint`, `link_2_joint`, `link_3_joint`) controlled via `JointTrajectory` messages. The arm actuation is triggered when the robot's x-position crosses −3.0 m during navigation. The robot stops for 5 seconds, the arm executes a pick-up and retract motion, then the robot resumes.

Arm trigger inside SQUARE_FWD:

```
ARM_X_TRIGGER = -3.0

if (not self.arm_triggered
        and self.odom_x <= ARM_X_TRIGGER
        and not self.front_detected()):
    self.arm_triggered  = True
    self.arm_stop_start = self.get_clock().now()
    self.stop()
    self.state = "ARM_STOP"
```

1. The robot stops for 5 seconds (ARM_STOP state) to simulate positioning at a trash collection point.
2. A trajectory is published with two waypoints — the arm extends to a pick-up pose ([0.5, −0.4, 0.6] rad) over 1.5 seconds, then retracts to a stowed pose ([−0.5, −0.4, 0.0] rad) over the next 1.5 seconds.
3. The arm demo runs for 4 seconds (ARM_DEMO state), after which the robot resumes its square path.

The arm joints are driven by position controllers within Gazebo, and all joint motion is visually rendered in the simulation.

Trajectory with two joint waypoints (extend → retract):

```
def send_arm_trajectory(self):
    traj = JointTrajectory()
    traj.joint_names = ['link_1_joint', 'link_2_joint', 'link_3_joint']

    p1 = JointTrajectoryPoint()
    p1.positions  = [0.5, -0.4, 0.6]        # extend / pick-up pose
    p1.time_from_start = Duration(sec=1, nanosec=500_000_000)

    p2 = JointTrajectoryPoint()
    p2.positions  = [-0.5, -0.4, 0.0]       # retract / stowed pose
    p2.time_from_start = Duration(sec=3, nanosec=0)
```

```
    traj.points = [p1, p2]
    self.arm_publisher.publish(traj)
```

FSM states for arm demo:

```
elif self.state == "ARM_STOP":
    self.stop()
    elapsed = (self.get_clock().now() - self.arm_stop_start).nanoseconds / 1e9
    if elapsed >= ARM_STOP_SEC:          # 5 seconds
        self.send_arm_trajectory()
        self.state = "ARM_DEMO"

elif self.state == "ARM_DEMO":
    self.stop()
    elapsed = (self.get_clock().now() - self.arm_demo_start).nanoseconds / 1e9
    if elapsed >= ARM_DEMO_SEC:          # 4 seconds
        self.state = "SQUARE_FWD"        # resume navigation
```

8. Start and Stop Signals

The robot implements a **start/stop control system** via two ROS 2 topics:

- **`/robot_start`** (`std_msgs/Bool`): The robot initializes in an **idle state** (`is_running = False`). It does not move until a `True` message is received on this topic. Upon receiving the start signal, the node logs the start time (used as the time reference for dynamic box spawning) and begins executing the FSM.
- **`/robot_stop`** (`std_msgs/Bool`): A `True` message on this topic immediately halts the robot — all velocity commands are zeroed, and the FSM is paused. The robot remains stopped until a new start signal is issued.

The start/stop check is the first condition evaluated in every control loop iteration. If `is_running` is `False`, the node publishes zero velocity and returns immediately, ensuring no motion occurs regardless of the FSM state. This makes the signals authoritative over all other behaviors.

Subscriptions and callbacks:

```
self.create_subscription(Bool, '/robot_start', self.start_callback, 10)
self.create_subscription(Bool, '/robot_stop',  self.stop_callback,  10)
 self.is_running = False   # robot starts idle
 def start_callback(self, msg: Bool):
    if msg.data and not self.is_running:
        self.is_running = True
        self.node_start_time = self.get_clock().now().nanoseconds / 1e9
        self.get_logger().info(">>> START signal received <<<")
 def stop_callback(self, msg: Bool):
    if msg.data and self.is_running:
        self.is_running = False
        self.stop()
        self.get_logger().info(">>> STOP signal received <<<")
```

Enforcement at the top of every control loop iteration:

```
def control_loop(self):
    if not self.is_running:
        self.stop()
        return
    # ... rest of FSM only executes when is_running == True
```

BILL OF MATERIALS

Table 7 bill of material

Category	Name	Quantity	Price for each (EGP)	Total Price (EGP)
Computing	Raspberry Pi 5 Model B 8GB RAM	1	6,800.00	6,800.00
Actuation	DC motor with encoder	2	475.00	950.00
Motor Driver	H-bridge	1	75.00	75.00
Sensing / Vision	Camera	1	490.00	490.00
Safety	Emergency switch	1	70.00	70.00
Sensing	IMU	1	175.00	175.00
Sensing	Ultrasonic sensor	3	40.00	120.00
Actuation	Servo motor (large)	2	700.00	1,400.00
Actuation	Servo motor (small)	1	85.00	85.00
Indicators	Buzzer	1	10.00	10.00
Power	Buck converter	3	80.00	240.00
Control	ESP32	1	320.00	320.00
Control	Shield	1	230.00	230.00
Electrical	Wago connectors	4	35.00	140.00
Power	On-off switch	1	20.00	20.00
Mechanical	Wheels	2	160.00	320.00
Mechanical	Caster wheels	2	35.00	70.00
Electrical	Wires	100	1.25	125.00
Fasteners	Bolts	50	4.00	200.00
Manufacturing	3D printing	1	5,000.00	5,000.00
Enclosure / Accessories	Raspberry Pi 5 ABS Case	1	210.90	210.90
Enclosure / Accessories	HDMI to Micro-HDMI Cable 1.8m	1	222.30	222.30
Storage	Micro SD Card 64GB	1	564.30	564.30
Power	8A buck module	1	165.00	165.00
Cooling	Raspberry Pi Active Cooler	1	490.00	490.00
Electrical	Jumper wires	1	90.00	90.00
Indicators	LED	2	0.50	1.00
Electrical	Resistor	6	0.20	1.20
Electrical	N-Channel MOSFET (55V, 49A)	2	15.00	30.00
Cooling	Heatsink	1	3.50	3.50
Fasteners	Screw M3x12mm	2	1.15	2.30
Fasteners	Nut M3	2	0.35	0.70
Electrical	BC547 Transistor (45V, 0.1A)	2	0.75	1.50
Electrical	Press Button 2 pin	1	1.50	1.50
Manufacturing	PCB implementation	1	100.00	100.00
Power	Adapter 12Vdc 2A	1	95.00	95.00
Power	Battery (12V 6A)	2	550.00	1,100.00
			Grand Total	**19,919.20**

VISION MODULE EVALUATION

The vision module was designed to provide intelligent perception capabilities for the autonomous trash-collection robot. Its primary objective is to detect and classify different types of recyclable waste and provide reliable information to the robot for autonomous decision-making.

To achieve high detection accuracy, a **custom dataset containing approximately 3000 images** was created specifically for this project, significantly exceeding the minimum project requirement of 100 images. The dataset includes three main waste categories: **plastic water bottles, aluminum cans, and paper cups**. Images were collected under different environmental conditions, including various lighting levels, backgrounds, distances, and object orientations, allowing the model to perform reliably in real-world scenarios.

After data collection, all images were carefully annotated and organized for training. A task-specific object detection model was trained and optimized using the custom dataset instead of relying solely on a generic pre-trained model. This approach improved the model's ability to recognize the target objects with higher accuracy and robustness.

The vision node was integrated into the ROS2 architecture and continuously processes camera frames in real time. For every detected object, the node publishes both the object class and its confidence score, allowing the high-level controller to make appropriate navigation and collection decisions. The vision system successfully supports the robot mission by identifying recyclable objects quickly and accurately during operation.

INTEGRATED MANUFACTURED MECHANICAL SYSTEM

The mechanical system was manufactured based on the approved CAD design and assembled into a complete autonomous mobile robot. The design process emphasized structural stability, lightweight construction, and modularity to simplify maintenance and future upgrades.

The robot body was fabricated using a combination of 3D-printed components and laser-cut acrylic sheets. The 3D-printed parts were utilized for custom brackets, robotic arm joints, motor mounts, and connection elements, while acrylic sheets formed the main chassis and supporting structure. This combination provides sufficient strength while maintaining a relatively low overall weight.

All robot components were properly assembled and securely fixed to ensure reliable operation during movement. The robotic arm and the extra collection mechanism were successfully integrated with the mobile platform, enabling the robot to detect, approach, pick, and transport trash objects.

All sensors, motors, battery packs, controllers, and electrical modules were carefully mounted in dedicated positions to improve accessibility, reduce vibrations, and maintain balanced weight distribution. The emergency stop button, system state indicators, and charging interface were also integrated into the chassis in accessible locations.

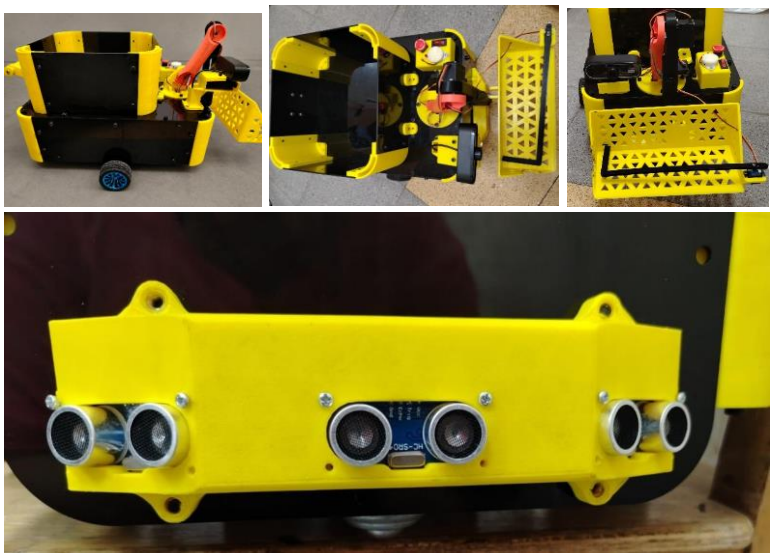


Figure 32 mechanical configuration

The final manufactured robot demonstrates excellent consistency with the submitted CAD model while providing a robust and professional mechanical implementation suitable for continuous operation.

FULLY INTEGRATED ELECTRICAL SYSTEM

A complete electrical architecture was designed and implemented to ensure stable operation, reliable communication, and safe power distribution throughout the robot.

The electrical system integrates the ESP32 microcontroller, Raspberry Pi, motor drivers, IMU sensor, wheel encoders, camera module, battery system, voltage regulators, and auxiliary electronic components into a single organized platform.

Special attention was given to cable routing and management. Power and signal wires were properly organized and secured to prevent interference, accidental disconnection, and maintenance difficulties. Cable labeling and structured wiring significantly improved system reliability and troubleshooting efficiency.

A dedicated **functional safety circuit** was implemented as a core component of the electrical design. The emergency stop system immediately disables robot motion while protecting hardware components and ensuring operator safety. Multiple state indicators provide visual feedback regarding system status, allowing users to quickly identify whether the robot is idle, operating, or in an emergency state.

The electrical system underwent extensive integration testing and demonstrated reliable operation under continuous task execution, providing stable power delivery and seamless interaction between all subsystems.

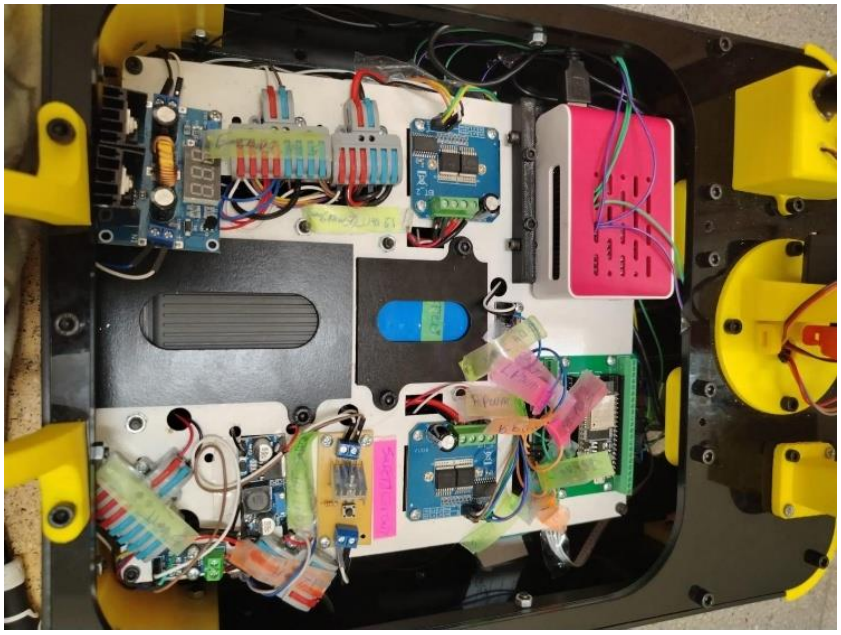


Figure 33 electrical implementation

MICRO-ROS COMMUNICATION SETUP

Communication between the ESP32 and Raspberry Pi was established using the **micro-ROS framework**, creating a distributed architecture that separates low-level embedded control from high-level intelligent processing.

The ESP32 operates as the real-time controller responsible for motor control, sensor acquisition, and hardware interaction, while the Raspberry Pi executes high-level ROS2 applications including computer vision and decision-making algorithms.

Publisher and subscriber nodes were successfully implemented and tested to exchange information between both processing units. Sensor readings, robot status, motor commands, and vision results are transmitted continuously through ROS2 topics, ensuring synchronized operation of all robot subsystems.

The communication framework provides reliable data transfer with low latency, allowing the robot to respond efficiently to environmental changes and mission requirements. The modular ROS2 architecture also simplifies debugging, future development, and integration of additional sensors or intelligent algorithms.

The successful implementation of micro-ROS communication enabled seamless cooperation between embedded hardware and high-level software, completing the integration of the autonomous trash-collection robot into a fully functional mechatronic system.

HARDWARE REQUIREMENTS

The hardware system was designed with a strong emphasis on safety, reliability, and ease of operation. All mechanical and electrical components were carefully integrated to ensure stable performance while maintaining a clean and organized layout suitable for continuous operation.

1. Main Power Control

A dedicated **ON/OFF switch** was integrated into the robot as the main power control interface. This switch allows the user to safely power the entire system on or off without disconnecting the battery, providing a simple and convenient method for startup and shutdown. The switch also enhances user safety and simplifies maintenance and transportation of the robot.

2. Emergency Stop System

A dedicated **Emergency Stop (E-Stop)** system was implemented as a primary safety feature of the robot. When the E-Stop button is activated, motion is immediately halted by disabling all actuators, including the **DC drive motors** responsible for robot navigation and the **servo motors** controlling the extra collection mechanism. This prevents any unexpected movement and protects both the user and the robot during testing or emergency situations.

The safety circuit was designed to provide a rapid response while maintaining the integrity of the electronic system, making it a reliable protection mechanism during robot operation.

3. State Indicators

To provide clear feedback about the robot's operating condition, multiple state indicators were integrated into the hardware system.

An **LED indicator** is used to represent the operation of the extra collection mechanism. Whenever the robot starts collecting a detected trash object, the LED is activated, providing a clear visual indication that the mechanism is currently in operation.

A **buzzer** is utilized as an audible warning indicator. Whenever the robot detects an obstacle or encounters a condition that prevents normal movement, the buzzer is activated to notify nearby users and indicate that the robot is performing an avoidance action or has temporarily stopped for safety reasons.

These visual and audio indicators allow the user to quickly understand the current robot status without relying on software monitoring.

4. Charging Interface

The robot is equipped with a dedicated charging port that enables battery charging without disassembling the robot structure. This feature improves usability, reduces maintenance time, and protects the internal electrical connections while providing convenient access for charging operations.

5. Wiring and Hardware Integration

Special attention was given to the physical integration of all hardware components. The ESP32, Raspberry Pi, motor drivers, battery, sensors, and power modules were securely mounted inside the chassis to minimize movement and vibration during operation.

Power and signal cables were carefully routed and organized to reduce electrical interference and prevent accidental disconnection. The clean wiring layout, secure mounting, and efficient cable management contribute to the robustness, maintainability, and professional appearance of the final system.

Overall, the hardware implementation successfully satisfies the project requirements by combining a main power control switch, functional safety mechanisms, clear state indicators, convenient charging capability, and clean subsystem integration into a reliable autonomous robotic platform.

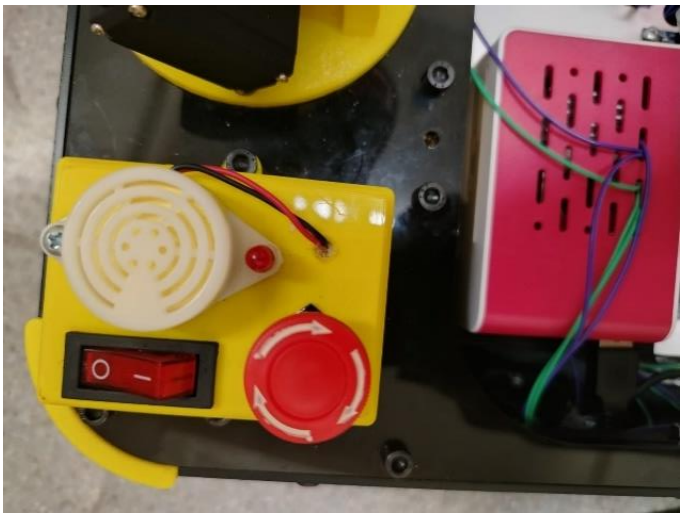


Figure 34 emergency switch and indicators

NAVIGATION

The navigation system was designed to enable the autonomous trash-collection robot to move safely and efficiently while completing its assigned mission. The robot combines precise motion control with obstacle detection and avoidance algorithms to ensure reliable operation in different environments.

1. Autonomous Maneuvering

The robot is capable of autonomous navigation using a closed-loop control system based on PID control, IMU feedback, and wheel encoder measurements. These components work together to maintain stable movement, accurate turning, and consistent path following throughout the mission.

The navigation algorithm continuously calculates the desired trajectory and compares it with the robot's current position and orientation. Any deviation is corrected in real time by adjusting the speed of the DC motors, allowing the robot to move smoothly while maintaining its intended path.

2. Static Obstacle Avoidance

An **ultrasonic sensor** is used to detect stationary obstacles located in front of the robot. During navigation, the sensor continuously measures the distance to nearby objects and sends this information to the control system.

When a static obstacle is detected within the predefined safety range, the robot automatically stops its forward movement and calculates an alternative path to avoid the obstacle. After successfully bypassing the object, the robot returns to its original trajectory and continues executing its mission without requiring any user intervention.

To provide clear feedback during this process, the **buzzer is activated**, informing nearby users that obstacle avoidance is currently in progress.

3. Dynamic Obstacle Avoidance

The navigation system also handles dynamic obstacles, such as moving objects or people crossing the robot's path. The ultrasonic sensor continuously monitors the environment, allowing the robot to detect changes in real time.

Unlike static obstacles, the robot does not attempt to generate a new path for moving objects. Instead, it immediately stops and remains in a safe position while continuously monitoring the

obstacle. Once the obstacle moves away and the path becomes clear, the robot automatically resumes its normal navigation and continues its assigned mission.

During the stopping period, the buzzer remains active as an audible safety warning to indicate that the robot has detected an obstacle and is temporarily paused.

Navigation Performance

The integration of PID control, IMU measurements, wheel encoder feedback, and ultrasonic sensing provides a reliable and robust autonomous navigation system. The robot demonstrates smooth path following, accurate turning, effective static obstacle avoidance through path replanning, and safe dynamic obstacle handling through temporary stopping and automatic mission resumption.

This integrated navigation strategy enables the robot to operate safely while maintaining consistent performance and successfully completing the autonomous trash-collection task.

GRAPHICAL USER INTERFACE (GUI)

A dedicated Graphical User Interface (GUI) was developed to provide an intuitive and user-friendly platform for monitoring and controlling the autonomous trash-collection robot. The interface enables the operator to interact with the robot in real time while displaying essential system information in a clear and organized layout.

1. Connection and Communication

The GUI communicates wirelessly with the Raspberry Pi through a Wi-Fi connection, allowing the operator to establish a remote connection by entering the IP address and login credentials. Once connected, the interface starts the robot software stack and continuously exchanges data with the robot, enabling real-time monitoring and control.

2. Robot Operating Modes

The interface provides two operating modes:

- Automatic Mode: The robot performs the complete trash-collection mission autonomously, including navigation, obstacle avoidance, object detection, and collection without user intervention.
- Manual Mode: The operator can directly control the robot for testing, calibration, or maintenance purposes using movement commands such as Forward, Backward, Left, and Right, in addition to stopping the robot whenever required.

Live Data Visualization

The GUI continuously displays live telemetry information, allowing the user to monitor the robot status during operation. The displayed information includes:

- Current robot state (Navigating, Static Avoidance, Obstacle Wait, Emergency Stop, etc.).
- Obstacle detection status.
- Trash detection status.
- Number of completed mission legs.
- Number of collected objects.

In addition, the interface visualizes the readings of the front proximity ultrasonic sensors (Left, Center, and Right) using graphical progress bars, enabling the user to easily observe the surrounding environment and monitor obstacle distances in real time.

3. Mission Control

The GUI provides dedicated control buttons for mission management, including:

- Start Mission to initiate the autonomous operation.
- Emergency Stop to immediately halt the robot for safety purposes.

These controls allow the operator to safely start, monitor, and terminate the mission whenever necessary.

System Monitoring and Alerts

A system alerts log is integrated into the interface to display real-time operational messages generated by the robot. The log records important events such as:

- Connection establishment.
- Robot startup and shutdown.
- ROS bridge initialization.
- State transitions.
- Emergency events.
- Background process status.

Additionally, a live state console displays the sequence of robot states, such as NAVIGATING, OBSTACLE_WAIT, STATIC_CHECK, STATIC_AVOIDANCE, and EMERGENCY_STOP, providing detailed insight into the robot's behavior throughout the mission.

4. GUI Performance

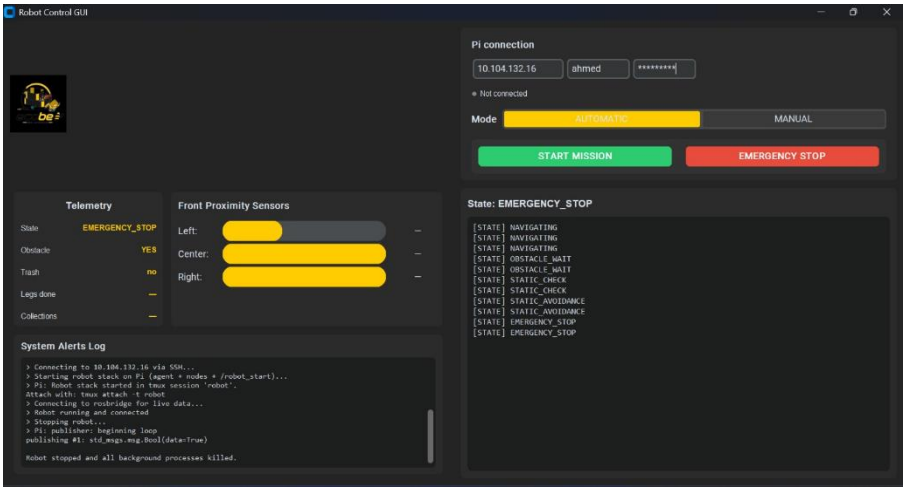


Figure 35 GUI interface

The developed GUI successfully integrates robot control, live telemetry visualization, sensor monitoring, wireless communication, and mission management into a single interface. Its organized layout and real-time feedback simplify robot operation, improve user interaction, and provide an efficient platform for monitoring the overall performance of the autonomous trash-collection system.

MISSION EXECUTION

The autonomous trash-collection robot was designed to execute a complete mission sequence with minimal human intervention by integrating the vision, navigation, control, and manipulation subsystems into a single intelligent platform.

1. Mission Workflow

The mission begins with the robot navigating autonomously along its predefined path while continuously scanning the surrounding environment using the onboard vision system. The camera processes the incoming images in real time and searches for mission-relevant objects, including plastic water bottles, aluminum cans, and paper cups.

Once a target object is successfully detected, the robot identifies it as the current mission objective and immediately initiates the collection sequence.

2. Detection and Collection Process

After detecting a trash object, the buzzer is activated to indicate the presence of an object in front of the robot and to notify nearby users that the collection process is about to begin. During this stage, the detected object is treated as an operational obstacle to prevent unnecessary robot movement while the collection mechanism is active.

The extra mechanism, consisting of a robotic arm equipped with a scoop-shaped end effector, is then activated. The robotic arm moves toward the detected object and performs the collection process by carefully lifting the trash item using the scoop mechanism.

After successfully picking up the object, the robotic arm transports it to the storage box located at the rear of the robot and releases it inside the container. Once the collection process is completed, the arm returns to its initial position, allowing the robot to continue its autonomous navigation and search for the next target.

3. Extra Mechanism Functionality

The extra mechanism was specifically designed to support the trash-collection mission. It consists of a robotic arm integrated with a custom scoop end effector, providing a simple and reliable solution for collecting recyclable waste objects without requiring a complex gripping mechanism.

The mechanism operates in synchronization with the vision and control systems, ensuring that object detection, arm movement, and trash placement are executed as a single coordinated process.

4. System Integration and Repeatability

The mission execution demonstrates the successful integration of the vision module, navigation system, PID motor control, robotic arm, electrical system, and micro-ROS communication architecture. All subsystems operate cooperatively to perform the complete trash-collection task.

Multiple experimental runs confirmed that the robot consistently follows the same operational sequence: autonomous navigation, object detection, collection, transportation to the rear storage box, and mission continuation. This repeatable behavior verifies the stability and reliability of the overall mechatronic system and highlights the effectiveness of the implemented hardware and software integration.

The successful completion of the mission validates the robot's capability to autonomously detect, collect, and store recyclable waste while maintaining safe operation and coordinated subsystem performance.

HIGH-LEVEL CONTROL

The high-level control architecture of the autonomous trash-collection robot was designed using the ROS2 framework, providing a modular, scalable, and distributed software structure. Each subsystem is implemented as an independent node responsible for a specific task, allowing all robot functions to operate simultaneously while maintaining efficient communication and synchronization.

1. Modular ROS2 Nodes

The software architecture consists of multiple ROS2 nodes, each handling a dedicated function within the robot system:

- **Vision Node:** Processes camera images, performs object detection, and publishes the detected object class along with its confidence score.
- **Navigation Node:** Receives sensor data and controls the robot navigation logic, including path following and obstacle avoidance.
- **Motor Control Node:** Communicates with the ESP32 through micro-ROS and executes movement commands using the PID controller.
- **GUI Node:** Provides a user-friendly interface for monitoring robot status, displaying telemetry data, and sending manual or automatic control commands.

This modular design simplifies debugging, maintenance, and future system expansion while improving the overall reliability of the robot.

2. Launch File Implementation

To simplify system startup, a dedicated **ROS2 Launch File** was developed. Instead of manually starting each node individually, the launch file initializes the complete robot software stack with a single command.

The launch file automatically starts the Vision, Navigation, Motor Control, Communication, and GUI nodes while establishing the required ROS2 communication channels between them. This approach reduces initialization time, minimizes user errors, and ensures that all subsystems are synchronized before mission execution.

3. micro-ROS Communication

A **micro-ROS communication framework** was implemented to establish reliable data exchange between the Raspberry Pi and the ESP32 microcontroller.

The Raspberry Pi executes the high-level ROS2 nodes responsible for vision processing, navigation, and decision making, while the ESP32 performs low-level real-time motor control and hardware interfacing. Publisher and subscriber nodes continuously exchange commands, sensor readings, and robot status information, allowing all subsystems to operate as a unified intelligent platform.

4. GPIO Utilization

The General Purpose Input/Output (GPIO) pins were utilized extensively to interface the processing units with the robot hardware.

ESP32 GPIO

The ESP32 GPIO pins are responsible for controlling the robot actuators and indicators, including:

- DC drive motors for autonomous navigation.
- Servo motors that operate the robotic arm and extra collection mechanism.
- LED indicators that display the operational status of the robot.
- Buzzer used as an audible warning system during obstacle detection and trash collection.

Raspberry Pi GPIO

The Raspberry Pi GPIO pins are connected to the three front ultrasonic sensors (Left, Center, and Right). These sensors continuously measure the distance to nearby objects and provide real-time data for obstacle detection and navigation decisions.

The integration of GPIO interfaces with ROS2 and micro-ROS enables efficient communication between software and hardware, allowing sensor readings and actuator commands to be processed with minimal latency.

High-Level System Performance

The modular ROS2 architecture, centralized launch file, efficient micro-ROS communication, and optimized GPIO utilization create a robust high-level control system capable of coordinating vision, navigation, motor control, and user interaction simultaneously. This architecture ensures reliable autonomous operation while maintaining flexibility for future development and additional robot functionalities.

SYSTEM INTEGRATION

The final robot represents the successful integration of mechanical, electrical, control, communication, vision, and user interface subsystems into a single autonomous platform. All components operate cooperatively to accomplish the trash-collection mission without human intervention.

The Vision Node detects recyclable objects and publishes the detection results through ROS2 topics. The Navigation Node processes obstacle information from the ultrasonic sensors and determines the appropriate robot movement. The Motor Control Node executes the required actions using PID control on the ESP32 through micro-ROS communication, while the robotic arm and scoop mechanism collect the detected object and place it into the rear storage box.

Simultaneously, the GUI provides live telemetry, robot status, sensor visualization, and manual control through a Wi-Fi connection, allowing the operator to monitor and interact with the robot in real time.

The successful integration of all subsystems demonstrates a reliable and robust mechatronic system capable of autonomous navigation, intelligent object detection, obstacle avoidance, and automated trash collection while maintaining safe and stable operation.



Figure 36 final prototype

CONCLUSION

This project successfully demonstrates the design, implementation, and integration of a semi-autonomous trash-collection robot that combines mechanical, electrical, and software engineering principles into a unified mechatronic system. Throughout the development process, individual subsystems were designed, tested, and integrated to achieve reliable and efficient operation.

A custom vision system was developed using a dataset of approximately 3000 images to detect plastic water bottles, aluminum cans, and paper cups with confidence-based object recognition. The manufactured mechanical structure, built using 3D-printed and acrylic components, provides a stable platform for the robotic arm and scoop end effector, enabling effective trash collection and transportation to the rear storage box.

The electrical system incorporates a functional safety circuit, emergency stop mechanism, state indicators, organized wiring, and a dedicated charging interface to ensure safe and reliable operation. Real-time motor control is achieved using RTOS-based PID controllers with IMU and encoder feedback, allowing smooth navigation and accurate movement. Static and dynamic obstacle avoidance are implemented using ultrasonic sensors, enhancing the robot's operational safety.

A modular ROS2 architecture with micro-ROS communication enables seamless interaction between the Raspberry Pi and ESP32, while a custom GUI provides wireless communication, live system monitoring, data visualization, and manual control capabilities. These integrated subsystems allow the robot to detect target objects, navigate the environment, activate the robotic arm, and collect trash with minimal operator intervention.

Overall, the developed system meets the project objectives and demonstrates a reliable, scalable, and well-integrated **semi-autonomous robotic platform**. The project reflects the successful application of robotics, embedded systems, computer vision, and control engineering concepts and provides a strong foundation for future improvements, including enhanced localization, advanced path planning, and increased autonomous decision-making capabilities.

Drive Link: https://drive.google.com/drive/folders/1cf_CfxXd5BI-DKrL7J2z1NhNBRvUgok7